

What is a language?

A language is a communication media through which we can communicate with each other.

What is a programming language?

A Programming language is an intermediate between the user and computer System.

Every language contains two important aspects :

- 1) Syntax (Rules given by the language)
- 2) Semantics (Meaning OR Structure of the language)

In English language, if we want to make a translation then the syntax is :

Subject + verb + Object

She is a girl. [Valid]

She is a box. [Invalid but still I am following the syntax]

In our programming language also :

```
int x = 12;
```

```
int y = 0;
```

```
int z = x / y;
```

In Java programming language we have 2 types of Security :

1) At Compilation level : Here our java compiler will verify whether the code is valid or not according to the syntax.

2) At Runtime Level : Here our runtime environment will verify the semantics of the code that means the code is meaningful or not?

Statically(Strongly) typed language :-

The languages where data type is compulsory before initialization of a variable are called statically typed language.

In these languages we can hold same kind of value during the execution of the program.

Ex:- C,C++,Core Java, C#

Dynamically(Loosely) typed language :-

The languages where data type is not compulsory and it is optional before initialization of a variable then it is called dynamically typed language.

In these languages we can hold different kind of value during the execution of the program.

Ex:- Visual Basic, Javascript, Python

What is a function :-

A function is a self defined block for any general purpose, calculation or printing some data.

The major benefits with function are :-

- 1) Modularity :- Dividing the bigger modules into number of smaller modules where each module will perform its independent task.
- 2) Easy understanding :- Once we divide the bigger task into number of smaller tasks then it is easy to understand the entire code.
- 3) Reusability :- We can reuse a particular module so many number of times so It enhances the reusability nature.

Note :- In java we always reuse our classes.

- 4) Easy Debugging :- Debugging means finding the errors, With function It is easy to find out the errors because each module is independent with another module.

Why we pass parameter to a function :-

We pass parameter to a function for providing more information regarding the function.

```
public void deposit()
{
}
```

Here we don't have parameter so, the information is partial, if we pass parameter then we will get complete information.

```
public void deposit(int amount)
{
}
```

Why functions are called method in java?

In C++ there is a facility to write a function inside the class as well as outside of the class by using :: (Scope resolution Operator), But in java all the functions must be declared inside the class only.

That is the reason member functions are called method in java.

Variable --> Field
function ---> Method

int data; Field

```
public void m1(int y)
{
    int x = 10;
}
```

Flavors of Java :

We have total 4 types of flavors :

- 1) JSE (Java Standard Edition) [Core Java]
- 2) JEE (Java Enterprise Edition) [Advanced Java]

3) JME (Java Micro Edition) [Android Application]

4) JavaFX (It is used to design or develop GUI Applications) [Outdated]

GUI = Graphical User interface

What is the difference between **Stand-alone program** and **web-related Program** ?

If the creation, compilation and execution of the program, everything is done in a single system then it is called **Stand-alone Programs OR Desktop Application OR Software.**

By using **JSE** we can develop stand-alone application.

On the other hand, if the creation of the program, compilation of the program and execution of the program, everything is done in different system in different places then it is called **web-related application OR Websites.**

By using JEE we can develop websites.

History of Java :

Father of Java : James Gosling

First Name of Java : Oak (Tree Name)

Birthday of Java : 23rd Jan 1996

Project Name : Green Project

Official Symbol : Coffee cup

java : Island in indonesia where coffee beans are prepared.

Why java become so popular in the IT Industry ?

The role of Java compiler :

- a) Compiler are used to check the syntax.
- b) It also check the compatibility issues(LHS = RHS)
- c) It converts the source code into machine code.

Java code :

- a) Java programs must be saved having extension .java
- b) java compiler(javac) is used to compile our code.
- c) After successful compilation we are getting .class file (bytecode)
- d) This .class file we submit to JVM for execution purpose (for executing my java code)

JVM :- It stands for Java Virtual Machine. It is a software in the form of interpreter written in 'C'

language.

Every browser contains JVM, Those browsers are known as JEB (Java Enabled Browsers) browsers.

C and C++ programs are platform dependent programs that means the .exe file created on one machine will not be executed on the another machine if the system configuration is different.

That is the reason C and C++ programs are not suitable for website development.

Where as on the other hand java is a platform independent language. Whenever we write a java program, the extension of java program must be .java.

Now this .java file we submit to java compiler (javac) for compilation process. After successful compilation the compiler will generate a very special machine code file i.e .class file (also known as bytecode). Now this .class file we submit to JVM for execution purpose.

The role of JVM is to load and execute the .class file. Here JVM plays a major role because It converts the .class file into appropriate machine code instruction (Operating System format) so java becomes platform independent language and it is highly suitable for website development.

Note :- We have different JVM for different Operating System that means JVM is platform dependent technology where as Java is platform Independent technology.

26-03-2024

What is the difference between the bit code and byte code.

Bit code is directly understood by Operating System but on the other hand byte code is understood by JVM, JVM is going to convert this byte code into machine understandable format.

Comments in JAVA :-

Comments are used to enhance the readability of the program. It is ignored by the compiler. In java we have 3 types of comments

- 1) Single line Comment (//)
- 2) Multiline Comment (/ * ----- */)
- 3) Documentation Comment (/ ** ----- */)

/**

Name of the Project : Online Shopping

Date created :- 12-12-2021

Last Modified - 16-01-2022

Author :- Ravishankar

Modules : - 10 Modules

*/

WAP in java to display welcome message :

At initial level we can write a java program with the help class and methods.

Syntax for a class

[Access Modifier] class [Name of the class]
{

}

Example :-
public class Test
{
}

Syntax for a method :

[Access modifier] [return type] [Name of the method] ()
{

}

Example :-
public void accept()
{
}

WAP in java to display welcome message :

Welcome.java

```
public class Welcome
{
    public static void main(String[] args)
    {
        System.out.println("Welcome to Java Batch 32");
    }
}
```

Note :-

- 1) In java whenever we write a program we need at least a main method.
- 2) In java the execution of the program always starts and ends with main method.

Description of main() method :

public :-

public is an access modifier in java. The main method must be declared as public otherwise JVM cannot execute our main method or in other words JVM can't enter inside the main method for execution of the program.

If main method is not declared as public then program will compile but it will not be executed by JVM.

Note :- From java compiler point of view there is no rule to declare our methods as public.

static :-

In java our main method is static so we need not to create an object to call the main method.

If a method is declared as a static then we need not to create an object to call that method. We can directly call the static methods with the help of class name (Object is not required).

If we don't declare the main method as static method then our program will compile but it will not be executed by JVM.

27-03-2024

void :-

It is a keyword. It means no return type. Whenever we define any method in java and if we don't want to return any kind of value from that particular method then we should write void before the name of the method.

Eg:

```
public void input()          public int accept()
{                             {
}                             return 15;
}
```

Note :- In the main method if we don't write void or any other kind of return type then it will generate a compilation error.

In java whenever we define a method then compulsory we should define return type of method.(Syntax rule)

main() :-

It is a user-defined method because a user is responsible to define some logic inside the main method.

main() method is very important method because every program execution will start from main() method only, as well as the execution of the program ends with main() method only.

We can write multiple main methods in our class but argument must be different.

Test.java

```
public class Test
{
    public static void main(String args)
    {
        System.out.println("Welcome to Java");
    }
}
```

```
public static void main(String [] args)
{
    System.out.println("Welcome to Hyd");
}
```

```
}  
}
```

Output : Welcome to Hyd

Command Line Argument (Introduction only) :-

Whenever we pass an argument/parameter to the main method then it is called Command Line Argument.

The argument inside the main method is String because String is a alpha-numeric collection of character so, It can accept numbers, decimals, characters, combination of number and character.

That is the reason java software people has provided String as a parameter inside the main method. (More Wider scope to accept the multiple values)

System.out.println() :-

It is an output statement in java, By using System.out.println() statement we can print anything on the console.

In System.out.println(), System is a predefined class available in java.lang package, out is a final, static reference variable of PrintStream class available in java.io package and println() is a predefined method available in PrintStream class.

In System.out, .(dot) is a member access operator. It is called as period. It is used to access the member of the class.

Actually, It creates HAS-A relation with System and PrintStream class.

WAP in java to add two numbers :

```
public class Addition  
{  
    public static void main(String[] args)  
    {  
        int x = 100;  
        int y = 200;  
        int z = x + y;  
        System.out.println(z);  
    }  
}
```

In the above program we are getting the output as 300 but it is not user friendly because proper message is not available.

How to provide user friendly message to our client ?

```
public class AdditionWithMessage  
{  
    public static void main(String[] args)  
    {  
        int x = 12;  
        int y = 24;
```

```
int z = x + y;
System.out.println("The Sum is :"+ z);
}
}
```

//WAP to add two numbers without 3rd variable

```
public class AdditionWithout3rdVariable
{
    public static void main(String[] args)
    {
        int x = 12;
        int y = 67;
        System.out.println("The Sum is :"+x+y);
        System.out.println(+x+y);
        System.out.println(""+x+y);
        System.out.println("The Sum is :"+(x+y));
    }
}
```

28-03-2024

//Program on static method to add two numbers which is in same class

```
public class Test
{
    public static void main(String[] args)
    {
        int result = add(12,24);
        System.out.println("The result is :"+result);

        result = add(100,200);
        System.out.println("The result is :"+result);
    }
}
```

```
public static int add(int x , int y)
{
    int sum = x + y;
    return sum;
}
}
```

//Program on static method to find out the area of Rectangle where
Rectangle is another class

```
public class Test
{
    public static void main(String[] args)
    {
        double areaOfRect = Rectangle.getAreaOfRectangle(2,7.9);
        System.out.println("Area of Rectangle is :"+areaOfRect);

        areaOfRect = Rectangle.getAreaOfRectangle(5,4);
        System.out.println("Area of Rectangle is :"+areaOfRect);
    }
}
```



```
class Rectangle
{
    public static double getAreaOfRectangle(double l, double b)
    {
        double area = l * b;
        return area;
    }
}
```

IQ :

```
public class Test
{
    public static void main(String[] args)
    {
        String str = 40 + 40 + "NIT" + 40 + 40;
        System.out.println(str);
    }
}
```

What is command Line Argument :

Whenever we pass any argument to the main method then it is called Command Line Argument.

By using Command Line Argument, we can pass some value at runtime.

The advantage of command line argument is, single time compilation and number of times execution.

//Program on Command Line Argument

```
public class Command
{
    public static void main(String[] args)
    {
        System.out.println(args[0]);
    }
}
```

```
javac Command.java (Compilation)
java Command Virat Rohit Bumrah
```

Note : Here It will print Virat.

```
public class Command
{
    public static void main(String[] args)
    {
        System.out.println(args[1]);
    }
}
```

```
javac Command.java (Compilation)
java Command 100 200 300
```

Note : Here It will print 200.

```
//Command Line Argument Program to print full name
public class Command
{
    public static void main(String[] args)
    {
        System.out.println(args[0]);
    }
}
```

```
javac Command.java (Compilation)
java Command "Virat Kohli"
```

Note : Here It will print Virat Kohli

```
public class Command
{
    public static void main(String[] args)
    {
        System.out.println(args[1]);
    }
}
```

```
javac Command.java (Compilation)
java Command 100
```

Note : Here It will generate an Exception i.e java.lang.ArrayIndexOutOfBoundsException

//WAP in java to add two numbers by using Command Line Argument

```
public class CommandAdd
{
    public static void main(String[] x)
    {
        System.out.println(x[0] + x[1]);
    }
}
```

```
javac CommandAdd.java
java CommandAdd 12 36
```

Output is : 1236

29-03-2024

Eclipse IDE :

IDE stands for Integrated Development Environment. By using Eclipse IDE, we can develop, compile and execute our program in a single window.

The main purpose of Eclipse IDE to reduce the development time, Once development time will be reduced, automatically the cost of the project will be reduced.

WAP in Eclipse IDE to print full name of user through command Line Argument

```
package com.nit.basic;
```

```
public class CommandFullName
{
    public static void main(String[] args)
    {
        System.out.println(args[0]);
    }
}
```

How to execute command line argument program by using eclipse IDE :

In order to execute command line Argument program, right click on the program -> run as -> run configuration -> verify project name and main class -> In the Argument tab, pass appropriate value -> click on the run button

What is a Package ?

A package is nothing but folder in windows or directory in DOS.

A package is of two types :

- 1) Predefined Package :- Created by java software people.
Example :- java.lang, java.util, java.io, java.text, java.awt and so on
- 2) Userdefined Package :- Created by user.
Example :- com.ravi.basic, com.ravi.introduction, com.ravi.oop and so on

The program which contains package statement is different style of compiling :

```
javac -d . Test.java
(javac space -d space . space FileName.java)
```

How to convert String into Integer :

If we want to convert any String value into corresponding integer then java software people has provided predefined class called Integer available in java.lang package.

In this class we have predefined static method, parseInt(String x) which is accepting a single parameter String as a parameter and convert this String into int value as shown in the below example.

CommandAddition.java

package com.nit.basic;

```
public class CommandAddition {

    public static void main(String[] x)
    {
        int num1 = Integer.parseInt(x[0]);
        int num2 = Integer.parseInt(x[1]);

        System.out.println(num1 + num2);
    }
}
```

```
}
```

```
}
```

Note :- At the time of passing the value from command line argument we need to pass the numeric value, other than numeric value will generate an exception `java.lang.NumberFormatException`.

30-03-2024

How `Integer.parseInt(String x)` working internally?

```
package com.ravi.command;
```

```
class Calculater
```

```
{  
    public static int doSum(int x, int y)  
    {  
        int z = x + y;  
        return z;  
    }  
}
```

```
    public static int getSquare(int num)  
    {  
        return (num*num);  
    }  
}
```

```
    public static int getCube(int num)  
    {  
        return (num*num*num);  
    }  
}
```

```
public class Main  
{
```

```
    public static void main(String[] args)  
    {  
        int result = Calculater.doSum(10, 20);  
        System.out.println("Sum is :"+result);
```

```
        result = Calculater.getSquare(10);  
        System.out.println("Square is :"+result);
```

```
        result = Calculater.getCube(5);  
        System.out.println("Cube is :"+result);  
    }
```

```
}
```

How to convert the String value based on the radix/base

In the `Integer` class, there is predefined overloaded method called `parseInt(String str, int base/radix)`, It will convert the String into integer using specified base.

```
package com.ravi.command;

public class Test
{
    public static void main(String[] args)
    {
        String str = "111";

        int value = Integer.parseInt(str,2);
        System.out.println("value is :"+value);

    }
}
```

How to find out the length of array variable :

We have a predefined variable/property called length available in array object to find out the length of the Array as bshown in the program below

```
package com.ravi.command;

public class Test
{
    public static void main(String[] args)
    {
        Object arr[] = {12, "NIT", true, 89.90};
        System.out.println("Length of the Array is :"+arr.length);
    }
}
```

```
package com.ravi.command;

public class Test
{
    public static void main(String[] args)
    {
        if(args.length == 0)
        {
            System.err.println("Length is 0");
        }
        else if(args.length ==1)
        {
            System.out.println(args[0]);
        }
        else if(args.length == 2)
        {
            int x = Integer.parseInt(args[0]);
            int y = Integer.parseInt(args[1]);
            int z = x+y;
            System.out.println("Sum is :"+z);
        }
    }
}
```

```
}  
}
```

```
}
```

*Naming convention in Java :

1) How to write a class in java

While writing a class in java we should follow pascal naming convention. A java class represents noun.

ThisIsExampleOfClass (Each word first letter is capital)

Example :

String
System
Integer
BufferedReader
DataInputStream
ClassNotFoundException
ArithmeticException

2) How to write a method in java :

In order to write methods in java we need to follow camel case naming convention. A java method represents verb.

thisIsExampleOfMethod()

Example:

read()
readLine()
toUpperCase()
charAt()
parseInt()

3) How to write variable(Fields) in java

In order to write variables in java we need to follow camel case naming convention.

rollNumber;
employeeName;
customerNumber;
customerBill;

4) How to write final variable(Field)

Final variables must be in upper-case letter.

final double PI = 3.14;
final int A = 90;

5) How to write final and static variable

MAX_VALUE;

MIN_VALUE;

Each character must be capital and in between every word _ symbol should be there.

6) How to write package

Package name must be in small character. It is reverse of company name.

com.ravi.basic;

com.tcs.shopping

com.wipro.chating

Token :

A token is the smallest unit of the program that is identified by the compiler.

Every Java statements and expressions are created using tokens.

A token can be divided into 5 types

1) Keywords

2) Identifiers

3) Literals

4) Punctuators

5) Operators

Keyword :-

A keyword is a predefined word whose meaning is already defined by the compiler.

In java all the keywords must be in lowercase only.

A keyword we can't use as a name of the variable, name of the class or name of the method.

true, false and null look like keywords but actually they are literals.

Identifiers :

A name in java program by default considered as identifiers.

Assigned to variable, method, classes to uniquely identify them.

We can't use keyword as an identifier.

Ex:-

```
class Fan
```

```
{  
    int coil ;
```

```
    void start()
```

```
{  
    }  
}
```

}

Here Fan(Name of the class), coil (Name of the variable) and start(Name of the function) are identifiers.

Rules for defining an identifier :

-
- 1) Can consist of uppercase(A-Z), lowercase(a-z), digits(0-9), \$ sign, and underscore (_)
 - 2) Begins with letter, \$, and _
 - 3) It is case sensitive
 - 4) Cannot be a keyword
 - 5) No limitation of length
-

Literals :-

Assigning some constant value to variable is called Literal.

Java supports 6 types of Literals :

- 1) Integral Literal Ex:- int x = 15;
- 2) Floating Point Literal Ex:- float x = 3.5f;
- 3) Character Literal Ex:- char ch = 'A';
- 4) Boolean Literal Ex:- boolean b = true;
- 5) String Literal Ex:- String x = "Naresh i Technology";

Note :- null is also a literal.

Integral Literal :

A numeric literal which does not contain any decimal or fraction is called Integral Literal.

Example : 15, 29, 45, 89

In Integral Literal we have 4 data types :

- 1) byte (8 bits)
- 2) short (16 bits)
- 3) int (32 bits)
- 4) long (64 bits)

An Integral Literal, we can represent in 4 different forms

- 1) Decimal Literal
- 2) Octal Literal
- 3) Hexadecimal Literal
- 4) Binary Literal (Introduced from java 1.7 onwards)

Decimal Literal :

By default literal is decimal only where digits are from 0-9.

Octal Literal :

If the Integral literal starts with 0 (zero) then it will become octal literal. Here the range of digit 0 - 7.

Example :

```
int x = 012; //valid
int y = 017; //valid
int z = 018; //Invalid [digit 8 is out of the range]
```

Hexadecimal Literal :

If any Integral Literal starts with 0X or 0x then it will become Hexadecimal literal. Here the allowed digits are 0 - 9 and A - f.

Example :

```
int a = 0x29; //Valid
int b = 0Xadd; //valid
int c = 0Xage; //Invalid
```

Binary Literal :

It was introduced from java 1.7v. Any Integeral literal starts with 0B or 0b then it will become Binary Literal. Here we can accept only 0 and 1.

Example :

```
int a = 0B101; //Valid
int b = 0b111; //valid
int c = 0B1102; //Invalid
```

```
//Octal literal
```

```
public class Test1
{
    public static void main(String[] args)
    {
        int one = 01;
        int six = 06;
        int seven = 07;
        int eight = 010;
        int nine = 011;
        System.out.println("Octal 01 = "+one);
        System.out.println("Octal 06 = "+six);
        System.out.println("Octal 07 = "+seven);
        System.out.println("Octal 010 = "+eight);
        System.out.println("Octal 011 = "+nine);
    }
}
```

```
//Hexadecimal
```

```
public class Test2
{
```

```

public static void main(String[] args)
{
    int i = 0x10;    //16
    int j = 0Xadd;   //2781
    System.out.println(i);
    System.out.println(j);
}
}

```

```

//Binary Literal
public class Test3
{
    public static void main(String[] args)
    {
        int i = 0b101;
        int j = 0B111;
        System.out.println(i); //5
        System.out.println(j); //7
    }
}

```

02-04-2024

By default every integral literal is of type int only but we can specify explicitly as long type by suffixing with l (small l) OR L (Capital L).

According to industry standard L is more preferable because l (small l) looks like 1(digit 1).

There is no direct way to specify byte and short literals explicitly. If we assign any integral literal to byte variable and if the value is within the range (-128 to 127) then it is automatically treated as byte literals because compiler internally converts from int to byte.

If we assign integral literals to short and if the value is within the range (-32768 to 32767) then automatically it is treated as short literals because compiler internally converts from int to short.

/* By default every integral literal is of type int only*/

```

public class Test4
{
    public static void main(String[] args)
    {
        byte b = 128;
        System.out.println(b);

        short s = 32768;
        System.out.println(s);
    }
}

```

//Assigning smaller data type value to bigger data type

```

public class Test5
{
    public static void main(String[] args)
    {

```

```

        byte b = 125;
    short s = b;
    System.out.println(s);
}
}

```

```

//Converting bigger type to smaller type
public class Test6
{
    public static void main(String[] args)
    {
        short s = 127;
        byte b = (byte) s;
        System.out.println(b);
    }
}

```

Note :- In explicit type casting, there is a chance loss of data.

```

public class Test7
{
    public static void main(String[] args)
    {
        byte x = (byte) 127L;
        System.out.println("x value = "+x);

        long l = 29L;
        System.out.println("l value = "+l);

        int y = (int) 18L;
        System.out.println("y value = "+y);
    }
}

```

Is java pure Object-Oriented language ?

No, Java is not a pure Object-Oriented language. In fact any language which accepts the primary data type like int, float, char is not a pure object oriented language hence java is also not a pure object oriented language.

Example of pure object oriented language : Ruby, smalltalk and so on

If we remove all 8 primitive data types from java then Java will become pure object oriented language.

In java we have a concept called Wrapper classes through which we can convert the primary data types into corresponding Wrapper Object.(Autoboxing 1.5V)

Primary data types	Corresponding Wrapper Object
byte - Byte	
short - Short	
int - Integer	
long - Long	
float - Float	
double - Double	

char - Character
boolean - Boolean

All these wrapper classes are available in java.lang package.

```
-----  
//Wrapper classes  
public class Test8  
{  
    public static void main(String[] args)  
    {  
        Integer x = 24;  
        Integer y = 24;  
        Integer z = x + y;  
        System.out.println("The sum is :"+z);  
  
        Boolean b = true;  
        System.out.println(b);  
  
        Double d = 90.90;  
        System.out.println(d);  
    }  
}
```

How to know the minimum and maximum value as well as size of integral literal data types:

These classes (Wrapper classes) are providing the static and final variables through which we can find out the minimum, maximum value as well as size of the data types

Ex:- I want to find out the range and size of Byte class

Byte.MIN_VALUE = -128

Byte.MAX_VALUE = 127

Byte.SIZE = 8 (in bits format)

Here MIN_VALUE, MAX_VALUE and SIZE these are static and final variables available in these classes (Byte, Short, Integer and Long).

```
-----  
//Program to find out the range and size of Integral Data type  
public class Test9  
{  
    public static void main(String[] args)  
    {  
        System.out.println("\n Byte range:");  
        System.out.println(" min: " + Byte.MIN_VALUE);  
        System.out.println(" max: " + Byte.MAX_VALUE);  
        System.out.println(" size :"+Byte.SIZE);  
  
        System.out.println("\n Short range:");  
        System.out.println(" min: " + Short.MIN_VALUE);  
        System.out.println(" max: " + Short.MAX_VALUE);  
        System.out.println(" size :"+Short.SIZE);  
  
        System.out.println("\n Integer range:");
```

```
System.out.println(" min: " + Integer.MIN_VALUE);
System.out.println(" max: " + Integer.MAX_VALUE);
System.out.println(" size :"+Integer.SIZE);
```

```
System.out.println("\n Long range:");
System.out.println(" min: " + Long.MIN_VALUE);
System.out.println(" max: " + Long.MAX_VALUE);
System.out.println(" size :"+Long.SIZE);
```

```
}
}
```

//We can provide _ in integral literal

```
public class Test10
{
    public static void main(String[] args)
    {
        long mobile = 9812_3456_78L;
        System.out.println("Mobile Number is :"+mobile);
    }
}
```

Note :- From java 1.7V now we can provide _ symbol in numeric literal just to increase the readability of the code.

public class Test11
{
 public static void main(String[] args)
 {
 final int x = 127;
 byte b = x;
 System.out.println(b);
 }
}

The above program will generate the output without any error

03-04-2024

// Converting from decimal to another number system

```
public class Test12
{
    public static void main(String[] argv)
    {
        //decimal to Binary
        System.out.println(Integer.toBinaryString(5)); //101

        //decimal to Octal
        System.out.println(Integer.toOctalString(15)); //17

        //decimal to Hexadecimal
        System.out.println(Integer.toHexString(2781)); //add
    }
}
```

Integer class has provided the following predefined static methods to convert decimal to another number system like binary, octal and hexadecimal toBinaryString(), toOctalString() , toHexString()

var keyword :

It is introduced from java 10v.

var keyword we can use inside the method only and initialization is compulsory at the same line.

```
public class Test
{
    public static void main(String[] args)
    {
        var x = 90;
        System.out.println(x);
    }
}
```

floating point literal :

If a numeric literal contains decimal or fraction then it is called floating point literal.

Example : 89.67, 23.78, 56.90

In floating point literal we have 2 data types

- float (32 bits)
- double (64 bits)

Every floating point literal is of type double so, the following statement will not compile

```
float f = 90.9; //error
```

```
float f1 = 90.9f; //valid
float f2 = 90.9F; //valid
float f3 = (float) 90.9; //valid
```

As we know by default every floating point literal is of type double but still compiler has provided two flavors to represent double value explicitly to enhance the readability of the code.

```
double d1 = 12d;
double d2 = 12D;
```

While working with integral literals, we can represent an integral literal in 4 different forms decimal, octal, hexadecimal and binary but while working with floating point literal we have only one form i.e decimal.

We can represent a floating point literal in exponent form also.

```
double d = 15e2;
        = 15 X 10 to the power 2
```

Integral literal we can assign to floating point literal but floating point literal we can't assign to integral literal.

```
public class Test
```

```
{
public static void main(String[] args)
{
    float f = 2.0; //error
    System.out.println(f);
}
}
```

```
public class Test1
{
    public static void main(String[] args)
    {
        float b = 15.29F;
        float c = 15.25f;
        float d = (float) 15.30;

        System.out.println(b+" : "+c+" : "+d);
    }
}
```

```
public class Test2
{
    public static void main(String[] args)
    {
        double d = 15.15;
        double e = 15d;
        double f = 15.15D;

        System.out.println(d+" , "+e+" , "+f);
    }
}
```

```
public class Test3
{
    public static void main(String[] args)
    {
        double x = 0129.89;

        double y = 0167;

        double z = 0178; //error

        System.out.println(x+" , "+y+" , "+z);
    }
}
```

```
class Test4
{
    public static void main(String[] args)
    {
        double x = 0X29;

        double y = 0X9.15; //error

        System.out.println(x+" , "+y);
    }
}
```

```
}  
}
```

```
-----  
public class Test5  
{  
    public static void main(String[] args)  
    {  
        double d1 = 15e-3;  
        System.out.println("d1 value is :"+d1);  
  
        double d2 = 15e3;  
        System.out.println("d2 value is :"+d2);  
    }  
}
```

```
-----  
public class Test6  
{  
    public static void main(String[] args)  
    {  
        double a = 0791; //error  
  
        double b = 0791.0;  
  
        double c = 0777;  
  
        double d = 0Xdead;  
  
        double e = 0Xdead.0; //error  
    }  
}
```

```
-----  
public class Test7  
{  
    public static void main(String[] args)  
    {  
        double a = 1.5e3;  
        float b = 1.5e3;  
        float c = 1.5e3F;  
        double d = 10;  
        int e = 10.0;  
        long f = 10D;  
        int g = 10F;  
        long l = 12.78F;  
    }  
}
```

```
-----  
//Range and size of floating point literal  
public class Test8  
{  
    public static void main(String[] args)  
    {  
        System.out.println("\n Float range:");  
        System.out.println(" min: " + Float.MIN_VALUE);  
        System.out.println(" max: " + Float.MAX_VALUE);  
        System.out.println(" size :"+Float.SIZE);  
    }  
}
```



```

System.out.println("\n Double range:");
System.out.println(" min: " + Double.MIN_VALUE);
System.out.println(" max: " + Double.MAX_VALUE);
System.out.println(" size :"+Double.SIZE);
}
}

```

boolean literal :

It is used to represent two states either true or false.

In boolean literal we have only one data type i.e boolean data type which accepts 1 bit of memory as well as it also depends on JVM implementation.

Unlike C and C++, we cannot assign 0 and 1 to boolean data type because in java any number without decimal is treated as integral literal

boolean isEmpty = 0; [Invalid in Java but valid in C and C++]

We cannot assign String to boolean type as shown below.

boolean isValid = "true"; //Invalid

```

public class Test1
{
    public static void main(String[] args)
    {
        boolean isValid = true;
        boolean isEmpty = false;

        System.out.println(isValid);
        System.out.println(isEmpty);
    }
}

```

```

public class Test2
{
    public static void main(String[] args)
    {
        boolean c = 0;
        boolean d = 1;
        System.out.println(c);
        System.out.println(d);
    }
}

```

```

public class Test3
{
    public static void main(String[] args)
    {
        boolean x = "true";
        boolean y = "false";
        System.out.println(x);
    }
}

```

```
        System.out.println(y);
    }
}
```

04-04-2024

Character Literal :

It is also known as char literal.

Here we have only one data type i.e char data type which accepts 2 bytes of memory.

We can represent char literal in different ways which are as follows :

a) Single character enclosed with single quotes.

Example :- char ch = 'A';

b) We can assign integral literal to char data type to represent the UNICODE value for that character.
In older language like C and C++, the range is 0 - 255 which is representing the ASCII value of the character.

But Java supports UNICODE where the range is 0 - 65535.

```
char ch = 65535; //Valid
char ch = 65536; //Invalid
```

c) The char literal we can assign to integral data type to know the UNICODE number value of that character.

```
int ch = 'A';    -> 65
```

d) All our escape sequences we can represent as char literal.

```
char ch = '\n';
```

e) We can represent char literal in 4 digit hexadecimal number to represent UNICODE value where the format is :

```
'\uXXXX'
```

Here \u stands for UNICODE where as X represents Digits.

Here '\u0000' : Minimum Value

'\uffff' : Maximum value becoz it is hexadecimal

```
public class Test1
{
    public static void main(String[] args)
    {
        char ch1 = 'a';
        System.out.println("ch1 value is :"+ch1);

        char ch2 = 97;
        System.out.println("ch2 value is :"+ch2);
    }
}
```

```
}  
}
```

```
class Test2
```

```
{  
    public static void main(String[] args)  
    {  
        int ch = 'A';  
        System.out.println("ch value is :"+ch);  
    }  
}
```

```
//The UNICODE value for ? character is
```

```
public class Test3  
{  
    public static void main(String[] args)  
    {  
        char ch1 = 63;  
        System.out.println("ch1 value is :"+ch1);  
  
        char ch2 = 64;  
        System.out.println("ch2 value is :"+ch2);  
  
        char ch3 = 65;  
        System.out.println("ch3 value is :"+ch3);  
    }  
}
```

The UNICODE value for ? is 63, @ is 64

```
public class Test4  
{  
    public static void main(String[] args)  
    {  
        char ch1 = 65535;  
        System.out.println("ch1 value is :"+ch1);  
  
        char ch2 = 0Xadd;  
        System.out.println("ch2 value is :"+ch2);  
    }  
}
```

Here we will get the output as ? because the equivalent language translator is not available to represent these characters so we are getting the ? symbol.

```
//Addition of two character in the form of Integer
```

```
public class Test5  
{  
    public static void main(String txt[ ])  
    {  
        int x = 'A';  
        int y = 'B';  
        System.out.println(x+y); //131  
        System.out.println('A' + 'A'); //130  
    }  
}
```

```
}  
}
```

//Range of UNICODE Value (65535)

```
class Test6
```

```
{  
    public static void main(String[] args)  
    {  
        char ch1 = 65535;  
        System.out.println("ch value is :"+ch1);  
  
        char ch2 = 65536; //error  
        System.out.println("ch value is :"+ch2);  
    }  
}
```

//WAP in java to describe unicode representation of char in hexadecimal format

```
public class Test7
```

```
{  
    public static void main(String[] args)  
    {  
        char ch1 = '\u0002';  
        System.out.println(ch1);  
  
        char ch2 = '\uffff';  
        System.out.println(ch2);  
  
        char ch3 = '\u0041';  
        System.out.println(ch3);  
  
        char ch4 = '\u0061';  
        System.out.println(ch4);  
    }  
}
```

class Test8

```
{  
    public static void main(String[] args)  
    {  
        char c1 = 'A';  
        char c2 = 65;  
        char c3 = '\u0041';  
  
        System.out.println("c1 = "+c1+", c2 ="+c2+", c3 ="+c3);  
    }  
}
```

class Test9

```
{  
    public static void main(String[] args)  
    {  
        int x = 'A';  
        int y = '\u0041';  
        System.out.println("x = "+x+" y ="+y);  
    }  
}
```

```

}
-----
//Every escape sequence is char literal
class Test10
{
    public static void main(String [] args)
    {
        char ch = '\n';
        System.out.println("Hello");
        System.out.println(ch);

    }
}
-----
public class Test11
{
    public static void main(String[] args)
    {
        System.out.println(Character.MIN_VALUE); //white space
        System.out.println(Character.MAX_VALUE); //?
        System.out.println(Character.SIZE); //16 bits

    }
}

```

```

-----
//Java Unicodes
public class Test12
{
    public static void main(String[] args)
    {
        System.out.println(" Java Unicodes\n");

        for (int i = 31; i < 126; i++)
        {
            char ch = (char)i; // Convert unicode to character
            String str = i + " " + ch;

            System.out.print(str + "\t\t");
            if ((i % 5) == 0) // Set 5 numbers per row
                System.out.println();
        }
    }
}

```

05-04-2024

String Literal :-

A string literal in Java is basically a sequence of characters. These characters can be anything like alphabets, numbers or symbols which are enclosed with double quotes. So we can say String is alpha-numeric collection of character.

How we can create String in Java :-

In java String can be created by using 3 ways :-

1) By using String Literal

```
String x = "Ravi";
```

2) By using new keyword

```
String y = new String("Hyderabad");
```

3) By using character array

```
char z[] = {'H','E','L','L','O'};
```

//Three Ways to create the String Object

```
public class StringTest1
```

```
{  
    public static void main(String[] args)  
    {  
        String s1 = "Hello World";    //Literal  
        System.out.println(s1);  
  
        String s2 = new String("Ravi"); //Using new Keyword  
        System.out.println(s2);  
  
        char s3[] = {'H','E','L','L','O'}; //Character Array  
        System.out.println(s3);  
  
    }  
}
```

//String is collection of alpha-numeric character

```
public class StringTest2
```

```
{  
    public static void main(String[] args)  
    {  
        String x="B-61 Hyderabad";  
        System.out.println(x);  
  
        String y = "123";  
        System.out.println(y);  
  
        String z = "67.90";  
        System.out.println(z);  
  
        String p = "A";  
        System.out.println(p);  
    }  
}
```

//IQ
public class StringTest3
{

```

public static void main(String []args)
{
    String s = 15+29+"Ravi"+40+40;
    System.out.println(s);

}
}

```

4) Punctuators :

It is also called separators.

It is used to inform the compiler how things are grouped in the code.

() {} [] ; , . @ (var args)

5) Operators :

Some Basic concepts related to Operators :

```

class Test
{
    public static void main(String[] args)
    {
        int x = 5;

        if(++x > 6 & ++x > 7)
        {
            ++x;
        }

        System.out.println(x);
    }
}

```

Note :- In logical AND operator, if the first expression is false then it will not evaluate all right hand side expression but in boolean AND operator it will evaluate all the right side expression.

```

class Test
{
    public static void main(String[] args)
    {
        int x = 5;
        int y = x++;
        System.out.println(x + " : " + y);

    }
}

```

Here first of all, x value will assign to y and then increment in x variable.

```

class Test
{
    public static void main(String[] args)

```

```

{
    int x = 5;
        int y = ++5; //error becoz variable is reqd
    System.out.println(x+": "+y);

}
}

```

```

class Test
{
    public static void main(String[] args)
    {
        int x = 15;
        int y = ++(++x);
        System.out.println(y);

    }
}

```

```

class Test
{
    public static void main(String[] args)
    {
        char ch = 'A';
        ch++;
        System.out.println(ch);

        double d = 20.78;
        d++;
        System.out.println(d);

    }
}

```

increment and decrement operator ww can apply with any data type except boolean.

Expression Conversion :

While working with Arithmetic Operator and Unary minus operator, the minimum data type required to hold the result of expression is int.

```

class Test
{
    public static void main(String[] args)
    {
        byte b = 1;
        short s = b + 2; //error
        System.out.println(s);

    }
}

```

```

class Test
{

```



```

public static void main(String[] args)
{
    byte b = 1;
    byte c = -b; //error
    System.out.println(c);

}
}

```

```

class Test
{
    public static void main(String[] args)
    {
        byte b = 1;
        b += 2;
        System.out.println(b);
    }
}

```

Code will compile and produce the output 3

Bitwise Complement Operator

* It will not work with boolean value.

```

class Test
{
    public static void main(String[] args)
    {
        System.out.println(~true); //error
        System.out.println(~5);
    }
}

```

```

class Test
{
    public static void main(String[] args)
    {
        System.out.println(5 & 6); //4
        System.out.println(5 | 6); //7
        System.out.println(5 ^ 6); //3
    }
}

```

06-04-2024

Limitation of Command Line Argument :

As we know by using Command Line Argument, we can pass some value at runtime, These values are stored in String array variable and then only the execution of the program will be started.

In Command line Argument we can't ask to enter the value from our end user as shown in the Program.

```

public class Test

```

```

{
    public static void main(String[] args)
    {
        System.out.println("Enter your Name :");
        System.out.println("Your Name is :"+args[0]);
    }
}

```

 Understanding Scanner class :

To avoid the drawback of Command Line Argument, Java Software people introduced to take the input from the user by using following ways :

- 1) DataInputStream class
- 2) BufferedRedaer class
- 3) Console class
- 4) System.in.read()
- 5) Scanner class

From java 1.5 onwards, Java software people has provided a predefined class called Scanner available in java.util package.

By using Scanner class we can read the data from our end user.

static variable of System class :

System class has provided 3 static variables which are as follows

- 1) System.out : Used to print normal message
- 2) System.err : Used to print error message (Red Color)
- 3) System.in : Used to take the input as a source

How to create an Object for Scanner class :

Scanner sc = new Scanner(System.in);

Methods available in Scanner class :

- 1) public byte nextByte() : Read the byte data
- 2) public short nextShort() : Read the short data
- 3) public int nextInt() : Read the int data
- 4) public long nextLong() : Read the long data
- 5) public float nextFloat() : Read the float data
- 6) public double nextDouble() : Read the double data
- 7) public boolean nextBooelan() : Read the boolean data
- 8) public String next() : Read a single word
- 9) public String nextLine() : Read multiple words or a complete line.
- 10) public char next().charAt(0) : Used to read a character.

WAP to read a character from the keyboard :

```

package com.ravi.keyboard_reading;

import java.util.Scanner;

public class ReadCharacter
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your Gender [M/F] : ");
        char gen = sc.next().charAt(0);
        System.out.println("You Are :"+gen);
        sc.close();
    }
}

```

WAP to read the employee data from the Scanner class :

```

package com.ravi.keyboard_reading;

import java.util.Scanner;

public class EmployeeData
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your Employee Id : ");
        int empld = sc.nextInt(); //123 \n

        System.out.print("Enter Employee Name :");
        String name = sc.nextLine(); //Buffer Problem
        name = sc.nextLine();

        System.out.print("Enter Employee Salary :");
        double salary = sc.nextDouble();

        System.out.println("Employee Id is :"+empld);
        System.out.println("Employee Name is :"+name);
        System.out.println("Employee Salary is :"+salary);
        sc.close();
    }
}

```

Note : While entering the data if client is providing some wrong data then we will get an exception i.e
java.util.InputMismatchException

Drawback of if condition :

What is drawback of if condition :-

The major drawback with if condition is, it checks the condition again and again so It increases the burden over CPU so we introduced switch-case statement to reduce the overhead of the CPU.

switch case :-

In switch case depending upon the parameter the appropriate case would be executed otherwise default would be executed.

In this approach we need not to check each and every case, if the appropriate case is available then directly it would be executed.

break keyword is optional here but we can use as per requirement. It will move the control outside of the body of the switch.

Note :- In the switch statement we can't pass long, float and double value.
Strings are allowed from JDK 1.7 version. enums are allowed from java 1.5 version.

import java.util.*;
public class SwitchDemo
{
 public static void main(String[] args)
 {
 Scanner sc = new Scanner(System.in);
 System.out.print("Please Enter a Character :");

 char colour = sc.next().toLowerCase().charAt(0);

 switch(colour)
 {
 case 'r' : System.out.println("Red") ; break;
 case 'g' : System.out.println("Green");break;
 case 'b' : System.out.println("Blue"); break;
 case 'w' : System.out.println("White"); break;
 default : System.out.println("No colour");
 }
 System.out.println("Completed") ;
 }
}

08-04-2024

import java.util.*;
public class SwitchDemo1
{
 public static void main(String args[])
 {
 System.out.println("\t\t**Main Menu**\n");
 System.out.println("\t\t**100 Police**\n");
 System.out.println("\t\t**101 Fire**\n");
 System.out.println("\t\t**102 Ambulance**\n");
 System.out.println("\t\t**139 Railway**\n");
 System.out.println("\t\t**181 Women's Helpline**\n");

 System.out.print("Enter your choice :");
 }
}

```

Scanner sc = new Scanner(System.in);
int choice = sc.nextInt();

switch(choice)
{
case 100:
System.out.println("Police Services");
break;
case 101:
System.out.println("Fire Services");
break;
case 102:
System.out.println("Ambulance Services");
break;
case 139:
System.out.println("Railway Enquiry");
break;
case 181:
System.out.println("Women's Helpline ");
break;
default:
System.out.println("Your choice is wrong");
}
}

```

```

import java.util.*;
public class SwitchDemo2
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.print("Enter the name of the season :");
String season = sc.next().toLowerCase();

switch(season) //String allowed from 1.7
{
case "summer" :
System.out.println("It is summer Season!!");
break;

case "rainy" :
System.out.println("It is Rainy Season!!");
break;
}
}
}

```

```

public class Test
{
public static void main(String[] args)
{
byte b = 125;

switch(b)

```

```

{
    case 125 :
        System.out.println("125 case executed");
        break;

    case 128 : //error
        System.out.println("130 case executed");
        break;
}

}
}

public class Test
{
    public static void main(String[] args)
    {
        short b = 125;

        switch(b)
        {
            case 32767 :
                System.out.println("32767 case executed");
                break;

            case 32768 :
                System.out.println("32768 case executed");
                break;
        }

    }
}

```

The above both the program will generate compilation error because case constant value is out of the range of byte and short.

Loops in java :

A loop is nothing but repetition of statements based on the specified condition.

In java we have 4 types of loops :

-
- 1) do-while loop
 - 2) while loop
 - 3) for loop
 - 4) for each loop

Program on do-while loop :

```

public class Test
{
    public static void main(String[] args)
    {
        do

```

```

    {
int x = 1; //Local block level variable (Error)
    System.out.println(x);
    x++;
    }
    while (x<=10);

}
}
-----

public class Test
{
    public static void main(String[] args)
    {
        int x = 1; //Local variable
        do
        {

            System.out.println(x);
            x++;
        }
        while (x<=10);

    }
}

```

Output of the program is 1-10.

What is a local variable in java ?

If a variable is declared inside the method body (not as a method parameter) then it is called local variable.

As far as it's accessibility is concerned, It is accessible within the same method body only that means we can't use local variables outside of the method.

We can't apply any kind of access modifier on local variable except final.

Local variables must be initialized by the developer before use.

```

public class Test
{
    public static void main(String[] args)
    {
        int x = 100; //local variable

    }
}
-----

```

Why we can't access local variable outside of the method ?

In java, all the objects are created in a special memory called HEAP Memory where as all the methods are executed in a special memory area STACK Area.

We know that STACK memory works on the LIFO basis.

```
-----  
public class Test  
{  
    public static void main(String[] args)  
    {  
        System.out.println("Main Method Started!!!");  
        m1();  
        System.out.println("Main Method Ended!!!");  
    }  
  
    public static void m1()  
    {  
        System.out.println("M1 Method Started!!!");  
        m2();  
        System.out.println("M1 Method Ended!!!");  
    }  
  
    public static void m2()  
    {  
        int x = 10;  
        System.out.println("Hi I am m2 method");  
    }  
}
```

Note :- In the above program x is a local variable, it is defined in the body of m2 method so once the m2 method execution is over x variable will be deleted from Stack Frame so another method which are executing in another STACK frame can't use this variable.

Program on while loop :

```
-----  
public class Test  
{  
    public static void main(String[] args)  
    {  
        int x = 0;  
        while(x>=-10)  
        {  
            System.out.println(x);  
            x--;  
        }  
    }  
}
```

```
-----  
public class Test  
{  
    public static void main(String[] args)  
    {  
        for(int i=1; i<=10; i++)  
        {  
            System.out.println(i);  
        }  
    }  
}
```



```
}
```

for each loop in java :

It is an enhanced for loop which is used to retrieve the values through the collection data.

It was introduced from java 1.5v.

It is used to fetch the values from collection data one by one so it is known as for each loop.

10-04-2024

```
package com.ravi.for_each_demo;

public class ForEachDemo {

    public static void main(String[] args)
    {
        int values[] = {50,40,30,20,10,9,2};

        for(int value : values)
        {
            System.out.println(value);
        }

    }
}
```

Note : In the above program output is according to the value available in the array.

How to sort an array variable Data :

In java there is a predefined class called Arrays available in java.util package.

Arrays class contains a predefined static method sort(Object arr[]) as a parameter through which we can sort an array in ascending order as shown in the Program

```
package com.ravi.for_each_demo;

import java.util.Arrays;

public class ForEachDemo {

    public static void main(String[] args)
    {
        int values[] = {50,40,30,20,10,9,2};

        Arrays.sort(values);

        for(int value : values)
        {
            System.out.println(value);
        }
    }
}
```

```
}
```

```
}
```

How to sort String data :

```
package com.ravi.for_each_demo;
```

```
import java.util.Arrays;
```

```
public class ForEachDedmo1 {
```

```
    public static void main(String[] args)
```

```
    {
```

```
        String fruits[] = {"Mango","Orange","Kiwi","Apple"};
```

```
        Arrays.sort(fruits);
```

```
        for(String fruit : fruits)
```

```
        {
```

```
            System.out.println(fruit);
```

```
        }
```

```
    }
```

```
}
```

Here fruits are sorted in alphabetical order.

```
package com.ravi.for_each_demo;
```

```
public class ForEachDemo2 {
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Object []values = {12,"true",45.90,"NIT",new String("India")};
```

```
        for(Object value : values)
```

```
        {
```

```
            System.out.println(value);
```

```
        }
```

```
    }
```

```
}
```

Note : Here Sorting is not possible because elements are hetrogeneous.

Working with Method parameter and return type :

What is Business Logic class (BLC) and Executable Logic class (ELC)

BLC :

It stands for Business Logic class. In this class we should write the logic of the program, It will never contain main method that's why it is called Business Logic class.

ELC :

It stands for Exceutable Logic Class, Here we should write main method, the execution of the program will start from ELC class only that's why it is called ELC class.

Working with static method and parameter :

//A static method can be directly call within the same class
package com.ravi.pack1;

```
public class Test1
{
    public static void main (String[] arg)
    {
        square(5);
    }

    public static void square(int x)
    {
        System.out.println(x*x);
    }
}
```

Note :- If a static method available in the same class then we can directly call the static method from another static method. (Object, even class name is not required).

Can we call a method from System.out.println() statement?

Yes, we can call any method from System.out.println() but that method return type must not be void otherwise we will get compilation error.

2 files :

FindSquare.java

package com.ravi.pack2;
//Finding the Square of the number

```
//BLC(Business Logic class)
public class FindSquare
{
    public static void getSquareOfTheNumber(int number)
    {
        System.out.println("Square of "+number+" is "+(number * number));
    }
}
```

//A static method available in another class can be
//call with class name

Test2.java

```
-----  
package com.ravi.pack2;  
  
import java.util.Scanner;  
  
//ELC(Executable Logic class)  
public class Test2  
{  
    public static void main (String[] args)  
    {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter a number :");  
        int num = sc.nextInt();  
  
        FindSquare.getSquareOfTheNumber(num);  
        sc.close();  
    }  
}
```

Reusability of the classes :

Points :

- 1) In a single java file, we can take only one public class.
- 2) The public class must be our file name otherwise compilation error.
- 3) We should not take multiple java classes in a single file otherwise the reusability of the classes is not possible from from different packages.

So, the conclusion we should take every class in a a single file.

12-04-2024

Finding the Square of the number by using method, where method return type is int

2 files :

FindSquare.java

```
-----  
//A static method returning integer value  
package com.ravi.pack3;  
  
//BLC  
public class FindSquare  
{  
    public static int getSquare(int x)  
    {  
        return (x*x);  
    }  
}
```

Test3.java

```
package com.ravi.pack3;
```

```
import java.util.Scanner;
```

```
//ELC
```

```
public class Test3
```

```
{
    public static void main (String[] arg)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("How many times you want Square :");
        int n = sc.nextInt();

        for(int i=1; i<=n; i++)
        {
            System.out.print("Enter the value for Square :");
            int num = sc.nextInt();

            System.out.println("Square of "+num+" is :"+(num*num));
        }
        sc.close();
    }
}
```

```
}
}
```

```
-----
/*Program to find out the square and cube of
the number by following criteria
```

```
*
a) If number is 0 or Negative it should return -1
b) If number is even It should return square of the number
c) If number is odd It should return cube of the number
*/
```

2 files :

Calculate.java

```
package com.ravi.pack4;
```

```
//BLC
```

```
public class Calculate
```

```
{
    public static int getSquareAndCube(int num)
    {
        if(num<0 || num==0)
        {
            return -1;
        }
        else if(num % 2==0)
        {
            return (num*num);
        }
    }
}
```

```

else
{
return (num*num*num);
}
}
}

```

Test4.java

```

package com.ravi.pack4;

import java.util.Scanner;

//ELC
public class Test4
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter a Number :");
        int num = sc.nextInt();

        int number = Calculate.getSquareAndCube(num);
        System.out.println("Number is :"+number);
        sc.close();
    }
}

```

How to provide formatting for Decimal number :

In java.text package, there is a predefined class called DecimalFormat through which we can provide formatting for Decimal number.

```

DecimalFormat df = new DecimalFormat("00.00");
System.out.println(df.format(double d));

```

Note :- format is non static method of DecimalFormat class which accpts double as a parameter, and return type of this method is String.

```

public String format(double number)

```

2 files :

Rectangle.java

```

package com.ravi.pack5;

//BLC
public class Rectangle
{
    public static double getAreaOfRectangle(double length, double breadth)

```

```
{  
    return (length * breadth);  
}  
  
}
```

Test5.java

```
package com.ravi.pack5;  
  
import java.text.DecimalFormat;  
import java.util.Scanner;  
  
public class Test5  
{  
    public static void main(String[] args)  
    {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter the length of the Rectangle :");  
        double length = sc.nextDouble();  
        System.out.print("Enter the breadth of the Rectangle :");  
        double breadth = sc.nextDouble();  
  
        double areaOfRectangle = Rectangle.getAreaOfRectangle(length, breadth);  
  
        DecimalFormat df = new DecimalFormat("000.000"); //format  
        System.out.println("Area of Rectangle is :"+df.format(areaOfRectangle));  
  
        sc.close();  
    }  
}
```

2 files :

EvenOrOdd.java

```
package com.ravi.pack6;  
  
//BLC  
public class EvenOrOdd  
{  
    public static boolean isEven(int num)  
    {  
        return (num % 2 == 0);  
    }  
}
```

Test6.java

```
package com.ravi.pack6;
```

```
//ELC
```

```

public class Test6
{
    public static void main(String[] args)
    {
        boolean isEven = EvenOrOdd.isEven(11);
        System.out.println("number is Even ?:"+isEven);

        isEven = EvenOrOdd.isEven(12);
        System.out.println("number is Even ?:"+isEven);
    }
}

```

Method return type as a String :

2 files :

Circle.java

//Area of Circle
//If the radius is 0 or Negative then return -1.

```

package com.ravi.pack7;
public class Circle
{
    public static String getAreaOfCircle(double rad)
    {
        if(rad<0 || rad==0)
        {
            return ""+(-1);
        }
        else
        {
            final double PI = 3.14;
            double area = PI * rad * rad;
            return " "+area;
        }
    }
}

```

}

Test7.java

package com.ravi.pack7;

import java.text.DecimalFormat;
import java.util.Scanner;

```

public class Test7
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the radius :");
    }
}

```



```
double rad = sc.nextDouble();
```

```
String circle = Circle.getAreaOfCircle(rad);
```

```
//Converting String into double
```

```
double areaOfCircle = Double.parseDouble(circle);
```

```
DecimalFormat df = new DecimalFormat("00.000");
```

```
System.out.println("Area of circle is :"+df.format(areaOfCircle));
```

```
sc.close();
```

```
}
```

```
}
```

2 files :

Student.java

package com.ravi.pack8;

```
//BLC
```

```
public class Student
```

```
{
```

```
    public static String getStudentDetails(int roll, String name, double fees)
```

```
    {
```

```
        //[Student name is : Ravi, roll is : 101, fees is :1200.90]
```

```
        return "[Student name is : "+name+", roll is :"+roll+", fees is :"+fees+"]";
```

```
    }
```

```
}
```

Test8.java

package com.ravi.pack8;

```
public class Test8
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        String details = Student.getStudentDetails(111, "Raj", 18000.90);
```

```
        System.out.println("Student Details are :"+details);
```

```
    }
```

```
}
```

Printing the Table :

2 files :

Table.java

package com.ravi.pack9;

```
//BLC // 5 X 1 = 5
```

```
public class Table
```

```

{
    public static void printTable(int num) //5
    {
        for(int i=1; i<=10; i++)
        {
            System.out.println(num+" X "+i+" = "+(num*i));
        }
        System.out.println(".....");
    }
}

```

Test9.java

package com.ravi.pack9;

```

//ELC
public class Test9
{
    public static void main(String[] args)
    {
        for(int i=1; i<=10; i++)
        {
            Table.printTable(i);
        }
    }
}

```

Types of variables in Java ?

In java, Based on the data type we have only 2 types of variables

1) Primitive variable

If any variable is declared with primitive data type like byte, short, int, long and so on then it is called primitive variable.

Example : int x = 90;

2) Non Primitive OR Reference variable

If any variable is declared with class name like String, Integer, Employee, Student then it is called as non-primitive OR reference variable.

Example : Customer a = null;
String b = "NIT";
Student s = new Student();

Now Based on the Declaration Position variables are divided into 4 types :

1) Instance variable OR Non static Field

- 2) Class Variable OR Static Field
- 3) Parameter variable
- 4) Local Variable

13-04-2024

Example of Primitive Variable :

```
package com.ravi.variable;

public class PrimitiveVariable
{
    int a = 100; //Instance Variable OR Non-static field
    static int b = 200; //Class Variable OR Static field

    public static void main(String[] args)
    {
        int y = 300;

        PrimitiveVariable p = new PrimitiveVariable();
        System.out.println("Instance Variable :"+p.a);
        System.out.println("Class Variable :"+b);
        System.out.println("Local variable :"+y);

        accept(400);
    }
    public static void accept(int x) //x is a parameter variable
    {
        System.out.println("Parameter Variable :"+x);
    }
}
```

Example of Reference Variable :

```
package com.ravi.variable;

import java.util.Scanner;

class Student
{
    public void m1()
    {
        System.out.println("M1 method");
    }

    public static void m2()
    {
        System.out.println("M2 method");
    }
}

public class ReferenceVariable
{
    static Student s1 = new Student(); //Class Variable (Ref var)
```

```

Scanner sc = new Scanner(System.in); //Instance variable (Ref var)

public static void main(String[] args)
{
    s1.m1();

    Student s2 = new Student(); //local variable
    acceptStudentData(s2);
    Student.m2();
}

public static void acceptStudentData(Student st) //st is parameter variable
{
    st.m1();
}
}

```

Object Oriented Programming (OOPs) :

What is an Object?

An object is a physical entity which exist in the real world.
 Example :- Pen, Car, Laptop, Mouse, Fan and so on

An Object is having 3 characteristics :

- a) Identification of the Object (Name of the Object)
- b) State of the Object (Data OR Properties OR Variable of Object)
- c) Behavior of the Object (Functionality of the Object)

OOP is a technique through which we can design or develop the programs using class and object.

Writing programs on real life objects is known as Object Oriented Programming.

Here in OOP we concentrate on objects rather than function/method.

Advantages of OOP :

-
- 1) Modularity (Dividing the bigger task into smaller task)
 - 2) Reusability (We can reuse the component so many times)
 - 3) Flexibility (Easy to maintain)

Features of OOP :

-
- 1) Class
 - 2) Object
 - 3) Abstraction
 - 4) Encapsulation
 - 5) Inheritance
 - 6) Polymorphism
-

What is a class?

A class is model/blueprint/template/prototype for creating the object.

A class is a logical entity which does not take any memory.

A class is a user-defined data type which contains data member and member function.

```
public class Employee
{
    Employee Data (Properties)
    +
    Employee behavior (Function/Method)
}
```

A CLASS IS A COMPONENT WHICH IS USED TO DEFINE OBJECT PROPERTIES AND OBJECT BEHAVIOR.

Write the first Object Oriented Program on Student Object:

2 files :

Student.java

```
package com.ravi.oop;
//BLC
public class Student
{
    //properties (Instance Variables)
    String regNumber;
    String studentName;
    int studentAge;

    public void talk()
    {
        System.out.println("Hello Everyone, My name is :"+studentName+" my age is :"+studentAge+" my
Registration number is :"+regNumber);
    }

    public void writeExam()
    {
        System.out.println("Hi I m "+studentName+" every saturday I am writing my java Exam");
    }
}
```

StudentDemo.java

```
package com.ravi.oop;

//ELC
public class StudentDemo
{
    public static void main(String[] args)
    {
        Student sachin = new Student();
        //Initializing the properties
    }
}
```

```
sachin.regNumber = "NIT0124009";
sachin.studentAge = 22;
sachin.studentName = "Sachin Kumar";
```

```
//Calling the behavior
sachin.talk();
sachin.writeExam();
```

```
System.out.println(".....");
```

```
Student priya = new Student();
//Initializing the properties
priya.regNumber = "NIT024008";
priya.studentAge = 21;
priya.studentName = "Priya Kumari";
```

```
//Calling the behavior
priya.talk();
priya.writeExam();
}
```

```
}
```

15-04-2024

Initializing the object properties :

The main purpose of our Object Oriented Programming to initialize the Object Properties through various ways :

In the last program, we initialize the object properties through object reference.

```
Student sachin = new Student();
//Initializing the properties
sachin.regNumber = "NIT0124009";
sachin.studentAge = 22;
sachin.studentName = "Sachin Kumar";
```

How to initialize the Object Properties through methods :

We can also initialize the Object Properties (instance variable) though methods as shown in the program below.

2 files :

Customer.java

```
package com.ravi.object_properties;
//BLC
public class Customer
{
    int customerId;
    String customerName;

    public void setCustomerData()
```

```

{
    customerId = 111;
    customerName = "Raj";
}

public void getCustomerData()
{
    System.out.println("Customer Id is :"+customerId);
    System.out.println("Customer Name is :"+customerName);
}
}

```

CustomerDemo.java

```

-----
package com.ravi.object_properties;
//ELC
public class CustomerDemo {

    public static void main(String[] args)
    {
        Customer raj = new Customer();
        raj.setCustomerData();
        raj.getCustomerData();

    }

}

```

What is Constrcutor? (Introduction Only)

If the name of the class and name of the method both are exactly same and it does not contain any return type then it is called Constructor.

Example :

```

-----
public class Employee
{
    public Employee() //Constructor
    {
    }
}

```

Class name and Method name same in the following but it is not constructor :

```

-----
public class Student
{
    public void Student() //It is a method, not a constructor
    {
    }
}

```

Default constructor added by the compiler :

In Java, whenever we write a class and if don't write any type of constructor in that class then

automatically one default constructor will be added by compiler.

*Every java class must contain at-least one constructor either explicitly written by user or implicitly added by compiler.

The access modifier of default constructor depends upon class access modifier.

Example :

case 1 :

class Student
{
}

javac Student.java (Compilation)
javap Student.class (To see the default constructor)

```
class Student
{
    Student() //default constructor added by the compiler
    {
    }
}
```

case 2:

public class Student
{
}

javac Student.java (Compilation)
javap Student.class (To see the default constructor)

```
class Student
{
    public Student() //default constructor added by the compiler
    {
    }
}
```

Why compiler is adding default constructor to our class :

We have 2 reasons that why compiler is adding default constructor :

- 1) Without default constructor, Object creation is not possible in java by using new keyword.
- 2) default constructor will initialize all the instance variables with default values.

Data type - Default value

byte - 0
short - 0
int - 0
long - 0
float - 0.0

double - 0.0
char - (space) '\u0000'
boolean - false
String - null
Object - null (For any class i.e reference variable the default value is null)

2 files :

Student.java

```
package com.ravi.object_properties;

public class Student
{
    int studentId;
    String studentName;

    public void showStudentData()
    {
        System.out.println("Student Id is :"+studentId);
        System.out.println("Student Name is :"+studentName);
    }
}
```

StudentDemo.java

```
package com.ravi.object_properties;

public class StudentDemo {

    public static void main(String[] args)
    {
        Student s = new Student(); //Nameless OR Anonymous Object
        s.showStudentData(); //0 and null

        s.studentId = 111;
        s.studentName = "Virat";
        s.showStudentData();

    }
}
```

16-04-2024

How to provide userdefined values for our instance variable :

The default constructor added by the compiler will provide the default values for the instance variable, but these default values are not useful for the user so, user will take a separate method setManagerData() to re-initialize the instance variable according to his/her choice as shown in the program.

2 files :

Manager.java

```
package com.ravi.default_value;
```

```
//BLC
```

```
public class Manager
```

```
{
```

```
    int id;
```

```
    String name;
```

```
    public void setManagerData()
```

```
    {
```

```
        id = 111;
```

```
        name = "Scott";
```

```
    }
```

```
    public void getManagerData()
```

```
    {
```

```
        System.out.println("Manager id is :"+id);
```

```
        System.out.println("Manager Name is :"+name);
```

```
    }
```

```
}
```

```
ManagerDemo.java
```

```
-----
```

```
package com.ravi.default_value;
```

```
//ELC
```

```
public class ManagerDemo
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Manager m = new Manager();
```

```
        m.setManagerData();
```

```
        m.getManagerData();
```

```
    }
```

```
}
```

```
-----  
Instance variable :
```

```
-----
```

If a non static variable declared outside of a method and inside the class then it is called Instance variable.

As far as its scope is concerned, it is accessible directly anywhere within the class as well as depends upon the accessibility level.(Access Modifier)

THE LIFE OF INSTANCE VARIABLE STARTS AT THE TIME OF CREATING THE OBJECT, IT IS THE PART OF THE OBJECT. WE CANNOT THINK ABOUT INSTANCE VARIABLE WITHOUT OBJECT.

```
public class Main
```

```
{
```

```
    int x = 100;
```

```
    public static void main(String [] args)
```

```
    {
```

```
        System.out.println(x); //error [cannot use instance variable
```

without object]

```
}  
}
```

Local variable :

If a variable is declared inside the method body (not as a method parameter) then it is called Local/Stack/Automatic/Temporary variable.

As far as its scope is concerned, it is accessible within the same method body only. (Not outside of the method)

A local variable must be initialized before use.

We cannot apply any kind of access modifier except final.

Program on Instance Variable :

2 files :

Customer.java

package com.nit.instance_var;

import java.util.Scanner;

public class Customer

```
{  
    int customerId;  
    String customerName;  
    double customerBill;  
  
    public void setCustomerData()  
    {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter Customer Id :");  
        customerId = sc.nextInt();  
        System.out.print("Enter Customer Name :");  
        customerName = sc.nextLine();  
        customerName = sc.nextLine();  
        System.out.print("Enter Customer Bill :");  
        customerBill = sc.nextDouble();  
        sc.close();  
    }  
  
    public void getCustomerData()  
    {  
        System.out.println("Customer id is :"+customerId);  
        System.out.println("Customer name is :"+customerName);  
        System.out.println("Customer bill is :"+customerBill);  
    }  
}
```

}

CustomerDemo.java

```
package com.nit.instance_var;

public class CustomerDemo {

    public static void main(String[] args)
    {
        Customer c = new Customer();
        c.setCustomerData();
        c.getCustomerData();

    }

}
```

What is a parameter variable :

If we declare a variable inside the method parameter(not inside the method body) then it is called parameter variable.

The scope of the parameter variable is within the same method body and it is executed inside the Stack Frame. Once the method execution is over parameter variables are deleted from stack frame.

Example :

```
public class Player
{
    int playerId;
    String playerName;

    public void setPlayerData(int id, String name) //id and name
    {
                                                parameter

    }

}
```

Program on Parameter Variable :

2 Files :

Player.java

```
package com.nit.parameter_var;

public class Player
{
    int playerId;
    String playerName;

    public void setPlayerData(int id , String name)
```

```

{
    playerId = id;
    playerName = name;
}

public void getPlayerData()
{
    System.out.println("Player id is :"+playerId);
    System.out.println("Player name is :"+playerName);
}
}

```

PlayerDemo.java

```

-----
package com.nit.parameter_var;

public class PlayerDemo {

    public static void main(String[] args)
    {
        Player virat = new Player();
        virat.setPlayerData(18, "Virat Kohli");

        Player rohit = new Player();
        rohit.setPlayerData(45, "Rohit Sharma");

        virat.getPlayerData();
        System.out.println(".....");
        rohit.getPlayerData();
    }
}

```

Program on calculation of Employee Grade :

2 files :

Employee.java

```

-----
package com.ravi.lab;

public class Employee
{
    int employeeId;
    String employeeName;
    double employeeSalary;
    char employeeGrade;

    public void setEmployeeData(int id, String name, double salary)
    {
        employeeId = id;
        employeeName = name;
        employeeSalary = salary;
    }
}

```

```

}

public void calculateEmployeeGrade()
{
    if(employeeSalary >=90000)
    {
        employeeGrade = 'A';
    }
    else if(employeeSalary >=75000)
    {
        employeeGrade = 'B';
    }
    else if(employeeSalary >=50000)
    {
        employeeGrade = 'C';
    }
    else
    {
        employeeGrade = 'D';
    }
}

public void getEmployeeData()
{
    System.out.println("Employee id is :"+employeeId);
    System.out.println("Employee name is :"+employeeName);
    System.out.println("Employee salary is :"+employeeSalary);
    System.out.println("Employee grade is :"+employeeGrade);
}

}

```

EmployeeDemo.java

```

-----
package com.ravi.lab;

public class EmployeeDemo {

    public static void main(String[] args)
    {
        Employee e1 = new Employee();
        e1.setEmployeeData(1, "Scott", 45000);
        e1.calculateEmployeeGrade();
        e1.getEmployeeData();

        Employee e2 = new Employee();
        e2.setEmployeeData(2, "Smith", 95000);
        e2.calculateEmployeeGrade();
        e2.getEmployeeData();

    }
}
-----

```

What is variable shadowing ?

If the name of instance variable and local /Parameter variable both are same then inside the method the local /Parameter variable will hide instance variable that is known as Variable Shadow as shown in the program below.

2 files :

Student.java

package com.nit.parameter_var;

```
public class Student
{
    int id = 100;
    String name = "Smith";

    public void display()
    {
        int id = 200;
        String name = "Scott";
        System.out.println(id);
        System.out.println(name);
    }
}
```

StudentDemo.java

package com.nit.parameter_var;

```
public class StudentDemo {

    public static void main(String[] args)
    {
        Student s = new Student();
        s.display();

    }
}
```

this keyword in java :

Whenever instance variable name and parameter variable name both are same then at the time of variable initialization our runtime environment will provide more priority to parameter variable, parameter variables are hiding the instance variables (Due to variable shadow)

To avoid the above said problem, Java software people introduced "this" keyword.

this keyword always refers to the current object and instance variables are the part of the object so by using this keyword we can refer to instance variable.

We cannot use this (non static member) keyword from static area (Static context).

2 files :

Book.java

```
package com.ravi.this_keyword;

public class Book
{
    String authorName;
    String bookTitle;

    public void setBookData(String authorName, String bookTitle)
    {
        this.authorName = authorName;
        this.bookTitle = bookTitle;

        //this.getBookData(); We can access the behavior of the object
        //so, It is working as a reference variable to the current
        //object
    }

    public void getBookData()
    {
        System.out.println("Author Name is :"+authorName);
        System.out.println("Title Name is :"+bookTitle);
    }

}
```

ThisKeywordDemo.java

```
package com.ravi.this_keyword;

public class ThisKeywordDemo
{
    public static void main(String[] args)
    {
        Book b1 = new Book();
        b1.setBookData("James", "Java");
        b1.getBookData();
    }

}
```

Role of instance variable while creating the Object :

Whenever we create an Object in java then a separate copy of all the instance variables are created with each and every object as shown in the Program (Diagram 18-APR)

```
public class Test
{
    int b = 10;

    public static void main(String args[])
```



```

{
Test t1 = new Test();
Test t2 = new Test();

    ++t1.b;
    --t2.b;

    System.out.println(t1.b); //11
    System.out.println(t2.b); //9
}
}

```

Role of static variable while creating the Object :

Whenever we create an object in java then a single copy of static variable will be created with each and every object.

```

public class Test
{
    static int x = 10;

    public static void main(String args[])
    {
        Test t1 = new Test();
        Test t2 = new Test();

        --t1.x;    --t2.x;

        System.out.println(t1.x);
        System.out.println(t2.x);
    }
}

```

So, the conclusion is

Instance variable = Multiple Copies
Class Variable = Single Copy

Note : STATIC VARIABLES ARE MAINLY USED FOR SAVING THE MEMORY

-----When we should declare a variable as an instance variable and when we should declare a variable as a static variable ?

Instance Variable :

If the value of the variable is different with respect to object then we should use instance variable.

Static Variable :

If the value of the variable is common with respect to all the objects then we should use static variable.

Example :

```

class Student

```

```
{
    int roll;
    String name;
    String address;
    static String collegeName = "NIT";
    static String courseName = "Java";
}
```

2 files :

Student.java

```
-----
package com.ravi;

public class Student
{
    int rollNumber;
    String studentName;
    String studentAddress;
    static String collegeName = "JNTU";
    static String courseName = "Java";

    public void setStudentData(int roll, String name, String addr)
    {
        this.rollNumber = roll;
        this.studentName = name;
        this.studentAddress = addr;
    }

    public void getStudentData()
    {
        System.out.println("Student roll is :"+this.rollNumber);
        System.out.println("Student name is :"+this.studentName);
        System.out.println("Student address is :"+this.studentAddress);
        System.out.println("College Name is :"+Student.collegeName);
        System.out.println("Course Name is :"+Student.courseName);
    }

}
```

StudentDemo.java

```
-----
package com.ravi;

public class StudentDemo {

    public static void main(String[] args)
    {
        Student s1 = new Student();
        s1.setStudentData(101, "Scott", "USA");
        s1.getStudentData();

        Student s2 = new Student();
        s2.setStudentData(102, "Smith", "London");
        s2.getStudentData();
    }
}
```

```
}
```

```
}
```

19-04-2024

How to print object properties by using toString() method :

If we want to print our object properties (Instance Variable) then we should generate(override) toString() method in our class from Object class.

Now with the help of toString() method we need not to write any display kind of method to print the object properties i.e instance variable.

In order to generate the toString() method we need to follow the steps
Right click on the program -> source -> generate toString()

In order to call this toString() method, we need to print the corresponding object reference by using System.out.println() statement.

```
Manager m = new Manager();  
System.out.println(m); //Calling toString() method of Manager class
```

```
Employee e = new Employee();  
System.out.println(e); //Calling toString() method of Employee class
```

2 files :

Person.java

```
package com.ravi.tostring_demo;
```

```
public class Person  
{  
    int personId;  
    String personName;  
  
    public void setPersonData(int id, String name)  
    {  
        this.personId = id;  
        this.personName = name;  
    }  
}
```

```
@Override
```

```
public String toString()  
{  
    return "Person [personId=" + this.personId + ", personName=" + this.personName + "];"  
}
```

```
}
```

PersonDemo.java

```
package com.ravi.tostring_demo;
```

```
public class PersonDemo
{
    public static void main(String[] args)
    {
        Person p = new Person();
        p.setPersonData(111, "Scott");
        System.out.println(p); //calling Person class toString()

    }
}
```

Data Hiding :

Data hiding is nothing but declaring our data members with private access modifier so our data will not be accessible from outer world that means no one can access our data directly from outside of the class.

*We should provide the accessibility of our data through methods so we can perform VALIDATION ON DATA which are coming from outer world.

2 files :

Customer.java

```
package com.ravi.data_hiding;
```

```
public class Customer
{
    private double balance = 1000; //data Hiding

    public void deposit(int amount)
    {
        //Validation of data
        if(amount <=0)
        {
            System.err.println("Amount can't be deposited");
        }
        else
        {
            this.balance = this.balance + amount;
            System.out.println("Amount After Deposit :"+this.balance);
        }
    }

    public void withdraw(int amount)
    {
        this.balance = this.balance - amount;
        System.out.println("Amount After Withdraw :"+this.balance);
    }
}
```

BankApplication.java

```

-----
package com.ravi.data_hiding;

public class BankApplication
{
    public static void main(String[] args)
    {
        Customer raj = new Customer();
        raj.deposit(1000);
        raj.withdraw(1000);
    }
}
-----

```

Abstraction :

Showing the essential details without showing the background details is called abstraction.

In our real world a user always interacts with functionality of the product but not the data so as a developer we should hide the data from end user by declaring them private.

On the other hand the function must be declared as public so our end user will interact with the function/method.

In Java we can achieve abstraction by using two ways :

- 1) Abstract class and abstract methods :- By using abstract class and abstract method we can achieve abstraction 0 to 100%
- 2) By using interface :- By using interface we can achieve 100% abstraction.

Example :

Partial Abstraction :

```

-----
public abstract class Lift
{
    abstract void keyOne();
    abstract void keyTwo();
    abstract void keyThree();
    abstract void keyFour();
    abstract void keyFive();
}

class SteelLift extends Lift
{

    @Override
    void keyOne() {
        // TODO Auto-generated method stub
    }
}

```

```

@Override
void keyTwo() {
    // TODO Auto-generated method stub

}

@Override
void keyThree() {
    // TODO Auto-generated method stub

}

@Override
void keyFour() {
    // TODO Auto-generated method stub

}

@Override
void keyFive() {
    // TODO Auto-generated method stub

}

}

```

Full Abstraction :

```

-----
package com.ravi.data_hiding;

interface ATMMachine
{
    void withdraw();
    void deposit();
    void pinChange();
}
class SBIATM implements ATMMachine
{

    @Override
    public void withdraw() {
        // TODO Auto-generated method stub

    }

    @Override
    public void deposit() {
        // TODO Auto-generated method stub

    }

    @Override
    public void pinChange() {
        // TODO Auto-generated method stub

```

```
}  
  
}  
public class FullAbstraction {  
  
}
```

20-04-2024

Program: 1

Create two BLC classes Person and PanCard, both must be declared with public access modifier.
Create one ELC class Test which contains main method to test the application

Designing of Person class :

Instance Variable :

private PanCard obj;

create a parameterized constructor which accept PanCard class as a parameter, In this constructor initialize the instance variable and call the non static display method of PanCard class.

Designing of PanCard class :

Instance Variable : private String panId;

create one no argument constructor where initialize the panId and create the object of Person class

Define display method to print panId; Define main method in ELC class Test to test the application.

3 files :

Person.java

package com.ravi.lab;

public class Person

```
{  
    private PanCard obj;
```

```
    public Person(PanCard obj) //obj = current pan card object
```

```
{  
    this.obj = obj;  
    obj.display();  
}  
}
```

PanCard.java

package com.ravi.lab;

public class PanCard

```
{  
    private String panId;
```

```
    public PanCard() //this -> pancard object
```

```
{  
    panId = "PQ12345U";  
    Person p = new Person(this);  
}
```

```
public void display()
{
    System.out.println(this.panId);
}
}
```

Test.java

```
-----
package com.ravi.lab;

public class Test {

    public static void main(String[] args) {

        PanCard p = new PanCard();

    }

}
```

Create a BLC class called A

Instance variable
private int data = 15;

Create a no-argument constructor, where create the object for class B (Another BLC class) and call the non-static display method available in class B.

Create an instance method show() in class A, which will print instance variable data.

Create an another BLC class B

Instance variable
private A obj;

Create a parameterized constructor which takes class A as a parameter to initialize the instance variable

Create a display method inside class B which internally calling the show() method of class A

Create an ELC class Test which contains main method to test the application.

Note :- BLC and ELC all the classes must be declared as public.

Expected Output:

instance variable data in A = 15

3 files :

A.java

```
-----
package com.ravi.lab;

public class A
{
    private int data = 15;
```



```

public A()
{
    B b1 = new B(this);
    b1.display();
}

public void show()
{
    System.out.println(this.data);
}
}

```

B.java

```

-----
package com.ravi.lab;

public class B {

    private A obj;

    public B(A obj) //obj = A class object
    {
        this.obj = obj;
    }

    public void display()
    {
        this.obj.show();
    }

}

```

ELC.java

```

-----
package com.ravi.lab;

public class ELC {

    public static void main(String[] args) {

        new A();

    }

}

```

What is Constructor ?

What is the advantage of writing constructor in our class ?

If we don't write a constructor in our program then variable initialization and variable re-initialization both are done in two different lines.

If we write constructor in our program then variable initialization and variable re-initialization both are done

in the same line i.e at the time of Object creation.

[Diagram 20-APR-24]

Constructor :

If the name of the class and name of the method both are exactly same and, It should not contain any return type then it is called Constructor.

*The main purpose of constructor to initialize the instance variable of the class. (Initializing the object)

Every class must contain at least one constructor either added by compiler (default constructor) or written by user.

The access modifier of default constructor depends upon the access modifier of the class.

Explicitly, a constructor never containing any return type including void also but implicitly it returns current class object (this keyword)

2 files :

Person.java

package com.ravi.basic;

```
public class Person
{
    public Person()
    {

    }

    public void m1()
    {
        System.out.println("M1 method");
    }
}
```

ELC.java

package com.ravi.basic;

```
public class ELC {

    public static void main(String[] args)
    {
        new Person().m1(); //Internally It is returning
                           current class object
    }

}
```

A constructor may contain return keyword but not return keyword with value.

public class Test

```

{
    public Test()
    {
        return ; //valid
    }

    public static void main(String[] args)
    {
        Test t1 = new Test();
    }
}

```

A constructor is automatically called and executed at the time of creating the object.

```

public class Demo
{
    public Demo()
    {
    }
}

```

```

Demo d1 = new Demo(); //Here only Constructor will invoked
d1.Demo(); //No need to call explicitly

```

A constructor is called once per object that means if we create two objects then constructor will be called 2 times.

22-04-2024

Types Of Constructor :

Constructors are of 3 types :

- 1) Default Constructor.
- 2) No Argument OR Parameter less OR Non Parameterized OR Zero Argument Constructor.
- 3) Parameterized Constructor.

1) Default Constructor :

The constructor added by compiler to help java programmer to create the object is called Default constructor.

Student.java

```

-----
public class Student
{
}

```

javac Student.java (Compilation)

Student.class

```

-----
public class Student
{
    public Student() //default Constructor added by compiler

```

```
{  
}  
}
```

2) No Argument Constructor :

A constructor written by user without parameter is called as No Argument constructor.
Default constructor and no argument constructor looks like same but default constructor is added by compiler and no argument constructor is written by user.

```
public class Employee  
{  
    private int employeeNumber;  
    public Employee() //No Argument Constructor  
    {  
        employeeNumber = 123;  
    }  
}
```

In this no argument approach, all the objects will be initialized with same value so it is not a recommended way because all objects must contain different value.

Program on No- Argument constructor :

2 files :

Person.java

```
package com.ravi.constructor;  
  
public class Person  
{  
    private int personId;  
    private String personName;  
  
    public Person() // No Argument constructor  
    {  
        personId = 111;  
        personName = "Raj";  
    }  
  
    @Override  
    public String toString() {  
        return "Person [personId=" + personId + ", personName=" + personName + "]";  
    }  
}
```

NoArgumentConstructor.java

```
package com.ravi.constructor;  
  
public class NoArgumentConstructor  
{
```

```

public static void main(String[] args)
{
    Person raj = new Person();
    System.out.println(raj);

    Person ram = new Person();
    System.out.println(ram);
}
}

```

3) Parameterized Constructor :

If we pass one or more argument to the constructor then it is called parameterized constructor. By using parameterized constructor we can initialize all our objects with different value.

```

public class Customer
{
    int cid;
    String cname;

    public Customer(int cid, String cname) //Parameterized Constructor
    {
        this.cid = cid;
        this.cname = cname;
    }
}

```

2 files :

Dog.java

```

package com.ravi.complete_oop;

```

```

public class Dog {
    private String dogName;
    private int dogAge;
    private double dogHeight;
    private String dogColor;

    public Dog(String dogName, int dogAge, double dogHeight, String dogColor) {
        super();
        this.dogName = dogName;
        this.dogAge = dogAge;
        this.dogHeight = dogHeight;
        this.dogColor = dogColor;
    }
}

```

```

@Override
public String toString() {
    return "Dog [dogName=" + dogName + ", dogAge=" + dogAge + ", dogHeight=" + dogHeight + ",
dogColor=" + dogColor
        + "]\n";
}

```

```
}
```

ParameterizedConstructor.java

```
-----  
package com.ravi.complete_oop;  
  
public class ParameterizedConstructor {  
  
    public static void main(String[] args)  
    {  
        Dog d1 = new Dog("Tiger", 3, 3.5, "Grey");  
        System.out.println(d1);  
  
        Dog d2 = new Dog("Lion", 5, 4.5, "Black");  
        System.out.println(d2);  
  
    }  
}
```

How to write setter and getter in java :

Setter :- To modify the existing object data.

getter :- To read the private data outside of the class.

Example :

```
-----  
package com.ravi.complete_oop;  
  
public class Student  
{  
    private int rollNumber;  
  
    //Initialization of rollNumber  
    public Student(int rollNumber)  
    {  
        super();  
        this.rollNumber = rollNumber;  
    }  
  
    //Will provide rollNumber value outside of this class  
    public int getRollNumber()  
    {  
        return rollNumber;  
    }  
  
    //Will modify the Student rollNumber  
    public void setRollNumber(int rollNumber) {  
        this.rollNumber = rollNumber;  
    }  
}
```

Note :- setter and getter we can automatically generate just like toString() method.

Program on setter and getter :

Employee.java

```
-----  
package com.ravi.setter_getter;  
  
public class Employee {  
    private int employeeId;  
    private String employeeName;  
    private double employeeSalary;  
  
    public Employee(int employeeId, String employeeName, double employeeSalary) {  
        super();  
        this.employeeId = employeeId;  
        this.employeeName = employeeName;  
        this.employeeSalary = employeeSalary;  
    }  
  
    public int getEmployeeId() {  
        return this.employeeId;  
    }  
  
    public void setEmployeeId(int employeeId) {  
        this.employeeId = employeeId;  
    }  
  
    public String getEmployeeName() {  
        return this.employeeName;  
    }  
  
    public void setEmployeeName(String employeeName) {  
        this.employeeName = employeeName;  
    }  
  
    public double getEmployeeSalary() {  
        return this.employeeSalary;  
    }  
  
    public void setEmployeeSalary(double employeeSalary) {  
        this.employeeSalary = employeeSalary;  
    }  
  
    @Override  
    public String toString() {  
        return "Employee [employeeId=" + employeeId + ", employeeName=" + employeeName + ",  
        employeeSalary=" + employeeSalary + "];"  
    }  
}
```

```

}

package com.ravi.setter_getter;

import java.util.Scanner;

public class SetterAndGetter {

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Employee e1 = new Employee(1, "Scott", 20000);
        System.out.println(e1);

        System.out.print("Enter your updated Salary :");
        double sal = sc.nextDouble();
        e1.setEmployeeSalary(sal);
        System.out.println(e1);

        //We have to decide whether Employee is
        //Developer --> >= 60000
        //Designer --> >=35000
        //Tester ----> >= 15000

        double salary = e1.getEmployeeSalary();
        String name = e1.getEmployeeName();

        if(salary >= 60000)
        {
            System.out.println(name+ " is a Developer");
        }
        else if(salary >= 35000)
        {
            System.out.println(name+ " is a a Designer");
        }
        else
        {
            System.out.println(name+ " is a a Tester");
        }
    }
}

```

23-04-2024

* Encapsulation :

Binding the data member with its associated function/method in a single unit is called encapsulation.

In other words we can say "Grouping the related things together is called Encapsulation". [Laptop]

In encapsulation data must be tightly coupled with associated function.

It provides us security because we can't access the data directly, data must be accessible via methods

only.

We can achieve encapsulation in our program by using following

- a) Declare all the data members as private (Tightly encapsulated class)
- b) Define getters and setters (Or any other kind of method) for each instance variable to perform read and write operation.

Note :

If we declare all the instance variables with private access modifier then it is called tightly encapsulated class.

On the other hand if some variables are declared with private access modifier and other variables are not declared with private access modifier then it is called loosely encapsulated class.

Pass By Value :

Java language always works with Pass by value only.

```
package com.ravi.instance_demo;
```

```
public class PassByValueTest
{
    public static void main(String[] args)
    {
        int x = 5;
        accept(x);
        System.out.println(x);
    }
}
```

```
static void accept(int x)
{
    x = x + 10;
}
```

```
}
```

While calling the accept method we are passing the copy of x to the accept method so the changes in the variable value will not modify the original value

```
package com.ravi.instance_demo;
```

```
public class PassByValueTest
{
    public static void main(String[] args)
    {
        String name = "Raj";
        modifyName(name);
        System.out.println(name);
    }
}
```

```
public static void modifyName(String name)
```

```

{
    name = "Ankit";

}

}

```

Here we will get the output as Raj because name parameter variable which is receiving the value Raj but in the modifyName stack frame name variable modified with Ankit value, once the method execution is over, parameter name variable will be delete from stack frame so we are getting Raj value.

```

-----
package com.ravi.instance_demo;

public class PassByValueTest
{
    public static void main(String[] args)
    {
        int x = 10;
        x = modifyValue(x);
        System.out.println(x);
    }

    public static int modifyValue(int x)
    {
        x = x + 90;
        return x;
    }

}

```

Here from modifyValue() stack Frame we are returning the value of x so finally, It will print 100.

```

-----
package com.ravi.instance_demo;

class Customer
{
    private double bill = 1200;

    public void setBill(double bill)
    {
        this.bill = bill;
    }

    public double getBill()
    {
        return this.bill;
    }

}

public class PassByValueTest

```

```

{
    public static void main(String[] args)
    {
        Customer c1 = new Customer();

        updateCustomerBill(c1);
        System.out.println(c1.getBill());
    }

    public static void updateCustomerBill(Customer cust)
    {
        cust.setBill(1800);
    }
}

```

In case of Object, If we modify the content from any method by using any reference variable, Original object content will modify as shown in above program.
[Here only one Object is created]

package com.ravi.instance_demo;

```

class Customer
{
    private double bill = 1200;

    public void setBill(double bill)
    {
        this.bill = bill;
    }

    public double getBill()
    {
        return this.bill;
    }
}

```

```

public class PassByValueTest
{
    public static void main(String[] args)
    {
        Customer c1 = new Customer();

        updateCustomerBill(c1);
        System.out.println(c1.getBill());
    }
}

```

```

public static void updateCustomerBill(Customer cust)
{
    cust = new Customer(); //Another Object
    cust.setBill(1800);
}

```

}

Here we will get the output as 1200 because 1800 assigned to another object, the old previous object is containing 1200 only.

HEAP and STACK Diagram :

In java all the objects and contents are stored in a very special memory called HEAP Memory.

All the methods (parameter and local variable) are stored in Stack memory.

What is Garbage Collector in java ?

Garbage Collector :

In C++, It is the responsibility of the programmer to allocate as well as to de-allocate the memory otherwise the corresponding memory will be blocked and we will get OutOfMemoryError.

In Java, Programmer is responsible to allocate the memory, memory de-allocation will be automatically done by garbage collector.

Garbage Collector will scan the heap area, identify which objects are not in use (The objects which does not contain any references) and it will delete those objects which are not in use.

24-04-2024

How many ways we can make an object eligible for GC :

There are 3 ways we can make an object eligible for GC.

1) Assigning null literal to reference variable :

```
Employee e1 = new Employee(111,"Ravi");  
e1 = null;
```

2) Creating an Object inside a method :

```
public void createObject()  
{  
    Employee e2 = new Employee();  
}
```

Here we are creating Employee object inside the method so, once the method execution is over then e2 will be deleted from the Stack Frame and the employee object will become eligible for GC.

3) Assigning new Object to the old existing reference variable:

```
Employee e3 = new Employee();  
e3 = new Employee();
```

Earlier e3 variable was pointing to Employee object after that a new Employee Object is created which is pointing to another memory location so the first object is eligible for GC.

HEAP and STACK Diagram for CustomerDemo.java

```

class Customer
{
    private String name;
    private int id;

    public Customer(String name , int id) //constructor
    {
        super();
        this.name=name;
        this.id=id;
    }

    public void setId(int id) //setter
    {
        this.id=id;
    }

    public int getId() //getter
    {
        return this.id;
    }
}

```

```

public class CustomerDemo
{
    public static void main(String[] args)
    {
        int val=100;

```

```

        Customer c = new Customer("Ravi",2);

```

```

        m1(c);

```

```

        //GC [Only 1 object 3000x is eligible for GC]

```

```

        System.out.println(c.getId());
    }

```

```

    public static void m1(Customer cust) //cust = c
    {
        cust.setId(5);

```

```

        cust = new Customer("Rahul",7);

```

```

        cust.setId(9);
        System.out.println(cust.getId());
    }
}

```

25-04-2024

Method return type as a class :

We can take method return type as a class so from that method we can return that particular type of Object as shown in the program below.

2 Files :

Book.java

```
package com.ravi.method_return_type;

import java.util.Scanner;

//BLC
public class Book
{
    private String bookTitle;
    private String authorName;

    public Book(String bookTitle, String authorName) {
        super();
        this.bookTitle = bookTitle;
        this.authorName = authorName;
    }

    //Method -> which will provide Book Object
    public static Book getBookObject()
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Book Title :");
        String title = sc.nextLine();
        System.out.print("Enter Book Author Name :");
        String author = sc.nextLine();
        Book b1 = new Book(title, author);
        return b1;
    }

    @Override
    public String toString() {
        return "Book [bookTitle=" + bookTitle + ", authorName=" + authorName + "]";
    }
}
```

BookDemo.java

```
package com.ravi.method_return_type;

import java.util.Scanner;

//ELC
public class BookDemo
{
    public static void main(String[] args)
    {

```

```

Scanner sc = new Scanner(System.in);
System.out.print("How Many Book Objects :");
int noOfobj = sc.nextInt();

for(int i=1; i<=noOfobj; i++)
{
    Book object = Book.getBookObject();
    System.out.println(object);
}

}

}

```

HEAP and STACK Diagram for Sample.java

```

public class Sample
{
    private Integer i1 = 900;

    public static void main(String[] args)
    {
        Sample s1 = new Sample();

        Sample s2 = new Sample();

        Sample s3 = modify(s2);

        s1 = null;

        //GC [4 Objects 1000x,2000x,5000x and 6000x are eligible]

        System.out.println(s2.i1);
    }
    public static Sample modify(Sample s) //s = s2
    {
        s.i1=9;
        s = new Sample();
        s.i1= 20;
        System.out.println(s.i1);
        s=null;
        return s;
    }
}

```

HEAP and STACK Program for Test.java

```

public class Test
{
    Test t;
    int val;

    public Test(int val)
    {
        this.val = val;
    }
}

```

```

}

public Test(int val, Test t)
{
    this.val = val;
    this.t = t;
}

public static void main(String[] args)
{
    Test t1 = new Test(100);

    Test t2 = new Test(200,t1);

    Test t3 = new Test(300,t1);

    Test t4 = new Test(400,t2);

    t2.t = t3;
    t3.t = t4;
    t1.t = t2.t;
    t2.t = t4.t;

    System.out.println(t1.t.val);
    System.out.println(t2.t.val);
    System.out.println(t3.t.val);
    System.out.println(t4.t.val);
}
}

```

26-04-2024

HEAP and STACK Diagram for Employee.java

```

public class Employee
{
    int id = 100;

    public static void main(String[] args)
    {
        int val = 200;

        Employee e1 = new Employee();

        e1.id = val;

        update(e1);

        System.out.println(e1.id);

        Employee e2 = new Employee();

        e2.id = 500;
    }
}

```



```
switchEmployees(e2,e1); //3000x, 1000x
```

```
//GC [2 objects 2000x and 4000x are eligible for GC]
```

```
System.out.println(e1.id);
    System.out.println(e2.id);
}
```

```
public static void update(Employee e) //e = e1
{
    e.id = 500;
    e = new Employee();
    e.id = 400;
    System.out.println(e.id);
}
```

```
public static void switchEmployees(Employee e1, Employee e2)
{
    int temp = e1.id;
    e1.id = e2.id; //500
    e2 = new Employee();
    e2.id = temp;
}
}
```

```
class Test
```

```
{
int x;
int y;
```

```
void m1(Test t) //t = t2 (2000x)
```

```
{
x=x+1;
y=y+2;
t.x=t.x+3;
t.y=t.y+4;
}
```

```
public static void main(String[] args)
```

```
{
Test t1=new Test();
Test t2=new Test();
t1.m1(t2);
System.out.println(t1.x+"... "+t1.y); //1 2
System.out.println(t2.x+"... "+t2.y); //3 4
```

```
t2.m1(t1);
System.out.println(t1.x+"... "+t1.y);
System.out.println(t2.x+"... "+t2.y);
```

```
t1.m1(t1);
System.out.println(t1.x+"... "+t1.y);
System.out.println(t2.x+"... "+t2.y);
```

```
t2.m1(t2);
```

```
System.out.println(t1.x+"... "+t1.y);
System.out.println(t2.x+"... "+t2.y);
}
}
```

Passing an object reference to the Constructor : (Copy Constructor)

We can pass an object reference to the constructor, the main purpose of passing an object reference to the constructor to copy the content of one object to another object.

Example :

```
public class Test
{
    int x, y;
    public Test(Test t) //Passing an object ref. to teh constructor
    {
    }
}
```

In the below program we are taking the data from Employee class to initialize the Manager class Data (From another clas to another class)

3 files :

Employee.java

```
package com.ravi.copy_construtcor;
```

```
//BLC
```

```
public class Employee
```

```
{
    private int employeeNumber;
    private String employeeName;
```

```
    public Employee(int employeeNumber, String employeeName) {
        super();
        this.employeeNumber = employeeNumber;
        this.employeeName = employeeName;
    }
```

```
    public int getEmployeeNumber() {
        return employeeNumber;
    }
```

```
    public String getEmployeeName() {
        return employeeName;
    }
```

```
}
```

Manager.java

```
-----
package com.ravi.copy_construtcor;

//BLC
public class Manager {
    private int managerId;
    private String managerName;

    public Manager(Employee emp) //emp = e1(1000x)
    {
        this.managerId = emp.getEmployeeNumber();
        this.managerName = emp.getEmployeeName();
    }

    @Override
    public String toString() {
        return "Manager [managerId=" + managerId + ", managerName=" + managerName + "];"
    }
}
```

CopyConstructor.java

```
-----
package com.ravi.copy_construtcor;

//ELC
public class CopyConstructor
{
    public static void main(String[] args)
    {
        Employee e1 = new Employee(111, "Scott"); //1000x
        Manager m1 = new Manager(e1);
        System.out.println(m1);
    }
}
```

In the below Program we want to copy the same class object data to another object of same class. Actually we copy the data of Player class to another Player class object.

2 files :

Player.java

```
-----
package com.ravi.copy_construtcor;

public class Player
{
    private String playerName1;
    private String playerName2;

    public Player(String playerName1, String playerName2)
    {
```

```

super();
this.playerName1 = playerName1;
this.playerName2 = playerName2;
}

public Player(Player p) //p = p1 (1000x)
{
    this.playerName1 = p.playerName2;
    this.playerName2 = p.playerName1;
}

@Override
public String toString() {
    return "Player [playerName1=" + playerName1 + ", playerName2=" + playerName2 + "]";
}
}

```

CopyConstructorDemo.java

```

package com.ravi.copy_construtcor;

public class CopyConstructorDemo {

    public static void main(String[] args)
    {
        Player p1 = new Player("Abhishek","Head");
        System.out.println(p1);
        Player p2 = new Player(p1);
        System.out.println(p2);

    }

}

```

27-04-2024

The payroll system of an organization involves calculating the gross salary of each type of employee and the tax applicable to each.

Note : Take parameterized constructor to initialize all fields.

Create the following BLC classes as described below.

Class Employee

Fields: employeeId: int, employeeName : String, basicSalary : double, HRAPer : double, DAPer : double

Public Method: calculateGrossSalary() - returns a double

Calculate the gross salary as : basicSalary +HRAPer +DAPer

Class Manager

Fields: managerId: int, managerName : String, basicSalary : double, HRAPer : double,DAPer : double, projectAllowance: double

public Method: calculateGrossSalary() - returns a double
Calculate the gross salary as : basicSalary +HRAPer +DAPer + projectAllowance

Class Trainer

Fields: trainerId: int, trainerName : String, basicSalary : double, HRAPer : double,DAPer : double,
batchCount: int, perkPerBatch: double

public Method: calculateGrossSalary() - returns a double
Calculate the gross salary as : basicSalary +HRAPer +DAPer +(batchCount * perkPerBatch)

Class Sourcing

Fields: sourceId: int, sourceName : String, basicSalary : double, HRAPer : double,DAPer : double,
enrollmentTarget: int, enrollmentReached: int, perkPerEnrollment: double

Public Method: calculateGrossSalary() - returns a double

Calculate the gross salary as : basicSalary +HRAPer +DAPer
+(((enrollmentReached/enrollmentTarget)*100)*perkPerEnrollment)

Class TaxUtil

Fields: None

Public Methods:

calculateTax(Employee e) - returns a double

calculateTax(Manager m) - returns a double

calculateTax(Trainer t) - returns a double

calculateTax(Sourcing s) - returns a double

Tax Calculation Logic: If gross salary is greater than 50000 tax is 20% else, tax is 5%.

An ELC class TaxCalculation is given to you with the main Method. Use this class to test your solution.

6 files :

Employee.java

package com.ravi.lab;

public class Employee

{

private int employeeId;

private String employeeName;

private double basicSalary;

private double HRAPer;

private double DAPer;

public Employee(int employeeId, String employeeName, double basicSalary, double hRAPer, double dAPer) {

super();

this.employeeId = employeeId;

this.employeeName = employeeName;

this.basicSalary = basicSalary;

HRAPer = hRAPer;

DAPer = dAPer;

}

```
public double calculateGrossSalary()
{
    return basicSalary +HRAPer +DAPer;
}
```

```
}
```

Manager.java

```
package com.ravi.lab;
```

```
public class Manager
```

```
{
```

```
    private int managerId;
```

```
    private String managerName;
```

```
    private double basicSalary;
```

```
    private double HRAPer;
```

```
    private double DAPer;
```

```
    private double projectAllowance;
```

```
    public Manager(int managerId, String managerName, double basicSalary, double hRAPer, double dAPer,
```

```
        double projectAllowance) {
```

```
        super();
```

```
        this.managerId = managerId;
```

```
        this.managerName = managerName;
```

```
        this.basicSalary = basicSalary;
```

```
        HRAPer = hRAPer;
```

```
        DAPer = dAPer;
```

```
        this.projectAllowance = projectAllowance;
```

```
    }
```

```
    public double calculateGrossSalary()
```

```
    {
```

```
        return basicSalary +HRAPer +DAPer + projectAllowance;
```

```
    }
```

```
}
```

Trainer.java

```
package com.ravi.lab;
```

```
public class Trainer
```

```
{
```

```
    private int trainerId;
```

```
    private String trainerName;
```

```
    private double basicSalary;
```

```

    private double HRAPer;
    private double DAPer;
    private int batchCount;
    private double perkPerBatch;
    public Trainer(int trainerId, String trainerName, double basicSalary, double hRAPer, double dAPer, int
batchCount,
    double perkPerBatch) {
    super();
    this.trainerId = trainerId;
    this.trainerName = trainerName;
    this.basicSalary = basicSalary;
    HRAPer = hRAPer;
    DAPer = dAPer;
    this.batchCount = batchCount;
    this.perkPerBatch = perkPerBatch;
}

    public double calculateGrossSalary()
    {
    return basicSalary +HRAPer +DAPer +(batchCount * perkPerBatch);
    }
}

```

Sourcing.java

```

package com.ravi.lab;

public class Sourcing
{
    private int sourceId;
    private String sourceName;
    private double basicSalary;
    private double HRAPer;
    private double DAPer;
    private int enrollmentTarget;
    private int enrollmentReached;
    private double perkPerEnrollment;
    public Sourcing(int sourceId, String sourceName, double basicSalary, double hRAPer, double dAPer,
int enrollmentTarget, int enrollmentReached, double perkPerEnrollment) {
    super();
    this.sourceId = sourceId;
    this.sourceName = sourceName;
    this.basicSalary = basicSalary;
    HRAPer = hRAPer;
    DAPer = dAPer;
    this.enrollmentTarget = enrollmentTarget;
    this.enrollmentReached = enrollmentReached;
    this.perkPerEnrollment = perkPerEnrollment;
}

    public double calculateGrossSalary()
    {
    return basicSalary +HRAPer +DAPer

```

```
+(((enrollmentReached/enrollmentTarget)*100)*perkPerEnrollment);  
}
```

```
}
```

TaxUtil.java

```
package com.ravi.lab;
```

```
public class TaxUtil
```

```
{
```

```
    public static double calculateTax(Employee e)
```

```
    {
```

```
        if(e.calculateGrossSalary()> 50000)
```

```
        {
```

```
            return e.calculateGrossSalary()*0.20;
```

```
        }
```

```
    else
```

```
    {
```

```
        return e.calculateGrossSalary()*0.05;
```

```
    }
```

```
}
```

```
    public static double calculateTax(Manager m)
```

```
    {
```

```
        if(m.calculateGrossSalary()> 50000)
```

```
        {
```

```
            return m.calculateGrossSalary()*0.20;
```

```
        }
```

```
    else
```

```
    {
```

```
        return m.calculateGrossSalary()*0.05;
```

```
    }
```

```
}
```

```
    public static double calculateTax(Trainer t)
```

```
    {
```

```
        if(t.calculateGrossSalary()> 50000)
```

```
        {
```

```
            return t.calculateGrossSalary()*0.20;
```

```
        }
```

```
    else
```

```
    {
```

```
        return t.calculateGrossSalary()*0.05;
```

```
    }
```

```
}
```

```
    public static double calculateTax(Sourcing s)
```

```
    {
```

```
        if(s.calculateGrossSalary()> 50000)
```

```
        {
```

```
            return s.calculateGrossSalary()*0.20;
```

```
        }
```

```
    else
```

```
    {
```

```
        return s.calculateGrossSalary()*0.05;
```

```
    }
```



```
}
```

```
}
```

TaxCalculation.java

```
-----  
package com.ravi.lab;  
  
public class TaxCalculation {  
  
    public static void main(String[] args)  
    {  
        Employee e1 = new Employee(1, "Scott", 45000, 4000, 2000);  
        double tax = TaxUtil.calculateTax(e1);  
        System.out.println("Employee Tax is :"+tax);  
  
        //The same we can do foe Manager, Trainer, Sourcing  
  
    }  
}
```

Program that shows return type of a method can be class :

2 files :

Employee.java

```
-----  
package com.ravi.method_return_type_as_class;  
  
import java.util.Scanner;  
  
public class Employee  
{  
    private int empId;  
    private String empName;  
    private double empSalary;  
  
    public Employee(int empId, String empName, double empSalary)  
    {  
        super();  
        this.empId = empId;  
        this.empName = empName;  
        this.empSalary = empSalary;  
    }  
  
    public static Employee getEmployeeObject()  
    {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter Employee Id :");  
        int id = sc.nextInt();  
    }  
}
```

```

System.out.print("Enter Employee Name :");
String name = sc.nextLine();
name = sc.nextLine();

System.out.print("Enter Employee Salary :");
double sal = sc.nextDouble();

Employee e1 = new Employee(id, name, sal);

return e1;

}

@Override
public String toString() {
return "Employee [empId=" + empId + ", empName=" + empName + ", empSalary=" + empSalary + "]";
}

}

```

EmployeeDemo.java

```

-----
package com.ravi.method_return_type_as_class;

public class EmployeeDemo
{
    public static void main(String[] args)
    {
        for(int i=1; i<=3; i++)
        {
            Employee empObject = Employee.getEmployeeObject();
            System.out.println(empObject);
        }
    }
}

```

Lab Program :

Lab Program :(Method return type as a class + Passing Object ref)

A class called Customer is given to you.

The task is to find the Applicable Credit card Type and create CardType object based on the Credit Points of a customer.

Define the following for the class.

Attributes :

customerName : String,private

creditPoints: int, private

Constructor :

parameterizedConstructor: for both cusotmerName & creditPoints in that order.

Methods :

Name of the method : getCreditPoints

Return Type : int

Modifier : public

Task : This method must return creditPoints

Name of the method : toString, Override it,

Return type : String

Task : return only customerName from this.

Create another class called CardType. Define the following for the class

Attributes :

customer : Customer, private

cardType : String, private

Constructor :

parameterizedConstructor: for customer and cardType attributes in that order

Methods :

Name of the method : toString Override this.

Return type : String

Modifier : public

Task : Return the string in the following format.

The Customer 'Rajeev' Is Eligible For 'Gold' Card.

Create One more class by name CardsOnOffer and define the following for the class.

Method :

Name Of the method : getOfferedCard

Return type : CardType

Modifiers: public,static

Arguments: Customer object

Task : Create and return a CardType object after logically finding cardType from creditPoints as per the below rules.

creditPoints cardType

100 - 500 - Silver

501 - 1000 - Gold

1000 > - Platinum

< 100 - EMI

Create an ELC class which contains Main method to test the working of the above.

4 files :

Customer.java

package com.ravi.credit_card;

```

public class Customer
{
    private String customerName;
    private int creditPoints;

    public Customer(String customerName, int creditPoints) {
        super();
        this.customerName = customerName;
        this.creditPoints = creditPoints;
    }

    public int getCreditPoints()
    {
        return this.creditPoints;
    }

    @Override
    public String toString()
    {
        return this.customerName;
    }
}

```

CardType.java

```

package com.ravi.credit_card;

public class CardType
{
    private Customer customer;
    private String cardType;

    public CardType(Customer customer, String cardType)
    {
        super();
        this.customer = customer;
        this.cardType = cardType;
    }

    @Override
    public String toString()
    {
        return "The Customer '"+this.customer+"' Is Eligible For '"+this.cardType+"' card";
    }
}

```

CardsOnOffer.java

```

package com.ravi.credit_card;

```

```

public class CardsOnOffer
{
    public static CardType getOfferedCard(Customer cust) //cust = c1
    {
        int creditPoint = cust.getCreditPoints();
        if(creditPoint >=100 && creditPoint <=500)
        {
            return new CardType(cust, "Silver");
        }
        else if(creditPoint >500 && creditPoint <=1000)
        {
            return new CardType(cust, "Gold");
        }
        else if(creditPoint >1000)
        {
            return new CardType(cust, "Platinum");
        }
        else
        {
            return new CardType(cust, "EMI");
        }
    }
}

```

Main.java

```

package com.ravi.credit_card;

import java.util.Scanner;

public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Customer Name :");
        String name = sc.nextLine();
        System.out.print("Enter Credit Point :");
        int cp = sc.nextInt();

        Customer c1 = new Customer(name, cp);
        CardType cardType = CardsOnOffer.getOfferedCard(c1);
        System.out.println(cardType);
        sc.close();

    }
}

```

instance Block non-static block in java :

What is Instance OR non-static block in Java ?

//Instance Block OR Non-static block

```
{  
  
}
```

It is a special block in java which is executed automatically whenever an object is created. [Depends upon object creation]

It will be automatically placed in the 2nd line of the constructor, if it is available in the class.

The main purpose of instance block to initialize the instance variable of the class before constructor body execution so, it is also known as instance initializer.

Always the instance block will be executed before the constructor body execution.

If we have n number of instance block available in the class then it would be executed according to the order.[top to bottom]

If we write the instance block in the body of the constructor then compiler will not place it in the 2nd line of Constructor.

```
package com.ravi.instance_ex;
```

```
class Test
```

```
{  
    public Test()  
    {  
        System.out.println("No Argument Constructor!!!");  
    }  
}
```

```
{  
    System.out.println("Instance Block in Java");  
}
```

```
}
```

```
public class InstanceDemo1 {
```

```
    public static void main(String[] args)  
    {  
        new Test(); new Test();  
    }  
}
```

```
}
```

Note :- From the above program, It is clear that first of all instance block will be executed then only the body of the constructor will be executed.

```
package com.ravi.instance_ex;
```

```
class Test
```

```

{
int x;

public Test()
{
x = 400;
System.out.println("x value is :"+x);
}

{
x = 100;
System.out.println("x value is :"+x);
}

{
x = 200;
System.out.println("x value is :"+x);
}

{
x = 300;
System.out.println("x value is :"+x);
}
}

public class InstanceDemo1
{

public static void main(String[] args)
{
new Test();
}

}

```

Note :- From the above program, It is clear that instance blocks are executed according to the order but before the constructor body execution.

```

package com.ravi.instance_ex;

class Test
{
Test()
{
System.out.println("Constructor");

{
System.out.println("Instance Block");
}
}
}

public class InstanceDemo1
{

```

```

public static void main(String[] args)
{
    new Test();
}
}

```

Note :- Compiler will not place the instance block in the 2nd line of constructor because it is written by user in the body of constructor.

Can we declare a constructor as private ?

We can declare a constructor with all access modifiers like private, default, protected and public.

We can't declare a constructor static and final.

If we declare our constructor with private access modifier then object can't be created from outside of the class because private members are accessible within the same class only.

We should declare a constructor as a private for the following 2 reasons :

- 1) To create a singleton class (Only one object is possible)
- 2) If we want to declare only static methods in the class then we should declare the constructor as private.

Math class has also private constructor because in Math class all the methods are static methods.

30-04-2024

Relationship between the classes :

In Java, in between the classes we have 2 types of relation

- 1) IS-A Relation
- 2) HAS-A Relation

IS-A relation we can achieve using Inheritance.
HAS-A relation we can achieve using Association.

Example of IS-A relation :

```

class Vehicle
{
}
class Car extends Vehicle
{
}

```

In between Vehicle and Car we have IS-A relation because Car IS-A Vehicle.

Example of HAS-A relation :
class Vehicle


```
{  
    private Engine engine;  
}
```

In between Vehicle and Engine, We have HAS-A relation because Vehicle has an engine.

Inheritance (IS-A Relation) :

Deriving a new class (child class) from existing class (parent class) in such a way that the new class will acquire all the properties and features (except private) from the existing class is called inheritance.

It is one of the most important feature of OOPs which provides "CODE REUSABILITY".

Using inheritance mechanism the relationship between the classes is parent and child. According to Java the parent class is called super class and the child class is called sub class.

In java we provide inheritance using 'extends' keyword.

*By using inheritance all the feature of super class is by default available to the sub class so the sub class need not to start the process from beginning onwards.

Inheritance provides IS-A relation between the classes. IS-A relation is tightly coupled relation (Blood Relation) so if we modify the super class content then automatically sub class content will also modify.

Inheritance provides us hierarchical classification of classes, In this hierarchy if we move towards upward direction more generalized properties will occur, on the other hand if we move towards downward more specialized properties will occur.

Types of Inheritance in java :

Java supports 5 types of inheritance :

- 1) Single level Inheritance
- 2) Multi level Inheritance
- 3) Hierarchical Inheritance
- 4) Multiple Inheritance (Not supported using class)
- 5) Hybrid Inheritance (Combination of two)

Program on Single Level Inheritance :

SingleLevelInheritance.java[Single File Approach]

```
package com.ravi.single_level;
```

```
class Father
```

```
{  
    public void home()  
    {  
        System.out.println("3 BHK home");  
    }  
}
```

```
class Son extends Father
```

```
{  
    public void car()
```

```
{  
    System.out.println("Audi car");  
}  
}
```

```
public class SingleLevelInheritance  
{  
    public static void main(String[] args)  
    {  
        Son s = new Son();  
        s.home();  
        s.car();  
    }  
}
```

Note :

- 1) In inheritance we should always create the object for more specialized class because it has all the properties of super class.
- 2) In java, there is a predefined class called Object available in java.lang package which is by default the super class of all the classes we have in java.

Test.java

```
public class Test  
{  
}
```

javac Test.java

```
public class Test extends Object  
{  
}
```

Program on Single level Inheritance : (Encapsulation Example :)

```
package com.ravi.single_level;
```

```
class Super  
{  
    private int x;  
    private int y;  
  
    public void setData()  
    {  
        x = 100;  
        y = 200;  
    }  
}
```

```
public int getX() {  
    return x;  
}
```

```
public int getY() {  
    return y;  
}
```

```
}  
}
```

```
class Sub extends Super  
{  
    public void getData()  
    {  
        System.out.println("x value is :"+getX());  
        System.out.println("y value is :"+getY());  
    }  
}
```

```
public class SingleDemo  
{  
    public static void main(String[] args)  
    {  
        Sub s = new Sub();  
        s.setData();  
        s.getData();  
    }  
}
```

```
}
```

Inheritance with Constructor :

Constructors are not inherited in java. In order to call super class constructor/member, java software people has provided super keyword.

super keyword we can use 3 ways in java :

- 1) To call the super class variable
2) To call the super class method
3) To call the super class constructor

1) To call super class variable :

a) To call the super class variable.

If the super class variable name and sub class variable name both are same then it is called variable shadow, Here sub class variable will hide super class variable.

If we create an object for sub class then by default it will access sub class variable, if we want to access super class variable then we should use super keyword.

super keyword always refers to its immediate super class.

Just like this keyword we cannot use super keyword from static context.

SuperVar.java

package com.ravi.single_level;

```
class Father  
{
```

```

protected double balance = 50000;
}
class Daughter extends Father
{
protected double balance = 18000;

public void getBalance()
{
    System.out.println("Daughter Balance is :"+this.balance);
    System.out.println("Father Balance is :"+super.balance);
}
}

```

```

public class SuperVar {

    public static void main(String[] args)
    {
        Daughter d = new Daughter();
        d.getBalance();

    }

}

```

b) To call the super class method :

Whenever super class method name and sub class method name both are same and if we create an object for sub class then by default it will access sub class method.

If we want to access super class method then we should use super keyword.

```

package com.ravi.single_level;

class Parent
{
    public void show()
    {
        System.out.println("Show method of Parent class");
    }
}
class Child extends Parent
{
    public void show()
    {
        super.show();
        System.out.println("Show method of Child class");
        super.show();
    }
}

```

```

public class SuperMethod {

    public static void main(String[] args)

```

```

{
    Child c = new Child();
    c.show();
}

}

```

01-05-2024

3) **To call the super class constructor : (Constructor Chaining)

Whenever we write a class in java and we don't write any kind of constructor to the class then the java compiler will automatically add one default constructor to the class.

THE FIRST LINE OF ANY CONSTRUCTOR IS RESERVERD EITHER FOR super() or this() keyword.

In the first line of any constructor if we don't specify either super() or this() then the compiler will automatically add super() to the first line of constructor.

Now the purpose of this super() [added by java compiler], to call the default constructor or No-Argument constructor of the super class.

In order to call the constructor of super class as well as same class, we have total 4 cases.

We have 4 cases :

case 1 :

super() :- Automatically added by compiler and it is used to access default or no argument constructor of super class.

ConstructorTest.java

```

package com.ravi.constructor_chaining;

```

```

class Alpha
{
    public Alpha()
    {
        System.out.println("Alpha class Constructor..");
    }
}

```

```

class Beta extends Alpha
{
    public Beta()
    {
        System.out.println("Beta class Constructor..");
    }
}

```

```

public class ConstructorTest
{

```

```
public static void main(String[] args)
{
    new Beta(); //nameless OR Anonymous object
}
}
```

```
package com.ravi.constructor_chaining;
```

```
class Alpha
{
    public Alpha()
    {
        System.out.println("Alpha class Constructor..");
    }
}
```

```
class Beta extends Alpha
{
}
```

```
class Gamma extends Beta
{
    public Gamma()
    {
        System.out.println("Gamma class Constructor..");
    }
}
```

```
public class ConstructorTest
{
    public static void main(String[] args)
    {
        new Gamma(); //nameless OR Anonymous object
    }
}
```

Note :- From the above program, It is clear that default constructor and super() is added by the compiler even the class is empty class.

Output : Alpha class Constructor..
Gamma class Constructor..

Case 2 :

super(15) : To call the parameterized constructor (one argument of type integer) of super class.

CallingParameterizedConstructor.java

```
package com.ravi.constructor_chaining;
```

```
class Super
{
    public Super(String name)
    {
        super();
    }
}
```

```
System.out.println("It's summer month name is :"+name);
}
}
```

```
class Sub extends Super
{
    public Sub()
    {
        super("May");
        System.out.println("After this rain will come");
    }
}
```

```
public class CallingParameterizedConstructor
{
    public static void main(String[] args)
    {
        new Sub();
    }
}
```

Case 3 :

this() : It is not added by compiler. The main purpose of this()
to call current class no argument constructor.

CallingNoArgumentConstructor.java

```
package com.ravi.constructor_chaining;

class Parent
{
    public Parent()
    {
        System.out.println("No Argument Constructor");
    }

    public Parent(String name)
    {
        this();
        System.out.println("Parameterized Constructor :"+name);
    }
}

class Child extends Parent
{
    public Child()
    {
        super("NIT");
        System.out.println("No Argument constructor of Child class");
    }
}

public class CallingNoArgumentConstructor {
```

```

public static void main(String[] args)
{
    new Child();
}
}

```

case 4 :

this(90) : Used to call parameterized constructor of current class.

CurrentClassParameterized.java

```

package com.ravi.constructor_chaining;

class Base
{
    public Base()
    {
        this("Hyd");
        System.out.println("Base class No argument");
    }
    public Base(String x)
    {
        System.out.println("Parmeterized Constructor :"+x);
    }
}
class Derived extends Base
{
    public Derived()
    {
        System.out.println("Derived class No argument");
    }
}

```

```

public class CurrentClassParameterized {

    public static void main(String[] args) {
        new Derived();
    }

}

```

Program on Single level Inheritance :

```

package com.ravi.constructor_chaining;

class TemporaryEmployee
{
    protected int employeeId;
    protected String employeeName;
    protected double employeeSalary;
}

```



```

public TemporaryEmployee(int employeeId, String employeeName, double employeeSalary)
{
    super();
    this.employeeId = employeeId;
    this.employeeName = employeeName;
    this.employeeSalary = employeeSalary;
}

@Override
public String toString() {
    return "TemporaryEmployee [employeeId=" + employeeId + ", employeeName=" + employeeName + ",
employeeSalary="
    + employeeSalary + "]";
}

}

class PermanentEmployee extends TemporaryEmployee
{
    protected String department;
    protected String designation;

    public PermanentEmployee(int employeeId, String employeeName, double employeeSalary, String
department,
    String designation) {
        super(employeeId, employeeName, employeeSalary);
        this.department = department;
        this.designation = designation;
    }

    @Override
    public String toString()
    {
        return super.toString()+ "PermanentEmployee [department=" + department + ", designation=" +
designation + "]";
    }

}

public class SingleLevelDemo
{
    public static void main(String[] args)
    {
        PermanentEmployee p = new PermanentEmployee(18, "Virat", 90000, "Cricket", "Batter");
        System.out.println(p);
    }
}

```

02-05-2024

Program on Single Level Inheritance :

```
package com.ravi.single_level_demo;

import java.util.Scanner;

class Shape
{
    protected int x;
    public Shape(int x) // x <- side
    {
        this.x = x;
    }
}

class Square extends Shape
{
    public Square(int side)
    {
        super(side);
    }

    public void getAreaOfSquare()
    {
        double area = x * x;
        System.out.println("Area of Square is :"+area);
    }
}

public class SingleLevelInheritance
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the side of the Square :");
        int side = sc.nextInt();

        Square ss = new Square(side);
        ss.getAreaOfSquare();
        sc.close();
    }
}
```

Program on Hierarchical Inheritance :

```
package com.ravi.hierarchical_inheritance;

import java.util.Scanner;

class Shape
{
```

```
protected int x;
```

```
public Shape(int x)
{
    this.x = x;
    System.out.println(x);
}
```

```
class Circle extends Shape
{
    final double PI = 3.14;
    public Circle(int radius)
    {
        super(radius);
    }
}
```

```
public void getAreaOfCircle()
{
    double area = PI * x * x;
    System.out.println("Area of Circle is :"+area);
}
```

```
class Rectangle extends Shape
{
    protected int breadth;
```

```
    public Rectangle(int length , int breadth)
    {
        super(length);
        this.breadth = breadth;
    }
```

```
    public void areaOfRectengle()
    {
        double area = super.x * this.breadth ;
        System.out.println("Area of Rectangle is :"+area);
    }
```

```
}
```

```
public class HierarchicalDemo
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the radius of the Circle :");
        int rad = sc.nextInt();
```

```
        Circle c = new Circle(rad);
        c.getAreaOfCircle();
```

```
        System.out.print("Enter the length of the Rect : ");
        int len = sc.nextInt();
        System.out.print("Enter the breadth of the Rect : ");
```

```
int bre = sc.nextInt();

Rectangle r = new Rectangle(len, bre);
r.areaOfRectangle();
sc.close();
}
}
```

----- HOW MANY WAYS WE CAN INITIALIZE THE OBJECT PROPERTIES ? -----

The following are the ways to initialize the object properties

```
public class Test
{
    int x,y;
}
```

1) At the time of declaration :

Example :

```
public class Test
{
    int x = 10;
    int y = 20;
}

Test t1 = new Test(); [x = 10 y = 20]
Test t2 = new Test(); [x = 10 y = 20]
```

Here the drawback is all objects will be initialized with same value.

2) By using Object Reference :

```
public class Test
{
    int x,y;
}

Test t1 = new Test(); t1.x=10; t1.y=20;
Test t2 = new Test(); t2.x=30; t2.y=40;
```

Here we are getting different values with respect to object but here the program becomes more complex.

3) By using methods :

A) First Approach (Method without Parameter)

```
public class Test
{
    int x,y;
```

```

    public void setData()
    {
        x = 100; y = 200;
    }
}

```

```

Test t1 = new Test(); t1.setData(); [x = 100 y = 200]
Test t2 = new Test(); t2.setData(); [x = 100 y = 200]

```

All the objects will be initialized with same value.

B) Second Approach (Method with Parameter)

```

public class Test
{
    int x,y;

    public void setData(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
}

```

```

Test t1 = new Test(); t1.setData(12,78); [x = 12 y = 78]
Test t2 = new Test(); t2.setData(15,29); [x = 15 y = 29]

```

Here the Drawback is initialization and re-initialization both are done in two different lines so Constructor introduced.

4) By using Constructor

A) First Approach (No Argument Constructor)

```

public class Test
{
    int x,y;

    public Test() //All the objects will be initialized with
    {               same value
        x = 100; y = 200;
    }
}

```

```

Test t1 = new Test(); [x = 100 y = 200]
Test t2 = new Test(); [x = 100 y = 200]

```

B) Second Approach (Parameterized Constructor)

```

public class Test
{
    int x,y;

```

```

    public Test(int x, int y)
    {
        this.x = x;
this.y = y;
    }
}

```

```

Test t1 = new Test(12,78);  [x = 12  y = 78]
Test t2 = new Test(15,29);  [x = 15  y = 29]

```

This is the best way to initialize our instance variable because variable initialization and variable re-initialization both will be done in the same line as well as all the objects will be initialized with different values.

C) Third Approach (Copy Constructor)

```

    public class Manager
    {
        private int managerId;
private String managerName;

    public Manager(Employee emp)
    {
        this.managerId = emp.getEmployeeId();
        this.managerName = emp.getEmployeeName();
    }
}

```

Here with the help of Object reference (Employee class) we are initializing the properties of Manager class. (Copy Constructor)

d) By using instance block (Instance Initializer)

```

public class Test
{
    int x,y;

    public Test()
    {
        System.out.println(x); //100
        System.out.println(y); //200
    }

    //Instance block
    {
        x = 100;
        y = 200;
    }
}

```

5) By using super keyword :

```

class Super

```

```

{
    int x,y;

    public Super(int x , int y)
    {
        this.x = x;
this.y = y;
    }
}
class Sub extends Super
{
    Sub()
    {
        super(100,200); //Initializing the properties of super class
    }
}

new Sub();

```

Why java does not support multiple inheritance ?

Multiple Inheritance is a situation where a sub class wants to inherit the properties two or more super classes and by using super keyword we have ambiguity issue. It is also known as Diamond Problem.

Java does not support multiple inheritance because in the sub class, automatically one super keyword will be added by compiler to the first line of constructor now we have ambiguity issue to call default constructor of super class as shown in the diagram (02-MAY-24)

Multiple inheritance we can't achieve using classes but same is possible by using interface.

Program to achieve multi-level Inheritance :

```

package com.ravi.multilevel;

class Student
{
    protected int studentId;
    protected String studentName;
    protected String studentAddress;

    public Student(int studentId, String studentName, String studentAddress) {
        super();
        this.studentId = studentId;
        this.studentName = studentName;
        this.studentAddress = studentAddress;
    }
}

class Science extends Student
{
    protected int physics;
    protected int chemistry;

    public Science(int studentId, String studentName, String studentAddress, int physics, int chemistry)

```

```

{
    super(studentId, studentName, studentAddress);
    this.physics = physics;
    this.chemistry = chemistry;
}
}
class PCM extends Science
{
    protected int math;

    public PCM(int studentId, String studentName, String studentAddress, int physics, int chemistry, int math)
    {
        super(studentId, studentName, studentAddress, physics, chemistry);
        this.math = math;
    }

    @Override
    public String toString()
    {
        return "PCM [math=" + math + ", physics=" + physics + ", chemistry=" + chemistry + ", studentId=" +
        studentId
        + ", studentName=" + studentName + ", studentAddress=" + studentAddress + "]";
    }

    public void calculateMarks()
    {
        int total = physics + chemistry + math;
        System.out.println("Total Marks :"+total);
    }

}

public class MultiLevelInheritance {

    public static void main(String[] args)
    {
        PCM p = new PCM(1, "A", "S R Nagar", 70, 60, 90);
        p.calculateMarks();
        System.out.println(p);
    }
}

```

03-05-2024

Access modifiers in java :

In order to define the accessibility level of the class as well as member of the class we have 4 access modifiers :

- 1) private (Within the same class)
- 2) default (Within the same package)
- 3) protected (Within the same package Or even from another package by using Inheritance)
- 4) public (No Restriction)

private :

It is an access modifier and it is the most restrictive access modifier because the member declared as private can't be accessible from outside of the class.

In Java we can't declare an outer class as a private or protected. Generally we should declare the data member(variables) as private.

In java outer class can be declared as public, abstract, final, sealed and non-sealed only.

default :-

It is an access modifier which is less restrictive than private. It is such kind of access modifier whose physical existence is not available that means when we don't specify any kind of access modifier before the class name, variable name or method name then by default it would be default.

As far as its accessibility is concerned, default members are accessible within the same folder(package) only.

protected :

It is an access modifier which is less restrictive than default because the member declared as protected can be accessible from the outside of the package (folder) too but by using inheritance concept.

Program :- Here we have two different packages :

Access.java [Available in pack1 package]
package pack1;

```
public class Access
{
    protected int x = 200;
}
```

ELC.java (Available in pack2 package)
package pack2;

```
import pack1.Access;
```

```
public class ELC extends Access
{
    public static void main(String[] args)
    {
        ELC e = new ELC();
        System.out.println(e.x);
    }
}
```

public :

It is an access modifier which does not contain any kind of restriction that is the reason the member declared as public can be accessible from everywhere without any restriction.

According to Object Oriented rule we should declare the classes and methods as public where as variables must be declared as private or protected according to the requirement.

Working with HAS-A relation :

HAS-A relation between the classes :

In order to achieve HAS-A relation concept we should use Association.

Association (Relationship between the classes through Object reference)

Association :

Association is a connection between two separate classes that can be built up through their Objects.

The association builds a relationship between the classes and describes how much a class knows about another class.

This relationship can be unidirectional or bi-directional. In Java, the association can have one-to-one, one-to-many, many-to-one and many-to-many relationships.

Example:-

One to One: A person can have only one PAN card

One to many: A Bank can have many Employees

Many to one: Many employees can work in single department

Many to Many: A Bank can have multiple customers and a customer can have multiple bank accounts.

3 files :

Student.java(C)

```
package com.ravi.association;
```

```
public class Student {  
    private int studentId;  
    private String studentName;  
    private String studentAddress;  
    private int totalMarks;
```

```
    public Student(int studentId, String studentName, String studentAddress, int totalMarks) {  
        super();  
        this.studentId = studentId;  
        this.studentName = studentName;  
        this.studentAddress = studentAddress;  
        this.totalMarks = totalMarks;  
    }
```

```
    @Override  
    public String toString() {  
        return "Student [studentId=" + studentId + ", studentName=" + studentName + ", studentAddress=" +  
studentAddress  
        + ", totalMarks=" + totalMarks + "];"  
    }
```

```
    public int getStudentId() {
```

```

return studentId;
}

public String getStudentName() {
return studentName;
}

public String getStudentAddress() {
return studentAddress;
}

public int getTotalMarks() {
return totalMarks;
}
}

```

Trainer.java(C)

```

package com.ravi.association;

import java.util.Scanner;

public class Trainer
{
    public static void viewStudentProfile(Student s)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Student Id :");
        int id = sc.nextInt();

        if(id == s.getStudentId())
        {
            System.out.println(s);
        }
        else
        {
            System.err.println("Student Id is invalid");
        }
        sc.close();
    }
}

```

Association.java

```

package com.ravi.association;

public class Association {

    public static void main(String[] args)
    {
        Student s1 = new Student(1, "Nikhil", "Ameerpet", 90);

        Trainer.viewStudentProfile(s1);

    }
}

```

}

Note :- Trainer and Student creates HAS-A Relation in the program, Trainer can view the profile of Student so, the Has-A relation is established through object reference.

A BLC class Student is given to you. It contains the following implementation.

Instance Variables: studentId : int , name :String , examFee:double

Methods: displayDetails(): String , payFee() :double

Define parameterized constructor and a toString method.

Create two BLC sub classes of Student called DayScholar and Hosteller.
Add the following implementations to each class.

DayScholar:

Instance Variables : transportFee:double ,

Methods: Define parameterized constructor

Hosteller:

Instance Variables: hostelFee :double

Methods: Define parameterized constructor

Add the following methods in BOTH classes.

Method: displayDetails(): This method should return a string containing the details of the student in the following format:

Student [name=John Smith,studentId=123,examFee=100.0] OR

DayScholar[transportFee=500, name=John Smith,studentId=123,examFee=100.0] and so on.

Method: payFee(amount): This method takes amount as parameter that represents the fee provided. You must calculate and then return the remaining amount paid in excess. If excess is paid, the returned amount will be negative. The total fees that each student must pay must include all the fees applicable to that Student(examFees, transportFees, hostelFees as applicable).

Subtract the given amount from total fee for each student and return the remaining amount.

An ELC class Tester is given to you with a main method. Use this class to test your solution's classes and methods.

Example Output:

Student[name=John Smith,studentId=1,examFee=25000.0]

DayScholar[TransportFee=5000.0,name=Brian Lara,studentId=2,examFee=25000.0]

Remaining amount to pay is: 12000.0

Hosteller[HostelFee=8000.0,name=Virat Kohli,studentId=3,examFee=25000.0]

All Fees are clear

Student.java(C)

package com.ravi.lab;

```
public class Student {  
    private int studentId;  
    private String studentName;  
    private double examFee;
```

```
    public Student(int studentId, String studentName, double examFee) {  
        super();  
        this.studentId = studentId;  
        this.studentName = studentName;  
        this.examFee = examFee;  
    }
```

```
    public String displayDetails()  
    {  
        return this.toString();  
    }
```

```
    public double payFee()  
    {  
        return this.examFee;  
    }
```

```
    @Override  
    public String toString()  
    {  
        //Student [name=John Smith,studentId=123,examFee=100.0]  
        return "Student[name "+this.studentName+", studentId "+this.studentId+",examFee"+this.examFee+"]";  
    }
```

```
}
```

DayScholer.java

package com.ravi.lab;

```
public class DayScholer extends Student  
{
```

```

private double transportFee;

public DayScholer(int studentId, String studentName, double examFee, double transportFee) {
    super(studentId, studentName, examFee);
    this.transportFee = transportFee;
}

public double payFee(double amount)
{
    double amountToPay = super.payFee() + this.transportFee;
    return amountToPay - amount;
}

public String displayDetails()
{
    return super.displayDetails()+" transport Fee"+this.transportFee;
}
}

```

Hosteller.java

```

package com.ravi.lab;

public class Hosteller extends Student {
    private double hostelFee;

    public Hosteller(int studentId, String studentName, double examFee, double hostelFee) {
        super(studentId, studentName, examFee);
        this.hostelFee = hostelFee;
    }

    public double payFee(double amount)
    {
        double amountToPay = super.payFee() + this.hostelFee;
        return amountToPay - amount;
    }

    public String displayDetails()
    {
        return super.displayDetails()+" Hostel Fee "+this.hostelFee;
    }
}

```

Tester.java

```

package com.ravi.lab;

public class Tester
{
    public static void main(String[] args)
    {
        DayScholer d = new DayScholer(1, "A", 3000, 10000);
        double payFee = d.payFee(12000);
    }
}

```

```

if(payFee < 0)
{
    System.out.println(payFee+" Amount will be refunded");
}
else
{
    System.out.println(payFee+" Amount must be paid to college");
}

```

```

System.out.println(d.displayDetails());

```

```

Hosteller h = new Hosteller(2, "B", 3000, 150000);
payFee = h.payFee(150000);
System.out.println(payFee);
System.out.println(h.displayDetails());

```

```

}

```

```

}

```

Composition (Strong reference) :

Composition in Java is a way to design classes such that one class contains an object of another class. It is a way of establishing a "HAS-A" relationship between classes.

Composition represents a strong relationship between the containing class and the contained class. If the containing object (Car object) is destroyed, all the contained objects (Engine object) are also destroyed.

A car has an engine. Composition makes strong relationship between the objects. It means that if we destroy the owner object, its members will be also destroyed with it. For example, if the Car is destroyed the engine will also be destroyed as well.

3 files :

Engine.java

package com.ravi.composition;

```

public class Engine
{
    private String engineType;
    private long horsepower;

    public Engine(String engineType, long horsepower)
    {
        super();
        this.engineType = engineType;
        this.horsePower = horsepower;
    }

```

@Override

```

public String toString() {
    return "Engine [engineType=" + engineType + ", horsepower=" + horsepower + "]";
}

```

```
}
```

Car.java

```
package com.ravi.composition;
```

```
public class Car
```

```
{
```

```
    private String carName;
```

```
    private int carModel;
```

```
    private final Engine engine; // HAS-A Relation
```

```
    public Car(String carName, int carModel)
```

```
    {
```

```
        super();
```

```
        this.carName = carName;
```

```
        this.carModel = carModel;
```

```
        this.engine = new Engine("Petrol", 1200); //Only one Engine Object
```

```
    }
```

```
    @Override
```

```
    public String toString() {
```

```
        return "Car [carName=" + carName + ", carModel=" + carModel + ", engine=" + engine + "];"
```

```
    }
```

```
}
```

Composition.java

```
package com.ravi.composition;
```

```
public class Composition
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Car c = new Car("Ford", 2024);
```

```
        System.out.println(c);
```

```
    }
```

```
}
```

Assignment :

Create a program on composition to establish a relation between
Department and University


```
class Department
{
}
```

```
class University
{
}
```

Aggregation (Weak Reference) :

Aggregation in Java is another form of association between classes that represents a "HAS-A" relationship, but with a weaker bond compared to composition.

In aggregation, one class contains an object of another class, but the contained object can exist independently of the container. If the container object is destroyed, the contained object can still exist.

3 files :

College.java

```
package com.ravi.weak_reference;
```

```
public class College
{
```

```
    private String collegeName;
    private String collgeLocation;
```

```
    public College(String collegeName, String collgeLocation) {
        super();
        this.collegeName = collegeName;
        this.collgeLocation = collgeLocation;
    }
```

```
    @Override
```

```
    public String toString() {
        return "College [collegeName=" + collegeName + ", collgeLocation=" + collgeLocation + "]\n";
    }
```

```
}
```

Student.java

```
package com.ravi.weak_reference;
```

```
public class Student
```

```
{
    private int StudentId;
    private String studentName;
    private College collge; //HAS-A Relation
```

```

public Student(int studentId, String studentName, College collge) //college = c1
{
    super();
    StudentId = studentId;
    this.studentName = studentName;
    this.collge = collge;
}

@Override
public String toString() {
    return "Student [StudentId=" + StudentId + ", studentName=" + studentName + ", collge=" + collge + "];"
}

}

```

AggregationDemo.java

```

-----
package com.ravi.weak_reference;

public class AggregationDemo
{
    public static void main(String[] args)
    {
        College c1 = new College("JNTU", "Hyderabad");

        Student s1 = new Student(1,"Scott",c1);
        System.out.println(s1);

        College c2 = new College("Osmania", "Hyderabad");

        Student s2 = new Student(2,"Smith",c1);
        System.out.println(s2);

    }
}

```

Assignment :

```

-----
class Order
{
    orderId;
    itemName;
    itemPrice;
}

class Customer
{
    //All customer related information
}

```

```
Order order; //HAS- A Realtion
}
```

Polymorphism :

Poly means "many" and morphism means "forms".

It is a Greek word whose meaning is "same object having different behavior".

In our real life a person or a human being can perform so many task, in the same way in our programming languages a method or a constructor can perform so many task.

Eg:-

```
void add(int a, int b)
```

```
void add(int a, int b, int c)
```

```
void add(float a, float b)
```

```
void add(int a, float b)
```

Polymorphism can be divided into two types :

1) Static polymorphism OR Compile time polymorphism OR Early binding

2) Dynamic Polymorphism OR Runtime polymorphism OR Late binding

1) Static Polymorphism :

The polymorphism which exist at the time of compilation is called Static OR compile time polymorphism.

In static polymorphism, compiler has very good idea that which method is invoked depending upon METHOD PARAMETER.

Here the binding of the method is done at compilation time so, it is known as early binding.

We can achieve static polymorphism by using Method Overloading concept.

Example of static polymorphism : Method Overloading.

2) Dynamic Polymorphism OR Runtime Polymorphism

The polymorphism which exist at runtime is called Dynamic polymorphism Or Runtime Polymorphism.

*Here compiler does not have any idea about method calling, at runtime JVM will decide which method will be invoked depending upon CLASS TYPE Object.

Here method binding is done at runtime so, it is also called Late Binding.

We can achieve dynamic polymorphism by using Method Overriding.

Example of Dynamic Polymorphism : Method Overriding

----- Method Overloading : -----

Writing two or more methods in the same class or even in the super and sub class in such a way that the method name must be same but the argument must be different.

While Overloading a method we can change the return type of the method.

If parameters are same but only method return type is different then it is not an overloaded method.

Method overloading is possible in the same class as well as super and sub class.

While overloading the method the argument must be different otherwise there will be ambiguity problem.

IQ

Can we overload the main method/static method ?

Yes, we can overload the main method OR static method but the execution of the program will start from main method which accept String [] array as a parameter.

Note :- The advantage of method overloading is same method name we can reuse for different functionality for refinement of the method.

Note :- In System.out.println() or System.out.print(int x), print() and println() methods are best example for Method Overloading.

2 files :

Addition.java

```
//Program on Constructor Overloading
package com.ravi.constructor_overloading;

public class Addition
{
    public Addition(int x, int y)
    {
        System.out.println("Sum of two integer is :"+(x+y));
    }

    public Addition(int x, int y, int z)
    {
        System.out.println("Sum of three integer is :"+(x+y+z));
    }

    public Addition(float x, float y)
    {
        System.out.println("Sum of two float is :"+(x+y));
    }
}
```

Main.java

```
package com.ravi.constructor_overloading;
```

```
public class Main {
```

```
    public static void main(String [] args)
    {
        new Addition(2.3f, 7.8F);
        new Addition(10, 20, 30);
        new Addition(12,90);
    }
```

```
}
```

2 files :

Addition.java

```
package com.ravi.constructor_overloading1;
```

```
public class Addition
```

```
{
    public Addition(int x, int y)
    {
        super();
        System.out.println("Sum of two integer is :"+(x+y));
    }
```

```
    public Addition(int x, int y, int z)
    {
        this(100,200);
        System.out.println("Sum of three integer is :"+(x+y+z));
    }
```

```
    public Addition(float x, float y)
    {
        this(10,20,30);
        System.out.println("Sum of two float is :"+(x+y));
    }
}
```

Main.java

```
package com.ravi.constructor_overloading1;
```

```
public class Main {
```

```
    public static void main(String [] args)
    {
        new Addition(2.3f, 7.8F);
```

```
    }
```

```
}
```

instance block role with constructor :

Our instance block will be placed in the constructor which contains super(), if we have multiple constructor and all the constructors are executed according to constructor chaining then compiler will place instance block to only one constructor which contains super()

Addition.java

```
package com.ravi.constructor_overloading2;

public class Addition
{
    public Addition()
    {
        this(10);
        System.out.println("No Argument Constructor");
    }

    public Addition(int x)
    {
        this(100,200);
        System.out.println("One Argument Constructor :"+x);
    }

    public Addition(int x, int y)
    {
        super();
        System.out.println("Two Argument Constructor :"+x+"-"+y);
    }

    {
        System.out.println("Instance Block");
    }
}
```

```
package com.ravi.constructor_overloading2;
```

```
public class Test {

    public static void main(String[] args)
    {
        new Addition();
    }
}
```

We can't use return keyword inside instance block

```
package com.ravi.cloneable;
```

```
public class CloneDemo
{
    int x = 100;
```

```

{
    System.out.println(x);
    return; //error
}

public static void main(String[] args)
{
    new CloneDemo();
}

}
-----
package com.ravi.constructor;

class Alpha
{
    public Alpha()
    {
        System.out.println("Alpha class Constructor");
    }

    {
        System.out.println("Alpha class instance block");
    }
}
class Beta extends Alpha
{
    public Beta()
    {
        System.out.println("Beta class Constructor");
    }

    {
        System.out.println("Beta class instance block");
    }
}

public class InstanceDemo
{
    public static void main(String[] args)
    {
        new Beta();
    }
}
-----

```

Program that describes we can change the return type of the method while overloading a method

2 files :

Sum.java

```

package com.ravi.method_overload;

```

```

public class Sum
{
    public int add(int x, int y)
    {
        int z = x+y;
        return z;
    }

    public String add(String x, String y) //data base
    {
        String z = x+y;
        return z;
    }

    public double add(double x, double y)
    {
        double z = x+y;
        return z;
    }
}

```

Main.java

```

-----
package com.ravi.method_overload;

public class Main
{
    public static void main(String[] args)
    {
        Sum s1 = new Sum();
        String add = s1.add("Data", "base");

        int x = s1.add(12, 12);

        double y = s1.add(12.89, 12.90);

        System.out.println(add+" : "+x+" : "+y);
    }
}
-----

```

08-05-2024

Var-Args :

It was introduced from JDK 1.5 onwards.

It stands for variable argument. It is an array variable which can hold 0 to n number of parameters of same type or different type by using Object class.

It is represented by exactly 3 dots (...) so it can accept any number of argument (0 to nth) that means now

we need not to define method body again and again, if there is change in method parameter value.

var-args must be only one and last argument.

We can use var-args as a method parameter only.

Program that describes var args can hold 0 to n number of parameters :

```
package com.ravi.var_args;

class Demo
{
    public void input(int ...x)
    {
        System.out.println("Method Executed");
    }
}
```

```
public class VarArgsDemo1
{
    public static void main(String[] args)
    {
        Demo d = new Demo();
        d.input();
        d.input(3);
        d.input(5,10);
        d.input(5,10,30);

    }
}
```

Program that describes we can fetch the value from var args to perform addition operation.

```
package com.ravi.var_args;

class Demo
{
    public void input(int ...x) //100 200 300 400
    {
        int sum = 0;

        for(int y : x)
        {
            sum = sum + y;
        }
        System.out.println("Sum of parameters :"+sum);

    }
}
```

```
public class VarArgsDemo1
{
```

```

public static void main(String[] args)
{
    Demo d = new Demo();
    d.input();
    d.input(3);
    d.input(5,10);
    d.input(5,10,30);
    d.input(100,200,300,400);

}

```

```

}
-----
//Var args can hold heterogeneous types of data also

```

```

package com.ravi.var_args;

class Demo
{
    public void input(Object ...x)
    {
        for(Object y : x)
        {
            System.out.println(y);
        }
        System.out.println(".....");
    }
}

```

```

public class VarArgsDemo1
{
    public static void main(String[] args)
    {
        Demo d = new Demo();
        d.input(10);
        d.input("Ravi","NIT",new String("32"),34.89, true);
        d.input(false, 12, 45.89, 45.6f);

    }

}

```

```

-----
Var args must be only one and last argument.

```

```

package com.ravi.var_args;

class Demo
{
    /*public void input(int ...x, int ...y) //invalid
    {
    }

    public void input(int ...x, int y) //invalid
    {

```

```

}*/

public void input(int x, int ...y)
{
    System.out.println("x value is :"+x);

    for(int a : y)
    {
        System.out.println(a);
    }
}
}
}

```

```

public class VarArgsDemo1
{
    public static void main(String[] args)
    {
        Demo d = new Demo();
        d.input(10,20,30,40,50);

    }
}

```

By using var args we can hold homogenous and hetrogeneous types of data.

```

package com.ravi.var_args;

```

```

class Demo
{
    public void input(Object ...x)
    {
        for(Object y : x)
        {
            System.out.println(y);
        }
    }
}

```

```

public class VarArgsDemo1
{
    public static void main(String[] args)
    {
        Demo d = new Demo();
        d.input("ravi", true, false, 90.78);
    }
}

```

Wrapper classes in java :

In java we have 8 primitive data types i.e byte, short, int, long, float, double, char and boolean.

Except these primitives, everything in java is an Object.

If we remove these 8 data types from java then Java will become pure Object Oriented language.

From java 1.5 onwards we can convert primitive to wrapper and wrapper to primitive by using following concept.

- a) Autoboxing
- b) Unboxing

Autoboxing

When we convert the primitive data types into corresponding wrapper object then it is called Autoboxing as shown below.

Primitive type	Wrapper Object
----------------	----------------

byte	- Byte
------	--------

short	- Short
-------	---------

int	- Integer
-----	-----------

long	- Long
------	--------

float	- Float
-------	---------

double	- Double
--------	----------

char	- Chracter
------	------------

boolean	- boolean
---------	-----------

Diagram (08-MAY-24)

09-05-2024

```
//Integer.valueOf(int);
public class AutoBoxing1
{
    public static void main(String[] args)
    {
        int a = 12;
        Integer x = Integer.valueOf(a); //Upto 1.4 version
        System.out.println(x);
```

```
        int y = 15;
        Integer i = y; //From 1.5 onwards compiler takes care
        System.out.println(i);
```

```
}  
}  
-----  
package com.ravi.var_args;  
  
public class VarArgsDemo1  
{  
    public static void main(String[] args)  
    {  
        byte b = 12;  
        Byte wrapB = Byte.valueOf(b);  
  
        short s = 56;  
        Short wrapS = Short.valueOf(s);  
  
        int i = 89;  
        Integer wrapI = Integer.valueOf(i);  
  
        long l = 90;  
        Long wrapL = Long.valueOf(l);  
  
        float f = 34.78f;  
        Float wrapF = Float.valueOf(f);  
  
        double d = 45.90;  
        Double wrapD = Double.valueOf(d);  
  
        char c = 'A';  
        Character wrapC = Character.valueOf(c);  
  
        boolean bool = true;  
        Boolean wrapBool = Boolean.valueOf(bool);  
  
    }  
}  
-----  
public class AutoBoxing2  
{  
    public static void main(String args[])  
    {  
        int y = 12;  
        Integer x = y;  
        System.out.println(x);  
  
        double e = 45.90;  
        Double d = e;  
        System.out.println(d);  
  
        boolean a = true;  
        Boolean b = a;  
        System.out.println(b);  
    }  
}  
-----
```

Overloaded valueOf() method :

The following are the commonly used overloaded methods of Integer class.

- 1) public static Integer valueOf(int x) : It will convert the primitive int type into wrapper object.
- 2) public static Integer valueOf(String str) : It will convert the String value into Wrapper object.
- 3) public static Integer valueOf(String str, int radix/base)
It will convert the String into Integer Wrapper object by using the specified base or radix as shown in the program.

Note :- We can pass base OR radix upto 36
i.e A to Z (26) + 0 to 9 (10) -> [26 + 10 = 36], It can be calculated by using Character.MAX_RADIX.
Output will be generated on the basis of radix

```
package com.ravi.value_of;

public class ValueOfMethod
{
    public static void main(String[] args)
    {
        Integer value = Integer.valueOf(12);
        System.out.println(value);

        String str = "234";
        Integer intObj = Integer.valueOf(str);
        System.out.println(intObj);

        String s1 = "29";
        Integer data = Integer.valueOf(s1,36);
        System.out.println(data);

        String s2 = "add";
        int x = Integer.parseInt(s2,16);
        System.out.println(x);
    }
}
```

```
public class AutoBoxing4
{
    public static void main(String[] args)
    {
        Integer i1 = new Integer(100);
        Integer i2 = new Integer(100);
        System.out.println(i1==i2);
    }
}
```

```

Integer a1 = Integer.valueOf(15);
Integer a2 = Integer.valueOf(15);
System.out.println(a1==a2);
}
}

```

== operator always compares the memory address so it is returning false (two object are created using new keyword), on the other hand valueOf() return the same object so it will provide true.

```

//Converting integer value to String
public class AutoBoxing5
{
    public static void main(String[] args)
    {
        int x = 12;
        String str = Integer.toString(x);
        System.out.println(str+2);
    }
}

```

We can also convert any int value to String, for that Integer class has provided a static method toString()

```

public static String toString(int x);

```

Unboxing :

Unboxing :

Converting wrapper object to corresponding primitive type is called Unboxing.

Wrapper Object	Primitive type
-------------------	-------------------

Byte	- byte
------	--------

Short	- short
-------	---------

Integer	- int
---------	-------

Long	- long
------	--------

Float	- float
-------	---------

Double	- double
--------	----------

Chracter	- char
----------	--------

Boolean	- boolean
---------	-----------

We have total 8 Wrapper classes.

Among all these 8, 6 Wrapper classes are the sub class of Number class so all the following six wrapper classes (Which are sub class of Number class) are providing the following common methods.

- 1) public byte byteValue()
 - 2) public short shortValue()
 - 3) public int intValue()
 - 4) public long longValue()
 - 5) public float floatValue()
 - 6) public double doubleValue()
-

//Converting Wrapper object into primitive

```
public class AutoUnboxing1
{
    public static void main(String args[])
    {
        Integer obj = 15; //Upto 1.4
        int x = obj.intValue();
        System.out.println(x);
    }
}
```

```
public class AutoUnboxing2
{
    public static void main(String[] args)
    {
        Integer x = 25;
        int y = x; //JDK 1.5 onwards
        System.out.println(y);
    }
}
```

```
public class AutoUnboxing3
{
    public static void main(String[] args)
    {
        Integer i = 15;
        System.out.println(i.byteValue());
        System.out.println(i.shortValue());
        System.out.println(i.intValue());
        System.out.println(i.longValue());
        System.out.println(i.floatValue());
        System.out.println(i.doubleValue());
    }
}
```

```
public class AutoUnboxing4
{
    public static void main(String[] args)
    {
        Character c1 = 'A';
        char ch = c1.charValue();
        System.out.println(ch);
    }
}
```



```
}  
}
```

```
-----  
public class AutoUnboxing5  
{  
    public static void main(String[] args)  
    {  
        Boolean b1 = true;  
        boolean b = b1.booleanValue();  
        System.out.println(b);  
    }  
}
```

```
-----  
package com.ravi.value_of;
```

```
public class Unboxing {  
  
    public static void main(String[] args)  
    {  
        Integer i1 = 127;  
        Integer i2 = 127;  
        System.out.println(i1==i2); //true  
  
        Integer i3 = 128;  
        Integer i4 = 128;  
        System.out.println(i3==i4); //false  
  
        System.out.println(i1.equals(i2));  
        System.out.println(i3.equals(i4));  
  
    }  
}
```

Note :- While comparing the Integer object it is always better to use equals(Object obj)

```
-----  
The type conversion is possible with primitive type but the same is not applicable for Wrapper class.
```

```
Long l = 12; //Invalid  
Float f = 89.89; //Invalid  
Double d = 78D; //Valid
```

```
package com.ravi.value_of;
```

```
public class Unboxing {  
  
    public static void main(String[] args)  
    {  
        Byte b = 12;  
        Short s = 15;  
        Integer i = 90;  
        Long l = 45L;  
        Float f = 89.67F;  
        Double d = 9.90d;
```

```
}
```

```
}
```

10-05-2024

Ambiguity issues while overloading a method :

Whenever we work with method overloading and if we have ambiguity while calling the method then compiler has provided 3 rules to work with Method overloading.

Rule 1:

Most specific type (Specific type will get more priority)

double > float
float > long
long > int
int > short
int > char
short > byte

Rule 2 :

While overloading a method, if we have an ambiguity issue then compiler provides the priority in the following order :

WAV [Widening -> Autoboxing -> Var args]

Rule 3 :

Nearest Type Rule

While Overloading a method if we are getting sub class reference in comparison to super class reference then sub class will be treated as Nearest type.

package com.ravi.method_over;

```
class Test
{
    public void accept(double ...d)
    {
        System.out.println("double Type");
    }
    public void accept(float ...f)
    {
        System.out.println("float Type");
    }
}
```

```
public class MethodOverloadingAmbiguity
{
    public static void main(String[] args)
```

```

{
    Test t1 = new Test();
    t1.accept();

}

}

```

Here float type will be executed because float is more specific than double.

```

-----
package com.ravi.method_over;

class Test
{
    public void accept(long ...d)
    {
        System.out.println("Long Type");
    }
    public void accept(float ...f)
    {
        System.out.println("float Type");
    }
}

public class MethodOverloadingAmbiguity
{
    public static void main(String[] args)
    {
        Test t1 = new Test();
        t1.accept();

    }

}

```

Here long is executed because long is more specific type

```

-----
package com.ravi.method_over;

class Test
{
    public void accept(int ...d)
    {
        System.out.println("Int Type");
    }
    public void accept(char ...f)
    {
        System.out.println("Char Type");
    }
}

public class MethodOverloadingAmbiguity
{
    public static void main(String[] args)
    {

```

```
Test t1 = new Test();
t1.accept();
```

```
}
```

```
}
```

char is executed because char is more specific than int

```
package com.ravi.method_over;
```

```
class Test
```

```
{
    public void accept(char ...d)
    {
        System.out.println("char Type");
    }
    public void accept(short ...f)
    {
        System.out.println("short Type");
    }
}
```

```
public class MethodOverloadingAmbiguity
```

```
{
    public static void main(String[] args)
    {
        Test t1 = new Test();
        t1.accept();
    }
}
```

Note :- error because in between char and short we don't have numeric relation, numeric relation is available with char and int

```
package com.ravi.method_over;
```

```
class Test
```

```
{
    public void accept(Number n)
    {
        System.out.println("Number Type");
    }
    public void accept(Integer i)
    {
        System.out.println("Integer Type");
    }
}
```

```
public class MethodOverloadingAmbiguity
```

```
{
    public static void main(String[] args)
    {
```

```
Test t1 = new Test();
t1.accept(9);
```

```
}
```

```
}
```

Here Integer will be executed because Integer is nearest type.

```
package com.ravi.method_over;
```

```
class Test
```

```
{
    public void accept(Number n)
    {
        System.out.println("Number Type");
    }
    public void accept(Object i)
    {
        System.out.println("Object Type");
    }
}
```

```
public class MethodOverloadingAmbiguity
```

```
{
    public static void main(String[] args)
    {
        Test t1 = new Test();
        t1.accept(9);

    }
}
```

Number will be executed

```
package com.ravi.method_over;
```

```
class Test
```

```
{
    public void accept(String n)
    {
        System.out.println("String Type");
    }
    public void accept(Object i)
    {
        System.out.println("Object Type");
    }
}
```

```
public class MethodOverloadingAmbiguity
```

```
{
    public static void main(String[] args)
    {
        Test t1 = new Test();
    }
}
```

```
t1.accept("NIT");
```

```
}
```

```
}
```

Here String will be executed

```
-----  
package com.ravi.method_over;
```

```
class Test
```

```
{
```

```
    public void accept(String n)
```

```
    {
```

```
        System.out.println("String Type");
```

```
    }
```

```
    public void accept(Object i)
```

```
    {
```

```
        System.out.println("Object Type");
```

```
    }
```

```
}
```

```
public class MethodOverloadingAmbiguity
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Test t1 = new Test();
```

```
        t1.accept(null);
```

```
    }
```

```
}
```

Here String is Executed

```
-----  
package com.ravi.method_over;
```

```
class Test
```

```
{
```

```
    public void accept(Integer l)
```

```
    {
```

```
        System.out.println("Integer Type");
```

```
    }
```

```
    public void accept(Double b)
```

```
    {
```

```
        System.out.println("Long Type");
```

```
    }
```

```
}
```

```
public class MethodOverloadingAmbiguity
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Test t1 = new Test();
```

```
        t1.accept(null);
```

```
}
```

```
}
```

Here we will get error, because unable to find the nearest type for classes Integer and Double are the sub class of Number

```
-----  
package com.ravi.method_over;
```

```
class Alpha{}
```

```
class Beta extends Alpha{}
```

```
class Test
```

```
{
```

```
    public void accept(Alpha a)
```

```
    {
```

```
        System.out.println("Alpha Type");
```

```
    }
```

```
    public void accept(Beta b)
```

```
    {
```

```
        System.out.println("Beta Type");
```

```
    }
```

```
}
```

```
public class MethodOverloadingAmbiguity
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Test t1 = new Test();
```

```
        t1.accept(null);
```

```
    }
```

```
}
```

```
-----  
package com.ravi.method_over;
```

```
class Test
```

```
{
```

```
    public void accept(long l)
```

```
    {
```

```
        System.out.println("long Type");
```

```
    }
```

```
    public void accept(int b)
```

```
    {
```

```
        System.out.println("int Type");
```

```
    }
```

```
}
```

```
public class MethodOverloadingAmbiguity
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Test t1 = new Test();
```

```
        t1.accept(9);
```

```
}
```

```
}
```

Here int type will be executed

```
-----  
package com.ravi.method_over;
```

```
class Test
```

```
{
```

```
    public void accept(long l)
```

```
    {
```

```
        System.out.println("long Type");
```

```
    }
```

```
    public void accept(double b)
```

```
    {
```

```
        System.out.println("double Type");
```

```
    }
```

```
}
```

```
public class MethodOverloadingAmbiguity
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Test t1 = new Test();
```

```
        t1.accept(9);
```

```
    }
```

```
}
```

Here long is excuted.

```
-----  
package com.ravi.method_over;
```

```
class Test
```

```
{
```

```
    public void accept(byte l)
```

```
    {
```

```
        System.out.println("byte Type");
```

```
    }
```

```
    public void accept(short b)
```

```
    {
```

```
        System.out.println("short Type");
```

```
    }
```

```
}
```

```
public class MethodOverloadingAmbiguity
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Test t1 = new Test();
```

```
        t1.accept(9);
```



```
}
```

```
}
```

Here Compilation error because 9 is of int type we can't assign to byte and short.

```
-----  
package com.ravi.method_over;
```

```
class Test
```

```
{
```

```
    public void accept(byte l)
```

```
    {
```

```
        System.out.println("byte Type");
```

```
    }
```

```
    public void accept(short b)
```

```
    {
```

```
        System.out.println("short Type");
```

```
    }
```

```
}
```

```
public class MethodOverloadingAmbiguity
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Test t1 = new Test();
```

```
        t1.accept((byte)9);
```

```
        t1.accept((short)15);
```

```
    }
```

```
}
```

```
-----  
package com.ravi.method_over;
```

```
class Test
```

```
{
```

```
    public void accept(long l)
```

```
    {
```

```
        System.out.println("long Type");
```

```
    }
```

```
    public void accept(Integer b)
```

```
    {
```

```
        System.out.println("Integer Type");
```

```
    }
```

```
}
```

```
public class MethodOverloadingAmbiguity
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Test t1 = new Test();
```

```
        t1.accept(29);
```

```
    }
```

}

Here Widening has more priority

```
package com.ravi.method_over;
```

```
class Test
{
    public void accept(int ...l)
    {
        System.out.println("Var args Type");
    }
    public void accept(Integer b)
    {
        System.out.println("Autoboxing Type");
    }
}

public class MethodOverloadingAmbiguity
{
    public static void main(String[] args)
    {
        Test t1 = new Test();
        t1.accept(29);

    }
}
```

Here Autoboxing has more priority

```
package com.ravi.method_over;
```

```
class Test
{
    public void accept(int ...l)
    {
        System.out.println("Var args Type");
    }
    public void accept(long b)
    {
        System.out.println("Long Type");
    }
}

public class MethodOverloadingAmbiguity
{
    public static void main(String[] args)
    {
        Test t1 = new Test();
        t1.accept(29);

    }
}
```

```
}
```

Here Widening has more priority

```
-----  
package com.ravi.method_over;
```

```
class Test
```

```
{  
    public void accept(int x, long y)  
    {  
        System.out.println("int-long");  
    }  
    public void accept(long x, int y)  
    {  
        System.out.println("long-int");  
    }  
}
```

```
public class MethodOverloadingAmbiguity
```

```
{  
    public static void main(String[] args)  
    {  
        Test t1 = new Test();  
        //t1.accept(5, 10); //error  
        t1.accept(5L, 9);  
  
    }  
}
```

```
-----  
* Method Overriding :
```

Writing two or more methods in the super and sub class in such a way that method signature(method name along with method parameter) of both the methods must be same in the super and sub classes.

While working with method overriding generally we can't change the return type of the method but from JDK 1.5 onwards we can change the return type of the method in only one case that is known as Co-Variant.

Without inheritance method overriding is not possible that means if there is no inheritance there is no method overriding.

What is the advantage of Method Overriding ?

```
-----  
The advantage of Method Overriding is, each class is specifying its own specific behavior.
```

14-05-2024

```
-----  
Upcasting and Downcasting :
```

```
-----  
Upcasting :-
```

```
-----  
It is possible to assign sub class object to super class reference variable using dynamic polymorphism. It is known as Upcasting.
```

Example:- `Animal a = new Lion(); //valid [upcasting]`

Downcasting :

By default we can't assign super class object to sub class reference variable.

```
Lion l = new Animal(); //Invalid
```

Even if we type cast `Animal` to `Lion` type then compiler will allow but at runtime JVM will not convert `Animal` object (Generic type) into `Lion` object (Specific type) and it will throw an exception `java.lang.ClassCastException`

```
Lion l = (Lion) new Animal(); //At runtime we will get  
ClassCastException
```

Rules for calling the method in upcasting :

[It is available in paint diagram 14-May]

Program on Method Overriding :

```
package com.ravi.overriding;

class Animal
{
    public void roam()
    {
        System.out.println("Generic Animal is roaming here and there");
    }
}
class Lion extends Animal
{
    public void roam()
    {
        System.out.println("Lion is roaming at night");
    }
}

public class MethodOverridingDemo1
{
    public static void main(String[] args)
    {
        Animal a = new Lion();
        a.roam();
    }
}
```

Program on Dynamic method Dispatch :

```
package com.ravi.overriding;
```

```
class Shape
{
    public void draw()
    {
        System.out.println("Generic Draw!!!");
    }
}
```

```
class Square extends Shape
{
    public void draw()
    {
        System.out.println("Square Draw!!!");
    }
}
```

```
class Rectangle extends Shape
{
    public void draw()
    {
        System.out.println("Rectangle Draw!!!");
    }
}
```

```
public class MethodOevrridingdemo2 {

    public static void main(String[] args)
    {
        Shape s = null;
        s = new Square(); s.draw(); //Dynamic Method Dispatch
        s = new Rectangle(); s.draw(); //Dynamic Method Dispatch
    }
}
```

@Override Annotation:

In Java we have a concept called Annotation, introduced from JDK 1.5 onwards. All the annotations must be start with @ symbol.

@Override annotation is optional but it is always a good practice to write @Override annotation before the Overridden method so compiler as well as user will get the confirmation that the method is overridden method and it is available in the super class.

If we use @Override annotation before the name of the overridden method in the sub class and if the method is not available in the super class then it will generate a compilation error so it is different from comment because comment will not generate any kind of compilation error if method is not an overridden method, so this is how it is different from comment.

```
package com.ravi.overriding;
```

```
class RBI
{
    public void loan()
    {
        System.out.println("Bank Should provide loan");
    }
}
```

```

}
}
class HDFC extends RBI
{
    @Override
    public void loan()
    {
        System.out.println("Providing loan @ 12.8% ROI");
    }
}

```

```

class ICICI extends RBI
{
    @Override
    public void loan()
    {
        System.out.println("Providing loan @ 11.6% ROI");
    }
}

```

```

public class MethodOverridingDemo3
{
    public static void main(String[] args)
    {
        RBI r;

        r = new HDFC(); r.loan();

        r = new ICICI(); r.loan();
    }
}

```

Variable Shadow in Method Overriding :

If super class variable name and sub class variable name both are same then it is called Variable Shadow.

In java variables are not overridden so, to get the output of the variable we depend upon the class reference variable.

```

package com.ravi.overriding;

```

```

class Bird
{
    String name= "Generic Bird";

    public String fly()
    {
        return "Generic Bird is flying";
    }
}
class Koyal extends Bird
{
    String name= "Koyal Bird"; //Variable Shadow

```

```

    @Override
    public String fly()
    {
        return "Koyal Bird is flying";
    }
}

public class MethodOverridingDemo4
{
    public static void main(String[] args)
    {
        Bird b = new Koyal();
        System.out.println(b.name + " : "+b.fly());
    }
}

```

Output :
Generic Bird : Koyal Bird is flying (variables are not overridden in java)

15-05-2024

Program that describes we cannot override private method:

private methods are available within the same class only, it is not visible to the child class so we cannot override.

```

package com.ravi.method_visibility;

class Super
{
    private void show()
    {
        System.out.println("Private Method of Super class ");
    }
}

class Sub extends Super
{
    @Override //error
    private void show() //Own method of sub class
    {
        System.out.println("Private Method of Sub class ");
    }
}

```

```

public class PrivateMethodOverriidng
{
    public static void main(String[] args)
    {

    }
}

```

```
}
```

Note :- private method of super class is not available or not inherited in the sub class so if the class declare the method with same signature then it is not overridden method, actually it is re-declared in the sub class.

Role of access modifier while overriding a method :

While overriding the method from super class, the access modifier of sub class method must be greater or equal in comparison to access modifier of super class method otherwise we will get compilation error.

In terms of accessibility, public is greater than protected, protected is greater than default (public > protected > default)
[default < protected < public]

So the conclusion is we can't reduce the visibility of the method while overriding a method.

Note :- private method is not available (visible) in sub class so it is not the part of method overriding.

package com.ravi.method_visibility;

```
class Super
{
    protected void show()
    {
        System.out.println("Super class show method");
    }
}
class Sub extends Super
{
    @Override
    void show() //error (Reducing the Visibility)
    {
        System.out.println("Sub class show method");
    }
}
```

```
public class VisibilityMode
{
    public static void main(String[] args)
    {

    }

}
```

Co-variant

In general we can't change the return type of method while overriding a method. if we try to change it will generate compilation error as shown in the program below.

```
class Super
```



```

{
    public void show()
    {
        System.out.println("Super class show method");
    }
}
class Sub extends Super
{
    @Override
    public int show()
    {
        System.out.println("Sub class show method");
        return 0;
    }
}
public class IncompatibleType
{
    public static void main(String[] args)
    {

    }

}
}

```

The above code will not compile because return type int is not compatible with void.

But from JDK 1.5 onwards we can change the return type of the method in only one case that the return type of both the METHODS(SUPER AND SUB CLASS METHODS) MUST BE IN INHERITANCE RELATIONSHIP (IS-A relationship so it is compatible) called Co-Variant as shown in the program below.

Note :- Co-variant will not work with primitive data type, it will work only with classes

```

class Alpha
{
}
class Beta extends Alpha
{
}
class Gamma extends Alpha
{
}

class Super
{
    public Alpha show()
    {
        System.out.println("Super class show method");
        return new Alpha();
    }
}
class Sub extends Super
{
    @Override
    public Gamma show()
    {

```

```

        System.out.println("Sub class show method");
        return new Gamma();
    }
}
public class Test
{
    public static void main(String[] args)
    {
        Super s = new Sub();
        s.show();
    }
}

```

Note :- Here Gamma and Alpha class both are in IS-A relationship so, this is called Co-variant(in the same direction).

```

-----
package com.ravi.method_visibility;

class Parent
{
    public Object show()
    {
        System.out.println("show method in Parent class");
        return null;
    }
}

class Child extends Parent
{
    public Boolean show()
    {
        System.out.println("show method in Child class");
        return null;
    }
}

public class CoVariant
{
    public static void main(String[] args)
    {
        Parent p = new Child();
        p.show();
    }
}

```

Note :- The above code will compile because in between Boolean and Object class we have IS-A relation

While working with co-variant, whenever we call a method then sub class method will be executed but the return type of method must be super class type to avoid ClassCastException

```

package com.ravi.method_visibility;

```

```

class Animal
{
    public Object eat()
    {
        System.out.println("Generic Animal is eating!!!");
        return new Object();
    }
}
class Dog extends Animal
{
    @Override
    public String eat()
    {
        System.out.println("Dog Animal is eating!!!");
        return new String();
    }
}

```

```

public class CoVariantDemo
{
    public static void main(String[] args)
    {
        Animal a = new Dog();
        Object obj = a.eat();
        System.out.println(obj);
    }
}

```

16-05-2024

- * Can we override static method ?
OR
- * Can we override main method?
OR
- * What is Method Hiding in java?

Point 1)

We cannot override static method but static methods can be inherited as shown in the program below that means static method of super class is by default available in sub class.

```

class Super
{
    public static void m1()
    {
        System.out.println("Static method of super class");
    }
}
class Sub extends Super
{

```

```

}

public class Test
{
    public static void main(String[] args)
    {
        //Sub.m1();

        Sub s1 = new Sub();
        s1.m1();
    }
}

```

Point 2)

We cannot override any static method with non-static method otherwise we will get compilation error i.e. Overridden method is static.

```

class Alpha
{
    public static void m1()
    {
        System.out.println("m1 static method");
    }
}
class Beta extends Alpha
{
    public void m1()
    {
        System.out.println("m1 non-static method");
    }
}

public class StaticMethodOvderridingWithNonStaticMethod
{
    public static void main(String[] args)
    {

    }
}

```

Point 3)

We cannot override any non-static method with static method otherwise we will get error Overriding method is static.

```

class Alpha
{
    public void m1()
    {
        System.out.println("m1 non static method");
    }
}
class Beta extends Alpha

```

```

{
    public static void m1()
    {
        System.out.println("m1 static method");
    }
}
public class OverridingNonStaticWithStatic
{
    public static void main(String[] args)
    {

    }
}

```

So, the conclusion is we cannot override static with non static method as well as non-static with static method because static method belongs to class and non-static method belongs to object.

Point 4 :

WAP that describes, if we write static method in the sub class with same as super class and if change the return type (return type must be compatible) then compiler will generate error, it is method hiding

```

class Super
{
    public static void m1()
    {
        System.out.println("Static method of super class");
    }
}
class Sub extends Super
{

    public static int m1() //method Hiding
    {
        System.out.println("Static method of super class");
        return 0;
    }
}

```

```

public class Test
{
    public static void main(String[] args)
    {

    }

}

```

Point 5 :

If we write static methods in the super and sub class with same signature and same return type then it

looks like static method is overridden but actually it is METHOD HIDING. The sub class static method is hidden from super class static method as shown in the program.

```
package com.ravi.method_hiding;

class Super
{
    public static void m1()
    {
        System.out.println("static method of super class!!!");
    }
}

class Sub extends Super
{

    public static void m1() //Method Hiding
    {
        System.out.println("static method of Sub class!!!");
    }
}

public class MethodHiding {

    public static void main(String[] args)
    {
        Super s = new Sub();
        s.m1();
    }
}
```

Note :- A static method of super class is by default available to the sub class but sub class can't override, It can only hide the method.

We can't apply @Override annotation on static methods.

private method of a class is not available to sub class but static method of super class is available to sub class so we can hide the static method.

NOte :- main method can be overloaded but main method cannot be overridden (Method Hiding).

final keyword in java :

It is used to provide some kind of restriction in our program.
We can use final keyword in ways 3 ways in java.

- 1) To declare a class as a final. (Inheritance is not possible)
- 2) To declare a method as a final (Overriding is not possible)
- 3) To declare a variable or field as a final (Re-assignment is not possible)

To declare a class as a final :

Whenever we declare a class as a final class then we can't extend or inherit that class otherwise we will get a compilation error.

We should declare a class as a final if the composition of the class (logic of the class) is very important and we don't want to share the feature of the class to some other developer to modify the original behavior of the existing class, In that situation we should declare a class as a final.

Declaring a class as a final does not mean that the variables and methods declared inside the class will also become as a final, only the class behavior is final that means we can modify the variables value as well as we can create the object for the final classes.

Note :- In java String and All wrapper classes are declared as final class.

```
-----  
final class A  
{  
    private int x = 100;  
    public void setData()  
    {  
        x = 120;  
        System.out.println(x);  
    }  
}  
class B extends A  
{  
}  
public class FinalClassEx  
{  
    public static void main(String[] args)  
    {  
        B b1 = new B();  
        b1.setData();  
    }  
}
```

```
-----  
final class Test  
{  
    private int data = 100;  
  
    public void setData(int data)  
    {  
        this.data = data;  
        System.out.println("Data value is :"+data);  
    }  
}  
public class FinalClassEx1  
{  
    public static void main(String[] args)  
    {  
        Test t1 = new Test();  
        t1.setData(200);  
    }  
}
```

Note :- for final class we can create object as well as we can modify the data.

Sealed class in Java :

It is a new feature introduced from java 15.

It is an improvement over final keyword.

By using sealed keyword we can declare classes and interfaces as sealed.

It is one kind of restriction that describes which classes and interfaces can extend or implement from Sealed class Or interface

It is similar to final keyword with less restriction because here we can permit the classes to extend from the original Sealed class.

The class which is inheriting from the sealed class must be final, sealed or non-sealed.

The sealed class must have at least one sub class.

We can also create object for Sealed class.

It provides the following modifier :

- 1) sealed : Can be extended only through permitted class.
- 2) non-sealed : Can be extended by any sub class, if a user wants to give permission to its sub classes
- 3) permits : We can provide permission to the sub classes, which are inheriting through Sealed class.
- 4) final : we can declare permitted sub class as final so, it cannot be extended further.

```
package com.ravi.sealed_demo;
```

```
sealed class Bird permits Parrot,Sparrow
{
    public void fly()
    {
        System.out.println("Generic Bird is flying");
    }
}
```

```
non-sealed class Parrot extends Bird
{
    public void fly()
    {
        System.out.println("Parrot Bird is flying");
    }
}
```

```
final class Sparrow extends Bird
{
    public void fly()
    {
        System.out.println("Sparrow Bird is flying");
    }
}
```

```
public class SealdDemo {
```



```

public static void main(String[] args)
{
    Bird b = null;
    b = new Parrot(); b.fly();
    b = new Sparrow(); b.fly();

}

}

```

```

package com.ravi.sealed_demo;

```

```

sealed class OnlineClass permits Mobile,Laptop

```

```

{
    public void attendOnlineClass()
    {
        System.out.println("Students are attending online class");
    }
}

```

```

}
non-sealed class Mobile extends OnlineClass

```

```

{
    public void attendOnlineClass()
    {
        System.out.println("Students are attending online class through mobile phone");
    }
}

```

```

non-sealed class Laptop extends OnlineClass

```

```

{
    public void attendOnlineClass()
    {
        System.out.println("Students are attending online class through Laptop");
    }
}

```

```

public class SelaedExample {

```

```

    public static void main(String[] args)
    {
        OnlineClass cls = null;
        cls = new Mobile(); cls.attendOnlineClass();
        cls = new Laptop(); cls.attendOnlineClass();

    }
}

```

2) To declare a method as a final (Overriding is not possible)

Whenever we declare a method as a final then we can't override that method in the sub class otherwise there will be a compilation error.

We should declare a method as a final if the body of the method i.e the implementation of the method is very important and we don't want to override or change the super class method body by sub class

method body then we should declare the super class method as final method.

```
class A
{
    protected int a = 10;
    protected int b = 20;

    public final void calculate()
    {
        int sum = a+b;
        System.out.println("Sum is :"+sum);
    }
}
class B extends A
{
    @Override
    public void calculate()
    {
        int mul = a*b;
        System.out.println("Mul is :"+mul);
    }
}
public class FinalMethodEx
{
    public static void main(String [] args)
    {
        A a1 = new B();
        a1.calculate();
    }
}
```

```
-----
class Alpha
{
    private final void accept()
    {
        System.out.println("Alpha class accept method");
    }
}
class Beta extends Alpha
{
    public void accept()
    {
        System.out.println("Alpha class accept method");
    }
}
public class FinalMethodEx1
{
    public static void main(String [] args)
    {
        new Beta().accept();
    }
}
```

If super class method is declared with private and final modifier then in the sub class method is not overridden, actually it is re-defined so, code will compile and execute.

3) To declare a variable OR Field as a final.

In older languages like C and C++ we use "const" keyword to declare a constant variable but in java const is a reserved word for future use so instead of const we should use "final" keyword.

If we declare a variable as a final then we can't perform re-assignment (i.e nothing but re-initialization) of that variable.

In java It is always a better practise to declare a final variable by uppercase letter according to the naming convention.

Some example of predefined final variables

Byte.MIN_VALUE -> MIN_VALUE is a static and final variable

Byte.MAX_VALUE -> MAX_VALUE is a static and final variable

Example:- final int DATA = 10; (Now we can not perform re-assignment)

class A
{
 final int A = 10;
 public void setData()
 {
 A = 10; //re-assignment is not possible
 System.out.println("A value is :"+A);
 }
}
class FinalVarEx
{
 public static void main(String[] args)
 {
 A a1 = new A();
 a1.setData();
 }
}

class FinalVarEx1
{
 public static void main(String[] args)
 {
 final int A = 127;
 byte b = A;
 System.out.println(b);
 }
}

What is blank final variable in java ?

A final variable must be initialized by the user.

If a final variable is not initialized at the time of declaration then it is called Blank final variable.

Example : final int A; //Blank final variable

A blank final variable can't be initialized by default constructor.

A blank final variable must be initialized by user only, till the completion of constructor body otherwise we will get compilation error.

A blank final variable we can't initialize through methods.

A blank final variable must be initialized by the user in the following 2 places :

- a) Inside instance block
- b) Inside constructor body

A blank final variable must be initialized in all the constructors available in the class.

A blank final variable has also default value.

```
-----  
class Sample  
{  
    final int A; //Blank final variable  
  
}  
class BlankFinalDemo1  
{  
    public static void main(String[] args)  
    {  
        Sample s = new Sample();  
        System.out.println(s.A);  
    }  
}
```

Note :- A blank final variable can't be initialized by default constructor.

```
-----  
class Sample  
{  
    final int A ;  
  
    {  
        A = 100;  
    }  
}  
public class BlankFinalDemo2 {  
  
    public static void main(String[] args)  
    {  
        System.out.println(new Sample().A);  
    }  
  
}
```

Note :- A blank final variable we can initialize inside instance block

```

class Demo
{
    final int A;

    Demo()
    {
        A = 400;
    }
}

public class BlankFinalDemo3 {

    public static void main(String[] args)
    {
        System.out.println(new Demo().A);
    }

}

```

Note :- A blank final variable we can initialize inside constructor

```

class Foo
{
    final int A;

    {
        m1();
        A = 100;
    }

    public void m1()
    {
        System.out.println("Default value :"+A);
    }

}

public class BlankFinalDemo4 {

    public static void main(String[] args)
    {
        System.out.println("User Value :"+new Foo().A);

    }

}

```

Note : A blank final variable has also default values.

```

class Employee
{
    private final int empNo;

    public Employee()
    {

```

```

    empNo = 111;
}

public Employee(int empld)
{
    this.empNo = empld;
}

public int getEmpNo()
{
    return this.empNo;
}

}

public class BlankFinalDemo5 {

    public static void main(String[] args)
    {
        Employee raj = new Employee();

        Employee scott = new Employee(222);

        System.out.println(scott.getEmpNo());
        System.out.println(raj.getEmpNo());
    }

}

```

A blank final variable must be initialized in all the constructor.

Different kinds of final variable :

```

class Student
{
    final int x = 100; //non static field

    static final int y = 200; //static field

    public void accept(final int a)
    {
        final int b = 400;
        System.out.println("Non static :"+this.x);
        System.out.println("Static :"+Student.y);
        System.out.println("Parameter :"+a);
        System.out.println("Local :"+b);
    }
}

public class DifferentTypesOfFinalVariable
{
    public static void main(String[] args)
    {

```

```

final Student s1 = new Student();
s1.accept(300);

}

}

```

Note :- On local and parameter variable we can apply only one keyword final.

How to create our own Immutable(Un-changed) class in java ?

In order to create our own immutable class we should follow the following steps :

- 1) Declare the class as a final
- 2) Declare all the instance variable with private and final modifier.
- 3) The class should not contain any setter
- 4) We should declare reference variable as a final so we can't assign another object.

//Program to create an immutable(unchanged) class

```

final class Citizen
{
    private final String aadharNumber;    //Blank final variable

    public Citizen(final String aadharNumber)
    {
        if(aadharNumber.length() == 12)
        {
            this.aadharNumber = aadharNumber;
        }
        else
        {
            this.aadharNumber = aadharNumber;
            System.err.println("Invalid Aadhar Number");
        }
    }

    public String getAadharNumber()
    {
        return aadharNumber;
    }
}

public class ImmutableClass
{
    public static void main(String ar[])
    {
        final Citizen ravi = new Citizen("987667897842");
        System.out.println("Ravi Aadhar Number is :"+ravi.getAadharNumber());

    }
}

```

18-05-2024

What is Method Chaining in java ?

By using this concept we can call n number of method in a single statement.

The next method call always depends upon current method return type.

The final return type of the method will depend upon last method call.

```
package com.ravi.method_chain;

public class MethodChaning1 {

    public static void main(String[] args)
    {
        String str = "india";
        char ch = str.concat(" is great").toUpperCase().charAt(7);
        System.out.println(ch);

        String s = "hyderabad";
        int len = s.toUpperCase().concat(" ameerpet").length();
        System.out.println(len);

    }

}
```

Working with Object class and its methods :

There is a predefined class called Object available in java.lang package, this Object class is by default the super class of all the classes we have in java.

```
class Test
{

}
```

Note :- Object is the super class for this Test class. by default this Object class is super class so explicitly we need not to mention.

Since, Object is the super class of all the classes in java that means we can override the method of Object class (Except final methods) as well as we can use the methods of Object class anywhere in java because every class is sub class of Object class.

The Object class provides some common behavior to each sub class Object like we can compare two objects , we can create clone (duplicate) objects, we can print object properties(instance variable), providing a unique number to each and every object(hashCode()) and so on.

1) public native final java.lang.Class getClass() :

It is a predefined method of Object class.

This method returns the runtime class of the object, the return type of this method is java.lang.Class.

This method will provide class keyword + Fully Qualified Name (package name + class name)

This getClass() method return type is java.lang.Class so further we can apply any other method of java.lang.Class class method.

We can't override this method because it is a final method.

2 files : (Available in different package)

Test.java

```
package com.nit;
```

```
public class Test  
{  
  
}
```

GetClassDemo1.java

```
package com.ravi.object;  
  
import com.nit.Test;  
  
public class GetClassDemo1 {  
  
    public static void main(String[] args)  
    {  
        Test t1 = new Test();  
        System.out.println(t1.getClass());  
  
    }  
  
}
```

getClass() method return type is java.lang.Class so we can apply the method of java.lang.Class.

java.lang.Class class has provided a predefined method called getName(), return type is String

```
public String getName();
```

```
String name = getClass().getName(); //Method Chaining
```

```
package com.ravi.object;
```

```
class Foo  
{  
  
}
```

```

public class GetClassDemo2
{
    public static void main(String[] args)
    {
        Foo f1 = new Foo();
        String name = f1.getClass().getName();
        System.out.println(name); //Fully Qualified name

    }

}

```

Here we are using getClass().getName() so it will provide fully qualified name.

```

public native int hashCode() :

```

It is predefined method of Object class.

It returns the hashCode of the object.

If two objects are same based on the equals(Object obj) method then the hashCode of both the object must be same.

hashCode is not used for comparing of two objects.

```

package com.ravi.object;

```

```

class Demo
{

}

```

```

public class HashCodeDemo1 {

    public static void main(String[] args)
    {
        Demo d1 = new Demo();
        Demo d2 = new Demo();
        Demo d3 = d2;

        System.out.println(d1.hashCode());
        System.out.println(d2.hashCode());
        System.out.println(d3.hashCode());

    }

}

```

```

package com.ravi.object;

```

```

public class HashCodeDemo2 {

    public static void main(String[] args)
    {

```

```
String s1 = "A";
Integer i1 = 65;

System.out.println(s1.hashCode());
System.out.println(i1.hashCode());
}
```

```
}
```

3) public String toString() :

It is a predefined method of Object class.

it returns a string representation of the object. In general, the toString method returns a string that "textually represents" this object. The result should be a concise but informative representation that is easy for a person to read

toString() method of Object class contains following logic.

```
public String toString()
{
    return getClass().getName()+" @ "+Integer.toHexString(hashCode());
}
```

Please note internally the toString() method is calling the hashCode() and getClass() method of Object class.

In java whenever we print any Object reference by using System.out.println() then internally it will invoke the toString() method of Object class as shown in the following program.

package com.ravi.object;

```
class Customer
{
```

```
}
```

```
class Employee
{
```

```
}
```

```
public class ToStringDemo1 {
```

```
    public static void main(String[] args)
    {
        Employee e1 = new Employee();
        System.out.println(e1);
```

```
        Customer c1 = new Customer();
        System.out.println(c1);
```

```
    }
```

```
}
```

We can override toString() method in our userdefined class

```
package com.ravi.object;
```

```
class Student
```

```
{
    @Override
    public String toString()
    {
        return "NIT Student";
    }
}
```

```
public class ToStringDemo2 {
```

```
    public static void main(String[] args)
    {
        Student s1 = new Student();
        System.out.println(s1);
    }
}
```

```
-----
public boolean equals(Object obj) :
-----
```

It is predefined method of Object class.

It is mainly used to compare two objects based on the memory address just like == operator because internally, it uses == operator only.

The following program explains how to use equals(Object obj) method for Employee comparison.

```
package com.ravi.equals;
```

```
class Employee
```

```
{
    private int employeeId;
    private String employeeName;
```

```
    public Employee(int employeeId, String employeeName)
    {
        super();
        this.employeeId = employeeId;
        this.employeeName = employeeName;
    }
}
```

```
public class EqualsMethodDemo1
```

```
{
    public static void main(String[] args)
    {
        Employee e1 = new Employee(111, "Scott");
        Employee e2 = new Employee(111, "Scott");
```

```
        System.out.println(e1==e2); //false
        System.out.println(e1.equals(e2)); //false
    }
}
```

```
}
```

```
}
```

Here we will get false and false, because equals(Object obj) method of Object class internally uses == operator which compares memory reference.

We want to compare two objects based on the content of the object but not based on the memory address then we should override equals(Object obj) method from Object class.

```
package com.ravi.equals;
```

```
class Employee
```

```
{  
    private int employeeId;  
    private String employeeName;
```

```
    public Employee(int employeeId, String employeeName)  
    {  
        super();  
        this.employeeId = employeeId;  
        this.employeeName = employeeName;  
    }  
}
```

```
//Overriding the equals(Object obj) method for content comparison
```

```
@Override
```

```
public boolean equals(Object obj) //obj = e2 (2nd Object)
```

```
{  
    //Retrieving 1st object data  
    int eid1 = this.employeeId;  
    String ename1 = this.employeeName;
```

```
    //Retrieving 2nd object data  
    Employee e2 = (Employee) obj;  
    int eid2 = e2.employeeId;  
    String ename2 = e2.employeeName;
```

```
    if(eid1 == eid2 && ename1.equals(ename2))  
    {  
        return true;  
    }  
    else  
    {  
        return false;  
    }  
}
```

```
}
```

```
}  
public class EqualsMethodDemo2  
{
```

```

public static void main(String[] args)
{
    Employee e1 = new Employee(111, "Scott");
    Employee e2 = new Employee(111, "Scott");

    System.out.println(e1==e2); //false
    System.out.println(e1.equals(e2)); //true

}

}

```

Here we have overridden equals(Object obj) method from Object class to change the behaviour from memory address comparison to content comparison.

//Comparing the Customer object by overriding equals(Object obj) method

```

package com.ravi.equals;

class Customer
{
    private int customerId;
    private String customerName;

    public Customer(int customerId, String customerName)
    {
        super();
        this.customerId = customerId;
        this.customerName = customerName;
    }

    @Override
    public boolean equals(Object obj)
    {
        Customer c2 = (Customer) obj;

        if(this.customerId == c2.customerId && this.customerName.equals(c2.customerName))
        {
            return true;
        }
        else
        {
            return false;
        }
    }

}

public class EqualsMethodDemo3
{
    public static void main(String[] args)
    {
        Customer c1 = new Customer(1, "ABC");
        Customer c2 = new Customer(1, "ABC");
    }
}

```

```
System.out.println(c1.equals(c2));
```

```
}
```

```
}
```

instanceof Operator :

It is a keyword as well as an operator.

It is used to verify whether a reference variable refers to a particular type of object or not?

It will return boolean value.

While working with instanceof operator there must be a IS-A relationship between reference variable and the class/interface type otherwise we will get compilation error.

For any null comparison instanceof operator will always return false.

```
class Alpha
{
}
class Beta extends Alpha
{
}
public class Test
{
    public static void main(String[] args)
    {
        String s1 = "India";

        if(s1 instanceof String)
        {
            System.out.println("s1 is pointing to String");
        }

        Integer i1 = 23;

        if(i1 instanceof Number)
        {
            System.out.println("i1 is pointing Integer Object");
        }

        Alpha a1 = new Alpha();
        if(a1 instanceof Object)
        {
        }

        Beta b1 = new Beta();
        if(b1 instanceof Object)
        {
        }
    }
}
```

```

    if(b1 instanceof Alpha)
    {
    }

}

}

```

WAP in java where we want to compare two objects of two different types

```

package com.ravi.equals;

class Player
{
    int playerId;
    String playerName;

    public Player(int playerId, String playerName)
    {
        super();
        this.playerId = playerId;
        this.playerName = playerName;
    }

    @Override
    public boolean equals(Object obj)
    {
        if(obj instanceof Player)
        {
            Player p2 = (Player) obj; //down-casting

            if(this.playerId == p2.playerId && this.playerName.equals(p2.playerName))
            {
                return true;
            }
            else
            {
                return false;
            }
        }
        else
        {
            System.err.println("Comparison is not possible");
            return false;
        }
    }
}

public class EqualsDemo4 {

```



```

public static void main(String[] args)
{
    Player p1 = new Player(111, "Rohit");
    Player p2 = new Player(111,"Rohit");

    Manager m1 = new Manager(111, "Rohit");

    System.out.println(p1.equals(p2)); //true
    System.out.println(p1.equals(m1)); //false
    System.out.println(p1.equals(null)); //false

}

}

class Manager
{
    int managerId;
    String managerName;

    public Manager(int managerId, String managerName)
    {
        super();
        this.managerId = managerId;
        this.managerName = managerName;
    }

}

```

Note :

1) If we compare two different objects then compiler will generate warning as well as by using instanceof operator we can avoid this comparison.

2) null with instanceof operator will always return false.

Record class in java :

public abstract class Record extends Object.

It is a new feature introduced from java 17.(In java 14 preview version)

As we know only objects are moving in the network from one place to another place so we need to write BLC class with necessary requirements to make BLC class as a Data carrier class.

Records are immutable data carrier so, now with the help of record we can send our immutable data from one application to another application.

It is also known as DTO (Data transfer object) OR POJO (Plain Old Java Object) classes.

It is mainly used to concise our code as well as remove the boiler plate code.

In record, automatically constructor will be generated which is known as canonical constructor and the variables which are known as components are by default final.

In order to validate the outer world data, we can write our own constructor which is known as compact constructor.

Record will automatically generate the implementation of toString(), equals(Object obj) and hashCode() method.

We can define static and non static method as well as static variable and static block inside the record. We cannot define instance variable and instance block inside the record.

We can't extend or inherit records because by default every record is implicitly final and it is extending from java.lang.Record class, which is an abstract class.

We can implement an interface by using record.

We don't have setter facility in record because by default components are final.

3 files :

EmployeeClass.java

```
package com.ravi.record_class;
```

```
import java.util.Objects;
```

```
public class EmployeeClass {  
    private int employeeId;  
    private String employeeName;
```

```
    public EmployeeClass(int employeeId, String employeeName) {  
        super();  
        this.employeeId = employeeId;  
        this.employeeName = employeeName;  
    }
```

```
    public int getEmployeeId() {  
        return employeeId;  
    }
```

```
    public void setEmployeeId(int employeeId) {  
        this.employeeId = employeeId;  
    }
```

```
    public String getEmployeeName() {  
        return employeeName;  
    }
```

```
    public void setEmployeeName(String employeeName) {  
        this.employeeName = employeeName;  
    }
```

```
    @Override  
    public String toString() {
```

```

return "EmployeeClass [employeeId=" + employeeId + ", employeeName=" + employeeName + "];"
}

@Override
public int hashCode() {
    return Objects.hash(employeeId, employeeName);
}

@Override
public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if (obj == null)
        return false;
    if (getClass() != obj.getClass())
        return false;
    EmployeeClass other = (EmployeeClass) obj;
    return employeeId == other.employeeId && Objects.equals(employeeName, other.employeeName);
}
}

```

EmployeeRecord.java

```

-----
package com.ravi.record_class;

//Canonical Constructor
public record EmployeeRecord(int empId, String empName)
{
    //To validate outer world data we can write compact constructor

    public EmployeeRecord
    {
        if(empId <=0)
        {
            System.err.println("Inavlid Employee Id");
        }

    }
}

```

MainClass.java

```

-----
package com.ravi.record_class;

public class MainClass {

    public static void main(String[] args)
    {
        EmployeeClass e1 = new EmployeeClass(111, "Scott");
        e1.setEmployeeId(99);
        System.out.println(e1);
        EmployeeClass e2 = new EmployeeClass(111, "Scott");
        System.out.println(e1.equals(e2));
        System.out.println(e1.getEmployeeId() + " : "+e1.getEmployeeName());
    }
}

```

```

System.out.println("-----");

EmployeeRecord r1 = new EmployeeRecord(-222, "Smith");
System.out.println(r1);
EmployeeRecord r2 = new EmployeeRecord(222, "Smith");
System.out.println(r1.equals(r2));
System.out.println(r1.empId() + " : "+r1.empName());
}

}

```

JVM Architecture with class loader sub system :

The entire JVM Architecture is divided into 3 sections

- 1) Class loader sub system
- 2) Runtime Data Areas (Memory Areas)
- 3) Execution Engine.

Class Loader sub System :

The Class loader sub system is responsible to load the .class file into JVM Memory.

In order to load the .class file into JVM memory class loader sub system follows Delegation Hierarchy Algorithm.

Class loader sub system internally performs 3 tasks :

- a) LOADING
- b) LINKING
- c) INITIALIZATION

LOADING :

In order to load the required .class file, JVM makes a request to class loader sub system. The class loader sub system follows delegation hierarchy algorithm to load the required .class files from different areas.

To load the required .class file we have 3 different kinds of class loaders.

- 1) Bootstrap/Primordial class loader
- 2) Extension/Platform class loader
- 3) Application/System class loader

Bootstrap/Primordial class Loader :-

It is responsible for loading all the predefined .class files that means all API level predefined classes are loaded by Bootstrap class loader.

It has the highest priority because Bootstrap class loader is the super class for Platform class loader.

It loads the classes from the following path

C -> Program files -> Java -> JDK -> lib -> jrt-fs.jar

Platform/Extension class loader :

It is responsible to load the required .class file which is given by some 3rd party in the form of jar file.

It is the sub class of Bootstrap class loader and super class of Application class loader so it has more priority than Application class loader.

It loads the required .class file from the following path.

C -> Program files -> Java -> JDK -> lib -> ext -> ThirdParty.jar

command to create the jar file :

jar cf FileName.jar FileName.class

[If we want to compile more than one java file at a time then the command is : javac *.java]

Application/System class loader :

It is responsible to load all userdefined .class file into JVM memory.

It has the lowest priority because it is the sub class Platform class loader.

It loads the .class file from class path level or environment variable.

How Delegation Hierarchy algorithm works :-

Whenever JVM makes a request to class loader sub system to load the required .class file into JVM memory, first of all, class loader sub system makes a request to Application class loader, Application class loader will delegate(by pass) the request to the Extension class loader, Extension class loader will also delegate the request to Bootstrap class loader.

Bootstrap class loader will load the .class file from lib folder(jrt.jar) and then by pass the request to extension class loader, Extension class loader will load the .class file from ext folder(*.jar) and by pass the request to Application class loader, It will load the .class file from environment variable into JVM memory.

Note :-

If all the class loaders are failed to load the .class file into JVM memory then we will get a Runtime exception i.e java.lang.ClassNotFoundException.

The following program explains that java.lang.Class can hold any .class file.

Example : java.lang.Class cls = AnyClass.class

```
package com.ravi.class_loading;
```

```
class Employee{
```

```
class Customer{
```

```
class Student{
```

```
public class MethdoAreaDemo {
```

```

public static void main(String[] args)
{
    Class cls = Employee.class;
    System.out.println(cls.getName());

    cls = Customer.class;
    System.out.println(cls.getName());

    cls = Student.class;
    System.out.println(cls.getName());

}
}

```

WAP in java that describes our user defined .class file is loaded by Application class loader.

```

package com.ravi.class_loading;

public class Test
{
    public static void main(String[] args)
    {
        System.out.println("This Test.class file is loaded by :"+Test.class.getClassLoader());

        System.out.println("Parent class for Application class loader is
        :"+Test.class.getClassLoader().getParent());

        System.out.println("Parent class for Platform class loader is
        :"+Test.class.getClassLoader().getParent().getParent());

    }
}

```

getClassLoader() is a predefined method of class called Class available in java.lang package and its return type is ClassLoader.

```

    public ClassLoader getClassLoader()

```

In order to get the information of parent class we can use getParent() method of ClassLoader class, return type is ClassLoader

```

    public ClassLoader getParent();

```

Note : Test.class.getClassLoader().getParent();

Note :- For Bootstrap CL, implementation is not provided by java software people because it is for internal use.

23-05-2024

Linking :

verify :-

It ensures the correctness of the .class files, If any suspicious activity is there in the .class file then It will stop the execution immediately by throwing an exception i.e java.lang.VerifyError.

There is something called ByteCodeVerifier(Component of JVM), responsible to verify the loaded .class file i.e byte code. Due to this verify module JAVA is highly secure language.

java.lang.VerifyError is the sub class of java.lang.linkageError

prepare: (static variable memory allocation + Initialization)

It will allocate the memory for all the static data members, here all the static data member will get the default values so if we have static int x = 100;

then for variable x memory will be allocated and now it will initialize with default value i.e 0, even the variable is final

Resolve :

All the symbolic references will be converted into direct references Or actual reference.

javap -verbose FileName.class

Note :- By using above command we can read the internal details of .class file.

Initialization :-

In Initialization, all the static data member will get their actual (Original) value as well as if any static block is present in the class then the static block will start executing from here.

Here, static variable and static block both are having same priority so it will executed according to the order (top to bottom).

static block :

static blocks are special block in java which is automatically executed at the time of loading the .class file into JVM memory.

Example :

```
static
{

}
```

static blocks are executed only once because class loading is possible only once.

Automatically a class will not be loaded into JVM memory, it will be loaded based on user request so, static block will not be executed if the class is not loaded.

The main purpose of static block to initialize the static data member so it is also known as static initializer.

We can have multiple static blocks in a class but it will be executed according to the order (top to bottom)

Blank static final field can also be initialized inside the static block only.

static blocks are executed before main method.

Without declaring a static variable (post declaration) if we want to use the static variable in static block then initialization is possible (Writing operation is possible) but reading the data from memory is not possible without declaration, if we try to

read without declaration we will get compilation error illegal forward reference.

We can perform read operation with the help of class name because now we are not reading the directly, we are reading through class reference.

```
-----  
//static block  
class Foo  
{  
    Foo()  
    {  
        System.out.println("No Argument constructor..");  
    }  
  
    {  
        System.out.println("Instance block..");  
    }  
  
    static  
    {  
        System.out.println("Static block...");  
    }  
}  
  
public class StaticBlockDemo  
{  
    public static void main(String [] args)  
    {  
        System.out.println("Main Method Executed ");  
    }  
}
```

Note :- In the above program static block will not be executed because Foo.class file is not loaded into JVM Memory.

```
-----  
class Test  
{  
    static int x;  
  
    static  
    {  
        x = 100;  
        System.out.println("x value is :"+x);  
    }  
  
    static  
    {  
        x = 200;
```



```

System.out.println("x value is :"+x);
}

static
{
    x = 300;
    System.out.println("x value is :"+x);
}

}

public class StaticBlockDemo1
{
    public static void main(String[] args)
    {
        System.out.println(Test.x);
    }
}

```

Note :- static blocks are executed according to the order.

```

-----
class Foo
{
    static int x;

    static
    {
        System.out.println("x value is :"+x);
    }
}

public class StaticBlockDemo2
{
    public static void main(String[] args)
    {
        new Foo();
    }
}

```

Note :- static variables are also initialized with default value.

Variable memory allocation + initialization :

1) Static Field OR Class Variable :

For any static field memory will be allocated at the time of class loading (in the prepare phase) and initialized with default value.

2) Non static Field OR Instance Variable :

For any non static field, memory will be allocated at the time of object creation (by new keyword -> Instantiation) and initialized with default value using constructor (Initialization)

3) Local Variable :

For local variable memory will be allocated at the time of calling the method inside the stack frame and it is initialized by developer before use.

4) Parameter Variable :

For parameter variable memory will be allocated at the time of calling the method inside the stack frame and it is initialized by end user.

```
class Demo
{

    final static int a ; //Blank static final variable

    static
    {
        a = 100;
        System.out.println(a);
    }
}

public class StaticBlockDemo3
{
    public static void main(String[] args)
    {
        System.out.println("a value is :"+Demo.a);
    }
}
```

A static blank final variable must be initialized inside static block only.

The following program describes that static blank final variable also have default values.

```
public class StaticBlockDemo8
{
    final static int x; //Blank static final field

    static
    {
        m1();
        x = 15;
    }

    public static void m1()
    {
        System.out.println("Default value of x is :"+x);
    }

    public static void main(String[] args)
    {
        System.out.println("After initialization :"+StaticBlockDemo8.x);
    }
}
```

```
class A    //AD BC EF
{
    static
```

```
{  
    System.out.println("A");  
}
```

```
{  
    System.out.println("B");  
}
```

```
A()  
{  
    System.out.println("C");  
}  
}
```

```
class B extends A  
{  
    static  
    {  
        System.out.println("D");  
    }  
}
```

```
{  
    System.out.println("E");  
}
```

```
B()  
{  
    System.out.println("F");  
}
```

```
}  
public class StaticBlockDemo4  
{  
    public static void main(String[] args)  
    {  
        new B();  
    }  
}
```

Note :- The above program describes that instance blocks are executed in the body of the constructor.

//illegal forward reference

```
class Demo  
{  
    static  
    {  
        i = 100;  
    }  
  
    static int i;  
}
```

```

public class StaticBlockDemo5
{
    public static void main(String[] args)
    {
        System.out.println(Demo.i);
    }
}

```

Note :- Without pre-declaration of static variable we can initialize the static inside static block

```

class Demo
{
    static
    {
        i = 100; //Initialization is possible
        System.out.println(i); //Invalid Illegal Forward Reference
        System.out.println(Demo.i); //valid
    }

    static int i;
}

```

```

public class StaticBlockDemo6
{
    public static void main(String[] args)
    {
        System.out.println(Demo.i);
    }
}

```

return keyword in the initializer like static and non-static block :

A static or non-static block must be executed normally then means we can't terminate the logic by writing return keyword.

return keyword is not allowed inside the static or non-static initializer.

```

class StaticBlockDemo7
{
    static
    {
        System.out.println("Static Block");
        return; //error
    }

    public static void main(String[] args)
    {
        System.out.println("Main Method");
    }
}

```

What is the difference between JDK,JRE,JVM and JIT compiler ?

Paint Diagram (24-May-24)

Can we execute a Java program without main method ?

We can't execute a java program without main method, Upto jdk 1.6 it was possible to execute a java program without main method by writing the static block.

From JDK 1.7 onwards now we can't execute java program without main method because JVM verifies the presence of the main method before initializing the class.

If main method method is not available then JVM will generate a message that main method not found in the particular class.

Eg:-

```
class WithoutMain
```

```
{
    static
    {
        System.out.println("Hello world");
        System.exit(0);
    }
}
```

The above program was possible to execute upto JDK 1.6.

How many ways we can load the .class file into JVM memory :

There are multiple ways to load the .class file into JVM memory. The following are the common examples :-

- 1) By using java tools [java command]
- 2) By using Constructor [Object creation]
- 3) By accessing the static member of the class.
- 4) By using Inheritance
- 5) By Reflection API

- 1) By using Java tools
javac Test.java
java Test [Load the Test.class file into JVM memory]

- 2) By using Constructor [Object creation]

- 3) By Calling static variable and static method using class name.

```
class Demo
```

```
{
    static int x = 10;
    static
    {
        System.out.println("Static Block of Demo class Executed!!! :"+x);
    }
}
```

```

}
public class ClassLoading
{
    public static void main(String[] args)
    {
        new Demo();
        System.out.println(Demo.x);
    }
}

```

4) By using Inheritance :

In java whenever we try to load sub class then first of super class will be loaded that is the reason Object is the first class to be loaded into the JVM memory.

```

class Alpha
{
    static
    {
        System.out.println("Static Block of super class Alpha!!");
    }
}
class Beta extends Alpha
{
    static
    {
        System.out.println("Static Block of Sub class Beta!!");
    }
}
class InheritanceLoading
{
    public static void main(String[] args)
    {
        new Beta();
    }
}

```

5) By Reflection API (Explicit class loading)

Java software people has provided a predefined class called "Class" available in java.lang package.

This class called Class contains a predefined static method `forName(String className)`, through which we can load the required .class file into JVM memory dynamically.

It throws a checked exception i.e `java.lang.ClassNotFoundException`

`Class.forName("com.ravi.Test");` //Fully Qualified Name

```

class Foo
{
    static
    {
        System.out.println("Foo class static block");
    }
}

```

```

}
}
public class Hello
{
    public static void main(String [] args) throws Exception
    {
        Class.forName("Foo");
    }
}

```

When we execute the above program in Eclipse IDE then fully Qualified name is reqd.

```

package com.ravi.io;

class Foo
{
    static
    {
        System.out.println("Foo class static block");
    }
}
public class Hello
{
    public static void main(String [] args) throws Exception
    {
        Class.forName("com.ravi.io.Foo"); //Fully Qualified Name
    }
}

```

 What is factory Method in java ?

 The method which returns the class name itself by creating the object for that particular class is called Factory Method.

```

Class cls = Class.forName("com.ravi.Test");

```

Here Test.class will be loaded into JVM memory and it will return Class class object so further we can call any method of java.lang.Class class.

```

public class FactoryMethod {

    public static void main(String[] args) throws Exception
    {
        Class name = Class.forName("java.lang.String");
        System.out.println(name.getName());
    }
}

```

Any method which is returning the class object then it is known as Factory Method.

 *What is the limitation of 'new' keyword ?

OR

What is the difference between new keyword and newInstance() method?

OR

How to create the Object for the classes which are coming dynamically from the database or from some file.

The limitation with new keyword is, It demands the class name at the begning or at the time of compilation so new keyword is not suitable to create the object for the classes which are coming from database or files at runtime.

In order to create the object for the classes which are coming at runtime from database or files, we should use newInstance() method available in java.lang.Class class.

CreateObjectDynamically.java

class Employee

```
{  
}
```

class Customer

```
{  
}
```

class Student

```
{  
}
```

public class CreateObjectDynamically

```
{  
    public static void main(String[] args) throws Exception  
    {  
        Object obj = Class.forName(args[0]).newInstance();  
        System.out.println("Object created for :"+obj.getClass().getName());  
    }  
}
```

javac CreateObjectDynamically.java

java CreateObjectDynamically Student [It will create Student
class object]

class Player

```
{  
    public void m1()  
    {  
        System.out.println("Greeting From Player");  
    }  
}
```

class Manager

```
{  
    public void greet()  
    {  
        System.out.println("Greeting From Manager");  
    }  
}
```

public class CreateObjectDynamically1


```

{
public static void main(String[] args) throws Exception
{
    Object obj = Class.forName(args[0]).newInstance();

    if(obj instanceof Manager)
    {
        Manager m = (Manager) obj;
        m.greet();
    }

    else if(obj instanceof Player)
    {
        Player p = (Player) obj;
        p.m1();
    }

}
}

```

* What is the difference between java.lang.ClassNotFoundException and java.lang.NoClassDefFoundError :

java.lang.ClassNotFoundException :-

It occurs when we try to load the required .class file at runtime by using Class.forName() statement or loadClass() static of ClassLoader class and if the required .class file is not available at runtime then we will get an exception i.e java.lang.ClassNotFoundException

Note :- It does not have any concern at compilation time, at run time, JVM will simply check whether the required .class file is available or not.

```

class Foo
{
    static
    {
        System.out.println("static block of Foo class");
    }
}

public class ClassNotFoundExceptionDemo
{
    public static void main(String[] args) throws Exception
    {
        Class.forName("Player");
    }
}

```

Note :- In the above program Player.class file is not available so, forName(String className) method is unable to load the Player.class file hence we will get ClassNotFoundException.

java.lang.NoClassDefFoundError :

It occurs when the class was present at the time of COMPILATION but at runtime the required .class file is not available(manually deleted by user) Or it is not available in the current directory (Misplaced) then

we will get an exception i.e java.lang.NoClassDefFoundError.

NoClassDefFoundErrorDemo.java

```
-----  
class Player  
{  
    public void m1()  
    {  
        System.out.println("Greeting From Player");  
    }  
}  
public class NoClassDefFoundErrorDemo  
{  
    public static void main(String[] args)  
    {  
        Player p1 = new Player();  
        p1.m1();  
    }  
}
```

After compilation delete Player.class file and execute the program.

28-05-2024

Runtime Data Areas :

Once the class is loaded in the JVM memory now we need to divide the memory into different section. We have total 5 memory areas

- 1) Method Area
- 2) HEAP Area
- 3) Stack Area
- 4) PC Register
- 5) Native Method Stack

Method Area :

Whenever a class is loaded then the class is dumped inside method area and returns java.lang.Class class.

It provides all the information regarding the class like name of the class, name of the package, static and non static fields available in the class, methods available in the class and so on.

We have only one method area per JVM that means for a single JVM we have only one Method area.

This Method Area OR Class Area is sharable by all the objects.

Program to Show From Method Area we can get complete information of the class. (Reflection API)

2 files :

Test.java

```
package com.ravi.method_area;
```

```
public class Test
{
    int x,y;
    static int a,b,c;

    public void input() {}

    public void accept() {}

    public void display() {}

    public static void show() {}

    public void get() {}

}
```

ClassDescription.java

```
-----
package com.ravi.method_area;
```

```
import java.lang.reflect.Field;
import java.lang.reflect.Method;
```

```
public class ClassDescription {

    public static void main(String[] args) throws Exception
    {
        Class cls = Class.forName(args[0]);
        System.out.println("Name of the class :"+cls.getName());
        System.out.println("Package Name :"+cls.getPackageName());

        System.out.println("METHODS ARE :");
        Method[] methods = cls.getDeclaredMethods();
        int count = 0;
        for(Method method : methods)
        {
            System.out.println(method.getName());
            count++;
        }
        System.out.println("Total Methods are :"+count);

        System.out.println("FIELDS ARE :");

        Field[] fields = cls.getDeclaredFields();
        count = 0;
        for(Field field : fields)
        {
            System.out.println(field.getName());
            count++;
        }
        System.out.println("Total fields are :"+count);
    }
}
```

```
}
```

Note :- `getDeclaredMethods()` is a predefined non static method available in `java.lang.Class` class , the return type of this method is `Method` array where `Method` is a predefined class available in `java.lang.reflect` sub package

`getDeclaredFields()` is a predefined non static method available in `java.lang.Class` class , the return type of this method is `Field` array where `Field` is a predefined class available in `java.lang.reflect` sub package.

`Field` and `Method` both the classes are providing `getName()` method to get the name of the field and `Method`.

HEAP Area :

It is a special area where we can store the objects as well as all properties.

We have only one HEAP AREA per JVM.

This HEAP area is sharable by Method Area. From java9 version onwards now we have PermGen memory so this heap area can be dynamically growable so now Method area is also the part of HEAP area in a `class_data` section.

STACK Area :

All the methods are executed as a part of Stack Area.

Whenever we call a method in java then internally one stack Frame will be created to hold method related information.

Every Stack frame contains 3 parts

- 1) Local Variable
- 2) Frame Data
- 3) Operand Stack.

We have multiple stack area for a single JVM.

JVM creates a separate thread for every runtime Stack.

HEAP and STACK Diagram :

HEAP and STACK Diagram for `Employee.java`

```
public class Employee
{
    int id = 100;

    public static void main(String[] args)
    {
```

```

int val = 200;

Employee e1 = new Employee();

e1.id = val;

update(e1);

System.out.println(e1.id);

    Employee e2 = new Employee();

e2.id = 900;

switchEmployees(e2,e1); //3000x and 1000x

    //GC [2 objects 2000x and 4000x both are eligible]

System.out.println(e1.id);
    System.out.println(e2.id);
}

public static void update(Employee e)
{
    e.id = 500;
    e = new Employee();
    e.id = 400;
    System.out.println(e.id);
}

public static void switchEmployees(Employee e1, Employee e2)
{
    int temp = e1.id;
    e1.id = e2.id;
    e2 = new Employee();
    e2.id = temp;
}
}

```

HEAP and STACK Diagram for Beta.java

```

class Alpha
{
    int val;
    static int sval = 200;
    static Beta b = new Beta();

    public Alpha(int val)
    {
        this.val = val;
    }
}

```

```

public class Beta

```

```

{
public static void main(String[] args)
{
Alpha am1 = new Alpha(9);
Alpha am2 = new Alpha(2);

Alpha []ar = fill(am1, am2);

ar[0] = am1;
    System.out.println(ar[0].val);
    System.out.println(ar[1].val);
}

public static Alpha[] fill(Alpha a1, Alpha a2)
{
a1.val = 15;

        Alpha fa[] = new Alpha[]{a2, a1};

    return fa;
}
}

```

PC Register :

It stands for Program counter Register.

In order to hold the current executing instruction of running thread we have separate PC register for each and every thread.

Native Method Stack :

Native method means, the java methods which are written by using native languages like C and C++. In order to write native method we need native method library support.

Native method stack will hold the native method information in a separate stack.

Execution Engine :

Interpreter

In java, JVM is an interpreter which executes the program line by line. JVM (Interpreter) is slow in nature because at the time of execution if we make a mistake at line number 9 then it will throw the exception at line number 9 and after solving the exception again it will start the execution from line number 1 so it is slow in execution that is the reason to boost up the execution java software people has provided JIT compiler.

JIT Compiler :

It stands for just in time compiler. The main purpose of JIT compiler to boost up the execution so the execution of the program will be completed as soon as possible.

JIT compiler holds the repeated instruction like method signature, variables, native method code and make it available to JVM at the time of execution so the overall execution becomes very fast.

30-05-2024

Abstraction :

Working with abstract class and abstract method :

Abstract class and abstract methods :

A class that does not provide complete implementation (partial implementation) is defined as an abstract class.

An abstract method is a common method which is used to provide easiness to the programmer because the programmer faces complexity to remember the method name.

An abstract method observation is very simple because every abstract method contains abstract keyword, abstract method does not contain any method body and at the end there must be a terminator i.e ; (semicolon)

In java whenever action is common but implementations are different then we should use abstract method, Generally we declare abstract method in the super class and its implementation must be provided in the sub classes.

if a class contains at least one method as an abstract method then we should compulsory declare that class as an abstract class.

Once a class is declared as an abstract class we can't create an object for that class.

*All the abstract methods declared in the super class must be overridden in the sub classes otherwise the sub class will become as an abstract class hence object can't be created for the sub class as well.

In an abstract class we can write all abstract method or all concrete method or combination of both the method.

It is used to achieve partial abstraction that means by using abstract classes we can achieve partial abstraction(0-100%).

*An abstract class may or may not have abstract method but an abstract method must have abstract class.

Note :- We can't declare an abstract method as final, private and static (illegal combination of modifiers)

We can't declare an abstract class as a final.

abstract class Shape
{
 public abstract void draw();
}

class Square extends Shape
{
 @Override
 public void draw()
{
 System.out.println("Drawing Square");
 }
}

```
}  
}
```

```
class Rectangle extends Shape  
{  
    @Override  
    public void draw()  
    {  
        System.out.println("Drawing Rectangle");  
    }  
}  
public class AbstractDemo  
{  
    public static void main(String[] args)  
    {  
        Shape s ;  
  
        s = new Square(); s.draw();  
  
        s = new Rectangle(); s.draw();  
    }  
}
```

Interview Question :

```
abstract class Car  
{  
    protected int speed = 100;  
  
    public Car()  
    {  
        System.out.println("Car class constructor");  
    }  
  
    public void getDetails()  
    {  
        System.out.println("Car has an Engine and 4 wheels");  
    }  
  
    public abstract void run();  
}  
  
class Honda extends Car  
{  
    @Override  
    public void run()  
    {  
        System.out.println("Running with Honda Car");  
    }  
}  
  
public class IQ  
{  
    public static void main(String[] args)
```



```
{
    Car c = new Honda();
    System.out.println("Car speed is :"+c.speed);
    c.getDetails();
    c.run();
}
}
```

31-05-2024

What is the advantage of writing constructor in abstract class?

If my abstract class contains any properties (state OR Data) then we can initialize those properties of abstract class with the help of sub class object by using super keyword as shown in the program below.

```
package com.nit.abstract_demo;

abstract class Shape
{
    protected String shapeType;

    public Shape(String shapeType)
    {
        super();
        this.shapeType = shapeType;
    }

    public abstract void draw();
}

class Square extends Shape
{
    public Square(String shapeType)
    {
        super(shapeType);
    }

    @Override
    public void draw()
    {
        System.out.println("Drawing "+shapeType);
    }
}

class Rectangle extends Shape
{
    public Rectangle(String shapeType)
    {
        super(shapeType);
    }

    @Override
    public void draw()
    {
```

```
System.out.println("Drawing "+shapeType);
}
}
```

```
public class AbstractDemo {

    public static void main(String[] args)
    {
        Square s = new Square("Square");
        s.draw();

        new Rectangle("Rectangle").draw();

    }

}
```

From the above program, It is clear that we can initialize the super class (abstract class) properties with sub class object using super keyword.

```
package com.nit.abstract_demo;

abstract class Alpha
{
    public abstract void show();
    public abstract void demo();
}

abstract class Beta extends Alpha
{
    @Override
    public void show()
    {
        System.out.println("Show method implemented in Beta class ");
    }
}

class Gamma extends Beta
{
    @Override
    public void demo()
    {
        System.out.println("demo method implemented in Gamma class ");
    }
}

public class AbstractExample
{
    public static void main(String[] args)
    {
        Gamma g = new Gamma();
        g.show(); g.demo();
    }

}
```

Note :- From the above program, It is clear that abstract methods of super class must be overridden in all the sub classes.

```
-----
package com.nit.abstract_demo;

abstract class Animal
{
    public abstract void checkup();
}
class Bird extends Animal
{
    @Override
    public void checkup()
    {
        System.out.println("Bird Checkup");
    }
}
class Lion extends Animal
{
    @Override
    public void checkup()
    {
        System.out.println("Lion Checkup");
    }
}
class Dog extends Animal
{
    @Override
    public void checkup()
    {
        System.out.println("Dog Checkup");
    }
}

public class AnimalDemo {

    public static void main(String[] args)
    {
        Bird birds[] = {new Bird(), new Bird(), new Bird()};

        Lion lions[] = {new Lion(), new Lion()};

        Dog dogs[] = {new Dog(), new Dog(), new Dog(), new Dog()};

        checkAnimals(dogs); checkAnimals(lions);
        checkAnimals(birds);

    }

    public static void checkAnimals(Animal []animals)
    {
        for(Animal animal : animals)
        {
```

```

        animal.checkup();
    }
}

}

```

----- interface upto java 1.7 -----

An interface is a keyword in java which is similar to a class which defines working functionality of a class.

Upto JDK 1.7 an interface contains only abstract method that means there is a guarantee that inside an interface we don't have concrete or general or instance methods.

From java 8 onwards we have a facility to write default and static methods.

By using interface we can achieve 100% abstraction concept because it contains only abstract methods.

In order to implement the member of an interface, java software people has provided implements keyword.

All the methods declared inside an interface is by default public and abstract so at the time of overriding we should apply public access modifier to sub class method.

All the variables declared inside an interface is by default public, static and final.

We should override all the abstract methods of interface to the sub classes otherwise the sub class will become as an abstract class hence object can't be created.

We can't create an object for interface, but reference can be created.

By using interface we can achieve multiple inheritance in java.

We can achieve loose coupling using interface.

Note :- inside an interface we can't declare any blocks (instance, static), instance variables (No properties) as well as we can't write constructor inside an interface.

01-06-2024

```
package com.ravi.interface_demo;
```

```
interface Moveable
{
    int SPEED = 100; //public + static + final
    void move(); //public + abstract
}

```

```
class Car implements Moveable
{
    @Override
    public void move()
    {
        //SPEED = 120; error
        System.out.println("Car is moving with "+ SPEED + " speed");
    }
}

```

```

}
public class InterfaceDemo
{
    public static void main(String[] args)
    {
        Moveable m = new Car();
        m.move();
        System.out.println("Car Speed is :"+Moveable.SPEED);

    }

}

```

```

package com.ravi.interface_demo;

```

```

interface Bank

```

```

{
    void deposit(int amount);
    void withdraw(int amount);
}

```

```

class Customer implements Bank

```

```

{
    private double balance;

```

```

    public Customer(double balance)

```

```

    {
        super();
        this.balance = balance;
    }

```

```

    @Override

```

```

    public void deposit(int amount)

```

```

    {
        if(amount <=0)
        {
            System.err.println("Can't deposit amount");
        }
        else
        {
            this.balance = this.balance + amount;
            System.out.println("Balance After Deposit :"+this.balance);
        }
    }

```

```

    @Override

```

```

    public void withdraw(int amount)

```

```

    {
        if(amount > this.balance)
        {
            System.err.println("Insufficient Balance");
        }
        else
        {
            this.balance = this.balance - amount;
            System.out.println("Balance After Withdraw :"+this.balance);
        }
    }

```

```

    }
}

}

public class BankDemo
{
    public static void main(String[] args)
    {
        Bank raj = new Customer(10000);
        raj.deposit(5000);
        raj.withdraw(12000);
    }
}

```

Assignment :

interface Calculate
{
 void doSum(int x, int y);
 void doSub(int x, int y);
 void doMul(int x, int y);
}

Program to achieve loose coupling using interface :

Program on loose coupling :

Loose Coupling :- If the degree of dependency from one class object to another class is very low then it is called loose coupling.

Tightly coupled :- If the degree of dependency of one class to another class is very high then it is called Tightly coupled.

According to IT industry standard we should always prefer loose coupling so the maintenance of the project will become easy.

6 files :

HotDrink.java(I)

package com.ravi.interface_demo;

```

public interface HotDrink
{
    void prepare();
}

```

Tea.java(C)

package com.ravi.interface_demo;

```

public class Tea implements HotDrink
{

```

```
@Override
public void prepare()
{
    System.out.println("Preparing Tea");
}
}
```

Coffee.java

```
-----
package com.ravi.interface_demo;

public class Coffee implements HotDrink
{
    @Override
    public void prepare()
    {
        System.out.println("Preparing Coffee");
    }
}
```

Horlicks.java

```
-----
package com.ravi.interface_demo;

public class Horlicks implements HotDrink
{
    @Override
    public void prepare()
    {
        System.out.println("Preparing Horlicks");
    }
}
```

Restaurant.java

```
-----
package com.ravi.interface_demo;

public class Restaurant
{
    public static void acceptObject(HotDrink hd) //hd = new Tea();
    {
        hd.prepare();
    }
}
```

LooseCoupling.java

```
-----
package com.ravi.interface_demo;

public class LooseCoupling {

    public static void main(String[] args)
    {
```

```

Restaurant.acceptObject(new Tea());
Restaurant.acceptObject(new Coffee());
Restaurant.acceptObject(new Horlicks());

}
}
-----

```

It is always better to take method return type as interface so we can return any implementer class object as shown in the example below

```

public HotDrink accept()
{
    return new Tea() OR new Coffee() OR new Horlicks() OR any future
    implementer class object.....
}
-----

```

Multiple Inheritance by using interface :

In a class we have a constructor so, it is providing ambiguity issue but inside an interface we don't have constructor so multiple inheritance is possible using interface.

The sub class constructor super keyword will move to Object class constructor.(01-JUNE)

```

package com.ravi.mi;

interface A
{
    void m1();
}

interface B
{
    void m1();
}

class C implements A,B
{
    @Override
    public void m1()
    {
        System.out.println("Multiple Inheritance is Possible");
    }
}

public class MultipleInheritance
{
    public static void main(String[] args)
    {
        C c1 = new C();
        c1.m1();
    }
}
-----

```


Extending interface :

One interface can extends another interface, it cannot implement because interface cannot provide implementation for the abstract method.

```
package com.ravi.mi;
```

```
interface Alpha
```

```
{  
    void m1();  
}
```

```
interface Beta extends Alpha
```

```
{  
    void m2();  
}
```

```
class Implementer implements Beta
```

```
{  
    @Override  
    public void m1()  
    {  
        System.out.println("m1 method implemented");  
    }  
}
```

```
    @Override  
    public void m2()  
    {  
        System.out.println("m2 method implemented");  
    }  
}
```

```
public class ExtendingInterface {
```

```
    public static void main(String[] args)  
    {  
        Implementer i = new Implementer();  
        i.m1(); i.m2();  
    }  
}
```

03-06-2024

Limitation of abstract method :

OR

Maintenance problem with interface in an Industry upto JDK 1.7

The major maintenance problem with interface is, if we add any new abstract method at the later stage of development inside an existing interface then all the implementer classes have to override that abstract method otherwise the implementer class will become as an abstract class so it is one kind of boundation.

We need to provide implementation for all the abstract methods available inside an interface whether it is required or not?

To avoid this maintenance problem java software people introduced default method inside an interface.

What is default Method inside an interface?

default method is just like concrete method which contains method body and we can write inside an interface from java 8 onwards.

default method is used to provide specific implementation for the implementer classes which are implementing from interface because we can override default method inside the sub classes to provide our own specific implementation.

*By using default method there is no boundation to override the default method in the sub class, if we really required it then we can override to provide my own implementation.

by default, default method access modifier is public so at the time of overriding we should use public access modifier.

default method we can write inside an interface only but not inside a class.

4 files :

Vehicle.java(I)

package com.ravi.interface_java_8;

public interface Vehicle
{
 void run();
 void horn();

 public default void digitalMeter() //java 8
 {
 System.out.println("Digital Meter facility");
 }

}

Car.java(C)

package com.ravi.interface_java_8;

public class Car implements Vehicle
{
 @Override
 public void run()
 {
 System.out.println("Car is Running");
 }

 @Override
 public void horn()
 {

```

        System.out.println("Car has horn");
    }

    @Override
    public void digitalMeter()
    {
        System.out.println("Car has Digital Meter facility");
    }
}

```

Bike.java(C)

```

-----
package com.ravi.interface_java_8;

public class Bike implements Vehicle
{
    @Override
    public void run()
    {
        System.out.println("Bike is Running");
    }

    @Override
    public void horn()
    {
        System.out.println("Bike has horn");
    }
}

```

InterfaceNewFeature.java

```

-----
package com.ravi.interface_java_8;

public class InterfaceNewFeature {

    public static void main(String[] args)
    {
        Vehicle v = null;
        v = new Car(); v.run(); v.horn(); v.digitalMeter();
        v = new Bike(); v.run(); v.horn();
    }

}

```

Note :- abstract method is a common method which is used to provide easiness to the programmer so, by looking the abstract method we will get confirmation that this is common behavior for all the sub classes and it must be implemented in all the sub classes.

The following program explains that default methods are having low priority than normal methods (Concrete Method). class is having more power than interface.

4 files :

A.java(I)

```
-----  
package com.ravi.interface_java_8;  
  
public interface A  
{  
    public default void m1()  
    {  
        System.out.println("interface m1 method");  
    }  
}
```

B.java(C)

```
-----  
package com.ravi.interface_java_8;  
  
public class B  
{  
    public void m1()  
    {  
        System.out.println("class m1 method");  
    }  
}
```

C.java(c)

```
-----  
package com.ravi.interface_java_8;  
  
public class C extends B implements A  
{  
  
}
```

Priority.java(C)

```
-----  
package com.ravi.interface_java_8;  
  
public class Priority {  
  
    public static void main(String[] args)  
    {  
        C c1 = new C();  
        c1.m1();  
    }  
}
```

Multiple inheritance using default method :

Multiple inheritance is possible in java by using default method inside an interface, here we need to use super keyword to differentiate the super interface methods.

I.java(I)

```
package com.ravi.mi_default;
```

```
public interface I
{
    public default void m1()
    {
        System.out.println("m1 method of I interface");
    }
}
```

```
J.java(I)
```

```
-----
```

```
package com.ravi.mi_default;
```

```
public interface J
{
    public default void m1()
    {
        System.out.println("m1 method of J interface");
    }
}
```

```
Implementer.java(C)
```

```
-----
```

```
package com.ravi.mi_default;
```

```
public class Implementer implements I,J
{
    @Override
    public void m1()
    {
        I.super.m1();
        J.super.m1();
        System.out.println("m1 method overridden");
    }
}
```

```
MultipleInheritanceUsingDefault.java
```

```
-----
```

```
package com.ravi.mi_default;
```

```
public class MultipleInheritanceUsingDefault {

    public static void main(String[] args)
    {
        Implementer i = new Implementer();
        i.m1();

    }

}
```

What is static method inside an interface?

We can define static method inside an interface from java 1.8 onwards.

static method is only available inside the interface, It is not available to the implementer classes.

It is used to provide common functionality which we can apply/invoke from any BLC/ELC class.

By default static method of an interface contains public access modifier.

//Program on static method inside an interface in two different packages.

Calculator.java(I) [Package name is -> com.ravi.interface_java_8]

```
package com.ravi.interface_java_8;
```

```
public interface Calculator
{
    static double doSum(int x, int y)
    {
        return (x+y);
    }

```

```
    static double getSquare(int x)
    {
        return (x*x);
    }

```

```
    static double getCube(int x)
    {
        return (x*x*x);
    }
}
```

Main.java(C) [Package is -> com.ravi.mi_default]

```
package com.ravi.mi_default;
```

```
import com.ravi.interface_java_8.Calculator;
```

```
public class Main {
```

```
    public static void main(String[] args)
    {
        double result = Calculator.doSum(12, 24);
        System.out.println("Sum is :"+result);

```

```
        result = Calculator.getSquare(5);
        System.out.println("Square is :"+result);

```

```
        result = Calculator.getCube(9);
    }
}
```

```
System.out.println("Cube is :"+result);
}
```

```
}
```

Program that describe that static method of an interface is only available to interface only that means we can access the static method of an interface by using only one way i.e interface name.

```
package com.ravi.static_demo;
```

```
interface Drawable
{
    public static void m1()
    {
        System.out.println("m1 static method");
    }
}
```

```
public class StaticInterfaceDemo implements Drawable
{
    public static void main(String[] args)
    {
        Drawable.m1();

        //m1(); //error

        //StaticInterfaceDemo.m1(); //error

        StaticInterfaceDemo i = new StaticInterfaceDemo();
        //i.m1(); error

    }
}
```

```
}
```

//From java 8 version we can write main method inside an interface and it will be executed.

```
package com.ravi.static_demo;
```

```
public interface Worker
{
    public static void main(String[] args)
    {
        System.out.println("Main method of an interface");
    }
}
```

Interface Static Method:

-
- a) Accessible using the interface name.
 - b) Cannot be overridden by implementing classes.(Not Available)
 - c) Can be called using the interface name only.

Class Static Method:

- a) Accessible using the class name.
- b) Can be hidden (not overridden) in subclasses by redeclaring a static method with the same signature.
- c) Can be called using the super class, sub class name as well as sub class object also as shown in the program below.

```
class A
{
    public static void m1()
    {
        System.out.println("Static method A");
    }
}
class B extends A
{
}
public class Demo
{
    public static void main(String [] args)
    {
        B.m1(); //valid
        new B().m1(); //valid
    }
}
```

04-06-2024

Anonymous inner class :

If we declare a class without any name then it is called anonymous inner class or nameless class.

The main purpose of anonymous inner class to extend a class or implement an interface that means for sub type purpose.

Program that describes how to override concrete class method using anonymous inner class.

```
package com.ravi.anonymous;

class Super
{
    public void show()
    {
        System.out.println("Show method of super class");
    }
}

public class AnonymousDemo1
{
    public static void main(String[] args)
    {
        //Anonymous inner class
        Super sub = new Super()
```



```

{
    @Override
    public void show()
    {
        System.out.println("Show method of Anonymous class");
    }
};

sub.show();

}

}

```

Anonymous inner class using abstract class :

```

package com.ravi.anonymous;

abstract class Animal
{
    public abstract void sleep();
}

public class AnonymousDemo2
{
    public static void main(String[] args)
    {
        Animal lion = new Animal()
        {
            @Override
            public void sleep()
            {
                System.out.println("Lion is sleeping");
            }
        };

        Animal dog = new Animal()
        {
            @Override
            public void sleep()
            {
                System.out.println("Dog is sleeping");
            }
        };

        lion.sleep();  dog.sleep();
    }
}

```

Anonymous inner class using interface :

```

package com.ravi.anonymous;

interface Drawable
{
    void draw();
}

public class AnonymousDemo3
{
    public static void main(String[] args)
    {
        Drawable rect = new Drawable()
        {
            @Override
            public void draw()
            {
                System.out.println("Drawing rectangle");
            }
        };

        Drawable square = new Drawable()
        {
            @Override
            public void draw()
            {
                System.out.println("Drawing Square");
            }
        };

        rect.draw();    square.draw();
    }
}

```

What is a Functional Interface in java ?

If an interface contains exactly one abstract method then that interface is known as Functional interface.

In order to restrict the user to take only one abstract method (Single Abstract Method = SAM) we can use @FunctionalInterface annoation.

```

@FunctionalInterface
interface Drawable
{
    void draw(); //SAM [Single Abstract method]
}

```

Inside a functional interface, we can define 'n' number of default and static methods but it must have only one abstract method.

Lambda Expression :

It is a new feature introduced in java from JDK 1.8 onwards.
It is an anonymous function i.e function without any name.

In java it is used to enable functional programming.

It is used to concise our code as well as we can remove boilerplate code.

It can be used with functional interface only.

If the body of the Lambda Expression contains only one statement then curly braces are optional.

We can also remove the variables type while defining the Lambda Expression parameter.

If the lambda expression method contains only one parameter then we can remove () symbol also.

In lambda expression return keyword is optional but if we use return keyword then {} are compulsory.

Independently Lamda Expression is not a statement.

It requires a target variable i.e functional interface reference only.

Lamda target can't be class or abstract class, it will work with functional interface only.

```
package com.ravi.lambda;
```

```
interface Printable
```

```
{
    void print();
}
```

```
public class LambdaDemo1
```

```
{
    public static void main(String[] args)
    {
        Printable p = ()-> System.out.println("Printing");
        p.print();
    }
}
```

```
package com.ravi.lambda;
```

```
@FunctionalInterface
```

```
interface Calculate
```

```
{
    void doSum(int x, int y);
}
```

```
public class LambdaDemo2 {
```

```
    public static void main(String[] args)
    {
        Calculate c1 = (c, d)-> System.out.println("Sum is :"+(c+d));
        c1.doSum(12, 34);
    }
}
```

```
package com.ravi.lambda;
```

```
import java.util.Scanner;
```

```
interface Length
{
    public int findLength(String str);
}
```

```
public class LambdaDemo3 {

    public static void main(String[] args)
    {
        Length l = str -> str.length();

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter any city name to find out the length :");
        String name = sc.nextLine();

        System.out.println("Length of "+name+ " is :"+l.findLength(name));
        sc.close();

    }

}
```

```
@FunctionalInterface
interface Calculate
{
    int getSquare(int num);
}
```

```
public class Lambda
{
    public static void main(String[] args)
    {
        Calculate c = x -> x*x;
        java.util.Scanner sc = new java.util.Scanner(System.in);
        System.out.print("Enter a Number :");
        int num = sc.nextInt();

        System.out.println("Square of "+num+" is "+c.getSquare(num));
    }
}
```

```
@FunctionalInterface
interface Moveable
{
    void move(); //SAM (Single Abstract Method)
}
```

```
public class Lambda1
{
    public static void main(String[] args)
    {
        Moveable car = ()-> System.out.println("Car is moving ");

        Moveable bike = ()-> System.out.println("Bike is moving ");
    }
}
```

```
Moveable bus = ()-> System.out.println("Bus is moving ");
```

```
car.move(); bike.move(); bus.move();  
}  
}
```

Working with predefined functional interface :

In java.lang package, there is a predefined functional interface called Runnable which contains only one abstract method i.e run()

```
@FunctionalInterface  
public interface Runnable  
{  
    public abstract void run();  
}
```

Lambda4.java

```
-----  
public class Lambda4  
{  
    public static void main(String[] args)  
    {  
        Runnable r1 = () -> System.out.println("Running");  
        r1.run();  
    }  
}
```

What is type parameter <T> ?

It is a technique through which we can make our application independent of data type. It is represented by <T>

In java we can pass Wrapper classes as well as User-defined class to this type parameter.

We cannot pass any primitive type to this type parameter.

Program on Type Parameter :

package com.ravi.type_parameter;

```
class Accept<T>  
{  
    private T x;  
  
    public Accept(T x) // Student x  
    {  
        super();  
        this.x = x;  
    }  
  
    public T getX()  
    {  
        return x;  
    }  
}
```

```
}  
}
```

```
public class TypeParameterDemo  
{  
    public static void main(String[] args)  
    {  
        Accept<Integer> intType = new Accept<Integer>(12);  
        System.out.println("Integer Type is :"+intType.getX());  
  
        Accept<Double> doubleType = new Accept<Double>(78.90);  
        System.out.println("Double Type is :"+doubleType.getX());  
  
        Accept<Boolean> booleanType = new Accept<Boolean>(false);  
        System.out.println("Boolean Type is :"+booleanType.getX());  
  
        Accept<Student> stuType = new Accept<Student>(new Student(1, "Scott"));  
  
        System.out.println("Student Type is :"+stuType.getX());  
    }  
}
```

```
record Student(int id, String name)  
{  
  
}
```

06-06-2024

Working with predefined functional interfaces :

In order to help the java programmer to write concise java code in day to day programming java software people has provided the following predefined functional interfaces

- 1) Predicate<T>
- 2) Consumer<T>
- 3) Function<T,R>
- 4) Supplier<T>
- 5) BiPredicate<T,U>
- 6) BiConsumer<T, U>
- 7) BiFunction<T,U,R>

Note :-

All these predefined functional interfaces are provided as a part of java.util.function sub package.

Predicate<T> functional interface :

It is a predefined functional interface available in java.util.function sub package.

It contains an abstract method test() which takes type parameter <T> and returns boolean value. The main purpose of this interface to test one argument boolean expression.

@FunctionalInterface

```
public interface Predicate<T>
{
    boolean test(T x);
}
```

Note :- Here T is a "type parameter" and it can accept any type of User defined class as well as Wrapper class like Integer, Float, Double and so on.

We can't pass primitive type.

Program on Predicate interface to verify a number is even or odd

```
package com.ravi.functional_demo;

import java.util.Scanner;
import java.util.function.Predicate;

public class PredicateDemo1
{
    public static void main(String[] args)
    {
        //Verify a number is even or odd
        Predicate<Integer> p1 = num -> num % 2 ==0;

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number to verify even or odd :");
        int x = sc.nextInt();

        System.out.println("Is "+x+" even number ?"+p1.test(x));
        sc.close();

    }
}
```

//name starts with 'P' Or not

```
package com.ravi.functional_demo;

import java.util.Scanner;
import java.util.function.Predicate;

public class PredicateDemo2
{
    public static void main(String[] args)
    {
        //Verify name starts with P or not

        Predicate<String> p2 = str -> str.startsWith("P");
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your Name :");
        String name = sc.nextLine();

        System.out.println(name+" starts with 'P' :"+p2.test(name));
    }
}
```

```

    sc.close();
}

}

-----
//name is Sachin or not ?

package com.ravi.functional_demo;

import java.util.Scanner;
import java.util.function.Predicate;

public class PredicateDemo3
{
    public static void main(String[] args)
    {
        //Verify a name is Sachin or not

        Predicate<String> p3 = str -> str.equals("Sachin");

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your Name :");
        String name = sc.nextLine();

        System.out.println("Are you Sachin ?"+p3.test(name));
        sc.close();

    }

}
}
-----

```

Assignment :

By using Predicate verify whether a year is leap year or not?

Consumer<T> functional interface :

It is a predefined functional interface available in java.util.function sub package.

It contains an abstract method accept() and returns nothing. It is used to accept the parameter value or consume the value.

```

@FunctionalInterface
public interface Consumer<T>
{
    void accept(T x);
}
-----

```

```

package com.ravi.functional_demo;

import java.util.function.Consumer;

public class ConsumerDemo1 {

    public static void main(String[] args)

```



```

{
    Consumer<Integer> c1 = x -> System.out.println(x);
    c1.accept(12);

    Consumer<Double> c2 = x -> System.out.println(x);
    c2.accept(234.90);

    Consumer<Product> c3 = prod -> System.out.println(prod);
    c3.accept(new Product(1, "Laptop"));

}
}

```

```

record Product(int productId, String productName)
{

}

```

Function<T,R> functional interface :

Function<T,R> functional interface :

Type Parameters:

T - the type of the input to the function.

R - the type of the result of the function.

It is a predefined functional interface available in java.util.function sub package.

It provides an abstract method apply that accepts one argument(T) and produces a result(R).

Note :- The type of T(input) and the type of R(Result) both will be decided by the user.

```

@FunctionalInterface
public interface Function<T,R>
{
    R apply(T x);
}

```

//Finding the cube of the number

```

package com.ravi.functional_demo;

```

```

import java.util.Scanner;
import java.util.function.Function;

```

```

public class FunctionDemo1 {

    public static void main(String[] args)
    {
        // cube of a number
        Function<Integer, Integer> fn1 = num -> num*num*num;

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number :");
    }
}

```

```
int n = sc.nextInt();

System.out.println("Cube of "+n +" is :"+fn1.apply(n));
sc.close();

}

}
```

```
//Finding the length of the String
package com.ravi.functional_demo;
```

```
import java.util.Scanner;
import java.util.function.Function;
```

```
public class FunctionDemo2 {
```

```
    public static void main(String[] args)
    {
        //Finding the length of name
        Function<String, Integer> fn2 = str -> str.length();
```

```
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your Name :");
        String name = sc.nextLine();
```

```
        System.out.println("Length of "+name+" is :"+fn2.apply(name));
        sc.close();
```

```
    }
```

```
}
```

```
//Verify name starts with 'R' or not
```

```
package com.ravi.functional_demo;
```

```
import java.util.function.Function;
```

```
public class FunctionDemo3 {
```

```
    public static void main(String[] args)
    {
        Function<String, Boolean> fn3 = str -> str.startsWith("R");
        System.out.println("Your name starts with 'R' :"+fn3.apply("Raj"));
    }
```

```
}
```

```
Supplier<T> predefined functional interface :
```

It is a predefined functional interface available in java.util.function sub package. It provides an abstract method get() which does not take any argument but produces/supply a value of type T.

```
@FunctionalInterface
public interface Supplier<T>
{
    T get();
}
```

```
package com.ravi.functional_demo;

import java.util.function.Supplier;

public class SupplierDemo1 {

    public static void main(String[] args)
    {
        Supplier<String> s1 = ()-> 90 + 90 + "NIT" + 34+45;
        System.out.println(s1.get());
    }

}
```

Here It is returning String value.

```
package com.ravi.functional_demo;

import java.util.Scanner;
import java.util.function.Supplier;

record Employee(int employeeId, String employeeName, double empSalary)
{

}
```

```
public class SupplierDemo2
{
    public static void main(String[] args)
    {
        Supplier<Employee> s2 = ()->
        {
            Scanner sc = new Scanner(System.in);
            System.out.print("Enter Employee id :");
            int id = sc.nextInt();

            System.out.print("Enter Employee Name :");
            String name = sc.nextLine();
            name = sc.nextLine();

            System.out.print("Enter Employee Salary :");
            double salary = sc.nextDouble();

            return new Employee(id, name, salary);
        };

        Employee emp = s2.get();
```

```
System.out.println(emp);
}
```

```
}
```

Here It returns Employee Object.

07-06-2024

BiPredicate<T,U> functional interface :

It is a predefined functional interface available in java.util.function sub package.

It is a functional interface in Java that represents a predicate (a boolean-valued function) OF TWO ARGUMENTS.

The BiPredicate interface has method named test, which takes two parameters and returns a boolean value, basically this BiPredicate is same with the Predicate, instead, it takes 2 arguments for the test.

```
@FunctionalInterface
public interface BiPredicate<T, U>
{
    boolean test(T t, U u);
}
```

Type Parameters:

T - the type of the first argument to the predicate
U - the type of the second argument the predicate

```
import java.util.function.*;
public class Lambda11
{
    public static void main(String[] args)
    {
        BiPredicate<String, Integer> filter = (x, y) ->
        {
            return x.length() == y;
        };

        boolean result = filter.test("Ravi", 4);
        System.out.println(result);

        result = filter.test("Hyderabad", 10);
        System.out.println(result);
    }
}
```

```
import java.util.function.BiPredicate;
```

```
public class Lambda12
{
    public static void main(String[] args)
    {
        // BiPredicate to check if the sum of two integers is even
```

```

BiPredicate<Integer, Integer> isSumEven = (a, b) -> (a + b) % 2 == 0;

        System.out.println(isSumEven.test(2, 3));
        System.out.println(isSumEven.test(5, 7));
    }
}

```

BiConsumer<T, U> functional interface :

It is a predefined functional interface available in java.util.function sub package.

It is a functional interface in Java that represents an operation that accepts two input arguments and returns no result.

It takes a method named accept, which takes two parameters and performs an action without returning any result.

```

@FunctionalInterface
public interface BiConsumer<T, U>
{
    void accept(T t, U u);
}

```

```

import java.util.function.BiConsumer;

```

```

public class Lambda13
{
    public static void main(String[] args)
    {
        BiConsumer<Integer, String> updateVariables = (num, str) ->
        {
            num = num * 2;
            str = str.toUpperCase();
            System.out.println("Updated values: " + num + ", " + str);
        };
    }
}

```

```

        int number = 15;
        String text = "nit";

        updateVariables.accept(number, text);

        // Values after the update (note that the original values are unchanged)
        System.out.println("Original values: " + number + ", " + text);
    }
}

```

BiFunction<T, U, R> Functional interface :

It is a predefined functional interface available in java.util.function sub package.

It is a functional interface in Java that represents a function that accepts two arguments and produces a result R.

The BiFunction interface has a method named apply that takes two arguments and returns a result R.

```
@FunctionalInterface
public interface BiFunction<T, U, R>
{
    R apply(T t, U u);
}
```

```
import java.util.function.BiFunction;
```

```
public class Lambda14
{
    public static void main(String[] args)
    {
        // BiFunction to concatenate two strings
        BiFunction<String, String, String> concatenateStrings = (str1, str2) -> str1 + str2;

        String result = concatenateStrings.apply("Hello", " Java");
        System.out.println(result);

        // BiFunction to find the length two strings
        BiFunction<String, String, Integer> concatenateLength = (str1, str2) -> str1.length() + str2.length();

        Integer result1 = concatenateLength.apply("Hello", "Java");
        System.out.println(result1);

    }
}
```

Interface from java 9v version

Yes, From java 9 onwards we can also write private static and private non-static methods inside an interface.

These private methods will improve code re-usability inside interfaces.

For example, if two default methods needed to share common and confidential code, a private method would allow them to do so, but without exposing that private method to its implementing classes.

Using private methods in interfaces have four rules :

- 1) private interface method cannot be abstract.
- 2) private method can be used only inside interface.
- 3) private static method can be used inside other static and non-static interface methods.
- 4) private non-static methods cannot be used inside private static methods.

```
package com.ravi.java_9_feature;
```

```
interface Drawable
{
    void m1(); // JDK 1.0

    default void m2() //JDK 1.8
    {
```

```

    m4();
    m5();
}

static void m3() //JDK 1.8
{
    m4();

}

private static void m4() //Private Static Method [Java 9]
{
    System.out.println("Common logic");
}

private void m5() //Private Non-Static Method [Java 9]
{
    System.out.println("Reusable Logic");
}

}
class Client implements Drawable
{
    @Override
    public void m1()
    {
        System.out.println("M1 method");
    }
}
public class InterfaceNewFaeture {

    public static void main(String[] args)
    {
        Drawable d = new Client();
        d.m1();
        d.m2();
        Drawable.m3();

    }

}

```

Can an interface extend a class like Object class :

No Interface does not inherits Object class,It can't extends a class. but it provide accessibility to all methods of Object class.

This is because, for every public method in Object class, there is an implicit abstract and public method declared in every interface which does not have direct super interfaces. (Java language Specification 9.2 about interface members)

```
package com.ravi.java_9_feature;
```

```
interface Printable
{
```

```
}
```

```
public class InterfaceMember {
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Printable p = null;
```

```
        p.equals(null);
```

```
        p.hashCode();
```

```
        p.toString();
```

```
    }
```

```
}
```

```
-----  
package com.ravi.java_9_feature;
```

```
interface Moveable
```

```
{
```

```
    public String toString();
```

```
    public int hashCode();
```

```
    public boolean equals(Object obj);
```

```
}
```

```
public class InteraceMember1 {
```

```
    public static void main(String[] args) {
```

```
        // TODO Auto-generated method stub
```

```
    }
```

```
}
```

```
-----  
package com.ravi.java_9_feature;
```

```
@FunctionalInterface
```

```
interface Worker
```

```
{
```

```
    void work();
```

```
    public String toString();
```

```
    public int hashCode();
```

```
    public boolean equals(Object obj);
```

```
}
```

```
public class FunctionalInterfaceDemo {
```

```
    public static void main(String[] args) {
```

```
        // TODO Auto-generated method stub
```

```
    }
```



```
}
```

*What is marker interface in java ?

An interface which does not contain any method and field is called marker interface. In other words, an empty interface is known as marker interface or tag interface.

*It describes run-time type information about objects, so the JVM have additional information about the object. [like object is clonable OR object is serializable OR Object is RandomAccessable]

Example :

public interface Drawable //Marker interface
{
}

Note :-In java we have Clonable, Serializable and RandomAccess are predefined marker interface.

```
interface Serializable //Marker interface  
{  
  
}
```

```
class Customer implements Serializable  
{  
  
}
```

```
Serializable c = new Customer();
```

```
if(c instanceof Customer)  
{  
    Now perform serialization  
}
```

****What is difference between abstract class and interface ?

The following are the differences between abstract class and interface.

- 1) An abstract class can contain instance variables but interface variables are by default public , static and final.
- 2) An abstract class can have state (properties) of an object but interface can't have state of an object.
- 3) An abstract class can contain constructor but inside an interface we can't define constructor.
- 4) An abstract class can contain instance and static blocks but inside an interface we can't define any blocks.
- 5) Abstract class can't refer Lambda expression but using Functional interface we can refer Lambda

Expression.

6) By using abstract class multiple inheritance is not possible but by using interface we can achieve multiple inheritance.

-----OOPs completed-----

Exception Handling
I/O and file Handling
enum
Inner classes
Multithreading
Collection Framework

Exception Handling :

What is an exception ?

An exception is a runtime error.

An exception is an abnormal situation or un-expected situation in a normal execution flow.

An exception encounter due to dependency, if one part of the program is dependent to another part then there might be a chance of getting Exception.

AN EXCEPTION ALSO ENCOUNTER DUE TO WRONG INPUT GIVEN BY THE USER.

Exception Hierarchy :

Available in the Diagram (Exception_Hierarchy)

Different Criteria of Exception :

1) java.lang.ArithmeticException

Whenever we divide a number by zero (an int value) then we will get
java.lang.ArithmeticException

Example :

```
int x = 100;  
int y = 0;  
int z = x/y; //java.lang.ArithmeticException
```

2) java.lang.NumberFormatException

If we try to convert a string value into corresponding numeric type
like int, float, double and so on, if the number is not in a proper format then we will get
java.lang.NumberFormatException

```
String str = "NIT";  
int no = Integer.parseInt(str); //java.lang.NumberFormatException
```

3) java.lang.NullPointerException

if we try to call non static method on null reference then we will get java.lang.NullPointerException

```
String str = null;  
System.out.println(str.length()); //java.lang.NullPointerException
```

4) java.lang.NegativeArraySizeException

At the time of creating array we can't pass negative size
int []arr = new int[-2]; //java.lang.NegativeArraySizeException

5) java.lang.ArrayIndexOutOfBoundsException

Whenever we try to access the index of an array but if the index is out of the range then we will get

```
int []arr = {10, 20,30};  
System.out.println(arr[3]); //java.lang.ArrayIndexOutOfBoundsException
```

6) java.util.InputMismatchException

Whenever we try to put an input but if the input is not in a proper format then we will get java.util.InputMismatchException

```
Scanner sc = new Scanner(System.in);  
System.out.println("Enter employee Id :");  
int id = sc.nextInt(); //Here we are passing character value then  
we will get java.util.InputMismatchException
```

Important points to remember :

1) As a developer we are responsible to handle the Exception. System admin is responsible to handle the error because we cannot recover from error.

2) Exception is the super class of all the exceptions we have in java

What is the format of Exception, if we print any exception object ?

The default format of any exception is as follows :

package name + class name [java.lang.ArithmeticException]

We can also pass errorMessage to the exception classes in String format to display the error message and the format is :

Package name + class name : error Message

WAP in java to show that Exception is the super class of all the Exceptions we have in java .

```
package com.ravi.exception;
```

```
public class ExceptionDemo {
```

```
    public static void main(String[] args)
```

```

{
Exception e1 = new ArithmeticException("Divide by zero");

Exception e2 = new ArithmeticException();

System.out.println(e1); //java.lang.AE:Divide by zero
System.out.println(e2); //java.lang.ArithmeticException

}

}

```

WAP in java that describes whenever an exception is encounter then our program will halt in the middle

```

package com.ravi.exception;

import java.util.Scanner;

public class ExceptionExample {

    public static void main(String[] args)
    {
        System.out.println("Main method started");

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the value of x :");
        int x = sc.nextInt();

        System.out.print("Enter the value of y :");
        int y = sc.nextInt();

        int result = x/y;
        System.out.println("Result is :"+result);

        System.out.println("Main method ended");
        sc.close();

    }

}

```

Note : Here we put the value of y as 0 (zero) then we will get java.lang.ArithmeticException and our program will halt in the middle which is known as Abnormal termination, Here JVM has default exception handler which will handle the exception and provide the message and terminate the program in the middle.

10-06-2024

In to work and handle the Exception, Java software people has provided the following keywords :

- 1) try block
- 2) catch block
- 3) finally block (try with resources 1.7)

- 4) throw
- 5) throws

Key points to remember :

- > With try block we can write either catch block or finally block or both.
- > In between try and catch we can't write any kind of statement.
- > try block will trace our program line by line.
- > If we have any exception inside the try block, With the help of JVM, try block will automatically create the appropriate Exception object and then throw the Exception Object to the nearest catch block.
- > In the try block whenever we get an exception the control will directly jump to the nearest catch block and the remaining code of try block will not be executed.
- > catch block is responsible to handle the exception.
- > catch block will only execute if there is an exception inside try block.

try block :

Whenever our statement is error suspecting statement OR Risky statement then we should write that statement inside the try block.

try block must be followed either by catch block or finally block or both.

*try block is responsible to trace our code line by line, if any exception encounter then with the help of JVM, TRY BLOCK WILL CREATE APPROPRIATE EXECPTION OBJECT, AND THROW THIS EXCEPTION OBJECT to the nearest catch block.

After the execption in the try block, the remaining code of try block will not be executed because control will directly transfer to the catch block.

In between try and catch block we cannot write any kind of statement.

catch block :

The main purpose of catch block to handle the exception which is thrown by try block.

catch block will only executed if there is an exception in the try block.

WAP that describes if we use Exception handling mechanism then our program will not be terminated abnormally.

```
package com.ravi.exception;

import java.util.Scanner;

public class TryDemo
{
    public static void main(String[] args)
    {
        System.out.println("Main method started");

        Scanner sc = new Scanner(System.in);
        try
        {
            System.out.print("Enter the value of x :");
            int x = sc.nextInt();
```

```

System.out.print("Enter the value of y :");
int y = sc.nextInt();

int result = x/y;
System.out.println("Result is :"+result);
System.out.println("End of try block");
}
catch(Exception e)
{
    System.out.println("Inside catch block");
    System.err.println(e);
}

System.out.println("Main method ended");
sc.close();

}

}

```

Note :- The above program is in the protection of try-catch so, even we have an exception (y=0) but program will be terminated normally.

```

-----
package com.ravi.exception;

public class ExceptionDemo {

    public static void main(String[] args)
    {
        try
        {
            //System.out.println(10/0);
            throw new ArithmeticException("Divide By zero");
        }
        catch(Exception e)
        {
            System.err.println(e);
        }
        System.out.println("Journey Completed");
    }

}

```

The above program describes how to create and throw an exception object explicitly. try block is doing the same implicitly.

The main purpose of Exception Handling to provide user-friendly message to the client.

```

package com.ravi.exception;

import java.util.Scanner;

public class CustomerDemo {

```

```

public static void main(String[] args)
{
    System.out.println("Hello Client, Welcome to my application.");

    Scanner sc = new Scanner(System.in);

    try
    {
        System.out.print("Please enetr the value of x :");
        int x = sc.nextInt();

        System.out.print("Please enetr the value of y :");
        int y = sc.nextInt();

        int result = x /y;
        System.out.println("Result is :"+result);

    }
    catch(Exception e)
    {
        System.err.println("Please don't put zero here, Try again");
    }
    System.out.println("Thank you 4 visting my application");
    sc.close();
}
}

```

 Throwable class method :

Throwable class has provided the following three methods :

- 1) public String getMessage() :- It will provide only error message.
- 2) public void printStackTrace() :- It will provide the complete details regarding exception like exception class name, exception error message, exception class location, exception method name and exception line number.
- 3) public String toString() :- It will convert the exception into String representation.

 package com.ravi.basic;

```

public class PrintStackTrace
{
    public static void main(String[] args)
    {
        System.out.println("Main method started...");
        try
        {
            String x = "Ravi";
            int y = Integer.parseInt(x);
            System.out.println(y);
        }
    }
}

```

```

catch(Exception e)
{
    e.printStackTrace(); //For complete Exception details
    System.out.println("-----");
    System.out.println(".....");
    System.out.println(e.getMessage()); //only for Exception message
    System.out.println(".....");
    System.out.println(e.toString());
}
System.out.println("Main method ended...");

}

}

```

11-06-2024

Working with Specific Exception :

While working with exception, in the corresponding catch block we can take Exception (super class) which can handle any type of Exception.

On the other hand we can also take specific type of exception (ArithmeticException, NullPointerException and so on) which will handle only one type i.e specific type of exception.

```

package com.ravi.basic;

import java.util.InputMismatchException;
import java.util.Scanner;

public class SpecificException
{
    public static void main(String[] args)
    {
        System.out.println("Main started");

        Scanner sc = new Scanner(System.in);

        try
        {
            System.out.print("Enter your Roll :");
            int roll = sc.nextInt();
            System.out.println("Your Roll is :"+roll);
        }
        catch(InputMismatchException e) //Specific Type
        {
            System.err.println("Input is not in proper format");
        }
        sc.close();
        System.out.println("Main ended");
    }
}

```

```

package com.ravi.basic;

```



```

public class SpecificException1
{
    public static void main(String[] args)
    {
        try
        {
            throw new OutOfMemoryError();
        }
        catch(Error e) //Specific type
        {
            System.out.println("Inside catch Block");
            System.err.println(e);
        }
        System.out.println("Main Method Completed");
    }
}

```

Working with Infinity and Not a number(NaN) :

10/0 -> Infinity (Java.lang.ArithmeticException)
 10/0.0 -> Infinity (POITIVE_INFINITY)

0/0 -> Undefined (Java.lang.ArithmeticException)
 0/0.0 -> Undefined (NaN)

While dividing a number with Integral literal in both the cases i.e Infinity (10/0) and Undefined (0/0) we will get java.lang.ArithmeticException because java software people has not provided any final, static variable support to deal with Infinity and Undefined.

On the other hand while dividing a number with with floating point literal in the both cases i.e Infinity (10/0.0) and Undefined (0/0.0) we have final, static variable support so the program will not be terminated in the middle which are as follows

10/0.0 = POSITIVE_INFINITY
 -10/0.0 = NEGATIVE_INFINITY
 0/0.0 = NaN

```

package com.ravi.basic;

```

```

public class InfinityFloatingPoint
{
    public static void main(String[] args)
    {
        System.out.println("Main Started");
        System.out.println(10/0.0);
        System.out.println(-10/0.0);
        System.out.println(0/0.0);
        System.out.println(10/0);
        System.out.println("Main Ended");
    }
}

```

Working with multiple try catch :

According to our application requirement we can provide multiple try-catch in my application to work with multiple exceptions.

```
package com.ravi.basic;
public class MultipleTryCatch
{
    public static void main(String[] args)
    {
        System.out.println("Main method started!!!!");

        try
        {
            int arr[] = {10,20,30};
            System.out.println(arr[3]);
        }
        catch(ArrayIndexOutOfBoundsException e)
        {
            System.err.println("Array index is out of limit!!!");
        }

        try
        {
            String str = null;
            System.out.println(str.length());
        }
        catch(NullPointerException e)
        {
            System.err.println("ref variable is pointing to null");
        }

        System.out.println("Main method ended!!!!");
    }
}
```

Here User will get all the Exception messages at a time so it is not a recommended way, to avoid this we introduced a new concept try with multiple catch blocks.

* Single try with multiple catch block :

According to industry standard we should write try with multiple catch blocks so we can provide proper information for each and every exception to the end user.

While working with multiple catch block always the super class catch block must be last catch block.

From java 1.7v this multiple exceptions we can write in a single catch block by using | symbol.

If try block is having more than one exception then always try block will handle only first exception because control will transfer to the nearest catch block.

```
package com.ravi.basic;
public class MultyCatch
{
```

```

public static void main(String[] args)
{
    System.out.println("Main Started...");
    try
    {
        int c = 10/0;
        System.out.println("c value is :"+c);

        int []x = {12,78,56};
        System.out.println(x[3]);
    }

    catch(ArrayIndexOutOfBoundsException e1)
    {
        System.err.println("Array is out of limit...");
    }
    catch(ArithmeticException e1)
    {
        System.err.println("Divide By zero problem...");
    }
    catch(Exception e1)
    {
        System.out.println("General");
    }

    System.out.println("Main Ended...");
}
}

```

```

package com.ravi.basic;

```

```

public class MultyCatch1
{
    public static void main(String[] args)
    {
        System.out.println("Main method started!!!");
        try
        {
            String str1 = null;
            System.out.println(str1.toUpperCase());

            String str2 = "Ravi";
            int x = Integer.parseInt(str2);
            System.out.println("Number is :"+x);
        }
        catch(NumberFormatException | NullPointerException e)
        {
            e.printStackTrace();
        }

        System.out.println("Main method ended!!!");
    }
}

```

finally block [100% Guaranteed for Execution]

finally is a block which is meant for Resource handling purposes.

According to Software Engineering, the resources are memory creation, buffer creation, opening of a database, working with files, working with network resources and so on.

Whenever the control will enter inside the try block always the finally block would be executed.

We should write all the closing statements inside the finally block because irrespective of exception finally block will be executed every time.

If we use the combination of try and finally then only the resources will be handled but not the exception, on the other hand if we use try-catch and finally then exception and resources both will be handled.

```
package com.ravi.basic;

public class FinallyBlock
{
    public static void main(String[] args)
    {
        System.out.println("Main method started");

        try
        {
            System.out.println(10/0);
        }

        finally
        {
            System.out.println("Finally Block");
        }

        System.out.println("Main method ended");
    }
}
```

Note :- In the above program finally block will be executed, even we have an exception in the try block but here only the resources will be handled but not the exception.

```
package com.ravi.basic;

public class FinallyWithCatch
{
    public static void main(String[] args)
    {

        try
        {
            int []x = new int[-2]; //We can't pass negative size of an array in negative
            x[0] = 12;
            x[1] = 15;
            System.out.println(x[0]+" : "+x[1]);
        }
    }
}
```

```

}
catch(NegativeArraySizeException e)
{
    System.err.println("Array Size is in negative value...");
}
finally
{
    System.out.println("Resources will be handled here!!!");
}
System.out.println("Main method ended!!!");
}
}

```

In the above program exception and resources both are handled because we have a combination of try-catch and finally.

Note :- In the try block if we write `System.exit(0)` and if this line is executed then finally block will not be executed.

12-06-2024

Limitation of finally Block :

The following are the limitation of finally block :

- 1) In order to close the resources, user is responsible to write finally block manually.
- 2) Due to finally block the length of the program will be increased.
- 3) In order to close the resources inside the finally block, we need to declare the resources outside of try block.

```

package com.ravi.exception;

import java.util.InputMismatchException;
import java.util.Scanner;

public class FinallyBlockLiomitation
{
    public static void main(String[] args)
    {
        Scanner sc = null;
        try
        {
            sc = new Scanner(System.in);
            System.out.print("Enter your Roll :");
            int roll = sc.nextInt();
            System.out.println("Your Roll number is :"+roll);
        }
        catch(InputMismatchException e)
        {
            System.out.println("Inside Catch");
            System.err.println("Input is not in a proper format");
        }
        finally

```

```

{
    System.out.println("Inside finally");
    sc.close();
}

}

}

```

To avoid the above said problem, java software people introduced automatic closing facility from java 1.7v using try with resources.

try with resources :

To avoid all the limitation of finally block, Java software people introduced a separate concept i.e try with resources from java 1.7 onwards.

Case 1:

```

try(resource1 ; resource2) //Only the resources will be handled
{
}

```

Case 2 :

```

//Resources and Exception both will be handled
try(resource1 ; resource2)
{
}
catch(Exception e)
{
}

```

Case 3 :

try with resources enhancement from java 9v

```

Resource r1 = new Resource();
Resource r2 = new Resource();

```

```

try(r1; r2)
{
}
catch(Exception e)
{
}

```

There is a predefined interface available in java.lang package called AutoCloseable which contains predefined abstract method i.e close() which throws Exception.

There is another predefined interface available in java.io package called Closeable, this Closeable interface is the sub interface for AutoCloseable interface.

```

public interface java.lang.AutoCloseable
{
    public abstract void close() throws Exception;
}
public interface java.io.Closeable extends java.lang.AutoCloseable
{
    void close() throws IOException;
}

```

Whenever we pass any resource class as part of try with resources then that class must implements either Closeable or AutoCloseable interface so, try with resources will automatically call the respective class close() method even an exception is encountered in the try block.

```

ResourceClass rc = new ResourceClass()
try(rc)
{
}
catch(Exception e)
{
}
}

```

//This ResourceClass must implements either Closeable or AutoCloseable interface so, try block will automatically call the close() method.

The following program explains how try block is invoking the close() method available in DatabaseResource class and FileResource class.

3 files :

DatabaseResource.java

```

-----
package com.ravi.try_resource;

public class DatabaseResource implements AutoCloseable
{
    @Override
    public void close() throws Exception
    {
        System.out.println("Database Resource Closed");
    }
}

```

FileResource.java

```

-----
package com.ravi.try_resource;

import java.io.Closeable;
import java.io.IOException;

public class FileResource implements Closeable
{
    @Override

```

```

public void close() throws IOException
{
    System.out.println("File resource closed");
}

}

```

Main.java

```

-----
package com.ravi.try_resource;

public class Main {

    public static void main(String[] args) throws Exception
    {
        DatabaseResource dr = new DatabaseResource();
        FileResource fr = new FileResource();

        try(dr; fr)
        {
            System.out.println(10/0);
        }
        catch(ArithmeticException e)
        {
            System.err.println("Don't divide a number by zero ");
        }

        System.out.println("Main method completed");
    }
}

```

//code to close Scanner class automatically using try with resources

```

package com.ravi.try_resource;

import java.util.InputMismatchException;
import java.util.Scanner;

public class TryWithResource
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        try(sc)
        {
            System.out.println("Enter your Employee code :");
            int code = sc.nextInt();
            System.out.println("Your Employee code is :"+code);
        }
        catch(InputMismatchException e)
        {
            System.err.println("Input is not in a proper format");
        }
    }
}

```



```
System.out.println("Main method completed");
}
```

```
}
```

Note :- Scanner class internally implementing Closeable interface so it is providing auto closing facility, as a user we need to pass the reference of Scanner class inside try with resources try()

Nested try block :

If we write a try block inside another try block then it is called Nested try block.

```
try //Outer try
{
    statement1;
    try //Inner try
    {
        statement2;
    }
    catch(Exception e) //Inner catch
    {
    }
}
catch(Exception e) //Outer Catch
{
}
```

The execution of inner try block depends upon outer try block that means if we have an exception in the Outer try block then inner try block will not be executed.

```
package com.ravi.basic;

public class NestedTryBlock
{
    public static void main(String[] args)
    {
        try //outer try
        {
            String x = null;
            System.out.println("It's length is :"+x.length());

            try //inner try
            {
                String y = "NIT";
                int z = Integer.parseInt(y);
                System.out.println("z value is :"+z);
            }
            catch(NumberFormatException e)
            {
                System.err.println("Number is not in a proper format");
            }
        }
        catch(NullPointerException e)
        {
        }
    }
}
```

```

        System.err.println("Null pointer Problem");
    }
}
}

```

Writing try-catch inside catch block :

We can write try-catch inside catch block but this try-catch block will be executed if the catch block will be executed that means if we have an exception in the try block.

```

package com.ravi.basic;

import java.util.InputMismatchException;
import java.util.Scanner;

public class TryWithCatchInsideCatch
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        try(sc )
        {
            System.out.print("Enter your Roll number :");
            int roll = sc.nextInt();
            System.out.println("Your Roll is :"+roll);
        }
        catch(InputMismatchException e)
        {
            System.err.println("Provide Valid input!!");
        }

        try
        {
            System.out.println(10/0);
        }
        catch(ArithmeticException e1)
        {
            System.err.println("Divide by zero problem");
        }
    }
}

```

try-catch with return statement

If we write try-catch block inside a method and that method is returning some value then we should write return statement in both the places i.e inside the try block as well as inside the catch block.

We can also write return statement inside the finally block only, if the finally block is present. After this return statement we cannot write any kind of statement. (Unreachable)

Always finally block return statement having more priority then try-catch return statement.

```
-----  
package com.ravi.advanced;  
  
public class ReturnExample1 {  
  
    public static void main(String[] args)  
    {  
        System.out.println(m1());  
    }  
  
    @SuppressWarnings("finally")  
    public static int m1()  
    {  
        try  
        {  
            System.out.println("Inside try");  
            return 100;  
        }  
        catch(Exception e)  
        {  
            System.out.println("Inside Catch");  
            return 200;  
        }  
        finally  
        {  
            System.out.println("Inside finally");  
            return 300;  
        }  
  
        // System.out.println("....");  Unreachable line  
    }  
}
```

We have return keyword in finally block so we cannot write any kind of statement after finally.

```
-----  
package com.ravi.advanced;  
public class ReturnExample  
{  
    public static void main(String[] args)  
    {  
        System.out.println(methodReturningValue());  
    }  
  
    public static int methodReturningValue()  
    {  
        try  
        {  
            System.out.println("Try block");  
            return 10/0;  
        }  
        catch (Exception e)  
        {  
            System.out.println("catch block");  
            return 20; //return statement is compulsory  
        }  
    }  
}
```

```
}
```

```
}
```

```
}
```

13-06-2024

Initialization of a variable in try and catch :

A local variable must be initialized inside try block as well as catch block OR at the time of declaration.

If we initialize inside the try block only then from catch block we cannot access local variable value, Here initialization is compulsory inside catch block.

```
package com.ravi.exception;
```

```
public class VariableInitialization {
```

```
    public static void main(String[] args)
```

```
    {  
        int i;
```

```
        try
```

```
        {  
            i = 100;  
            System.out.println(i);
```

```
        }  
        catch(Exception e)
```

```
        {  
            i = 200;  
            System.out.println(i);  
        }
```

```
    }
```

```
}
```

**Difference between Checked Exception and Unchecked Exception :

Checked Exception :

In java some exceptions are very common exceptions are called Checked exception here compiler takes very much care and wanted the clarity regarding the exception by saying that, by using this code you may face some problem at runtime and you did not report me how would you handle this situation at runtime are called Checked exception, so provide either try-catch or declare the method as throws.

All the checked exceptions are directly sub class of java.lang.Exception

Eg:

FileNotFoundException, IOException, InterruptedException, ClassNotFoundException, SQLException, CloneNotSupportedException, EOFException and so on

Unchecked Exception :-

The exceptions which are rarely occurred in java and for these kinds of exception compiler does not take any care are called unchecked exception.

Unchecked exceptions are directly entertained by JVM because they are rarely occurred in java.

All the un-checked exceptions are sub class of RuntimeException

RuntimeException is also Unchecked Exception.

Eg:

ArithmeticException, ArrayIndexOutOfBoundsException, NullPointerException, NumberFormatException, ClassCastException, ArrayStoreException and so on.

Some Bullet points regarding Checked and Unchecked :

Checked Exception :

- 1) Common Exception
- 2) Compiler takes care (Will not compile the code)
- 3) Handling is compulsory (try-catch OR throws)
- 4) Directly the sub class of java.lang.Exception

Unchecked Exception :

- 1) Rare Exception
- 2) Compiler will not take any care
- 3) Handling is not Compulsory
- 4) Sub class of RuntimeException

When to provide try-catch or declare the method as throws :-

We should provide try-catch if we want to handle the exception by own as well as if we want to provide user-defined messages to the client but on the other hand we should declare the method as throws when we are not interested to handle the exception and try to send it to the JVM for handling purpose.

Note :- It is always better to use try catch so we can provide appropriate user defined messages to our client.

*Why compiler takes very much care regarding the checked Exception ?

As we know Checked Exceptions are very common exception so in case of checked exception "handling is compulsory" because checked Exception depends upon other resources as shown below.

IOException (we are depending upon System Keyboard OR Files)

FileNotFoundException(We are depending upon the file)

InterruptedException (Thread related problem)

ClassNotFoundException (class related problem)

SQLException (SQL related or database related problem)

CloneNotSupportedException (Object is the resource)

EOFException(We are depending upon the file)

* What is the difference between throw and throws :

throw :

try block is responsible to create the exception object and throw the exception object to the nearest catch block but it works with predefined exception implicitly.

If a user wants to throw an exception based on his own requirement and specification by using userdefined exception then we should write throw keyword to throw the exception object explicitly. (throw new InvalidMarksException())

THROWING THE EXCEPTION OBJECT EXPLICITLY.

throws :-

In case of checked Exception if a user is not interested to handle the exception and wants to throw the exception to JVM, wants to skip from the current situation then we should declare the method as throws. It is mainly used to work with Checked Exception.

14-06-2024

Types of exception in java :

Exception can be divided into two types :

- 1) Predefined Exception OR Built-in Exception
- 2) Userdefined Exception OR Custom Exception

Predefined Exception :-

The Exceptions which are already defined by Java software people for some specific purposes are called predefined Exception or Built-in exception.

Ex :

IOException, ArithmeticException and so on

Userdefined Exception :-

The exceptions which are defined by user according to their own use and requirement are called User-defined Exception.

Ex:-

InvalidAgeException, GreaterMarksException

Steps to develop userdefined Exception :

In order to develop user-defined Exception, we need to follow the following steps :

- 1) Our userdefined exception class must be extended either from java.lang.Exception (In case of Checked Exception) OR java.lang.RuntimeException (In case of un-checked Exception)

Example of Checked Exception

public class InvalidAgeException extends Exception

```
{  
}
```

Example of Un-Checked Exception

```
public class GreaterMarksException extends RuntimeException  
{  
}
```

- 2) We should define No Argumenet (No error message) and parameterized constructor (In case, we want to pass some errorMessage at the time of creating the object) in userdefined Exception class.
- 3) In parameterized constructor we should use super() to pass the errorMessage in the super class.
- 4) In order to throw the exception object we need to use throw keyword which will throw the exception object explicitly.

Program to Develop checked Exception (Hanling is compulsory)

```
-----  
package com.ravi.exception;  
  
import java.util.Scanner;  
  
@SuppressWarnings("serial")  
class InvalidAgeException extends Exception //Checked  
{  
    public InvalidAgeException()  
    {  
  
    }  
    public InvalidAgeException(String errorMessage)  
    {  
        super(errorMessage);  
    }  
}
```

```
public class UserDefinedChecked {  
  
    public static void main(String[] args)  
    {  
        System.out.println("Main method started");  
        Scanner sc = new Scanner(System.in);  
  
        try(sc)  
        {  
            System.out.print("Enter your Age :");  
            int age = sc.nextInt();  
  
            if(age<18)  
            {
```

```

        throw new InvalidAgeException("Age is invalid");
    }
    else
    {
        System.out.println("Welcome to vote!!!");
    }
}
catch(InvalidAgeException e)
{
    System.err.println(e); //com.ravi.exception.InvalidAgeException
}
System.out.println("Main method ended");

}

}
-----
//Program on Unchecked Exception

package com.ravi.exception;

import java.util.Scanner;

@SuppressWarnings("serial")
class GreaterMarksException extends RuntimeException //Un-checked
{
    public GreaterMarksException()
    {
    }

    public GreaterMarksException(String errorMessage)
    {
        super(errorMessage);
    }

}

public class CustomUnchecked {

    public static void main(String[] args)
    {
        System.out.println("Main started");

        Scanner sc = new Scanner(System.in);

        try(sc)
        {
            System.out.print("Enter your Marks :");
            int marks = sc.nextInt();

            if(marks > 100)
            {
                throw new GreaterMarksException("Invalid Marks");
            }

```



```

else
{
    System.out.println("Your Marks is :"+marks);
}
}
catch(GreaterMarksException e)
{
    System.err.println(e.getMessage());
    System.out.println(".....");
    e.printStackTrace();
}
System.out.println("Main ended");
}
}

```

Note :- If we want to throw any exception explicitly by using throw keyword then that exception class must be sub class of Throwable either directly or in-directly.

```

class Ravi extends Throwable [Here Throwable OR Exception OR
{
    RuntimeException
}

```

```

public class Test
{
    public static void main(String[] args) throws Ravi
    {
        throw new Ravi();
    }
}

```

Some important rules while working with Checked Exception :

a) If the try block does not throw any checked exception then in the corresponding catch block we can't handle checked exception. It will generate compilation error i.e "exception never thrown from the corresponding try statement"

Example :-

```

public class Test
{
    public static void main(String[] args)
    {
        try
        {
            //try block is not throwing checked exception
            //i.e. InterruptedException
        }
        catch (InterruptedException e) //error
        {
        }
    }
}

```

```
}
```

```
}
```

Note :- The above rule is not applicable for Unchecked Exception

```
    try
    {

    }
    catch(ArithmeticException e) //Valid
    {
        e.printStackTrace();
    }
}
-----
```

b) If the try block does not throw any exception then in the corresponding catch block we can write Exception, Throwable because both are the super classes for all types of Exception whether it is checked or unchecked.

```
package com.ravi.method_related_rule;

public class CatchingWithSuperClass
{
    public static void main(String[] args)
    {

        try
        {
            //try block, not throwing any checked exception
        }
        catch(Exception e) //Here Throwable and Exception
        {
            both are valid
            e.printStackTrace();
        }

    }

}
}
-----
```

c) At the time of method overriding if the super class method does not reporting or throwing checked exception then the overridden method of sub class not allowed to throw checked exception. otherwise it will generate compilation error but overridden method can throw Unchecked Exception.

```
package com.ravi.method_related_rule;

import java.io.IOException;

class Super
{
    public void show()
    {
        System.out.println("Super class method not throwing checked Exception");
    }
}
```

```

}
class Sub extends Super
{
    @Override
    public void show() throws IOException [error]
    {
        System.out.println("Sub class method should not throw checked Exception");
    }
}

```

```

public class MethodOverridingWithChecked {

```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub

```

```

    }

```

```

}

```

d) If the super class method declare with throws keyword to throw a checked exception, then at the time of method overriding, sub class method may or may not use throws keyword.

If the Overridden method is also using throws

keyword to throw checked exception then it must be either same exception class or sub class, it should not be super class as well as we can't add more exceptions in the overridden method.

```

package com.ravi.method_related_rule;

```

```

import java.io.FileNotFoundException;
import java.io.IOException;

```

```

class Base
{
    public void show() throws FileNotFoundException
    {
        System.out.println("Super class method ");
    }
}

```

```

class Derived extends Base
{
    public void show() throws IOException //error
    {
        System.out.println("Sub class method ");
    }
}

```

```

public class MethodOverridingWithThrows

```

```

{
    public static void main(String[] args)
    {
        System.out.println("Overridden method may or may not throw checked exception but if it is throwing
then must be same or sub class");
    }
}

```

```

}

```

Here we will get compilation error because IOException is the super class of FileNotFoundException

e) Just like return keyword we can't use throw keyword inside static and non static block because all initializers must be executed normally.

```
public class Test
{

    {
        throw new ArithmeticException(); //error
    }

}
```

```
public static void main(String[] args)
{

}

}
}
-----
```

```
public class Test
{
    static
    {
        try
        {
            throw new Exception("My Error Message");
        }
        catch(Exception e)
        {
            System.err.println("Inside Catch block");
            System.err.println(e);
        }
    }
}
```

```
public static void main(String[] args)
{

}

}
```

This code is valid.

17-06-2024

Exception propagation :- [Exception object will shift from callee to caller]

Whenever we call a method and if the the callee method contains any kind of exception and if callee method doesn't contain any kind of exception handling mechanism (try-catch) then JVM will propagate the exception to caller method for handling purpose. This is called Exception Propagation.

If the caller method also does not contain any exception handling mechanism then JVM will terminate the method from the stack frame hence the remaining part of the method(m1 method) will not be executed even if we handle the exception in another caller method like main.

If any of the the caller method does not contain any exception handling mechanism then exception will be handled by JVM, JVM has default exception handler which will provide the exception message and terminates the program abnormally. [17-JUN]

```
package com.ravi.exception;

public class ExceptionPropagation
{
    public static void main(String[] args)
    {
        System.out.println("Main method started!!!!");
        try
        {
            m1();
        }
        catch(ArithmeticException e)
        {
            System.err.println("Handled in main");
        }
        System.out.println("Main method ended!!!!");
    }
    public static void m1()
    {
        System.out.println("M1 method started!!!!");
        m2();
        System.out.println("M1 method ended!!!!"); //This line not executed
    }

    public static void m2()
    {
        System.out.println(10/0);
    }
}
```

In the above program exception is hamdled by main method so m2 and m1 both the methods are terminated by JVM so the remaining part of m1() method will not be executed, even we handled the exception in main method.

Handling Exception while calling a method ?

Whenever we are calling a method and if that method is throwing any checked Exception by using throws keyword then at the time of calling caller method must be protected either by try-catch or declare the method as throws

```
package com.ravi.exception;

public class MethodCalling {

    public static void main(String[] args)
    {
        try
```

```

{
    accept(); //At the time of calling, try-catch
            or throws required
}
catch(InterruptedException e)
{

}
}
}

public static void accept() throws InterruptedException
{

}
}

```

The same rule is not applicable for Unchecked Exception

```

public class MethodCalling {

    public static void main(String[] args)
    {
        accept(); //valid
    }

    public static void accept() throws ArithmeticException
    {

    }

}

```

In the below program method is throwing Exception OR Throwable so here also protection is required, either try-catch or declare the method as throws because by using Exception or Throwable we can represent checked and Unchecked both the Exception.

```

public class MethodCalling
{

    public static void main(String[] args)
    {
        accept(); //error
    }

    public static void accept() throws Exception
    {

    }

}

```

Input Output in Java :

Java software people has provided input and output concept by using java.io package.

By using this package we can provide various input and output operation like reading the data from the

keyboard, creating a file, reading and writing the data to the file, reading and writing the primitive data, object into the file.

Reading the data from the keyboard :

As we know, Scanner class is available from JDK 1.5v onwards, before Scanner class we have the following two classes to read the data from the keyboard and these 2 classes are available in java.io package.

- 1) DataInputStream
- 2) BufferedReader

How to create the Object for these classes :

DataInputStream :

DataInputStream dis = new DataInputStream(System.in);

Note : readLine() method of DataInputStream class is deprecated now.

BufferedReader :

InputStreamReader isr = new InputStreamReader(System.in);
BufferedReader br = new BuffereedReader(isr);

OR

BufferedReader br = new BuffereedReader(new InputStreamReader(System.in));

Whenever we are reading the using BufferedReader then it provieds fast accessibility because we are reading the data through buffer, It is always recommended to read the data through buffer only.

Methods available in these classes :

-
- 1) public int read() : It is used to read a single character from the source and returns the ASCII/UNICODE value of the character. If the data is not available then It will return -1 (Which represents EOF).
 - 2) public String readLine() : It is used to read multiple characters or a complete line from the source.
The retun type is String.

//WAP to read name from the keyboard

```
import java.io.*;
public class ReadName
{
    public static void main(String[] args) throws IOException
    {
        DataInputStream d = new DataInputStream(System.in);

        System.out.println("ENTER YOUR NAME :");
```

```
String name = d.readLine();
System.out.println("Your Name is :"+name);
}
}
```

WAP to read your age from the keyboard :

```
import java.io.*;
public class ReadAge
{
    public static void main(String[] args)
    {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        try(br)
        {
            System.out.println("Enter your Age :");
            String ag = br.readLine();
            System.out.println("Your Age is :"+ag);
            int age = Integer.parseInt(ag);

            if(age>18)
            {
                System.out.println("Go for a Movie");
            }
            else
            {
                System.err.println("Try after some time");
            }
        }
        catch (IOException e)
        {
        }
    }
}
```

```
import java.io.*;
public class ReadSalary
{
    public static void main(String[] args)
    {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        try(br)
        {
            System.out.println("Enter your Salary :");
            String sal = br.readLine();
            float salary = Float.parseFloat(sal);
            System.out.println("Your Salary is :"+salary);

        }
        catch (IOException e)
        {
        }
    }
}
```



```

    }
}
}
-----
package com.ravi.io;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;

public class ReadGender {

    public static void main(String[] args)
    {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        try(br)
        {
            System.out.print("Enter your Gender :");
            char gen = (char) br.read();
            System.out.println("Your Gender is :"+gen);
        }
        catch(IOException e)
        {
            e.printStackTrace();
        }

    }

}
}
-----

```

18-06-2024

File Handling ?

What is the need of File Handling ?

As we know variables are used to store some meaningful value in our program but once the execution of the program is over, now we can't get those values so to hold those values permanently in our memory we use files.

Files are stored in the secondary storage devices so, we can use/read the data stored in the file anytime according to our requirement.

In order to work with File system java software people has provided number of predefined classes like File, FileInputStream, FileOutputStream and so on. All these classes are available in java.io package. We can read and write the data in the form of Stream.

Streams in java :

A Stream is nothing but flow of data or flow of characters to both the end.
Stream is divided into two categories

1) byte oriented Stream :-

It used to handle characters, images, audio and video file in binary format.

2) character oriented Stream :-

It is used to handle the data in the form of characters or text.

Now byte oriented or binary Stream can be categorized as "InputStream" and "OutputStream". input streams are used to read or receive the data where as output streams are used to write or send the data.

Again Character oriented Stream is divided into Reader and Writer. Reader is used to read() the data from the file where as Writer is used to write the data to the file.

All Streams are represented by classes in java.io package.

InputStream is the super class for all kind of input operation where as OutputStream is the super class for all kind of output Operation for byte oriented stream.

Where as Reader is the super class for all kind reading operation where as Writer is the super class for all kind of writing operation in character oriented Stream.

File :

It is a predefined class in java.io package through which we can create file and directory. By using this class we can verify whether the file is existing or not.

```
File f = new File("abc.txt");
```

The above statement will not create any file, It actually create the file object and perform one of the following two task.

- a) If abc.txt does not exist, It will not create it
- b) if abc.txt does exist, the new file object will be refer to the referenec variable f

Now if the file does not exist then to create the file and verify whether file is existing or not, we should use the following two methods :

1) public boolean exists() : Will verify whether file is existing or not and based on that, It will return true or false.

2) public boolean createNewFile() : Will Create a new file only if file is not available, if file is already available then return false.

File class has also a predefined method called getName(), to get the name of the file.

```
import java.io.*;
public class File0
{
    public static void main(String[] args)
    {
        try
        {
            File f = new File("C:\\new\\Batch32.txt");

            if(f.exists())
            {
```

```

        System.out.println("File is existing");
    }
    else
    {
        System.out.println("File is not existing");
    }

    if (f.createNewFile())
    {
        System.out.println("File created: " + f.getName());
    }
    else
    {
        System.out.println("File is already existing....");
    }
}
catch (IOException e)
{
    System.err.println(e);
}
}
}
}

```

Assignment :

Create a folder/directory by using File class.

FileOutputStream :

It is a predefined class available in java.io package.

It is used to create the file as well as write the data to the files.

It will override the existing file that means if file is already available then it will replace the file.

It is binary oriented Stream, so we can only write binary data to the files or in other words our data must be available in binary format.

How to convert the String data into binary format :

String class has provided a predefined method `getBytes()` through which we can convert String data into binary format.

```

public byte[] getBytes();

```

Example :

```

public class ConvertToBinary {

    public static void main(String[] args)
    {
        String str = "ABCDEF";

        byte[] b = str.getBytes();
    }
}

```

```

for(byte c : b)
{
    System.out.println(c); //65 66 67 68..70
}

}

}

-----
//Creating and writing the data to the file

import java.io.*;
public class File1
{
    public static void main(String args[]) throws IOException
    {
        var fout = new FileOutputStream("C:\\new\\Hyderabad.txt");
        try(fout)
        {
            String s = "Hyd is a popular IT City in India ";
            byte b[] = s.getBytes();

            fout.write(b);

            System.out.println("Success....");
        }
        catch(Exception e)
        {
            e.printStackTrace();
        }
    }
}

-----

```

19-06-2024

FileInputStream :

It is a predefined class available in java.io package. It is used to read the data from the existing file.

It is a binary stream.

If we want to print the file data in console then data must be available in char format.

Note :- Whenever we want to write the data in the file then data must be available in byte format where as If we want to print the data to the console then the data must be converted into char format.

```

-----
//Reading the data from the file
import java.io.*;
public class File2
{
    public static void main(String s[]) throws IOException
    {

```


a separate Stream called `SequenceInputStream`.

`SequenceInputStream` :

It is a predefined class available in `java.io` package. This class is used to read the data from two files at the same time using constructor.

`SequenceInputStream` class has a parameterized constructor which is accepting two parameters as shown below

```
var sis = new SequenceInputStream(InputStream i1, InputStream i2);
```

//Program to read the data from two files at the same time

```
import java.io.*;
public class File4
{
    public static void main(String args[]) throws IOException
    {
        var f1 = new FileInputStream("File1.java");
        var f2 = new FileInputStream("File2.java");

        var s = new SequenceInputStream(f1,f2);

        try(f1; f2; s)
        {
            int i;
            while(true)
            {
                i = s.read();
                if(i== -1)
                    break;
                System.out.print((char)i);
            }
        }
        catch(IOException e)
        {
            e.printStackTrace();
        }
    }
}
```

//Reading the data from two files and writing the data to a single file

```
import java.io.*;
public class File5
{
    public static void main(String x[]) throws IOException
    {
        var f1 = new FileInputStream("File3.java");
        var f2 = new FileInputStream("File4.java");

        var fout = new FileOutputStream("C:\\new\\FileData.txt");

        var s = new SequenceInputStream(f1,f2);

        int i;
```

```

try(f1; f2; fout; s)
{
    while(true)
    {
        i = s.read();
        if(i== -1)
            break;
        System.out.print((char)i);
        fout.write((byte)i);
    }
}
catch(IOException e)
{
    e.printStackTrace();
}
System.out.println("File Created Successfully");
}
}

```

Limitation of FileOutputStream :

By using this FileOutputStream class, we can write the data to single file only. If we want to write the data to multiple files at the same time then we should use a separate stream called ByteArrayOutputStream.

ByteArrayOutputStream :-

It is a predefined class available in java.io package. By using this class we can write the data to multiple files.

ByteArrayOutputStream class provides a method called writeTo(), through which we can write the data to multiple files.

//Program to write the data on multiple files.

```

import java.io.*;
public class File6
{
    public static void main(String args[]) throws IOException
    {
        var fin = new FileInputStream("File1.java");

        var f1 = new FileOutputStream("C:\\new\\a1.txt");
        var f2 = new FileOutputStream("C:\\new\\b1.txt");
        var f3 = new FileOutputStream("C:\\new\\c1.txt");

        var bout = new ByteArrayOutputStream();

        try(fin; f1; f2; f3; bout)
        {
            int i;
            while((i = fin.read()) != -1)
            {
                bout.write((byte)i); //writing the data to ByteArrayOutputStream
            }
        }
    }
}

```

```

        bout.writeTo(f1);
        bout.writeTo(f2);
        bout.writeTo(f3);

        bout.flush(); //clear the buffer for reusing of ByteArrayOutputStream
        System.out.println("Success");
    }
    catch(IOException e)
    {
        e.printStackTrace();
    }
}
}

```

```

//Working with images
import java.io.*;
public class File7
{
    public static void main(String[] args) throws IOException
    {
        var fin = new FileInputStream("C:\\new\\abc.jpg");

        var f1 = new FileOutputStream("C:\\new\\a1.jpg");
        var f2 = new FileOutputStream("C:\\new\\a2.jpg");
        var f3 = new FileOutputStream("C:\\new\\a3.jpg");

        var bout = new ByteArrayOutputStream();

        try(fin; f1; f2; f3; bout)
        {
            int i;
            while((i = fin.read()) != -1)
            {
                bout.write((byte)i);
            }

            bout.writeTo(f1);
            bout.writeTo(f2);
            bout.writeTo(f3);
            System.out.println("success...");
            bout.flush();
        }
        catch(IOException e)
        {
            e.printStackTrace();
        }
    }
}

```

BufferedOutputStream :-

It is a predefined class available in java.io package.

Whenever we use the class FileOutputStream the data will be available on the Stream but not in the

buffer so there may be chance of miss memory management, It is always preferable that we should perform read and write operation by using buffer.

By using this BufferedOutputStream now the data is in the buffer so the execution will become more faster.

//Program to put the data in the buffer for fast execution

```
import java.io.*;
class File8
{
    public static void main(String args[]) throws IOException
    {
        var fout = new FileOutputStream("C:\\new\\Hyderabad.txt");

        var bout = new BufferedOutputStream(fout);

        try(fout ; bout)
        {
            String s = "Hyderabad is a nice city";
            byte b[] = s.getBytes();
            bout.write(b);
            System.out.print("success...");
        }
        catch(IOException e)
        {
            e.printStackTrace();
        }
    }
}
```

BufferedInputStream :-

It is a predefined class available in java.io package.

Whenever we use FileInputStream to read the data/content from the file the data will be available on the Stream but not in the buffer so there may be a chance of miss memory management so we should take the data into the buffer by using BufferedInputStream class so overall the execution will become faster.

//BufferedInputStream

```
import java.io.*;
public class File9
{
    public static void main(String args[]) throws IOException
    {
        var fin = new FileInputStream("File1.java");
        var bin = new BufferedInputStream(fin);

        try(fin ; bin)
        {
            int i;
            while((i = bin.read()) != -1)
            {
                System.out.print((char)i);
            }
        }
    }
}
```

```

    catch(IOException e)
    {
        e.printStackTrace();
    }
    System.out.println();
}
}

```

20-06-2024

Writing and Reading the primitive data to the files :-

It is possible to write the primitive data(byte,short,int, long, float, double, char and boolean) to the file.

In order to write primitive data to the file we should use a predefined class available in java.io package called DataOutputStream.

```

var fos = new FileOutputStream("data.txt");
var dos = new DataOutputStream(fos);

```

now by using this we can write primitive data to the file.

It provides various methods like writeByte(), writeShort(), writeInt() and so on to write the primitive data to the file.

If we want to read the primitive data from the file we can use a predefined class available in java.io package called DataInputStream, this class provides various methods like readByte(), readShort(), readInt() and so on.

```

var fin = new FileInputStream("data.txt");
var dis = new DataInputStream(fin);

```

Note :- For writing String into a file we have writeBytes() and to read the String data from the file we have readLine() method.

[DataInputStream class readLine() method is deprecated now, so compilation warning]

Note :- The writing and reading format of primitive data should be same format otherwise we will get wrong format data.

```

//DataOutputStream and DataInputStream
import java.io.*;
public class File10
{
    public static void main(String args[]) throws IOException
    {
        var fout = new FileOutputStream("C:\\new\\Primitive.txt");
        var dout = new DataOutputStream(fout);

        try(fout ; dout)
        {
            dout.writeBoolean(true);
            dout.writeChar('A');

```

```

        dout.writeByte(Byte.MAX_VALUE);
        dout.writeShort(Short.MAX_VALUE);
        dout.writeInt(Integer.MAX_VALUE);
        dout.writeLong(Long.MAX_VALUE);
        dout.writeFloat(Float.MAX_VALUE);
        dout.writeDouble(Math.PI); //PI is a final static variable
    dout.writeBytes("Hello India...");
        dout.flush(); //For reuse purpose
    }
    catch(IOException e)
    {
        e.printStackTrace();
    }
}

```

```

System.out.println("Reading the Primitive data from the file!!!");

```

```

var fin = new FileInputStream("C:\\new\\Primitive.txt");
var din = new DataInputStream(fin);

```

```

try(fin ; din)
{
    boolean f = din.readBoolean();
    char c = din.readChar();
    byte b = din.readByte();
    short s = din.readShort();
    int i = din.readInt();
    long l = din.readLong();
    float ft = din.readFloat();
    double d = din.readDouble();
    String x = din.readLine(); //for reading String (deprecated)

    System.out.println(f + "\n" + c + "\n" + b + "\n" + s + "\n" + i + "\n" + l + "\n" + ft + "\n" + d + "\n" + x);
}
catch(IOException e)
{
    e.printStackTrace();
}

}
}

```

----- **** Serialization and De-serialization :** -----

It is a technique through which we can store the object data in a file. Storing the object data into a file is called Serialization on the other hand Reading the object data from a file is called De-serialization.

In order to perform serialization, a class must implements Serializable interface, predefined marker interface in java.io package.

Java.io package has also provided a predefined class called ObjectOutputStream to perform serialization i.e writing Object data to a file using writeObject() method.

```

public void writeObject(Object obj)

```

where as ObjectInputStream is also a predefined class available in java.io package through which we can read the Object data from a file using readObject(). The return type of readObject() is Object.

```
public Object readObject()
```

While reading the object data from the file, if the object is not available in the file then it will throw checked exception java.io.EOFException. (End of file Exception). It is a checked Exception.

3 files :

Employee.java

```
package com.ravi.serialization_deserialization;
```

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.Serializable;
import java.time.LocalDate;
```

```
@SuppressWarnings("serial")
public class Employee implements Serializable
{
    private int employeeId;
    private String employeeName;
    private double employeeSalary;
    private LocalDate hireDate; // HAS-A relation
```

```
    public Employee(int employeeId, String employeeName, double employeeSalary, LocalDate hireDate) {
        super();
        this.employeeId = employeeId;
        this.employeeName = employeeName;
        this.employeeSalary = employeeSalary;
        this.hireDate = hireDate;
    }
```

```
@Override
public String toString() {
    return "Employee [employeeId=" + employeeId + ", employeeName=" + employeeName + ",
employeeSalary="
        + employeeSalary + ", hireDate=" + hireDate + "]\n";
}
```

```
public static Employee getEmployeeObject() throws NumberFormatException, IOException
{
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
```

```
    System.out.print("Enter Employee Id :");
    int id = Integer.parseInt(br.readLine());
```

```
    System.out.print("Enter Employee Name :");
    String name = br.readLine();
```

```
System.out.print("Enter Employee Salary :");  
double salary = Double.parseDouble(br.readLine());
```

```
LocalDate date = LocalDate.now();
```

```
return new Employee(id, name, salary, date);  
}
```

```
}
```

StoreEmployeeObject.java

```
package com.ravi.serialization_deserialization;
```

```
import java.io.FileOutputStream;  
import java.io.IOException;  
import java.io.ObjectOutputStream;  
import java.util.Scanner;
```

```
public class StoreEmployeeObject {
```

```
public static void main(String[] args) throws NumberFormatException, IOException  
{  
    var fos = new FileOutputStream("C:\\new\\Employee.txt");  
    var oos = new ObjectOutputStream(fos);  
    var sc = new Scanner(System.in);
```

```
try(fos; oos; sc)  
{
```

```
System.out.print("How many objects U want to write :");  
int obj = sc.nextInt();
```

```
for(int i=1; i<=obj; i++)  
{  
    Employee emp = Employee.getEmployeeObject();  
    oos.writeObject(emp);  
}
```

```
}  
catch(Exception e)  
{  
    System.err.println("Exception raised :"+e);  
}  
System.out.println("Employee object data stored successfully");  
}
```

```
}
```

RetrieveEmployeeData.java

```
package com.ravi.serialization_deserialization;
```

```
import java.io.EOFException;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
```

```
public class RetrieveEmployeeData {

    public static void main(String[] args) throws IOException, ClassNotFoundException
    {
        var fin = new FileInputStream("C:\\new\\Employee.txt");
        var ois = new ObjectInputStream(fin);

        try(fin ; ois)
        {
            Employee emp = null;

            while((emp = (Employee) ois.readObject())!=null)
            {
                System.out.println(emp);
            }
        }
        catch(EOFException e)
        {
            System.err.println("End of file is reached ");
        }

    }

}
```

21-06-2024

transient keyword role at the time of Serialization :

While performing serialization i.e storing the object data into the file, if we don't want to serialized any variable or filed then we can declare those variables as a transient variable.

Once a variable is declared with transient keyword then it will not be serialized and we will get default value for those variables.

Product.java

package serialization_de_serialization;

```
import java.io.Serializable;
import java.time.LocalDate;

@SuppressWarnings("serial")
public class Product implements Serializable
{
    private transient int productId;
    private transient String productName;
    private double productPrice;
    private LocalDate manufactureDate;
```

```

public Product(int productId, String productName, double productPrice, LocalDate manufactureDate) {
    super();
    this.productId = productId;
    this.productName = productName;
    this.productPrice = productPrice;
    this.manufactureDate = manufactureDate;
}

@Override
public String toString() {
    return "Product [productId=" + productId + ", productName=" + productName + ", productPrice=" +
productPrice
    + ", manufactureDate=" + manufactureDate + "]";
}
}

```

SerializationAndDeSerialization.java

```

package serialization_de_serialization;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.time.LocalDate;

public class SerializationAndDeSerialization {

    public static void main(String[] args) throws IOException, ClassNotFoundException
    {
        //Serialization
        var fos = new FileOutputStream("C:\\new\\Product.txt");
        var oos = new ObjectOutputStream(fos);

        try(fos;oos)
        {
            Product p1 = new Product(1, "Laptop", 23900.89, LocalDate.now());
            oos.writeObject(p1);
        }
        catch(Exception e)
        {
            e.printStackTrace();
        }
        System.out.println("Object Stored Successfully");

        //De-Serialization

        var fin = new FileInputStream("C:\\new\\Product.txt");
        var ois = new ObjectInputStream(fin);

        try(fin; ois)

```

```

{
    Product p1 = (Product) ois.readObject();
    System.out.println(p1);
}
catch(Exception e)
{
    e.printStackTrace();
}

}

}

```

Working With Character Oriented Stream :

In order to work with character oriented Stream, if we two super classes Reader and Writer.

Both are abstract classes and Reader is the super class for all types of reading operation, on the other hand Writer is the super class for all types of writing operation.

Here we can directly work with characters.

FileWriter class :

It is a predefined class available in java.io package, By using this class we can create a file and write character data to the file.

By using this class we can directly write String (collection of characters) Or character array to the file.

Actually It is a character oriented Stream where as if we work with FileOutputStream class, It is byte oriented Stream.

```

//FileWriter
import java.io.*;
public class File11
{
    public static void main(String args[]) throws IOException
    {
        var fw = new FileWriter("C:\\new\\HelloIndia.txt");
        var bw = new BufferedWriter(fw);

        try(fw; bw)
        {
            bw.write("India, It is in Asia");
            System.out.println("Success....");
        }
        catch(IOException e)
        {
            e.printStackTrace();
        }
    }
}

```

//FileWriter


```

import java.io.*;
class File12
{
    public static void main(String args[]) throws IOException
    {
        var fw = new FileWriter("C:\\new\\Data.txt");
        var bw = new BufferedWriter(fw);

        try(fw;bw)
        {
            char c[] = {'H','E','L','L','O',' ',' ','W','O','R','L','D'};

            bw.write(c);
            System.out.println("Success....");
        }
        catch(Exception e)
        {
            e.printStackTrace();
        }
    }
}

```

FileReader class :

It is a predefined class available in java.io package, It is a character oriented Stream. The main purpose of this class to read the data in the character format directly from the file.

```

//FileReader
import java.io.*;
public class File13
{
    public static void main(String args[]) throws IOException
    {
        var fr = new FileReader(args[0]); //Command Line Arg
        var br = new BufferedReader(fr);

        try(fr ; br)
        {
            while(true)
            {
                int i = br.read();
                if(i == -1)
                    break;
                System.out.print((char)i);
            }
        }
        catch(IOException e)
        {
            e.printStackTrace();
        }
    }
}

```

```

import java.io.*;
public class File14

```

```

{
    public static void main(String[] args) throws IOException
    {
        var fr = new FileReader("C:\\new\\abc.jpg");
        var fw = new FileWriter("C:\\new\\NIT.jpg");

        try(fr;fw)
        {
            int i;
            while((i=fr.read())!= -1)
            {
                fw.write(i);
            }
        }
        catch(Exception e)
        {
        }
        System.out.println("Success");
    }
}

```

Note :- In the above program the image file will not be created in a proper format because images are created by using binary data but FileReader and FileWriter can work with Character data.

 PrintWriter :

 It is a predefined class available in java.io package under Writer abstract class.

It is used to write primitive data into the file in text format.

By using PrintWriter class we can create a file as well as write the primitive data into text format.

It contains a predefined non static method called printf() which accept two parameters

- 1) Formatted String
- 2) Actual Data.

```

//PrintWriter
import java.io.*;
public class File15
{
    public static void main(String[] args) throws IOException
    {
        PrintWriter writeData = new PrintWriter("C:\\new\\Roll.txt");

        try(writeData)
        {
            int roll = 15;
            //Writing primitive data into text format
            writeData.printf("My roll number is : %d ", roll);
        }
        catch(Exception e)
        {
            e.printStackTrace();
        }
    }
}

```

```
}
```

```
-----  
import java.io.FileWriter;  
import java.io.BufferedWriter;  
import java.io.IOException;
```

```
public class File16
```

```
{  
    public static void main(String[] args) throws IOException  
    {
```

```
        var filename = "C:\\new\\append.txt";  
        //wants to write the data in the existing file  
        var fileWriter = new FileWriter(filename, true);
```

```
        var bufferedWriter = new BufferedWriter(fileWriter);
```

```
        try(fileWriter;bufferedWriter)
```

```
{
```

```
    // Append text to the file  
    String textToAppend = "My Name is Raj";  
    bufferedWriter.write(textToAppend);
```

```
    //Moving the cursor to the next line  
    bufferedWriter.newLine();
```

```
    textToAppend = "I lives in hyderabad";  
    bufferedWriter.write(textToAppend);
```

```
    System.out.println("Text appended successfully to the file.");
```

```
}
```

```
catch (IOException e)
```

```
{
```

```
    System.out.println("An error occurred while appending the text to the file: " + e.getMessage());
```

```
}
```

```
}
```

```
}
```

```
-----  
enum in java :
```

```
-----  
An enum is class in java that is used to represent group of universal constants. It is introduced from JDK 1.5 onwards.
```

In order to craete an enum, we should use enum keyword and all the univarsal constants of the enum must be separeted by comma. Semicolon is optional at the end.

Example:-

```
enum Color
```

```
{  
    RED, BLUE, BLACK, PINK    //public + static + final
```

```
}
```

The enum constants are by default public, static and final.

An enum we can define inside the class, outside of the class and even inside of the method.

If we define an enum inside the class then we can apply public, private, protected and static.

Every enum in java extends java.lang.Enum class so an enum can implement many interfaces but can't extend a class.

By default every enum is implicitly final so we can't inherit an enum.

In order to get the constant value of an enum we can use values() method which returns enum array, but this method is added by compiler to each and every enum at the time of compilation.

In order to get the order position of enum constants we can use ordinal() method which is given inside the enum class and the return type of this method is int. The order position of enum constant will start from 0.

As we know an enum is just like a class so we can define any method, constructor inside an enum. Constructor must be either private or default.

*All the enum constants are by default object of type enum.

Enum constants must be declared at the first line of enum otherwise we will get compilation error.

From java 1.5 onwards we can pass an enum in a switch statement.

In order to compare two enum constants we have final equals(Object obj) method in the java.lang.Enum class which will compare two objects based on the memory reference same as == operator.

```
-----  
public class Test1  
{  
    public static void main(String[] args)  
    {  
        enum Month  
        {  
            JANUARY, FEBRUARY, MARCH    //public + static + final  
        }  
  
        System.out.println(Month.MARCH);  
    }  
}
```

```
-----  
enum Month  
{  
    JANUARY, FEBRUARY, MARCH  
}  
public class Test2  
{  
    enum Color { RED, BLUE, BLACK }  
  
    public static void main(String[] args)  
    {  
        enum Week {SUNDAY, MONDAY, TUESDAY }  
    }  
}
```

```
System.out.println(Month.FEBRUARY);
System.out.println(Color.RED);
System.out.println(Week.SUNDAY);
}
}
```

Note :- From the above Program it is clear that we can define an enum inside a class, outside of a class and inside a method as well.

```
//Comparing the constant of an enum
public class Test3
{
    enum Color { RED,BLUE }

    public static void main(String args[])
    {
        Color c1 = Color.RED;
        Color c2 = Color.RED;

        if(c1 == c2)
        {
            System.out.println("==");
        }
        if(c1.equals(c2))
        {
            System.out.println("equals");
        }
    }
}
```

Here == operator and equals(Object obj) method both will return true because equals(Object obj) method internally uses == operator only.

```
public class Test4
{
    private enum Season //private, public, protected, static
    {
        SPRING, SUMMER, WINTER, RAINY;
    }

    public static void main(String[] args)
    {
        System.out.println(Season.RAINY);
    }
}
```

```
//Interview Question
class Hello
{
    int x = 100;
}
```

```
enum Direction extends Hello
{
    EAST, WEST, NORTH, SOUTH
}
```

```
}
```

```
class Test5
```

```
{  
    public static void main(String[] args)  
    {  
        System.out.println(Direction.SOUTH);  
    }  
}
```

Note : Every enum by default extends from java.lang.Enum class so enum can't extend any other class.

//All enums are by default final so can't inherit

```
enum Color
```

```
{  
    RED, BLUE, PINK;  
}
```

```
class Test6 extends Color
```

```
{  
    public static void main(String[] args)  
    {  
        System.out.println(Color.RED);  
    }  
}
```

Note : Every enum is implicitly final so any class can't extend enum

```
package com.ravi.enum_demo;
```

```
enum Season
```

```
{  
    SPRING, SUMMER, WINTER, FALL, RAINY  
}
```

```
public class EnumDemo
```

```
{  
    public static void main(String[] args)  
    {  
        Season[] seasons = Season.values();  
  
        for(Season season : seasons)  
        {  
            System.out.println(season.name());  
        }  
    }  
}
```

values() static method is used to take all the enum values so the return type is enum array.

//ordinal() to find out the order position

```
class Test8
```

```
{
```

```
static enum Season
{
    SPRING, SUMMER, WINTER, FALL, RAINY
}
```

```
public static void main(String[] args)
{
    Season s1[] = Season.values();

    for(Season x : s1)
        System.out.println(x+" order is :"+x.ordinal());
}
```

Note : public int ordinal() method is used to find out the order position.

//We can take main () inside an enum

```
enum Test9
{
    TEST1, TEST2, TEST3; //Semicolon is compulsory

    public static void main(String[] args)
    {
        System.out.println("Enum main method");
    }
}
```

//constant must be in first line of an enum

```
enum Test10
{

    public static void main(String[] args)
    {
        System.out.println("Enum main method");
    }

    HR, SALESMAN, MANAGER;

}
```

//Writing constructor in enum

```
enum Season
{
    WINTER, SUMMER, SPRING, RAINY; //All are object of type enum

    private Season()
    {
        System.out.println("Constructor is executed....");
    }
}

class Test11
```

```
{
public static void main(String[] args)
{
    System.out.println(Season.WINTER);
    System.out.println(Season.SUMMER);

}
}
```

```
-----
//Writing constructor with message
enum Season
{
    SPRING("Pleasant"), SUMMER("UnPleasant"), RAINY("Rain"), WINTER;

    String msg;

    private Season(String msg)
    {
        this.msg = msg;
    }

    private Season()
    {
        this.msg = "Cold";
    }

    public String getMessage()
    {
        return msg;
    }
}

class Test12
{
    public static void main(String[] args)
    {
        Season s1[] = Season.values();

        for(Season x : s1)
            System.out.println(x+" is :"+x.getMessage());
    }
}
```

```
-----
package com.ravi.enum_demo;
```

```
enum Direction
{
    EAST
    {
        @Override
        public String toString()
        {
            return "EAST Direction";
        }
    },
}
```



```

WEST
{
    @Override
    public String toString()
    {
        return "West Direction";
    }
}
}

```

```

public class Test13 {

    public static void main(String[] args)
    {
        Direction[] directions = Direction.values();

        for(Direction direction : directions)
        {
            System.out.println(direction.name());
            System.out.println(direction.toString());
        }

    }

}

```

```

public class Test14
{
    enum Day
    {
        SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY
    }

    public static void main(String args[])
    {
        Day day = Day.MONDAY;

        switch(day)
        {
            case SUNDAY:
                System.out.println("Sunday");
                break;
            case MONDAY:
                System.out.println("Monday");
                break;
            default:
                System.out.println("other day");
        }

    }

}

```

----- Working with Nested classes in java ? -----

In java it is possible to define a class (inner class) inside another class (outer class) called nested class.

In the same way it is also possible to define an interface OR enum OR Annotation or record inside another interface OR enum OR Annotation or record is called inner class or nested class.

Inner classes in Java create a strong encapsulation and HAS-A relationship between the inner class and its outer class.

An inner class, .class file will be represented by \$ symbol.

Advantages of inner class :

- 1) It is a way of logically grouping classes that are only used in one place.
- 2) It is used to achieve strong encapsulation.
- 3) It enhance the readability and maintainability of the code.

Types of Inner classes in java :

There are 4 types of inner classes in java :

- 1) Non Static Nested class OR Regular class
- 2) Static Nested class
- 3) Method / Method local class
- 4) Anonymous inner class

Non static Nested class :

If a non static class define inside a class and outside of the method then it is called Non static nested class.

Outer class object is compulsory for non static nested class.

From non static nested class, we can directly access the private data of outer class. (Encapsulation)

A non static nested class we can declare with private, default, protected, public, abstract and final.

It is also known as Member Regular class.

```
class Outer
{
    private int a = 15;

    class Inner
    {
        public void displayValue()
        {
            System.out.println("Value of a is " + a);
        }
    }
}
```

```

}
public class Test1
{
    public static void main(String... args)
    {
        Outer out = new Outer();

        Outer.Inner in = out.new Inner();

        in.displayValue();

    }
}

```

Note : An inner class can directly access the private data of Outer class.

```

class MyOuter
{
    private int x = 7;
    public void makeInner()
    {
        MyInner in = new MyInner();
        System.out.println("Inner y is "+in.y);
        in.seeOuter();
    }

    class MyInner
    {
        private int y = 15;
        public void seeOuter()
        {
            System.out.println("Outer x is "+x);
        }
    }
}

public class Test2
{
    public static void main(String args[])
    {
        MyOuter m = new MyOuter();
        m.makeInner();
    }
}

```

```

class MyOuter
{
    private int x = 15;
    class MyInner
    {
        public void seeOuter()
        {
            System.out.println("Outer x is "+x);
        }
    }
}

```

```

public class Test3
{
    public static void main(String args[])
    {
        //Creating inner class object in a single line
        MyOuter.MyInner m = new MyOuter().new MyInner();
        m.seeOuter();
    }
}

```

```

-----
class MyOuter
{
    static int x = 7;
    class MyInner
    {
        public static void seeOuter() //MyInner.seeOuter();
        {
            System.out.println("Outer x is "+x);
        }
    }
}

```

```

public class Test4
{
    public static void main(String args[])
    {
        MyOuter.MyInner.seeOuter();
    }
}

```

```

-----
class Car
{
    private String make;
    private String model;
    private Engine engine;

    public Car(String make, String model, int horsepower)
    {
        this.make = make;
        this.model = model;
        this.engine = new Engine(horsePower);
    }

    //Inner class
    private class Engine
    {
        private int horsepower;

        public Engine(int horsepower)
        {
            this.horsePower = horsepower;
        }

        public void start()
    {

```

```

        System.out.println("Engine started! Horsepower: " + horsePower);
    }

    public void stop()
    {
        System.out.println("Engine stopped.");
    }
}

public void startCar()
{
    System.out.println("Starting " + make + " " + model);
    this.engine.start();
}

public void stopCar()
{
    System.out.println("Stopping " + make + " " + model);
    this.engine.stop();
}
}
public class Test5
{

    public static void main(String[] args) {

        Car myCar = new Car("Swift", "Desire", 1200);

        myCar.startCar();

        myCar.stopCar();
    }
}
-----
class Laptop
{
    private String brand;
    private String model;
    private Motherboard motherboard;

    public Laptop(String brand, String model, String motherboardModel, String chipset)
    {
        this.brand = brand;
        this.model = model;
        this.motherboard = new Motherboard(motherboardModel, chipset);
    }

    public void switchOn()
    {
        System.out.println("Turning on " + brand + " " + model);
        this.motherboard.boot();
    }

    //Motherboard inner class

```

```

public class Motherboard
{
    private String model;
    private String chipset;

    public Motherboard(String model, String chipset)
    {
        this.model = model;
        this.chipset = chipset;
    }

    public void boot()
    {
        System.out.println("Booting " + brand + " " + model + " with " + chipset + " chipset");
    }
}

public class Test6
{
    public static void main(String[] args)
    {

        Laptop laptop = new Laptop("HP", "ENVY", "IRIS", "Intel");

        laptop.switchOn();
    }
}

```

25-06-2024

```

class Person
{
    private String name;
    private int age;
    private Heart heart;

    public Person(String name, int age)
    {
        this.name = name;
        this.age = age;
        this.heart = new Heart();
    }

    public void describe()
    {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Heart beats per minute: " + heart.getBeatsPerMinute());
    }

    // Inner class representing the Heart
    private class Heart
    {

```

```

        private int beatsPerMinute;

        public Heart()
        {
            this.beatsPerMinute = 72;
        }

        public int getBeatsPerMinute()
        {
            return this.beatsPerMinute;
        }

        public void setBeatsPerMinute(int beatsPerMinute)
        {
            this.beatsPerMinute = beatsPerMinute;
        }
    }
}
public class Test7
{
    public static void main(String[] args)
    {
        Person person = new Person("Virat", 30);
        person.describe();
    }
}

```

```

class University
{
    private String name;

    public University(String name)
    {
        this.name = name;
    }

    public void displayUniversityName()
    {
        System.out.println("University Name: " + name);
    }
}

```

```

public class Department
{
    private String name;
    private String headOfDepartment;

    public Department(String name, String headOfDepartment)
    {
        this.name = name;
        this.headOfDepartment = headOfDepartment;
    }

    // Method to display department details
    public void displayDepartmentDetails()
    {

```

```

        displayUniversityName();
        System.out.println("Department Name: " + name);
        System.out.println("Head of Department: " + headOfDepartment);
    }
}
}
public class Test8
{
    public static void main(String[] args)
    {
        University university = new University("JNTU");

        University.Department cs = university.new Department("Computer Science", "Dr. John");

        University.Department ee = university.new Department("Electrical Engineering", "Dr. Scott");

        cs.displayDepartmentDetails();
        ee.displayDepartmentDetails();
    }
}

```

```

class OuterClass
{
    private int x = 200;

    class InnerClass
    {
        public void display() //Inner class display method
        {
            System.out.println("Inner class display method");
        }

        public void getValue()
        {
            display();
            System.out.println("Can access outer private var :"+x);
        }
    }

    public void display() //Outer class display method
    {
        System.out.println("Outer class display");
    }
}
public class Test9
{
    public static void main(String [] args)
    {
        OuterClass.InnerClass inobj = new OuterClass().new InnerClass();
        inobj.getValue();

        new OuterClass().display();
    }
}

```



```
}  
}
```

Note :- inner class display() will be invoked from inner class object, if we want to access display() method of Outer class then Outer class object is required.

```
class OuterClass  
{  
    int x;  
    public class InnerClass //private, def, prot, public, abstract  
    {  
        //final  
        int x;  
    }  
}  
public class Test10  
{  
}
```

Method OR Method Local inner class :

If a class is defined inside a method with class name then it is called Method local inner class.

On method local inner class, we can apply only abstract and final modifier.

A method local inner class must be accessible within the same method only that means we can't access method local inner class outside of the method.

```
//program on method local inner class  
class MyOuter3  
{  
    private String x = "Outer class private data";  
  
    public void doSttuff()  
    {  
        String z = "local variable";  
  
        class MyInner //only final and abstract is possible  
        {  
            public void seeOuter()  
            {  
                System.out.println("Outer x is "+x);  
                System.out.println("Local variable z is : "+z);  
            }  
        }  
  
        MyInner mi = new MyInner();  
        mi.seeOuter();  
    }  
}  
  
}  
public class Test11  
{
```

```

    public static void main(String args[])
    {
        MyOuter3 m = new MyOuter3();
        m.doStuff();
    }
}

```

 //local inner class we can't access outside of the method

```

class MyOuter3
{
    private String z = "Outer class Data";

    public void doStuff()
    {
        String x = "local variable";

        class MyInner
        {
            String z = "CLASS instance variable";
            public void seeOuter()
            {
                System.out.println("Outer x is "+MyOuter3.this.z);
                System.out.println("Class variable z is : "+this.z);
                System.out.println("Local variable z is : "+x);
            }
        }
        MyInner mi = new MyInner();
        mi.seeOuter();
    }
}

public class Test12
{
    public static void main(String args[])
    {
        MyOuter3 m = new MyOuter3();
        m.doStuff();
    }
}

```

 //Variable Shadow

```

public class Test
{
    private int x = 100;

    public void methodLocal()
    {
        class Hello
        {
            private int x = 200; //Variable Shadow

            public void access()
            {
                System.out.println("x value is :"+this.x);
            }
        }
    }
}

```

```

        System.out.println("x value is :"+Test.this.x);
    }
}

    Hello h = new Hello();
    h.access();
}

```

```

public static void main(String[] args)
{
    new Test().methodLocal();
}
}

```

 static Nested class :

If we declare a static class inside an outer class then it is called static nested class.

For static nested inner class, Outer class object is not required.

If the static nested inner class, contains only static member then to access those members, static nested inner class object is also not required.

If the static nested inner class contains non static member then object is required for static nested inner class.

From static nested inner class, we can't access non static member of Outer class.

Example :

```

public class Outer
{
    static class Inner //static nested inner class
    {
    }
}

```

```

Outer.Inner in = new Outer.Inner();

```

 //static nested inner class
 class BigOuter
 {
 static class Nest //static nested inner class
 {
 void go() //Instance method of static inner class
 {
 System.out.println("Hello welcome to static nested class");
 }
 }
 }
}

```

class Test13
{
    public static void main(String args[])
    {
        BigOuter.Nest n = new BigOuter.Nest();
        n.go();

    }
}

```

```

class Outer
{
    static int x = 15;

    static class Inner
    {
        void msg()
        {
            System.out.println("x value is "+x);
        }
    }
}

```

```

class Test14
{
    public static void main(String args[])
    {
        Outer.Inner obj=new Outer.Inner();
        obj.msg();
    }
}

```

```

class Outer
{
    static int x = 25;
    static class Inner
    {
        static void msg()
        {
            System.out.println("x value is "+x);
        }
    }
}

```

```

class Test15
{
    public static void main(String args[])
    {
        Outer.Inner.msg();
    }
}

```

```

class Outer
{
    int x=15; //error (not possible because try to access instance variable)
    static class Inner

```

```

    {
    void msg()
    {
        System.out.println("x value is "+x);
    }
    }
}
class Test16
{
    public static void main(String args[])
    {
        Outer.Inner obj=new Outer.Inner();
        obj.msg();
    }
}

```

 Anonymous inner class :

If we define class inside a method body without any name then it is called anonymous inner class.

The main purpose of anonymous inner class to extend a class or implement an interface that means creating sub type of a class or interface.

Anonymous inner class object will be created at the time of declaration the class body.

Anonymous inner class is used to create singleton object because for 1 anonymous inner class only 1 object will be created.

A normal concrete class can extend only one class as well as implement many number of interfaces but an anonymous inner class can either extend only one class OR can implement only one interface only.

Inside an anonymous inner class we can write static , non static variable, static block and non-static block. Here we can't write abstract method and constructor.

 //Anonymous inner class
 class Tech
 {
 public void tech()
 {
 System.out.println("Tech");
 }
 }
 public class Test17
 {
 public static void main(String... args)
 {

 Tech a = new Tech() //Anonymous inner class
 {

 @Override
 public void tech()
 {
 System.out.println("anonymous tech");
 }
 }
 }
 }

```

    }

};
a.tech();
}
}

```

```

@FunctionalInterface
interface Vehicle
{
    void move(); //SAM(Single Abstract Method)
}

```

```

class Test18
{
    public static void main(String[] args)
    {
        Vehicle car = new Vehicle()
        {
            @Override
            public void move()
            {
                System.out.println("Moving with Car...");
            }
        };
        car.move();

        Vehicle bike = new Vehicle()
        {
            @Override
            public void move()
            {
                System.out.println("Moving with Bike...");
            }
        };
        bike.move();
    }
}

```

26-06-2024

Multithreading :

Uniprocessing :-

In uniprocessing, only one process can occupy the memory So the major drawbacks are

- 1) Memory is wastage
- 2) Resources are wastage
- 3) Cpu is idle

To avoid the above said problem, multitasking is introduced.

In multitasking multiple task can concurrently work with CPU so, our task will be completed as soon as possible.

Multitasking is further divided into two categories.

- a) Process based Multitasking
- b) Thread based Multitasking

Process based Multitasking :

If a CPU is switching from one subtask(Thread) of one process to another subtask of another process then it is called Process based Multitasking.

Thread based Multitasking :

If a CPU is switching from one subtask(Thread) to another subtask within the same process then it is called Thread based Multitasking.

Thread :-

A thread is light weight process and it is the basic unit of CPU which can run concurrently with another thread within the same context (process).

It is well known for independent execution. The main purpose of multithreading to boost the execution sequence.

A thread can run with another thread at the same time so our task will be completed as soon as possible.

In java whenever we define main method then JVM internally creates a thread called main thread.

Program that describes that main is a Thread :

Whenever we define main method then JVM will create main thread internally under main group, the purpose of this main thread to execute the entire main method.

In java there is a predefined class called Thread available in java.lang package, this class contains a predefined static method `currentThread()` which will provide currently executing Thread.

`Thread t = Thread.currentThread(); //Factory Method`

Thread class has provided predefined method `getName()` to get the name of the Thread.

```
package com.ravi.thread;
```

```
public class ThreadName {
```

```
    public static void main(String[] args)
    {
```

```
        Thread currentThread = Thread.currentThread();
        System.out.println("Current Thread Name is :"+currentThread.getName());
```

```
        //OR
```

```
        System.out.println("Name of the current thread :"+Thread.currentThread().getName());
```

```
}
```

```
}
```

How to create user-defined thread ?

We can create user-defined thread by using the following two packages

- 1) By using java.lang package
- 2) By using java.util.concurrent package

Creating user-defined Thread by using java.lang package :

By using java.lang package we can create user-defined thread by using anyone of the following two approaches :

- 1) By extending java.lang.Thread class
- 2) By implementing java.lang.Runnable interface

Note :- Thread is a predefined class available in java.lang package where as Runnable is a predefined interface available in java.lang Package.

27-06-2024

//Program to create user thread by extending Thread class

```
package com.ravi.multithreading;
```

```
class MyThread extends Thread
```

```
{  
    @Override  
    public void run()  
    {  
        System.out.println("Child Thread is running");  
    }  
}
```

```
public class UserThread
```

```
{  
    public static void main(String [] args)  
    {  
        System.out.println("Main Thread started...");  
        MyThread mt = new MyThread();  
        mt.start();  
        System.out.println("Main Thread ended...");  
    }  
}
```

In the above program main thread and Thread-0 threads are created both threads are executing their own Stack Memory as shown in the diagram.

[27-JUNE]

```
public synchronized void start() :
```

start() is a predefined non static method of Thread class which internally performs the following two tasks :
:

1) It will make a request to the O.S to assign a new thread for concurrent execution.

2) It will implicitly call run() method.

public boolean isAlive() :-

It is a predefined non static method of Thread class through which we can find out whether a thread has started or not ?

As we know a new thread is created/started after calling start() method so if we use isAlive() method before start() method, it will return false but if the same isAlive() method if we invoke after the start() method, it will return true.

We can't restart a thread in java if we try to restart then It will generate an exception i.e java.lang.IllegalThreadStateException

```
package com.ravi.basic;
```

```
class Foo extends Thread
{
    @Override
    public void run()
    {
        System.out.println("Child thread is running...");
        System.out.println("It is running with separate stack");
    }
}

public class IsAlive
{
    public static void main(String[] args)
    {
        System.out.println("Main Thread is started..");
        Foo f = new Foo();
        System.out.println("Thread has not started yet so :"+f.isAlive());

        f.start(); //new Thread has created

        System.out.println("Thread has started so :"+f.isAlive());
        f.start(); //java.lang.IllegalThreadStateException

    }
}
```

package com.ravi.basic;

```
class Stuff extends Thread
{
    @Override
    public void run()
    {
        System.out.println("Child Thread is Running!!!!");
    }
}

public class ExceptionDemo
```

```

{
public static void main(String[] args)
{
    System.out.println("Main Thread Started");

    Stuff s1 = new Stuff();
    Stuff s2 = new Stuff();

    s1.start();
    s2.start();

    System.out.println(10/0);

    System.out.println("Main Thread Ended");
}
}

```

Note :- Here main thread is interrupted due to AE but still child thread will be executed because child thread is executing with separate Stack

```

-----
package com.ravi.basic;

class Sample extends Thread
{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();

        for(int i=1; i<=10; i++)
        {
            System.out.println("i value is :"+i+" by "+name+ " thread");
        }
    }
}

public class ThreadLoop
{
    public static void main(String[] args)
    {

        new Sample().start();

        String name = Thread.currentThread().getName();

        for(int i=1; i<=10; i++)
        {
            System.out.println("i value is :"+i+" by "+name+ " thread");
        }

        int x = 1;
        do
        {
            System.out.println("India");

```

```

    x++;
}
while(x<=10);
}
}

```

Note : Here processor is frequently switching from main thread to Thread-0 thread so output is un-predicatable

How to set and get the name of the Thread :

Whenever we create a userdefined Thread in java then by default JVM assigns the name of thread is Thread-0, Thread-1, Thread-2 and so on.

If a user wants to assign some user defined name of the Thread, then Thread class has provided a predefined method called setName(String name) to set the name of the Thread.

On the other hand we want to get the name of the Thread then Thread class has provided a predefined method called getName().

```

public final void setName(String name) //setter

```

```

public final String getName() //getter

```

```

package com.ravi.basic;
class DoStuff extends Thread
{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();
        System.out.println(name +" thread is running Here!!!!");
    }
}
public class ThreadName
{
    public static void main(String[] args)
    {
        DoStuff t1 = new DoStuff();
        DoStuff t2 = new DoStuff();

        t1.start();
        t2.start();

        System.out.println(Thread.currentThread().getName()+" thread is running.....");
    }
}

```

Note :- If we don't provide our user-defined name for the thread then by default the name would be Thread-0, Thread-1 and so on.

```

package com.ravi.basic;
class Demo extends Thread
{

```

```

@Override
public void run()
{
    System.out.println(Thread.currentThread().getName()+" thread is running.....");
}
}
public class ThreadName1
{
    public static void main(String[] args)
    {
        Thread t = Thread.currentThread();
        t.setName("Parent"); //Changing the name of the main thread

        Demo d1 = new Demo();
        Demo d2 = new Demo();

        d1.setName("Child1");
        d2.setName("Child2");

        d1.start(); d2.start();

        String name = Thread.currentThread().getName();
        System.out.println(name + " Thread is running..");
    }
}

```

In the above program we have assigned user-defined name to the Thread so it will take the name given by the user.

28-06-2024

Thread.sleep(long millisecond) :

If we want to put a thread into temporarily waiting state then we should use sleep() method.

The waiting time of the Thread depends upon the time specified by the user in millisecond as parameter to sleep() method.

It is a static method of Thread class.

Thread.sleep(1000); //Thread will wait for 1 second

It is throwing a checked Exception i.e InterruptedException because there may be chance that this sleeping thread may be interrupted by a thread so provide either try-catch or declare the method as throws.

package com.ravi.basic;

```

class Sleep extends Thread
{
    @Override
    public void run()
    {
        for(int i = 1; i<=10; i++)

```

```

    {
        System.out.println("i value is :"+i);
        try
        {
            Thread.sleep(1000);
        }
        catch(InterruptedException e)
        {
            System.err.println("Thread is Interrupted");
        }
    }
}
}

public class SleepDemo
{
    public static void main(String[] args)
    {
        Sleep s = new Sleep();
        s.start();

    }
}

```

```

package com.ravi.basic;

```

```

class MyTest extends Thread
{
    @Override
    public void run() //11 22 33 44 55
    {
        for(int i=1; i<=5; i++)
        {
            System.out.println("i value is :"+i);
            try
            {
                Thread.sleep(1000);
            }
            catch(InterruptedException e)
            {
                System.err.println("Thread has Interrupted");
            }
        }
    }
}

public class SleepDemo1
{
    public static void main(String[] args)
    {
        MyTest m1 = new MyTest();
        MyTest m2 = new MyTest();

        m1.start();
        m2.start();
    }
}

```

```
}  
}
```

Assignment :

Thread.sleep(long millis, int nanos) :

Thread life cycle :

As we know a thread is well known for Independent execution and it contains a life cycle which internally contains 5 states (Phases).

During the life cycle of a thread, It can pass from these 5 states. At a time a thread can reside to only one state of the given 5 states.

- 1) NEW State (Born state)
- 2) RUNNABLE state (Ready to Run state) [Thread Pool]
- 3) RUNNING state
- 4) WAITING / BLOCKED state
- 5) EXIT/Dead state

New State :-

Whenever we create a thread instance(Thread Object) a thread comes to new state OR born state. New state does not mean that the Thread has started yet only the object or instance of Thread has been created.

Runnable state :-

Whenever we call start() method on thread object, A thread moves to Runnable state i.e Ready to run state. Here Thread scheduler is responsible to select/pick a particular Thread from Runnable state and sending that particular thread to Running state for execution.

Running state :-

If a thread is in Running state that means the thread is executing its own run() method.

From Running state a thread can move to waiting state either by an order of thread scheduler or user has written some method(wait(), join() or sleep()) to put the thread into temporarily waiting state.

From Running state the Thread may also move to Runnable state directly, if user has written Thread.yield() method explicitly.

Waiting state :-

A thread is in waiting state means it is waiting for its time period to complete. Once the time period will be completed then it will re-enter inside the Runnable state to complete its remaining task.

Dead or Exit :

Once a thread has successfully completed its run method then the thread will move to dead state. Please remember once a thread is dead we can't restart a thread in java.

IQ :- If we write Thread.sleep(1000) then exactly after 1 sec the Thread will re-start?

Ans :- No, We can't say that the Thread will directly move from waiting state to Running state.

The Thread will definitely wait for 1 sec in the waiting mode and then again it will re-enter into Runnable state which is control by Thread Scheduler so we can't say that the Thread will re-start just after 1 sec.

Anonymous inner class approach with Thread class :

As we know, In order to create a thread we need to take Thread as a super class so, instead of making Thread as a super class we can use Anonymous inner class approach as shown below :

a) Anonymous inner class with Thread class with reference :

```
package com.nit.thread;

public class AnonymousWithReference
{
    public static void main(String[] args)
    {
        //Anonymous inner class with reference
        Thread t1 = new Thread()
        {
            @Override
            public void run()
            {
                String name = Thread.currentThread().getName();
                System.out.println("Child thread is :"+name);
            }
        };

        t1.start();

        String name = Thread.currentThread().getName();
        System.out.println("Parent thread is :"+name);

    }
}
```

b) Anonymous inner class with Thread class without reference :

```
package com.nit.thread;

public class AnonymousInnerWithoutRef {

    public static void main(String[] args)
    {
        new Thread()
```

```

{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();
        System.out.println("Child thread is :"+name);
    }
}.start();

}

}

```

29-06-2024

join() method of Thread class :

The main purpose of join() method to put the current thread into waiting state until the other thread finish its execution.

Here the currently executing thread stops its execution and the thread goes into the waiting state. The current thread remains in the wait state until the thread on which the join() method is invoked has achieved its dead state.

It also throws checked exception i.e InterruptedException so better to use try catch or declare the method as throws.

It is a non static method so we can call this method with the help of Thread object reference.

```

package com.nit.thread;

```

```

class Join extends Thread

```

```

{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();
        for(int i=1; i<=5; i++)
        {
            System.out.println("i value is :"+i+" by "+name);
            try
            {
                Thread.sleep(1000);
            }
            catch(InterruptedException e)
            {
                System.err.println("Thread is interrupted");
            }
        }
    }
}

```

```

public class JoinDemo

```

```

{
    public static void main(String[] args) throws InterruptedException

```



```

{
    System.out.println("Main Thread started!!!");

    Join j1 = new Join();
    Join j2 = new Join();
    Join j3 = new Join();

    j1.setName("J1"); j2.setName("J2"); j3.setName("J3");

    j1.start();

    j1.join(); //Putting the main thread into waiting mode

    j2.start();
    j3.start();

    System.out.println("Main Thread ended!!!");

}

}
-----
package com.ravi.basic;

class Alpha extends Thread
{
    @Override
    public void run()
    {
        Thread t = Thread.currentThread();
        String name = t.getName(); //Alpha_Thread is current thread

        Beta b1 = new Beta();
        b1.setName("Beta_Thread");
        b1.start();
        try
        {
            b1.join(); //Alpha thread is Blocked

            System.out.println("-----");
        }
        catch (InterruptedException e)
        {
            e.printStackTrace();
        }

        for(int i=1; i<=10; i++)
        {
            System.out.println(i+" by "+name);
        }

    }
}

```

```

public class JoinDemo2
{
    public static void main(String[] args)
    {
        Alpha a1 = new Alpha();
        a1.setName("Alpha_Thread");
        a1.start();
    }
}

```

```

class Beta extends Thread
{
    @Override
    public void run()
    {
        Thread t = Thread.currentThread();
        String name = t.getName();
        for(int i=1; i<=10; i++)
        {
            System.out.println(i+" by "+name);
            try
            {
                Thread.sleep(1000);
            }
            catch(InterruptedException e) {

            }
        }
        System.out.println(".....");
    }
}

```

Note : Here Beta thread is calling join() method so first of all Beta thread will be completed then only Alpha thread can start its execution(that means Alpha thread can join)

```

-----
package com.ravi.basic;

public class JoinDemo1
{
    public static void main(String[] args) throws InterruptedException
    {
        System.out.println("Main Thread Started");

        Thread t = Thread.currentThread();
        String name = t.getName();

        for(int i=1; i<=10; i++)
        {
            System.out.println("i value is :"+i+" by "+name);
            Thread.sleep(500);
        }
        t.join(); //Deadlock [Main thread is waiting for main thread
                // complete]
    }
}

```

```
System.out.println("Main Thread Ended");
```

```
}
```

```
}
```

Here, It is a deadlock state because main thread is waiting for main thread to complete.

Assignment :

```
join(long millis)
```

```
join(long millis, long nanos)
```

Creating Thread by using Runnable interface : [2nd Approach]

package com.ravi.basic;

```
class Ravi implements Runnable
```

```
{
```

```
    public void run()
```

```
    {
```

```
        System.out.println("Running");
```

```
    }
```

```
}
```

```
public class RunnableDemo
```

```
{
```

```
    public static void main(String [] args)
```

```
    {
```

```
        Thread t1 = new Thread(new Ravi());
```

```
        t1.start();
```

```
    }
```

```
}
```

01-07-2024

Assigning the target through Runnable

```
package com.ravi.mt;
```

```
class General implements Runnable
```

```
{
```

```
    @Override
```

```
    public void run()
```

```
    {
```

```
        String name = Thread.currentThread().getName();
```

```
        System.out.println(name + " is booking General Ticket");
```

```
    }
```

```
}
```

```
class Tatkal implements Runnable
```

```
{
```

```
    @Override
```

```
    public void run()
```

```

{
String name = Thread.currentThread().getName();
System.out.println(name +" is booking Tatkal Ticket");
}
}

```

```

public class AssigningTarget
{
public static void main(String[] args)
{
Thread general = new Thread(new General(),"Scott");

Thread tatkal = new Thread(new Tatkal(), "Smith");

general.start(); tatkal.start();

}

}

```

Thread class Constructor :

In Thread class we have total 9 constructors are available but commonly we are using the following 7 constructors

- 1) Thread t1 = new Thread();
- 2) Thread t2 = new Thread(String name);
- 3) Thread t3 = new Thread(Runnable target);
- 4) Thread t4 = new Thread(Runnable target, String name);
- 5) Thread t5 = new Thread(ThreadGroup tg, String name);
- 6) Thread t6 = new Thread(ThreadGroup tg, Runnable target);
- 7) Thread t7 = new Thread(ThreadGroup tg, Runnable target, String name);

Anonymous inner class by using Runnable interface :

```

package com.ravi.mt;

public class RunnableInnerClass {

public static void main(String[] args)
{
Runnable r1 = new Runnable()
{
@Override
public void run()
{
String name = Thread.currentThread().getName();
System.out.println("Child Thread name is :"+name);
}
}
}

```

```
};

String name = Thread.currentThread().getName();
System.out.println("Parent Thread name is :"+name);

Thread t1 = new Thread(r1,"Scott"); t1.start();
}
```

```
}
```

Anonymous inner class by using Lambda :

```
package com.ravi.mt;

public class RunnableUsingLambda {

    public static void main(String[] args)
    {
        Runnable r1 = () -> System.out.println("Current Thread name is :"+Thread.currentThread().getName());

        new Thread(r1,"Smith").start();

    }

}
```

Anonymous inner class using Thread class constructor without ref. :

```
package com.ravi.mt;

public class RunnableUsingLambda {

    public static void main(String[] args)
    {
        new Thread(new Runnable()
        {
            @Override
            public void run()
            {
                String name = Thread.currentThread().getName();
                System.out.println("Child Thread name is :"+name);
            }
        }, "Scott").start();

    }

}
```

package com.ravi.mt;

```
public class RunnableUsingLambda {

    public static void main(String[] args)
    {
        new Thread(()-> System.out.println(Thread.currentThread().getName())).start();
    }

}
```

```
}
```

```
}
```

In between extends Thread and implements Runnable which one is better and why?

In between extends Thread and implements Runnable, implements Runnable is more better due to the following reasons :

1) In extend Thread class approach, further we can't extend any other class but while working with implements Runnable we can extend a single class.

```
class Ravi extends Thread
```

```
class Raj extends MyClass implements Runnable
```

2) By using extends Thread our target is run method of sub class
but by using implements Runnable we can assign different targets as per our requirement so it provides loose coupling.

```
Thread t1 = new Thread(Runnable target);
```

3) By using extends Thread, we can't implement Lambda Expression but the same we can achieve by using Runnable interface.

Conclusion :

Implements Runnable Advantage

- 1) extend a single class
- 2) Assigning different targets
- 3) Implementing Lambda

Drawback of Multithreading :

Multithreading is very good to complete our task as soon as possible but in some situation, It provides some wrong data or wrong result.

In Data Race or Race condition, all the threads try to access the resource at the same time so the result will be corrupted.

In multithreading if we want to perform read operation and data is not updatable then multithreading is good but if the data is updatable data (modifiable data) then multithreading may produce some wrong result or wrong data as shown in the diagram.(01-JULY)

```
package com.ravi.mt;
```

```
class Customer implements Runnable
```

```
{  
    private int availableSeat = 1;  
    private int wantedSeat;
```

```
    public Customer(int wantedSeat)  
    {  
        this.wantedSeat = wantedSeat;
```

```

    }

    @Override
    public void run()
    {
        String name = null;
        if(availableSeat >= wantedSeat)
        {
            name = Thread.currentThread().getName();
            System.out.println(wantedSeat+ " seat is booked for : "+name);
            availableSeat = availableSeat - wantedSeat;
        }
        else
        {
            name = Thread.currentThread().getName();
            System.err.println("Sorry !" +name+" berth is not available");
        }
    }
}

}

public class RailwayReservation
{
    public static void main(String[] args) throws InterruptedException
    {
        Customer cust =new Customer(1);

        Thread t1 = new Thread(cust,"Virat");
        Thread t2 = new Thread(cust,"Rohit");

        t1.start();
        t2.start();
    }
}

```

Note :- From the above program, It is clear that both the threads are getting the ticket.

```

package com.ravi.mt;

```

```

public class Customer
{
    private int balance = 20000;
    private int withAmount;

    public Customer(int withAmount)
    {
        this.withAmount = withAmount;
    }

    public static void main(String[] args)
    {

```

```
Customer c1 = new Customer(20000);
```

```
Runnable r1 = ()->
```

```
{  
    String name = null;  
    if(c1.balance >= c1.withAmount)  
    {  
        name = Thread.currentThread().getName();  
        System.out.println(c1.withAmount+ " Amount is withdraw by : "+name);  
        c1.balance = c1.balance - c1.withAmount;  
    }  
    else  
    {  
        name = Thread.currentThread().getName();  
        System.out.println("Sorry "+name+ "you have in-sufficient balance");  
    }  
};
```

```
Thread t1 = new Thread(r1,"Scott");  
Thread t2 = new Thread(r1,"Smith");
```

```
t1.start(); t2.start();
```

```
}
```

```
}
```

02-07-2024

****Synchronization :**

In order to solve the problem of multithreading java software people has introduced synchronization concept.

In order to acheive synchronization in java we have a keyword called "synchronized".

It is a technique through which we can control multiple threads but accepting only one thread at all the time.

Synchronization allows only one thread to enter inside the synchronized area for a single object.

Synchronization can be divided into two categories :-

1) Method level synchronization

2) Block level synchronization

1) Method level synchronization :-

In method level synchronization, the entire method gets synchronized so all the thread will wait at method level and only one thread will enter inside the synchronized area as shown in the diagram.(02-JUL-24)

2) Block level synchronization

In block level synchronization the entire method does not get synchronized, only the part of the method gets synchronized so all the thread will enter inside the method but only one thread will enter inside the synchronized block as shown in the diagram (02-JUL-24)

Note :- In between method level synchronization and block level synchronization, block level synchronization is more preferable because all the threads can enter inside the method so only the PART OF THE METHOD GETS synchronized so only one thread will enter inside the synchronized block.

How synchronization logic controls multiple threads ?

Every Object has a lock(monitor) in java environment and this lock can be given to only one Thread at a time.

Actually this lock is available with each individual object provided by Object class.

The thread who acquires the lock from the object will enter inside the synchronized area, it will complete its task without any disturbance because at a time there will be only one thread inside the synchronized area(for single Object). *This is known as Thread-safety in java.

The thread which is inside the synchronized area, after completion of its task while going back will release the lock so the other threads (which are waiting outside for the lock) will get a chance to enter inside the synchronized area by again taking the lock from the object and submitting it to the synchronization mechanism.

This is how synchronization mechanism controls multiple Threads.

Note :- Synchronization logic can be done by senior programmers in the real time industry because due to poor synchronization there may be chance of getting deadlock.

//Program on Method Level Synchronization :

```
package com.ravi.synchronization;

class Table
{
    public synchronized void printTable(int num)
    {
        for(int i=1; i<=10; i++)
        {
            System.out.println(num+" X "+i+" = "+(num*i));
            try
            {
                Thread.sleep(1000);
            }
            catch(InterruptedException e)
            {
            }
        }
        System.out.println(".....");
    }
}
```

```
public class MethodLevelSynchronization
{
```

```
public static void main(String[] args)
{
    Table obj = new Table(); //Lock is created
```

```

    Thread t1 = new Thread()
    {
        @Override
        public void run()
        {
            obj.printTable(5);
        }
    };

```

```

    Thread t2 = new Thread()
    {
        @Override
        public void run()
        {
            obj.printTable(10);
        }
    };

```

```

    Thread t3 = new Thread()
    {
        @Override
        public void run()
        {
            obj.printTable(15);
        }
    };

```

```

    t1.start(); t2.start(); t3.start();
}

}
```

```
//Program on Block Level Synchronization:
package com.ravi.advanced;
```

```
//Block level synchronization
```

```
class ThreadName
{
    public void printThreadName()
    {
        String name = Thread.currentThread().getName();
        System.out.println("Thread inside the method is :"+name);

        synchronized(this) //synchronized Block
        {
            for(int i=1; i<=9; i++)
            {
                System.out.println("i value is :"+i+" by :"+name);
            }
        }
    }
}
```

```

        System.out.println(".....");
    }
}
}
public class BlockSynchronization
{
    public static void main(String[] args)
    {
        ThreadName obj1 = new ThreadName(); //lock is created

        Runnable r1 = () -> obj1.printThreadName();

        Thread t1 = new Thread(r1,"Child1");
        Thread t2 = new Thread(r1,"Child2");
        t1.start(); t2.start();
    }
}

```

 Drawback with Object level synchronization :

From the given diagram it is clear that there is no interference between t1 and t2 thread because they are passing through Object1 where as on the other hand there is no interference even in between t3 and t4 threads because they are also passing through Object2 (another object).

But there may be chance that with t1 Thread, t3 or t4 thread can enter inside the synchronized area at the same time, similarly it is also possible that with t2 thread, t3 or t4 thread can enter inside the synchronized area so the conclusion is, synchronization mechanism does not work with multiple Objects.(Diagram 02-JULY-24)

```

package com.ravi.advanced;
class PrintTable
{
    public synchronized void printTable(int n)
    {
        for(int i=1; i<=10; i++)
        {
            System.out.println(n+" X "+i+" = "+(n*i));
            try
            {
                Thread.sleep(500);
            }
            catch(Exception e)
            {
            }
        }
        System.out.println(".....");
    }
}

```

```

public class ProblemWithObjectLevelSynchronization
{
    public static void main(String[] args)
    {
        PrintTable pt1 = new PrintTable(); //lock1
        PrintTable pt2 = new PrintTable(); //lock2
    }
}

```

```
Thread t1 = new Thread() //Anonymous inner class concept
```

```
{
    @Override
    public void run()
    {
        pt1.printTable(2); //lock1
    }
};

Thread t2 = new Thread()
{
    @Override
    public void run()
    {
        pt1.printTable(3); //lock1
    }
};

Thread t3 = new Thread()
{
    @Override
    public void run()
    {
        pt2.printTable(8); //lock2
    }
};

Thread t4 = new Thread()
{
    @Override
    public void run()
    {
        pt2.printTable(9); //lock2
    }
};

t1.start(); t2.start(); t3.start(); t4.start();
}
}
```

Note : Here we have two objects so two locks are available hence inside the synchronized method we have two threads.

static synchronization :

If we make our synchronized method as a static method then it is called static synchronization.

Now for static synchronization, to call the static synchronized method object is not required.

The thread will acquire the lock from the class instead of object.

Unlike objects we can write only one class per application. [Inside a package we can define only one class but for a single class we can create multiple objects]

```

package com.ravi.advanced;
class MyTable
{
    public static synchronized void printTable(int n) //static synchronization
    {
        for(int i=1; i<=10; i++)
        {
            try
            {
                Thread.sleep(100);
            }
            catch(InterruptedException e)
            {
                System.err.println("Thread is Interrupted...");
            }
            System.out.println(n+" X "+i+" = "+(n*i));
        }
        System.out.println("-----");
    }
}

public class StaticSynchronization
{
    public static void main(String[] args)
    {
        Thread t1 = new Thread()
        {
            @Override
            public void run()
            {
                MyTable.printTable(5);
            }
        };

        Thread t2 = new Thread()
        {
            @Override
            public void run()
            {
                MyTable.printTable(10);
            }
        };

        Runnable r3 = new Runnable()
        {
            @Override
            public void run()
            {
                MyTable.printTable(15);
            }
        };

        Thread t3 = new Thread(r3);

        t1.start();
        t2.start(); t3.start();
    }
}

```

```
}  
}
```

Here the thread will take the lock from MyTable class.

** Inter Thread Communication(ITC) :

It is a mechanism to communicate two synchronized threads within the context to achieve a particular task.

In ITC we put a thread into wait mode by using wait() method and other thread will complete its corresponding task, after completion of the task it will call notify() method so the waiting thread will get a notification to complete its remaining task.

ITC can be implemented by the following method of Object class.

1) public final void wait() throws InterruptedException

2) public native final void notify()

3) public native final void notifyAll()

public final void wait() throws InterruptedException :-

It will put a thread into temporarily waiting state and it will release the lock.

It will wait till the another thread invokes notify() or notifyAll() method from the same object.

public native final void notify() :-

It will wake up the single thread that is waiting on the same object. It will not release the lock immediately, once synchronized area is completed then only it will release the lock.

public native final void notifyAll() :-

It will wake up all the threads which are waiting on the same object. It will not release the lock immediately, once synchronized area is completed then only it will release the lock.

*Note :- wait(), notify() and notifyAll() methods are defined in Object class but not in Thread class because these methods are related to lock(because we can use these methods from the synchronized area ONLY) and Object has a lock so, all these methods are defined inside Object class.

The following program explains, if we don't use these methods from synchronized area then we will get java.lang.IllegalMonitorStateException

```
public class ExceptionDemo  
{  
    public static void main(String[] args) throws InterruptedException  
    {  
        Object obj = new Object();  
        obj.wait(); //java.lang.IllegalMonitorStateException  
    }  
}
```

*What is the difference between sleep() and wait()

(Given in the diagram 03-JUL-24)

//Program that describes if we don't use ITC then there is no communication between two threads so we will not get proper output.

```
package com.ravi.itc;

class Test extends Thread
{
    int val = 0;

    @Override
    public void run()
    {
        for(int i=1; i<=10; i++)
        {
            val = val + i; //0 1 3 6 10 15 21
        }
    }
}

public class ProblemWithoutITC
{
    public static void main(String[] args) throws InterruptedException
    {
        Test t1 = new Test();
        t1.start();

        System.out.println(t1.val);
    }
}
```

//Communication between main thread and child thread using ITC

```
class SecondThread extends Thread
{
    int x = 0;

    @Override
    public void run()
    {
        //Child thread is waiting for lock
        synchronized(this)
        {
            for(int i=1; i<=100; i++)
            {
                x = x + i;
            }
        }
    }
}
```

```

        System.out.println("Sending notification");
        notify();
    }
}
}
public class InterThreadComm
{
public static void main(String [] args)
{
    SecondThread b = new SecondThread();
    b.start();

    synchronized(b) //Main thread has taken the lock
    {
        //Main thread is suspended by Thread Scheduler
        try
        {
            System.out.println("Waiting for b to complete...");
            b.wait(); //main thread is waiting, lock is released

            System.out.println("Main thread wake up");
        }
        catch (InterruptedException e)
        {
        }
        System.out.println("Value is: " + b.x);
    }
}
}
}

```

04-07-2024

//Program on wait and notify method for balance updation

```

package com.ravi.mt;

class Customer
{
    private double balance = 10000;

    public synchronized void withdraw(double amount)
    {
        System.out.println("Going to withdraw the amount");
        if(amount > balance)
        {
            System.out.println("Insufficient Balance, Waiting 4 deposit");
            try
            {
                wait();//Thread will wait and release the lock
            }
            catch (InterruptedException e)
            {
                e.printStackTrace();
            }
        }
    }
}

```



```
    balance = balance - amount;
    System.out.println("Balance after withdraw is :"+balance);
}
```

```
public synchronized void deposit(double amount)
{
    System.out.println("Going for deposit ");
    balance = balance + amount;
    System.out.println("Deposit Completed");
    System.out.println("After deposit amount is :"+balance);
    notify();
}
}
```

```
public class InterThreadBalance
{
    public static void main(String[] args)
    {
        Customer cust = new Customer();
```

```
        Thread son = new Thread()
        {
            @Override
            public void run()
            {
                cust.withdraw(5000);
            }
        };
        son.start();
```

```
        Thread father = new Thread()
        {
            @Override
            public void run()
            {
                cust.deposit(9000);
            }
        };
        father.start();
    }
}
```

```
class Resource
{
    private boolean flag = false;

    public synchronized void waitMethod()
    {
        System.out.println("Wait");
        while (!flag)
        {
            try
            {

```

```

        System.out.println(Thread.currentThread().getName() + " is waiting...");
        wait();
    }
    catch (InterruptedException e)
    {
        e.printStackTrace();
    }
    }
    System.out.println(Thread.currentThread().getName() + " thread completed!!!");
}

    public synchronized void setMethod()
    {
        System.out.println("notifyAll");
        this.flag = true;
        System.out.println(Thread.currentThread().getName() + " has make flag value as a true");
        notifyAll(); // Notify all waiting threads that the signal is set
    }
}

public class InterThreadNotifyAll
{
    public static void main(String[] args)
    {
        Resource r1 = new Resource();

        Thread t1 = new Thread(() -> r1.waitMethod(), "Child1");
        Thread t2 = new Thread(() -> r1.waitMethod(), "Child2");
        Thread t3 = new Thread(() -> r1.waitMethod(), "Child3");

        Thread setter = new Thread(() -> r1.setMethod(), "Setter_Thread");

        t1.start();
        t2.start();
        t3.start();

        try
        {
            Thread.sleep(2000);
        }
        catch (InterruptedException e)
        {
            e.printStackTrace();
        }

        setter.start();
    }
}

```

Thread Priority :

It is possible in java to assign priority to a Thread. Thread class has provided two predefined methods `setPriority(int newPriority)` and `getPriority()` to set and get the priority of the thread respectively.

In java we can set the priority of the Thread in numbers from 1- 10 only where 1 is the minimum priority and 10 is the maximum priority.

Whenever we create a thread in java by default its priority would be 5 that is normal priority.

The user-defined thread created as a part of main thread will acquire the same priority of main Thread.

Thread class has also provided 3 final static variables which are as follows :-

Thread.MIN_PRIORITY :- 01

Thread.NORM_PRIORITY : 05

Thread.MAX_PRIORITY :- 10

Note :- We can't set the priority of the Thread beyond the limit(1-10) so if we set the priority beyond the limit (1 to 10) then it will generate an exception java.lang.IllegalArgumentException.

WAP that describes whenever we create a thread by default its priority would be 5

```
public class ThreadPrior
{
    public static void main(String [] args)
    {
        System.out.println("Main thread priority is :"+Thread.currentThread().getPriority()); //5

        Thread t = new Thread();
        System.out.println(t.getPriority());//5
        t.setPriority(8);
        System.out.println(t.getPriority()); //8

        t.setPriority(Thread.MAX_PRIORITY);
        System.out.println(t.getPriority()); //10
    }
}
```

//program that describes if we change main thread priority then child thread will acquire same priority.

```
package com.ravi.mt;
```

```
public class InterThreadBalance
{
    public static void main(String[] args)
    {
        Thread t = Thread.currentThread();
        t.setPriority(Thread.MAX_PRIORITY);
        System.out.println(t.getPriority());

        Thread t1 = new Thread();
        System.out.println(t1.getPriority());
    }
}
```

```

}
-----
class ThreadPrior1 extends Thread
{
    public void run()
    {
        int count = 0;
        for (int i=1; i<=1000000 ; i++ )
        {
            count++;
        }

        System.out.println("running thread name is:"+Thread.currentThread().getName());

        System.out.println("running thread priority is:"+Thread.currentThread().getPriority());
    }

    public static void main(String args[])
    {
        ThreadPrior1 m1 = new ThreadPrior1();
        ThreadPrior1 m2 = new ThreadPrior1();

        m1.setPriority(Thread.MIN_PRIORITY);//1
        m2.setPriority(Thread.MAX_PRIORITY);//10

        m1.setName("Last");
        m2.setName("First");

        m1.start();
        m2.start();
    }
}

```

Most of time the thread having highest priority will complete its task but we can't say that it will always complete its task first.

```

-----
Thread.yield() :
-----

```

It is a static method of Thread class.

It will send a notification to thread scheduler to stop the currently executing Thread (In Running state) and provide a chance to Threads which are in Runnable state to enter inside the running state having same priority or higher priority. Here The running Thread will directly move from Running state to Runnable state.

The Thread scheduler can ignore this notification message given by currently executing Thread.

Here there is no guarantee that after using yield() method the running Thread will move to Runnable state and from Runnable state the thread can move to Running state.[That is the reason yield() method is throwing InterruptedException]

If the thread which is in runnable state is having low priority then the current executing thread in Running state, will continue its execution.

It is mainly used to avoid the over-utilisation a CPU by the current Thread.

```
class Test implements Runnable
{
    @Override
    public void run()
    {
        for(int i=1; i<=10; i++)
        {
            String name = Thread.currentThread().getName();

            System.out.println("i value is :"+i+" by thread :"+name);

            if(name.equals("Child1"))
            {
                Thread.yield(); //Give a chance to Child2 Thread
            }
        }
    }
}

public class ThreadYieldMethod
{
    public static void main(String[] args)
    {
        Test obj = new Test();

        Thread t1 = new Thread(obj, "Child1");
        Thread t2 = new Thread(obj, "Child2");

        t1.start(); t2.start();
    }
}
```

Note : By using yield() we can make a request to thread scheduler to avoid over utilisation of CPU by current Thread.

ThreadGroup :-

There is a predefined class called ThreadGroup available in java.lang package.

In Java it is possible to group multiple threads in a single object so, we can perform a particular operation on a group of threads by a single method call.

The target of thread may be same or may be different.

The Thread class has the following constructor for ThreadGroup

```
new Thread(ThreadGroup groupName, Runnable target, String name);
```

ThreadGroup class has provided the following two methods :

public String getName() : Will provide group name

public int activeCount() : Will provide total number of active threads

in that group

```
package com.ravi.thread_group_demo;
```

```
class MyGroup implements Runnable
```

```
{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();
        for(int i=1; i<=5; i++)
        {
            System.out.println(i+" by "+name);
            try
            {
                Thread.sleep(500);
            }
            catch(InterruptedException e)
            {
                e.printStackTrace();
            }
        }
    }
}
```

```
public class ThreadGroupDemo1
```

```
{
    public static void main(String[] args)
    {
        ThreadGroup tg = new ThreadGroup("NIT_THREAD");

        var t1 = new Thread(tg, new MyGroup(), "Thread1");
        var t2 = new Thread(tg, new MyGroup(), "Thread2");
        var t3 = new Thread(tg, new MyGroup(), "Thread3");
        var t4 = new Thread(tg, new MyGroup(), "Thread4");

        t1.start(); t2.start(); t3.start(); t4.start();

        System.out.println("Name of the group is :"+tg.getName());
        System.out.println("Currently active threads in this group :"+tg.activeCount());

    }
}
```

```
package com.ravi.thread_group_demo;
```

```
class TatkalBooking implements Runnable
```

```
{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();
        System.out.println("Tatkal ticket booked by :"+name);
    }
}
```

```

}
}
class PremiumBooking implements Runnable
{
    @Override
    public void run()
    {
        String name = Thread.currentThread().getName();
        System.out.println("Premium ticket booked by :"+name);
    }
}

public class ThreadGroupDemo2
{
    public static void main(String[] args)
    {
        ThreadGroup tg1 = new ThreadGroup("Tatkal");
        ThreadGroup tg2 = new ThreadGroup("General");

        Thread t1 = new Thread(tg1, new TatkalBooking(), "Scott");
        Thread t2 = new Thread(tg1, new TatkalBooking(), "Smith");

        Thread t3 = new Thread(tg2, new PremiumBooking(), "John");
        Thread t4 = new Thread(tg2, new PremiumBooking(), "Martin");

        t1.start(); t2.start(); t3.start(); t4.start();
    }
}

```

```

package com.ravi.thread_group_demo;

```

```

public class MainThreadGroup
{
    public static void main(String[] args)
    {
        Thread t = Thread.currentThread();
        System.out.println(t.toString());

    }
}

```

Thread[main, 5, main] : Here toString() method describes that main thread is running under main group.

interrupt Method of Thread class :

It is a predefined non static method of Thread class. The main purpose of this method to disturb the execution of the Thread, if the thread is in waiting or sleeping state.

Whenever a thread is interrupted then it throws InterruptedException so the thread (if it is in sleeping or waiting mode) will get a chance to come out from a particular logic.

Points :-

If we call interrupt method and if the thread is not in sleeping or waiting state then it will behave normally.

If we call interrupt method and if the thread is in sleeping or waiting state then we can stop the thread gracefully.

*Overall interrupt method is mainly used to interrupt the thread safely so we can manage the resources easily.

Methods :

1) public void interrupt () :- Used to interrupt the Thread but the thread must be in sleeping or waiting mode.

2) public boolean isInterrupted() :- Used to verify whether thread is interrupted or not.

```
class Interrupt extends Thread
{
    @Override
    public void run()
    {
        Thread t = Thread.currentThread();
        System.out.println(t.isInterrupted());

        for(int i=1; i<=10; i++)
        {
            System.out.println(i);
            try
            {
                Thread.sleep(1000);
            }
            catch (InterruptedException e)
            {
                System.err.println("Thread is Interrupted ");
                e.printStackTrace();
            }
        }
    }
}

public class InterruptThread
{
    public static void main(String[] args)
    {
        Interrupt it = new Interrupt();
        System.out.println(it.getState());
        it.start();
        it.interrupt(); //main thread is interrupting the child thread
    }
}
```

```
-----
class Interrupt extends Thread
{
    public void run()
    {
        try
        {
```



```

        Thread.currentThread().interrupt();

        for(int i=1; i<=10; i++)
        {
            System.out.println("i value is :"+i);
            Thread.sleep(1000);
        }

    }
    catch (InterruptedException e)
    {
        System.err.println("Thread is Interrupted :"+e);
    }
    System.out.println("Child thread completed...");
}
}
public class InterruptThread1
{
    public static void main(String[] args)
    {
        System.out.println("Main thread is started");
        Interrupt it = new Interrupt();
        it.start();
        System.out.println("Main thread is ended");
    }
}

-----

public class InterruptThread2
{
    public static void main(String[] args)
    {
        Thread thread = new Thread(new MyRunnable());
        thread.start();

        try
        {
            Thread.sleep(3000);
        }
        catch (InterruptedException e)
        {
            e.printStackTrace();
        }

        thread.interrupt();
    }
}

class MyRunnable implements Runnable
{
    @Override
    public void run()
    {
        try
        {
            while (!Thread.currentThread().isInterrupted())

```

```

    {
        System.out.println("Thread is running by locking the resource");
        Thread.sleep(500);
    }
}
catch (InterruptedException e)
{
    System.out.println("Thread interrupted gracefully.");
}
finally
{
    System.out.println("Thread resource can be release here.");
}
}
}

```

08-07-2024

Deadlock :

It is a situation where two or more than two threads are in blocked state forever, here threads are waiting to acquire another thread resource without releasing its own resource.

This situation happens when multiple threads demands same resource without releasing its own attached resource so as a result we get Deadlock situation and our execution of the program will go to an infinite state as shown in the diagram. (08-JULY-24)

```

public class DeadlockExample
{
    public static void main(String[] args)
    {
        String resource1 = "Ameerpet";
        String resource2 = "S R Nagar";

        // t1 tries to lock resource1 then resource2

        Thread t1 = new Thread()
        {
            @Override
            public void run()
            {
                synchronized (resource1)
                {
                    System.out.println("Thread 1: locked resource 1");
                    try
                    {
                        Thread.sleep(1000);
                    }
                    catch (Exception e)
                    {}

                    synchronized (resource2) //Nested synchronized block
                    {
                        System.out.println("Thread 1: locked resource 2");
                    }
                }
            }
        };
    }
}

```

```

    }
}
};

// t2 tries to lock resource2 then resource1
Thread t2 = new Thread()
{
    @Override
    public void run()
    {
        synchronized (resource2)
        {
            System.out.println("Thread 2: locked resource 2");
            try
            {
                Thread.sleep(1000);
            }
            catch (Exception e)
            {}

            synchronized (resource1) //Nested synchronized block
            {
                System.out.println("Thread 2: locked resource 1");
            }
        }
    }
};
t1.start();
t2.start();
}
}

```

Note : Here this situation is known as Deadlock situation because both the threads are waiting for infinite state.

----- Daemon Thread [Service Level Thread] -----

Daemon thread is a low- priority thread which is used to provide background maintenance.

The main purpose of of Daemon thread to provide services to the user thread.

JVM can't terminate the program till any of the non-daemon (user) thread is active, once all the user thread will be completed then JVM will automatically terminate all Daemon threads, which are running in the background to support user threads.

The example of Daemon thread is Garbage Collection thread, which is running in the background for memory management.

In order to make a thread as a Daemon thread , we should use `setDaemon(true)` which is a non static method Thread class.

```

public class DaemonThreadDemo1
{
    public static void main(String[] args)

```

```

{
    System.out.println("Main Thread Started...");

    Thread daemonThread = new Thread(() ->
    {
        while (true)
        {
            System.out.println("Daemon Thread is running...");
            try
            {
                Thread.sleep(1000);
            }
            catch (InterruptedException e)
            {
                e.printStackTrace();
            }
        }
    });

    daemonThread.setDaemon(true);
    daemonThread.start();

    Thread userThread = new Thread(() ->
    {
        for (int i = 1; i <=19; i++)
        {
            System.out.println("User Thread: " + i);
            try
            {
                Thread.sleep(2000);
            }
            catch (InterruptedException e)
            {
                e.printStackTrace();
            }
        }
    });

    userThread.start();

    System.out.println("Main Thread Ended...");
}
}

```

Remaining methods of Object class :

Object cloning in java :

Object cloning is the process of creating an exact copy of an existing object in the memory.

Object cloning can be done by the following process :

1) Creating Shallow copy

2) Creating Deep copy

3) Using clone() method of java.lang.Object class

4) Passing Object reference to the Constructor.

Shallow Copy :

In shallow copy, we create a new reference variable which will point to same old existing object so if we make any changes through any of the reference variable then original object content will be modified.

Here we have one object and multiple reference variables.

HashCode of the object will be same because all the reference variables are pointing to the same object.

```
package com.ravi.clone_method;
```

```
class Student
```

```
{  
    int id;  
    String name;
```

```
    @Override
```

```
    public String toString()
```

```
{  
    return "Id is :" + id + "\nName is :" + name ;  
}
```

```
}  
public class ShallowCopy
```

```
{  
    public static void main(String[] args)
```

```
{  
        Student s1 = new Student();  
        s1.id = 111;  
        s1.name = "Ravi";
```

```
        System.out.println(s1);
```

```
        System.out.println("After Shallow Copy");
```

```
        Student s2 = s1; //shallow copy  
        s2.id = 222;  
        s2.name = "Shankar";
```

```
        System.out.println(s1);  
        System.out.println(s2);
```

```
        System.out.println(s1.hashCode());  
        System.out.println(s2.hashCode());  
    }
```

```
}
```

Deep Copy :

In deep copy, We create a copy of object in a different memory location. This is called a Deep copy.

Here objects are created in two different memory locations so if we modify the content of one object it will not reflect another object.

Here hashcode of both the objects will be different.

```
package com.ravi.clone_method;

class Employee
{
    int id;
    String name;

    @Override
    public String toString()
    {
        return "Employee [id=" + id + ", name=" + name + "]";
    }
}

public class DeepCopy
{
    public static void main(String[] args)
    {
        Employee e1 = new Employee();
        e1.id = 111;
        e1.name = "Ravi";

        Employee e2 = new Employee();
        e2.id = e1.id;
        e2.name = e1.name;

        System.out.println(e1 + " : "+e2);

        e2.id = 222;
        e2.name = "shankar";
        System.out.println(e1 + " : "+e2);

        System.out.println(e1.hashCode() + " : "+e2.hashCode());
    }
}
```

protected native Object clone() throws CloneNotSupportedException

Object cloning in Java is the process of creating an exact copy of the original object. In other words, it is a way of creating a new object by copying all the data and attributes from the original object.

The clone method of Object class creates an exact copy of an object.

In order to use clone() method , a class must implements Cloneable interface because we can perform cloning operation on Cloneable objects only [JVM must have additional information].

We can say an object is a Cloneable object if the corresponding class implements Cloneable interface.

It throws a checked Exception i.e CloneNotSupportedException

Note :- clone() method is not the part of Cloneable interface[marker interface], actually it is the method of Object class.

clone() method of Object class follow deep copy concept so hashcode will be different as well as if we modify one object content then another object content will not be modified.

clone() method of Object class has protected access modifier so we need to override clone() method in sub class.

```
package com.ravi.clone_method;
```

```
class Customer implements Cloneable
```

```
{
    int id;
    String name;
```

```
    @Override
```

```
    public String toString()
```

```
{
    return "Customer [id=" + id + ", name=" + name + "]";
}
```

```
    @Override
```

```
    public Object clone() throws CloneNotSupportedException
```

```
{
    return super.clone();
}
```

```
}
```

```
public class CloneMethod
```

```
{
    public static void main(String[] args) throws CloneNotSupportedException
```

```
{
    Customer c1 = new Customer();
    c1.id = 111;
    c1.name = "Scott";
    System.out.println(c1);
```

```
    System.out.println("After calling clone method");
```

```
        Customer c2 = (Customer) c1.clone();
```

```
        System.out.println(c2);
```

```
        c2.id = 222;
```

```
        c2.name = "Smith";
```

```
        System.out.println(".....");
```

```
        System.out.println(c1);
```

```
        System.out.println(c2);
```

```

        System.out.println(c1.hashCode() + " : "+c2.hashCode());
    }

}

```

protected void finalize() throws Throwable :

It is a predefined method of Object class.

Garbage Collector automatically call this method just before an object is eligible for garbage collection to perform clean-up activity.

Here clean-up activity means closing the resources associated with that object like file connection, database connection, network connection and so on we can say resource de-allocation.

Note :- JVM calls finalize method only one per object.

Diagram (08-JULY-24)

```

package com.ravi.finalize_method;

```

```

public class Student
{
    int id;
    String name;

    public Student(int id, String name)
    {
        this.id= id;
        this.name = name;
    }
}

```

```

@Override
public String toString()
{
    return "Id is :"+id+"\nName is :"+name;
}

```

```

@Override
protected void finalize()
{
    System.out.println("JVM call this finalize method...");
    System.out.println("Just before Student object destruction");
}

```

```

public static void main(String[] args) throws InterruptedException
{
    Student s1 = new Student(111,"Ravi");
    System.out.println(s1.hashCode());
    System.out.println(s1);

    s1 = null;
    System.gc(); //Explicitly calling Garbage Collector
}

```



```
Thread.sleep(3000);
System.out.println(s1);
}

}
```

*What is the difference between final, finally and finalize

final :- It is a keyword which is used to provide some kind of restriction like class is final, Method is final, variable is final.

finally :- if we open any resource as a part of try block then that particular resource must be closed inside finally block otherwise program will be terminated ab-normally and the corresponding resource will not be closed (because the remaining lines of try block will not be executed)

finalize() :- It is a method which JVM is calling automatically just before object destruction so if any resource (database, file and network) is associated with that particular object then it will be closed or de-allocated by JVM by calling finalize().

09-07-2024

Collection Framework in java (40 - 45% IQ)

Collections framework is nothing but handling individual Objects(Collection Interface) and Group of objects(Map interface).

We know only object can move from one network to another network.

A collections framework is a class library to handle group of Objects.

It is implemented by using java.util package.

It provides an architecture to store and manipulate group of objects.

All the operations that we can perform on data such as searching, sorting, insertion and deletion can be done by using collections framework because It is the data structure of Java.

The simple meaning of collections is single unit of Objects.

It provides the following sub interfaces :

- 1) List (Accept duplicate elements)
- 2) Set (Not accepting duplicate elements)
- 3) Queue (Storing and Fetching the elements based on some order i.e FIFO)

Note : Collection is a predefined interface available in java.util package where as Collections is a predefined class which is available from JDK 1.2V which contains only static methods (Constructor is private)

10-07-2024

Methods of Collection interface :

- a) public boolean add(E element) :- It is used to add an item/element in the in the collection.
- b) public boolean addAll(Collection c) :- It is used to insert the specified collection elements in the existing collection(For merging the Collection)
- c) public boolean retainAll(Collection c) :- It is used to retain all the elements from existing element.
(Common Element)
- d) public boolean removeAll(Collection c) :- It is used to delete all the elements from the existing collection.
- e) public boolean remove(Object element) :- It is used to delete an element from the collection based on the object.
- f) public int size() :- It is used to find out the size of the Collection [Total number of elements available]
- g) public void clear() :- It is used to clear all the elements at once from the Collection.

All the above methods of Collection interface will be applicable to all the sub interfaces like List, Set and Queue.

List interface :

It is the sub interface of Collection interface.

It stores the elements on the basis of index.

It accepts duplicate elements.

We can perform sorting operation manually.

List interface Hierarchy :

Hierarchy is given in the paint Diagram [10-JULY]

Behavior of List interface implemented classes :

- 1) It stores the elements based on the index.
- 2) It can accept duplicate, homogeneous, heterogeneous as well as null.
- 3) It stores everything in the form of Object.
- 4) By using generic we can eliminate compilation warning and still we can take heterogeneous by using <Object>
- 5) Most of the List interface classes are dynamically Growable.

11-07-2024

Methods of List interface :

- 1) public boolean isEmpty() :- Verify whether List is empty or not
- 2) public void clear() :- Will clear all the elements

- 3) `public int size()` :- To get the size of the Collections
- 4) `public void add(int index, Object o)` :- Insert the element based on the index position.
- 5) `public boolean addAll(int index, Collection c)` :- Insert the Collection based on the index position
- 6) `public Object get(int index)` :- To retrieve the element based on the index position
- 7) `public Object set(int index, Object o)` :- To override or replace the existing element based on the index position
- 8) `public Object remove(int index)` :- remove the element based on the index position
- 9) `public boolean remove(Object element)` :- remove the element based on the object element, It is the Collection interface method extended by List interface
- 10) `public int indexOf()` :- index position of the element
- 11) `public int lastIndex()` :- last index position of the element
- 12) `public Iterator iterator()` :- To fetch or iterate or retrieve the elements from Collection in forward direction only.
- 13) `public ListIterator listIterator()` :- To fetch or iterate or retrieve the elements from Collection in forward and backward direction.

**How many ways we can fetch the Collection Object :

There are 9 ways to fetch the Collection Object which are as follows :

- 1) By using `toString()` method of respective class [1.0]
- 2) By using Ordinary for loop [1.0]
- 3) By using for Each loop [1.5]
- *4) By using Enumeration interface [1.0]
- *5) By using Iterator interface [1.2]
- *6) By using ListIterator interface [1.2]
- *7) By using SplitIterator interface [1.8]
- 8) By using `forEach(Consumer<T> cons)` method [1.8]
- 9) By using Method Reference (`::`) [1.8]

Note : Among all these 9, Enumeration, Iterator, ListIterator and SplitIterator are the cursors [Cursor means it can move from one direction to another direction]

Enumeration :

It is a predefined interface available in `java.util` package from JDK 1.0 onwards (Legacy interface).

We can use Enumeration interface to fetch or retrieve the Objects one by one from the Collection because it is a cursor.

We can create Enumeration object by using `elements()` method of the legacy Collection class.

```
public Enumeration elements();
```

Enumeration interface contains two methods :

1) public boolean hasMoreElements() :- It will return true if the Collection is having more elements.

2) public Object nextElement() :- It will return collection object so return type is Object and move the cursor to the next line.

Note :- It will only work with legacy Collections classes.

Iterator interface :

It is a predefined interface available in java.util package available from 1.2 version.

It is used to fetch/retrieve the elements from the Collection in forward direction only because it is also a cursor.

```
public Iterator iterator();
```

Example :

```
Iterator itr = vector.iterator();
```

Now, Iterator interface has provided two methods

public boolean hasNext() :-

It will verify, the element is available in the next position or not, if available it will return true otherwise it will return false.

public Object next() :- It will return the collection object.

ListIterator interface :

It is a predefined interface available in java.util package and it is the sub interface of Iterator available from JDK 1.2v.

It is used to retrieve the Collection object in both the direction i.e in forward direction as well as in backward direction.

```
public ListIterator listIterator();
```

Example :

```
ListIterator lit = v.listIterator();
```

1) public boolean hasNext() :-

It will verify the element is available in the next position or not, if available it will return true otherwise it will return false.

2) public Object next() :- It will return the next position collection object.

3) public boolean hasPrevious() :-

It will verify the element is available in the previous position or not, if available it will return true otherwise it will return false.

4) public Object previous () :- It will return the previous position collection object.

Note :- Apart from these 4 methods we have add(), set() and remove() method in ListIterator interface

SplitIterator :

Spliterator interface :

It is a predefined interface available in java.util package from java 1.8 version.

It is a cursor through which we can fetch the elements from the Collection [Collection, array, Stream]

It is the combination of hasNext() and next() method.

It is using forEachRemaining(Consumer <T>) method for fetching the elements.

By using forEach() method :

From java 1.8 onwards every collection class provides a method forEach() method, this method takes Consumer functional interface as a parameter.
This method is available in java.lang.Iterable interface.

package com.ravi.collection;

import java.util.Vector;
import java.util.function.Consumer;

public class ForEachInternalImplementation {

```
public static void main(String[] args)
{
    Vector<String> fruits = new Vector<>();
    fruits.add("Orange");
    fruits.add("Apple");
    fruits.add("Mango");
    fruits.add("Banana");
    fruits.add("Gauva");
    fruits.add("KIWI");
```

```
    Consumer<String> cons = new Consumer<String>()
    {
        @Override
        public void accept(String t)
        {
            System.out.println(t.toUpperCase());
        }
    };
    fruits.forEach(cons);
```

```
    fruits.forEach(cons);
```

```
}
```

```
}
```

```
-----  
package com.ravi.collection;
```

```
import java.util.Vector;  
import java.util.function.Consumer;
```

```
public class ForEachInternalImplementation {
```

```
    public static void main(String[] args)  
    {  
        Vector<String> fruits = new Vector<>();  
        fruits.add("Orange");  
        fruits.add("Apple");  
        fruits.add("Mango");  
        fruits.add("Banana");  
        fruits.add("Gauva");  
        fruits.add("KIWI");
```

```
        Consumer<String> cons = str -> System.out.println(str);
```

```
        fruits.forEach(cons);
```

```
    }
```

```
}
```

```
-----  
package com.ravi.collection;
```

```
import java.util.Vector;  
import java.util.function.Consumer;
```

```
public class ForEachInternalImplementation {
```

```
    public static void main(String[] args)  
    {  
        Vector<String> fruits = new Vector<>();  
        fruits.add("Orange");  
        fruits.add("Apple");  
        fruits.add("Mango");  
        fruits.add("Banana");  
        fruits.add("Gauva");  
        fruits.add("KIWI");
```

```
        fruits.forEach(str -> System.out.println(str.toUpperCase()));
```

```
    }
```

```
}
```

//Program to shown how to fetch Colollection object by using 9 ways :

```
package com.ravi.collection;

import java.util.Enumeration;
import java.util.Iterator;
import java.util.ListIterator;
import java.util.Spliterator;
import java.util.Vector;

public class RetrievingCollectionObject {

    public static void main(String[] args)
    {
        Vector<String> fruits = new Vector<>();
        fruits.add("Orange");
        fruits.add("Apple");
        fruits.add("Mango");
        fruits.add("Banana");
        fruits.add("Gauva");
        fruits.add("KIWI");

        System.out.println("BY USING TOSTRING METHOD :");
        System.out.println(fruits.toString());

        System.out.println("BY USING ORDINARY FOR LOOP :");

        for(int i=0; i<fruits.size(); i++)
        {
            System.out.println(fruits.get(i));
        }

        System.out.println("BY USING FOR EACH LOOP :");

        for(String fruit : fruits)
        {
            System.out.println(fruit);
        }

        System.out.println("BY USING ENUMERATION INTERFACE:");

        Enumeration<String> ele = fruits.elements();
        while(ele.hasMoreElements())
        {
            System.out.println(ele.nextElement());
        }

        System.out.println("BY USING ITERATOR INTERFACE :");

        Iterator<String> itr = fruits.iterator();

        while(itr.hasNext())
        {
            System.out.println(itr.next());
        }
    }
}
```

```

System.out.println("BY USING LISTITERATOR INTERFACE :");

ListIterator<String> litlitr = fruits.listIterator();
System.out.println("IN FORWARD DIRECTION");

while(litlitr.hasNext())
{
    System.out.println(litlitr.next());
}

    System.out.println("IN BACKWARD DIRECTION");

while(litlitr.hasPrevious())
{
    System.out.println(litlitr.previous());
}

System.out.println("BY USING SPLITITERATOR INTERFACE :");

    Spliterator<String> spllitr = fruits.splitIterator();
    spllitr.forEachRemaining(fruit -> System.out.println(fruit));

System.out.println("BY USING FOR EACH METHOD :");
fruits.forEach(fruit -> System.out.println(fruit.toUpperCase()));

System.out.println("BY USING METHOD REFERENEC :");
fruits.forEach(System.out::println);

}

}

```

12-07-2024

Working with List interface specific classes :

Vector<E> :

public class Vector<E> extends AbstractList<E> implements List<E>, Serializable, Clonable, RandomAccess

Vector is a predefined class available in java.util package under List interface.

Vector is always from java means it is available from jdk 1.0 version.

It can accept duplicate, null. homogeneous as well as hetrogeneous elements.

Vector and Hashtable, these two classes are available from jdk 1.0, remaining Collection classes were added from 1.2 version. That is the reason Vector and Hashtable are called legacy(old) classes.

The main difference between Vector and ArrayList is, ArrayList methods are not synchronized so multiple threads can access the method of ArrayList where as on the other hand most the methods are synchronized in Vector so performance wise Vector is slow.

*We should go with ArrayList when ThreadSafety is not required on the other hand we should go with Vector when we need ThreadSafety for retrieval operation.

It also stores the elements on index basis. It is dynamically growable with initial capacity 10. The next capacity will be 20 i.e double of the first capacity.

new capacity = current capacity * 2;

It implements List, Serializable, Cloneable, RandomAccess interfaces.

Constructors in Vector :

We have 4 types of Constructor in Vector

1) Vector v1 = new Vector();
It will create the vector object with default capacity is 10

2) Vector v2 = new Vector(int initialCapacity);
Will create the vector object with user specified capacity.

3) Vector v3 = new Vector(int initialCapacity, int capacityIncrement);
Eg :- Vector v = new Vector(1000,5);

Initially It will create the Vector Object with initial capacity 1000 and then when the capacity will be full then increment by 5 so the next capacity would be 1005, 1010 and so on.

4) Vector v4 = new Vector(Collection c);
We can achieve loose coupling.

Program that shows performance wise Vector is not good in comparison to ArrayList

System is a predefined class available in java.lang package and it contains a predefined static method currentTimeMillis() , the return type of this method is long, actually it returns the current time of the system in ms.

```
public static native long currentTimeMillis()
```

PerformanceComparison.java

```
package com.ravi.collection;
```

```
import java.util.ArrayList;  
import java.util.Vector;
```

```
public class PerformanceComparison {
```

```
    public static void main(String[] args)  
    {
```

```
        ArrayList<Integer> al = new ArrayList<>();
```

```
        long startTime = System.currentTimeMillis();
```

```
        for(int i=0; i<=1000000; i++)  
        {
```

```

        al.add(i);
    }

    long endTime = System.currentTimeMillis();
    System.out.println("Time taken by ArrayList :"+(endTime - startTime)+ " ms");

    Vector<Integer> v = new Vector<>();

    startTime = System.currentTimeMillis();

    for(int i=0; i<=1000000; i++)
    {
        v.add(i);
    }

    endTime = System.currentTimeMillis();
    System.out.println("Time taken by Vector :"+(endTime - startTime)+ " ms");

}

}

```

Note :

 From the above program, It is clear that performance wise ArrayList is more better than Vector.

 //Vector Program on capacity

```

package com.ravi.vector;

import java.util.*;

public class VectorDemo1 {
    public static void main(String[] args)
    {
        Vector<Integer> v = new Vector<>(100,10); // initial capacity is 100
        System.out.println("Initial capacity is :"+ v.capacity());

        for (int i = 0; i < 100; i++)
        {
            v.add(i);
        }

        System.out.println("After adding 100 elements capacity is :"+ v.capacity()); // 100
        v.add(101);
        System.out.println("After adding 101th elements capacity is :"+ v.capacity()); // 110

        for(Integer i : v)
        {
            System.out.print(i+"\t");
            if(i%5==0)
            {
                System.out.println();
            }
        }
    }
}

```

```

}
}
-----
package com.ravi.collection;

import java.util.Collections;
import java.util.Vector;

public class VectorDemo2 {

    public static void main(String[] args)
    {
        Vector<String> fruits = new Vector<>();
        fruits.add("Orange");
        fruits.add("Apple");
        fruits.add("Mango");
        fruits.add("Grapes");
        fruits.add("Gauva");
        fruits.add("Mango");

        fruits.remove(3);
        fruits.remove("Gauva");
        System.out.println("After removing the elements are :");
        fruits.forEach(System.out::println);

        Collections.sort(fruits);
        System.out.println("After Sorting the elements are :");
        fruits.forEach(System.out::println);

        System.out.println("Size is :"+fruits.size());
        System.out.println("Is the List is empty ?"+fruits.isEmpty());

    }
}
-----

```

Working with custom object :

```

-----
package com.ravi.collection;

import java.util.Vector;

record Product(Integer productId, String productName)
{
}

public class VectorDemo3
{
    public static void main(String[] args)
    {
        Vector<Product> listOfProducts = new Vector<>();
    }
}

```

```
listOfProducts.add(new Product(333, "Laptop"));
listOfProducts.add(new Product(222, "Mobile"));
listOfProducts.add(new Product(111, "Camera"));
```

```
listOfProducts.forEach(System.out::println);
```

```
}
```

```
}
```

15-07-2024

```
package com.ravi.vector;
```

```
import java.util.Scanner;
```

```
import java.util.Vector;
```

```
public class VectorDemo4
```

```
{
    public static void main(String[] args)
    {
        Vector<String> toDoList = new Vector<>();

        Scanner scanner = new Scanner(System.in);

        int choice;
        do
        {
            System.out.println("ToDo List Menu:");
            System.out.println("1. Add Task");
            System.out.println("2. View Tasks");
            System.out.println("3. Mark Task as Completed");
            System.out.println("4. Exit");
            System.out.print("Enter your choice: ");

            choice = scanner.nextInt();
            scanner.nextLine();

            switch (choice)
            {
                case 1:
                    // Add Task
                    System.out.print("Enter task description: ");
                    String task = scanner.nextLine();
                    toDoList.add(task);
                    System.out.println("Task added successfully!\n");
                    break;
                case 2:
                    // View Tasks
                    System.out.println("ToDo List:");
                    for (int i = 0; i < toDoList.size(); i++)
                    {
                        System.out.println((i + 1) + ". " + toDoList.get(i));
                    }
                    System.out.println();
            }
        }
    }
}
```

```

        break;
    case 3:
        // Mark Task as Completed
        System.out.print("Enter task number to mark as completed: ");
        int taskNumber = scanner.nextInt();
        if (taskNumber >= 1 && taskNumber <= toDoList.size())
        {
            String completedTask = toDoList.remove(taskNumber - 1);
            System.out.println("Task marked as completed: " + completedTask + "\n");
        } else {
            System.out.println("Invalid task number!\n");
        }
        break;
    case 4:
        System.out.println("Exiting ToDo List application. Goodbye!");
        break;
    default:
        System.out.println("Invalid choice. Please enter a valid option.\n");
}

} while (choice != 4);

scanner.close();
}
}

```

```

package com.ravi.vector;
//Array To Collection
import java.util.*;
public class VectorDemo5
{
    public static void main(String args[])
    {
        Vector<Integer> v = new Vector<>();

        int x[]={22,20,10,40,15,58};

        //Adding array values to Vector
        for(int i=0; i<x.length; i++)
        {
            v.add(x[i]);
        }
        Collections.sort(v);
        System.out.println("Maximum element is :"+Collections.max(v));
        System.out.println("Minimum element is :"+Collections.min(v));
        System.out.println("Vector Elements :");
        v.forEach(y -> System.out.println(y));
        System.out.println(".....");
        Collections.reverse(v);
        v.forEach(y -> System.out.println(y));
    }
}

```

Stack :


```

        System.out.println("After deletion elements are :"+s);

        System.out.println("Is the Stack empty ? :"+s.empty());
    }
    catch(EmptyStackException e)
    {
        e.printStackTrace();
    }
}
}

```

```

//add(Object obj) is the method of Collection
package com.ravi.stack;
import java.util.*;
public class Stack2
{
    public static void main(String args[])
    {
        Stack<Integer> st1 = new Stack<>();
        st1.add(10);
        st1.add(20);
        st1.forEach(x -> System.out.println(x));

        Stack<String> st2 = new Stack<>();
        st2.add("Java");
        st2.add("is");
        st2.add("programming");
        st2.add("language");
        st2.forEach(x -> System.out.println(x));

        Stack<Character> st3 = new Stack<>();
        st3.add('A');
        st3.add('B');
        st3.forEach(x -> System.out.println(x));

        Stack<Double> st4 = new Stack<>();
        st4.add(10.5);
        st4.add(20.5);
        st4.forEach(x -> System.out.println(x));
    }
}

```

```

package com.ravi.stack;
import java.util.Stack;

public class Stack3
{
    public static void main(String[] args)
    {
        Stack<String> stk= new Stack<>();
        stk.push("Apple");
        stk.push("Grapes");
        stk.push("Mango");
    }
}

```

```

stk.push("Orange");
System.out.println("Stack: " + stk);

String fruit = stk.peek();
System.out.println("Element at top: " + fruit);
System.out.println("Stack elements are : " + stk);
}
}

```

//Searching an element in the Stack

```

package com.ravi.stack;
import java.util.Stack;
public class Stack4
{
    public static void main(String[] args)
    {
        Stack<String> stk= new Stack<>();
        stk.push("Apple");
        stk.push("Grapes");
        stk.push("Mango");
        System.out.println("Offset Position is : " + stk.search("Mango")); //1
        System.out.println("Offser Position is : " + stk.search("Banana")); //-1
        System.out.println("Is stack empty ? "+stk.empty()); //false

        System.out.println("Index Position is : " + stk.indexOf("Mango")); //2
    }
}

```

ArrayList<E> :

```

public class ArrayList<E> extends AbstractList<E> implements List<E>, Serializable, Clonable,
RandomAccess

```

It is a predefined class available in java.util package under List interface from java 1.2v.

It accepts duplicate elements and null values.

It is dynamically growable array.

It stores the elements on index basis so it is simillar to dynamic array.

Initial capacity of ArrayList is 10. The new capacity of Arraylist can be calculated by using the formula
 $\text{new capacity} = (\text{current capacity} * 3) / 2 + 1$ [Almost 50% increment]

*All the methods declared inside an ArrayList is not synchronized so multiple thread can access the method of ArrayList.

*It is highly suitable for fetching or retriving operation when duplicates are allowed and Thread-safety is not required.

It implements List,Serializable, Clonable, RandomAccess interfcaes

Constructor of ArrayList :

In ArrayList we have 3 types of Constructor:

Constructor of ArrayList :

We have 3 types of Constructor in ArrayList

1) ArrayList al1 = new ArrayList();

Will create ArrayList object with default capacity 10.

2) ArrayList al2 = new ArrayList(int initialCapacity);

Will create an ArrayList object with user specified Capacity

3) ArrayList al3 = new ArrayList(Collection c)

We can copy any Collection interface implemented class data to the current object reference (Copying one Collection data to another)

```
package com.ravi.arraylist;

import java.util.*;
public class ArrayListDemo
{
    public static void main(String... a)
    {
        ArrayList<String> arl = new ArrayList<>();//Generic type
        arl.add("Apple");
        arl.add("Orange");
        arl.add("Grapes");
        arl.add("Mango");
        arl.add("Guava");
        arl.add("Mango");
```

```
        Collections.sort(arl);
        System.out.println("In Ascending Order");
        arl.forEach(System.out::println);
```

```
        Collections.reverse(arl);
        System.out.println("In reverse Order");
        arl.forEach(System.out::println);
```

```
    }
}
```

//Working with custom object

```
package com.ravi.arraylist;
```

```
import java.util.ArrayList;
```

```
record Customer(Integer customerId, String customerName, Double custBill)
```

```
{
```

```
}
```

```
public class ArrayListDemo1
```

```
{
```

```

public static void main(String[] args)
{
    ArrayList<Customer> al = new ArrayList<>();
    al.add(new Customer(333, "Rohan", 29789.90));
    al.add(new Customer(111, "Satish", 23789.90));
    al.add(new Customer(222, "Aryan", 25789.90));
    al.add(new Customer(444, "Zuber", 27789.90));

    al.forEach(System.out::println);

}
}

```

```

package com.ravi.arraylist;

//Program to merge and retain of two collection
import java.util.*;
public class ArrayListDemo2
{
    public static void main(String args[])
    {
        ArrayList<String> al1=new ArrayList<>();
        al1.add("Ravi");
        al1.add("Rahul");
        al1.add("Rohit");

        ArrayList<String> al2=new ArrayList<>();
        al2.add("Pallavi");
        al2.add("Sweta");
        al2.add("Puja");

        al1.addAll(al2);

        al1.forEach(x -> System.out.println(x.toUpperCase()));

        System.out.println(".....");

        ArrayList<String> al3=new ArrayList<>();
        al3.add("Ravi");
        al3.add("Rahul");
        al3.add("Rohit");

        ArrayList<String> al4=new ArrayList<>();
        al4.add("Pallavi");
        al4.add("Rahul");
        al4.add("Raj");

        al3.retainAll(al4);

        al3.forEach(x -> System.out.println(x));
    }
}

```

How to Create an Immutable and Fixed length List :

1) How to create a fixed length list :

If we want to create a fixed length list then we have a predefined method called `asList(T ...x)` available in `Arrays` class, the return type of this method is `List` interface. It will create a fixed length array of type `List` so further we can't add any element but we can replace existing element. If we try to add more elements then we will get `java.lang.UnsupportedOperationException` as shown below.

```
package com.ravi.arraylist;

import java.util.Arrays;
import java.util.List;

public class ImmutableList {

    public static void main(String[] args)
    {
        List<Integer> listOfNumbers = Arrays.asList(12,90,56,89,45);
        System.out.println(listOfNumbers);
        listOfNumbers.set(0,100); //valid
        System.out.println(listOfNumbers);
        listOfNumbers.add(120);

    }

}
```

2) Creating immutable List :

If we want to create an immutable list then we can use a predefined static method of `List` interface called `of(T ...x)` which will create immutable `List` so further we can't perform any kind of operation even we can't replace an existing element otherwise we will get `java.lang.UnsupportedOperationException` as shown below.

```
package com.ravi.arraylist;

import java.util.List;

public class ImmutableList1 {

    public static void main(String[] args)
    {
        List<String> listOfNames = List.of("Rahul","Ankit","Rohan");
        System.out.println(listOfNames);

        listOfNames.add(0,"Rohit"); //Invalid

    }

}
```

//Program to fetch the elements in forward and backward

//direction using ListIterator interface

```
package com.ravi.arraylist;

import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import java.util.ListIterator;

public class ArrayListDemo3
{
    public static void main(String args[])
    {
        List<String> listOfName = Arrays.asList("Rohit","Akshar","Pallavi","Sweta"); //Length is fixed

        Collections.sort(listOfName);

        //Fetching the data in both the direction
        ListIterator<String> lst = listOfName.listIterator();

        System.out.println("In Forward Direction..");
        while(lst.hasNext())
        {
            System.out.println(lst.next());
        }
        System.out.println("In Backward Direction..");
        while(lst.hasPrevious())
        {
            System.out.println(lst.previous());
        }
    }
}
```

//Serialization and De-serialization on ArrayList Object

```
package com.ravi.arraylist;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.util.ArrayList;

public class ArrayListDemo4
{
    public static void main(String[] args) throws IOException
    {
        ArrayList<String> cityName = new ArrayList<>();
        cityName.add("Hyderabad");
        cityName.add("Pune");
        cityName.add("Banglore");
        cityName.add("Chennai");

        //Serialization
        var fout = new FileOutputStream("C:\\new\\CityName.txt");
```

```

var oos = new ObjectOutputStream(fout);

try(fout ; oos)
{
    oos.writeObject(cityName);
    System.out.println("Object stored successfully");
}
catch(Exception e)
{
    e.printStackTrace();
}

//De-serialization
var fin = new FileInputStream("C:\\new\\CityName.txt");
var ois = new ObjectInputStream(fin);

try(fin; ois)
{
    ArrayList<String> list = (ArrayList<String>) ois.readObject();
    System.out.println(list);
}
catch(Exception e)
{
    e.printStackTrace();
}

}
}

```

```

package com.ravi.arraylist;

```

```

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;
import java.util.ArrayList;

```

```

record Employee(Integer employeeId, String employeeName) implements Serializable
{
}

```

```

public class ArrayListSerialization
{
    public static void main(String[] args) throws IOException
    {
        ArrayList<Employee> listOfEmployees = new ArrayList<>();
        listOfEmployees.add(new Employee(111, "A"));
        listOfEmployees.add(new Employee(222, "B"));
        listOfEmployees.add(new Employee(333, "C"));
        listOfEmployees.add(new Employee(444, "D"));
    }
}

```

```

//Serialization

```

```

var fout = new FileOutputStream("C:\\new\\EmployeeData.txt");
var oos = new ObjectOutputStream(fout);

try(fout;oos)
{
    oos.writeObject(listOfEmployees);
    System.out.println("Object Stored Successfully");
}
catch(Exception e)
{
    e.printStackTrace();
}

//De-serialization
var fin = new FileInputStream("C:\\new\\EmployeeData.txt");
var ois = new ObjectInputStream(fin);

try(fin;ois)
{
    ArrayList<Employee> list = (ArrayList<Employee>)ois.readObject();
    System.out.println(list);
}
catch(Exception e)
{
    e.printStackTrace();
}

}

}

```

```
public void ensureCapacity(int minimumCapacity) :
```

As we know we don't have capacity() method with ArrayList class.

In ArrayList, once we create the object then the default capacity is 10. After object creation, if we want to resize our arraylist capacity then we can use ensureCapacity(int minimumCapacity) method of ArrayList class.

This method will ensure that the ArrayList must hold the minimum capacity element which is specified as a parameter to ensureCapacity()

Even after resizing, It is dynamically Growable.

```

package com.ravi.arraylist;

import java.util.ArrayList;
import java.util.LinkedList;

public class ArrayListDemo5
{
    public static void main(String[] args)
    {
        ArrayList<String> city= new ArrayList<>();
    }
}

```

```
city.ensureCapacity(3); //resize the capacity of ArrayList
city.add("Hyderabad");
city.add("Mumbai");
city.add("Delhi");
```

```
city.add("Kolkata");
System.out.println("ArrayList: " + city);
}
}
```

```
package com.ravi.arraylist;
```

//Program on ArrayList that contains null values as well as we can pass the element based on the index position

```
import java.util.ArrayList;
import java.util.LinkedList;
public class ArrayListDemo6
{
    public static void main(String[] args)
    {
        ArrayList<Object> al = new ArrayList<>(); //Generic type
        al.add(12);
        al.add("Ravi");
        al.add(12);
        al.add(3,"Hyderabad"); //add(int index, Object o)method of List interface
        al.add(1,"Naresh");
        al.add(null);
        al.add(11);
        System.out.println(al); //12 Naresh Ravi 12 Hyderabad
    }
}
```

```
package com.ravi.arraylist;
```

```
import java.util.ArrayList;
import java.util.List;
```

```
class Professor
{
    private String name;
    private String specialization;

    public Professor(String name, String specialization)
    {
        this.name = name;
        this.specialization = specialization;
    }

    public String getName()
    {
        return name;
    }
}
```

```

    }

    public String getSpecialization()
    {
        return specialization;
    }

    @Override
    public String toString() {
        return "Professor [name=" + name + ", specialization=" + specialization + "]";
    }
}

class Department
{
    private String name;
    private final List<Professor> professors; // Department "HAS-A" relationship with Professor

    public Department(String name)
    {
        this.name = name;
        this.professors = new ArrayList<>();
    }

    public void addProfessor(Professor prof)
    {
        professors.add(prof);
    }

    public String getName()
    {
        return name;
    }

    public List<Professor> getProfessors()
    {
        return professors;
    }
}

public class ArrayListDemo7
{
    public static void main(String[] args)
    {
        Professor prof1 = new Professor("Scott", "Java");
        Professor prof2 = new Professor("Rahul", "Python");
        Professor prof3 = new Professor("Samir", ".Net");

        Department csd = new Department("Computer Science");
        csd.addProfessor(prof1);
        csd.addProfessor(prof2);
        csd.addProfessor(prof3);

        // Accessing properties through the "HAS-A" relationship
    }
}

```



```

        System.out.println("Department Name: " + csd.getName());

        System.out.println("Professors in " + csd.getName() + ":");

        List<Professor> professors = csd.getProfessors();
        professors.forEach(System.out::println);

    }
}

```

Limitation of ArrayList :

Time Complexcity of ArrayList :

The time complexcity of ArrayList to insert and delete an element from the middle would be $O(n)$ [Big O of n] because 'n' number of elements will be re-located so it is not a good choice to perform insertion and deletion operation in the middle of the List.

On the other hand time complexcity of ArrayList to retrieve an element from the List would be $O(1)$ because by using `get(int index)` method we can retrieve the element randomly from the list. ArrayList class implements RandomAccess marker interface which provides the facility to fetch the elements Randomly.

[16-JUL-24]

In order to perform frequent insertion and deletion operation, we introduced LinkedList.

ConcurrentModificationException :

```

package com.ravi.linked_list;
import java.util.ArrayList;
import java.util.Iterator;

class Concurrent extends Thread
{
    ArrayList<String> arl = null;

    public Concurrent(ArrayList<String> arl)
    {
        super();
        this.arl = arl;
    }
    @Override
    public void run()
    {
        try
        {
            Thread.sleep(2000);
        }
        catch (InterruptedException e)
        {
            e.printStackTrace();
        }
    }
}

```

```

    }
    arl.add("Goa");
  }
}

```

```

public class ConcurrentModificationException
{
    public static void main(String[] args) throws InterruptedException
    {
        ArrayList<String> listOfCity = new ArrayList<>();
        listOfCity.add("Hyderabad");
        listOfCity.add("Kolkata");
        listOfCity.add("Bhubneswar");
        listOfCity.add("Indore");
        listOfCity.add("Mumbai");

        Concurrent cc = new Concurrent(listOfCity);
        cc.start();

        Iterator<String> itr = listOfCity.iterator();

        while(itr.hasNext())
        {
            System.out.println(itr.next());
            Thread.sleep(500);
        }

    }
}

```

17-07-2024

LinkedList :

public class LinkedList<E> extends AbstractSequentialList<E> implements List<E>, Deque<E>, Cloneable, Serializable

It is a predefined class available in java.util package under List interface from JDK 1.2v.

It is ordered by index position like ArrayList except the elements (nodes) are doubly linked to one another. This linkage provide us new method for adding and removing the elements from the middle of LinkedList.

*The important thing is, LikedList may iterate more slowly than ArrayList but LinkedList is a good choice when we want to insert or delete the elements frequently in the list.

From jdk 1.6 onwards LinkedList class has been enhanced to support basic queue operation by implementing Deque<E> interface.

LinkedList methods are not synchronized.

It inserts the elements by using Doubly linked List so insertion and deleteion is very easy.

ArrayList is using Dynamic array data structure but LinkedList class is using LinkedList (Doubly LinkedList) data structure.

Constructor:

It has 2 constructors

- 1) `LinkedList list1 = new LinkedList();`
It will create a LinkedList object with 0 capacity.
- 2) `LinkedList list2 = new LinkedList(Collection c);`
Interconversion between the collection

Methods of LinkedList class:

- 1) `void addFirst(Object o)`
- 2) `void addLast(Object o)`
- 3) `Object getFirst()`
- 4) `Object getLast()`
- 5) `Object removeFirst()`
- 6) `Object removeLast()`

Note :- It stores the elements in non-contiguous memory location.

The time complexity for insertion and deletion is $O(1)$

The time complexity for searching $O(n)$ because it searches the elements using node reference.

```
package com.ravi.linked_list;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;
public class LinkedListDemo
{
    public static void main(String args[])
    {
        List<Object> list=new LinkedList<>();
        list.add("Ravi");
        list.add("Vijay");
        list.add("Ravi");
        list.add(null);
        list.add(42);

        System.out.println("1st Position Element is :"+list.get(1));

        //Iterator interface

        Iterator<Object> itr = list.iterator();
        itr.forEachRemaining(System.out::println); //JDK 1.8
```

```

}
}
-----
package com.ravi.linked_list;

import java.util.*;
public class LinkedListDemo1
{
    public static void main(String args[])
    {
        LinkedList<String> list= new LinkedList<>(); //generic
        list.add("Item 2");//2
        list.add("Item 3");//3
        list.add("Item 4");//4
        list.add("Item 5");//5
        list.add("Item 6");//6
        list.add("Item 7");//7

        list.add("Item 9");//10

        list.add(0,"Item 0");//0
        list.add(1,"Item 1");//1

        list.add(8,"Item 8");//8
        list.add(9,"Item 10");//9
        System.out.println(list);

        list.remove("Item 5");

        System.out.println(list);

        list.removeLast();
        System.out.println(list);

        list.removeFirst();
        System.out.println(list);

        list.set(0,"Ajay"); //set() will replace the existing value
        list.set(1,"Vijay");
        list.set(2,"Anand");
        list.set(3,"Aman");
        list.set(4,"Suresh");
        list.set(5,"Ganesh");
        list.set(6,"Ramesh");
        list.forEach(x -> System.out.println(x));

    }
}
-----

```

```

package com.ravi.linked_list;

```

```

//Methods of LinkedList class
import java.util.LinkedList;

```

```

public class LinkedListDemo2
{
    public static void main(String[] argv)
    {
        LinkedList<String> list = new LinkedList<>();

        list.addFirst("Ravi");
        list.add("Rahul");
        list.addLast("Anand");

        System.out.println(list.getFirst());
        System.out.println(list.getLast());

        list.removeFirst();
        list.removeLast();

        System.out.println(list);
    }
}

```

```

package com.ravi.linked_list;
//ListIterator methods
import java.util.*;
public class LinkedListDemo3
{
    public static void main(String[] args)
    {
        LinkedList<String> city = new LinkedList<> ();
        city.add("Kolkata");
        city.add("Bangalore");
        city.add("Hyderabad");
        city.add("Pune");
        System.out.println(city);

        ListIterator<String> It = city.listIterator();

        while(It.hasNext())
        {
            String cityName = It.next();

            if(cityName.equals("Kolkata"))
            {
                It.remove();
            }
            else if(cityName.equals("Hyderabad"))
            {
                It.add("Ameerpet");
            }
            else if(cityName.equals("Pune"))
            {
                It.set("Mumbai");
            }
        }
        city.forEach(System.out::println);
    }
}

```

```
}
```

The method methods add(Object obj) ,remove() and set() are from ListIterator interface.

```
package com.ravi.linked_list;
```

```
//Insertion, deletion, displaying and exit
```

```
import java.util.LinkedList;
```

```
import java.util.List;
```

```
import java.util.Scanner;
```

```
public class LinkedListDemo4
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        List<Integer> linkedList = new LinkedList<>();
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        while (true)
```

```
        {
```

```
            System.out.println("Linked List: " + linkedList); //[]
```

```
            System.out.println("1. Insert Element");
```

```
            System.out.println("2. Delete Element");
```

```
            System.out.println("3. Display Element");
```

```
            System.out.println("4. Exit");
```

```
            System.out.print("Enter your choice: ");
```

```
            int choice = scanner.nextInt();
```

```
            switch (choice)
```

```
            {
```

```
                case 1:
```

```
                    System.out.print("Enter the element to insert: ");
```

```
                    int elementToAdd = scanner.nextInt();
```

```
                    linkedList.add(elementToAdd);
```

```
                    break;
```

```
                case 2:
```

```
                    if (linkedList.isEmpty())
```

```
                    {
```

```
                        System.out.println("Linked list is empty. Nothing to delete.");
```

```
                    }
```

```
                else
```

```
                {
```

```
                    System.out.print("Enter the element to delete: ");
```

```
                    int elemenetToDelete = scanner.nextInt();
```

```
                    boolean remove = linkedList.remove(Integer.valueOf(elemenetToDelete));
```

```
                    if(remove)
```

```
                    {
```

```
                        System.out.println("Element "+elemenetToDelete+ " is deleted Successfully" );
```

```
                    }
```

```
                    else
```

```
                    {
```

```
                        System.out.println(elemenetToDelete+" not available is the LinkedList");
```

```

        }
    }
    break;
case 3:
    System.out.println("Elements in the linked list.");
    linkedList.forEach(System.out::println);
    break;
case 4:
    System.out.println("Exiting the program.");
    scanner.close();
    System.exit(0);
default:
    System.out.println("Invalid choice. Please try again.");
}
}
}
}
}

```

```

package com.ravi.linked_list;

```

```

//Insertion, deletion, displaying and exit

```

```

import java.util.LinkedList;
import java.util.List;
import java.util.Scanner;

```

```

public class LinkedListDemo4
{
    public static void main(String[] args)
    {
        List<Integer> linkedList = new LinkedList<>();
        Scanner scanner = new Scanner(System.in);

        while (true)
        {
            System.out.println("Linked List: " + linkedList); //[]
            System.out.println("1. Insert Element");
            System.out.println("2. Delete Element");
            System.out.println("3. Display Element");
            System.out.println("4. Exit");
            System.out.print("Enter your choice: ");

            int choice = scanner.nextInt();
            switch (choice)
            {
                case 1:
                    System.out.print("Enter the element to insert: ");
                    int elementToAdd = scanner.nextInt();
                    linkedList.add(elementToAdd);
                    break;
                case 2:
                    if (linkedList.isEmpty())
                    {
                        System.out.println("Linked list is empty. Nothing to delete.");
                    }

```

```

    }
else
{
    System.out.print("Enter the element to delete: ");
    int elemenetToDelete = scanner.nextInt();
    boolean remove = linkedList.remove(Integer.valueOf(elemenetToDelete));

    if(remove)
    {
        System.out.println("Element "+elemenetToDelete+ " is deleted Successfully" );
    }
    else
    {
        System.out.println(elemenetToDelete+" not available is the LinkedList");
    }

}
break;
case 3:
    System.out.println("Elements in the linked list.");
    linkedList.forEach(System.out::println);
    break;
    case 4:
        System.out.println("Exiting the program.");
        scanner.close();
        System.exit(0);
    default:
        System.out.println("Invalid choice. Please try again.");
}
}
}
}
}
}

```

```

package com.ravi.linked_list;

```

```

import java.util.Arrays;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.List;
import java.util.Optional;
import java.util.stream.Stream;

```

```

public class LinkedListDemo5 {

```

```

    public static void main(String[] args)
    {

```

```

        List<String> listOfName = Arrays.asList("Ravi","Rahul","Ankit", "Rahul");

```

```

        LinkedList<String> list = new LinkedList<>(listOfName);
        list.forEach(System.out::println);

```



```
}
```

```
}
```

```
-----  
import java.util.LinkedList;  
import java.util.Iterator;  
import java.util.List;
```

```
record Dog(String name)  
{  
}
```

```
public class LinkedListDemo5  
{  
    public static void main(String[] args)  
    {  
        List<Dog> d = new LinkedList<>();  
        Dog dog = new Dog("Tiger");  
        d.add(dog);  
        d.add(new Dog("Tommy"));  
        d.add(new Dog("Rocky"));  
  
        Iterator<Dog> i3 = d.iterator();  
        i3.forEachRemaining(x -> System.out.println(x.name().toUpperCase())); //java 8  
  
        System.out.println("size " + d.size());  
        System.out.println("Get 1st Position Object " + d.get(1).name());  
  
    }  
}
```

```
-----  
import java.util.Deque;  
import java.util.LinkedList;
```

```
public class LinkedListDemo6  
{  
    public static void main(String[] args)  
    {  
        // Create a LinkedList and treat it as a Deque  
        Deque<String> deque = new LinkedList<>();  
  
        deque.addFirst("Ravi"); //Raj Ravi Pallavi Sweta  
        deque.addFirst("Raj");  
  
        deque.addLast("Pallavi");  
        deque.addLast("Sweta");  
  
        System.out.println("Deque: " + deque);  
    }  
}
```

```

String first = deque.removeFirst();
String last = deque.removeLast();

System.out.println("Removed first element: " + first);
System.out.println("Removed last element: " + last);
System.out.println("Updated Deque: " + deque);
}
}

```

18-07-2024

Set<E> interface :

Set(I)

Set interface is the sub interface of Collection(I) which came from JDK 1.2v

Set interface never accept duplicate elements, Here internally equals(Object obj) method is working to verify two object are identical or not, if identical then it will accept only 1.

Set interface does not use any indexing technique.

ListIterator interface does not work with Set interface.

Set interface uses methods of Collection interface as well as It has more methods from java 9.

Set interface Hierarchy :

Available in the paint diagram (18-JULY)

What is hashing algorithm ?

Hashing algorithm is a technique through which we can search, insert and delete an element in more efficient way in comparison to our classical indexing approach.

Hashing algorithm, internally uses Hashtable data structure, Hashtable data structure internally uses Bucket data structure.

Here elements are inserted by using hashing algorithm so the time complexity to insert, delete and search an element would be $O(1)$

HashSet (UNSORTED, UNORDERED , NO DUPLICATES)

public class HashSet<E> extends AbstractSet<E> implements Set<E>, Cloneable, Serializable

It is a predefined class available in java.util package under Set interface and introduced from JDK 1.2V.

It is an unsorted and unordered set.

It accepts heterogeneous kind of data.

*It uses the hashcode of the object being inserted into the Collection. Using this hashcode it finds the bucket location.

It doesn't contain any duplicate elements as well as It does not maintain any order while iterating the elements from the collection.

It can accept one null value.

HashSet methods are not synchronized.

HashSet is used for fast searching operation.

It contains 4 types of constructors

1) HashSet hs1 = new HashSet();

It will create the HashSet Object with default capacity is 16. The default load factor or Fill Ratio is 0.75 (75% of HashSet is filled up then new HashSet Object will be created having double capacity)

2) HashSet hs2 = new HashSet(int initialCapacity);

will create the HashSet object with user specified capacity

3) HashSet hs3 = new HashSet(int initialCapacity, float loadFactor);

we can specify our own initialCapacity and loadFactor(by default load factor is 0.75)

4) HashSet hs = new HashSet(Collection c);

Interconversion of Collection

```
-----  
//Unsorted, Unordered and no duplicates  
import java.util.*;  
public class HashSetDemo  
{  
    public static void main(String args[])  
    {  
        HashSet<Integer> hs = new HashSet<>();  
        hs.add(67);  
        hs.add(89);  
        hs.add(33);  
        hs.add(45);  
        hs.add(12);  
        hs.add(35);  
  
        hs.forEach(str-> System.out.println(str));  
    }  
}
```

19-07-2024

```
import java.util.*;  
public class HashSetDemo1  
{  
    public static void main(String[] argv)  
    {  
        HashSet<String> hs=new HashSet<>();  
        hs.add("Ravi");  
        hs.add("Vijay");  
        hs.add("Ravi");  
        hs.add("Ajay");  
    }  
}
```

```

hs.add("Palavi");
hs.add("Sweta");
hs.add(null);
hs.add(null);
hs.forEach(str -> System.out.println(str));

    }
}

```

//add() method return type is boolean

```

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

```

```

public class HashSetDemo2 {

```

```

    public static void main(String[] args)
    {

```

```

        Boolean []arr = new Boolean[5];

```

```

        HashSet<Object> hs = new HashSet<>();

```

```

        arr[0] = hs.add(12);
        arr[1] = hs.add(new String("NIT"));
        arr[2] = hs.add(12);
        arr[3] = hs.add(40);
        arr[4] = hs.add("NIT");

```

```

        System.out.println(Arrays.toString(arr));

```

```

        if(hs.contains("NIT"))
        {
            System.out.println("NIT is available in the Collection ");
        }
        else
        {
            System.err.println("NIT is not available in the Collection ");
        }

```

```

        hs.forEach(System.out::println);
    }

```

```

}

```

Note : Collection interface add() method return type is boolean.

//add, delete, display and exit

```

import java.util.HashSet;
import java.util.Scanner;

```

```

public class HashSetDemo3
{

```

```

    public static void main(String[] args)
    {

```

```

        HashSet<String> hashSet = new HashSet<>();

```

```

Scanner scanner = new Scanner(System.in);

while (true)
{
    System.out.println("Options:");
    System.out.println("1. Add element");
    System.out.println("2. Delete element");
    System.out.println("3. Display HashSet");
    System.out.println("4. Exit");

    System.out.print("Enter your choice (1/2/3/4): ");
    int choice = scanner.nextInt();

    switch (choice)
    {
        case 1:
            System.out.print("Enter the element to add: ");
            String elementToAdd = scanner.next();
            if (hashSet.add(elementToAdd))
            {
                System.out.println("Element added successfully.");
            }
            else
            {
                System.out.println("Element already exists in the HashSet.");
            }
            break;
        case 2:
            System.out.print("Enter the element to delete: ");
            String elementToDelete = scanner.next();
            if (hashSet.remove(elementToDelete))
            {
                System.out.println("Element deleted successfully.");
            }
            else
            {
                System.out.println("Element not found in the HashSet.");
            }
            break;
        case 3:
            System.out.println("Elements in the HashSet:");
            hashSet.forEach(System.out::println);
            break;
        case 4:
            System.out.println("Exiting the program.");
            scanner.close();
            System.exit(0);
        default:
            System.out.println("Invalid choice. Please try again.");
    }

    System.out.println();
}
}
}

```

```

-----
//Converting from ArrayList to HashSet
package com.ravi.collection;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.Set;

public class HashSetDemo4 {

    public static void main(String[] args)
    {
        ArrayList<String> cityName = new ArrayList<>();
        cityName.add("Hyderabad");
        cityName.add("Mumbai");
        cityName.add("Pune");
        cityName.add("Ameerpet");
        cityName.add("Hyderabad");
        System.out.println(cityName);

        System.out.println(".....");

        Set<String> s = new HashSet<>(cityName);
        System.out.println(s);

    }
}
-----

```

LinkedHashSet :

public class LinkedHashSet extends HashSet implements Set, Clonable, Serializable

It is a predefined class in java.util package under Set interface and introduced from java 1.4v.

It is the sub class of HashSet class.

It is an orderd version of HashSet that maintains a doubly linked list across all the elements.

It internally uses Hashtable and LinkedList data structures.

We should use LinkedHashSet class when we want to maintain an order.

When we iterate the elements through HashSet the order will be unpredictable, while when we iterate the elments through LinkedHashSet then the order will be same as they were inserted in the collection.

It accepts hetrogeneous and null value is allowed.

It has same constructor as HashSet class.

```

import java.util.*;
public class LinkedHashSetDemo
{
    public static void main(String args[])
    {
        LinkedHashSet<String> lhs=new LinkedHashSet<>();
        lhs.add("Ravi");
        lhs.add("Vijay");
        lhs.add("Ravi");
        lhs.add("Ajay");
        lhs.add("Pawan");
        lhs.add("Shiva");
        lhs.add(null);
        lhs.add("Ganesh");
        lhs.forEach(str -> System.out.println(str));
    }
}

```

```

import java.util.*;

public class LinkedHashSetDemo1
{
    public static void main(String[] args)
    {
        LinkedHashSet<Integer> linkedHashSet = new LinkedHashSet<>();

        linkedHashSet.add(10);
        linkedHashSet.add(5);
        linkedHashSet.add(15);
        linkedHashSet.add(20);
        linkedHashSet.add(5);

        System.out.println("LinkedHashSet elements: " + linkedHashSet);

        System.out.println("LinkedHashSet size: " + linkedHashSet.size());

        int elementToCheck = 15;
        if (linkedHashSet.contains(elementToCheck))
        {
            System.out.println(elementToCheck + " is present in the LinkedHashSet.");
        }
        else
        {
            System.out.println(elementToCheck + " is not present in the LinkedHashSet.");
        }

        int elementToRemove = 10;
        linkedHashSet.remove(elementToRemove);
        System.out.println("After removing " + elementToRemove + ", LinkedHashSet elements: " +
linkedHashSet);

        linkedHashSet.clear();
        System.out.println("After clearing, LinkedHashSet elements: " + linkedHashSet); //[]
    }
}

```

```
}
```

SortedSet interface :

As we know Collections.sort(List list) method accept list as a parameter so, we can't perform sorting operation by using sort() method on HashSet and LinkedHashSet.

In order to provide automatic sorting facility, Set interface has provided one more interface i.e SortedSet interface available from JDK 1.2.

SortedSet interface provided default natural sorting order, default natural sorting order means, if it is number then ascending order but if it is String then alphabetical /dictionary order.

In order to sort the element either in default natural sorting order or user-defined sorting order we are using Comparable or Comparator interfaces.

***** What is the difference between Comparable and Comparator :

Difference is available in paint diagram 19-JULY-24

20-07-2024

Program on Comparable interface :

2 files :

Employee.java(R)

```
package com.ravi.comparable_demo;
```

```
public record Employee(Integer employeeId, String employeeName) implements Comparable<Employee>
```

```
{
    //Sorting based on the Name
    @Override
    public int compareTo(Employee e2)
    {
        return this.employeeName.compareTo(e2.employeeName);
    }
}
```

EmployeeComparable.java(C)

```
package com.ravi.comparable_demo;
```

```
import java.util.ArrayList;
import java.util.Collections;
```

```
public class EmployeeComparable
{
    public static void main(String[] args)
    {
        ArrayList<Employee> listOfEmployees = new ArrayList<>();
    }
}
```



```
listOfEmployees.add(new Employee(333, "Aryan"));
listOfEmployees.add(new Employee(222, "Zuber"));
listOfEmployees.add(new Employee(111, "Scott"));
```

```
Collections.sort(listOfEmployees);
```

```
listOfEmployees.forEach(System.out::println);
```

```
}
```

```
}
```

Limitation of Comparable interface :

1) We can write only one sorting logic using comparable (We can sort based on ID or based on name but not both)

2) If the BLC class is given by some 3rd party in .class format then we can't modify the source code hence Comparable will not work here.

To avoid the above said problems we introduced Comparator interface :

Program on Comparator interface :

2 files :

Product.java(R)

```
package com.ravi.comparator;
```

```
public record Product(Integer productId, String productName)
```

```
{
```

```
}
```

ProductComparator.java(C)

```
package com.ravi.comparator;
```

```
import java.util.ArrayList;
```

```
import java.util.Collections;
```

```
import java.util.Comparator;
```

```
public class ProductComparator {
```

```
    public static void main(String[] args)
```

```
    {
        ArrayList<Product> listOfProduct = new ArrayList<>();
```

```
        listOfProduct.add(new Product(333, "Camera"));
```

```
        listOfProduct.add(new Product(111, "Mobile"));
```

```
        listOfProduct.add(new Product(222, "Laptop"));
```

```
//Anonymous inner class with Comparator to sort product using ID
```

```

Comparator<Product> compId = new Comparator<Product>()
{
    @Override
    public int compare(Product p1, Product p2)
    {
        return p1.productId() - p2.productId();
    }
};

```

```

Collections.sort(listOfProduct, compId);
System.out.println("Sorting Product based on the ID :");
listOfProduct.forEach(System.out::println);

```

```

System.out.println(".....");

```

```

//Sorting based on the name
Comparator<Product> compName = (p1,p2)-> p1.productName().compareTo(p2.productName());
Collections.sort(listOfProduct, compName);
System.out.println("Sorting Product based on the Name :");
listOfProduct.forEach(System.out::println);
}

```

```

}

```

Comparable is not a Functional interface because due to current object support(this keyword) we need to write the sorting logic in the BLC class only so BLC class must implements Comparable interface. We can't write the logic of compareTo(T x) method in another class because It accept only one parameter for second parameter we need to depend upon the Current Object.

Program to sort the Integer object in descending order :

```

package com.ravi.comparator;

```

```

import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;

```

```

public class IntegerDescending {

```

```

    public static void main(String[] args)
    {
        ArrayList<Integer> al = new ArrayList<>();
        al.add(89);
        al.add(63);
        al.add(29);
        al.add(15);

```

```

        Comparator<Integer> desc = (i1,i2)-> i2 - i1;

```

```

        Collections.sort(al,desc);
        al.forEach(System.out::println);

```

```

    }

```

```
}
```

22-07-2024

TreeSet<E> :

public class TreeSet<E> extends AbstractSet<E> implements NavigableSet<E>, Clonable, Serializable

It is a predefined class available in java.util package under Set interface available from JDK 1.2v.

TreeSet, TreeMap and PriorityQueue are the three sorted collection in the entire Collection Framework so these classes never accepting non comparable objects.

It will sort the elements in natural sorting order i.e ascending order in case of number , and alphabetical order or Dictionary order in the case of String. In order to sort the elements according to user choice, It uses Comparable/Comparator interface.

It does not accept duplicate and null value (java.lang.NullPointerException)

It does not accept non comparable type of data if we try to insert it will throw a runtime exception i.e java.lang.ClassCastException

TreeSet implements NavigableSet.

NavigableSet extends SortedSet.

It contains 4 types of constructors :

-
- 1) TreeSet t1 = new TreeSet();
create an empty TreeSet object, elements will be inserted in using Comparable/Comparator
 - 2) TreeSet t2 = new TreeSet(Comparator c);
Customized sorting order
 - 3) TreeSet t3 = new TreeSet(Collection c);
loose coupling
 - 4) TreeSet t4 = new TreeSet(SortedSet s);
Another implemented class of SortedSet we can pass as a parameter.
-

22-07-2024

```
package com.ravi.collection;
```

```
import java.util.TreeSet;
```

```
public class TreeSetExample  
{  
    public static void main(String[] args)  
    {  
        TreeSet<Integer> ts = new TreeSet<>();  
        ts.add(19);  
        ts.add(15);  
        ts.add(11);  
        ts.add(20);  
    }  
}
```

```

        System.out.println(ts);
    }
}
-----
package com.ravi.collection;

import java.util.TreeSet;

class Dog implements Comparable<Dog>
{
    private String name;

    public Dog(String name)
    {
        super();
        this.name = name;
    }

    @Override
    public String toString() {
        return "Dog [name=" + name + "]";
    }

    @Override
    public int compareTo(Dog d2)
    {
        return this.name.compareTo(d2.name);
    }

}

```

```

public class TreeSetExample
{
    public static void main(String[] args)
    {
        TreeSet<Dog> d = new TreeSet<>();
        d.add(new Dog("Tiger"));
        d.add(new Dog("Tommy"));
        d.add(new Dog("Jimmy"));

        System.out.println(d);
    }
}

```

Note : It is clear that TreeSet sort the elements here on the basis of Comparable interface.

```

-----
//program that describes TreeSet provides default natural sorting order
import java.util.*;

```

```

public class TreeSetDemo
{
    public static void main(String[] args)
    {
        SortedSet<Integer> t1 = new TreeSet<>();
        t1.add(4);
        t1.add(7);
        t1.add(2);
        t1.add(1);
        t1.add(9);
        System.out.println(t1);

        NavigableSet<String> t2 = new TreeSet<>();
        t2.add("Orange");
        t2.add("Mango");
        t2.add("Banana");
        t2.add("Grapes");
        t2.add("Apple");
        System.out.println(t2);
    }
}

```

```

import java.util.*;
public class TreeSetDemo1
{
    public static void main(String[] args)
    {
        TreeSet<String> t1 = new TreeSet<>();
        t1.add("Orange");
        t1.add("Mango");
        t1.add("Pear");
        t1.add("Banana");
        t1.add("Apple");
        System.out.println("In Ascending order");
        t1.forEach(i -> System.out.println(i));

        TreeSet<String> t2 = new TreeSet<>();
        t2.add("Orange");
        t2.add("Mango");
        t2.add("Pear");
        t2.add("Banana");
        t2.add("Apple");

        System.out.println("In Descending order");
        Iterator<String> itr2 = t2.descendingIterator(); //for descending order

        itr2.forEachRemaining(x -> System.out.println(x));
    }
}

```

Note :- descendingIterator() is a predefined method of TreeSet class which will traverse in the descending order and return type of this method is Iterator interface.

```

public Iterator descendingIterator()

```

```

import java.util.*;
public class TreeSetDemo2
{
    public static void main(String[] args)
    {
        Set<String> t = new TreeSet<>();
        t.add("6");
        t.add("5");
        t.add("4");
        t.add("2");
        t.add("9");
        Iterator<String> iterator = t.iterator();
        iterator.forEachRemaining(x -> System.out.println(x));

        //From 1.8 to replace hasNext() and next() method
    }
}

```

```

import java.util.*;

public class TreeSetDemo3
{
    public static void main(String[] args)
    {
        Set<Character> t = new TreeSet<>();
        t.add('A');
        t.add('C');
        t.add('B');
        t.add('E');
        t.add('D');
        Iterator<Character> iterator = t.iterator();
        iterator.forEachRemaining(x -> System.out.println(x));
    }
}

```

Here Character wrapper class is also implementing Comparable interface so provides compareTo() method support.

Converting From one Collection to another Collection (TreeSet to AL)

```

package com.ravi.collection;

import java.util.ArrayList;
import java.util.SortedSet;
import java.util.TreeSet;

public class TreeSetDemo4 {

    public static void main(String[] args)
    {
        SortedSet<String> cityName = new TreeSet<>();
        cityName.add("Hyderabad");
        cityName.add("Mumbai");
        cityName.add("Pune");
        cityName.add("Ameerpet");
    }
}

```

```
TreeSet<String> nameOfCity = new TreeSet<>(cityName);
System.out.println(nameOfCity);
```

```
System.out.println(".....");
```

```
ArrayList<String> listOfCity = new ArrayList<>(cityName);
System.out.println(listOfCity);
```

```
}
```

```
}
```

Sorting the element by using Custom comparator :

2 files :

Employee.java(R)

package com.ravi.collection;

```
public record Employee(Integer employeeId, String employeeName, Double employeeSalary)
{
```

```
}
```

TreeSetCustomComparator.java(C)

package com.ravi.collection;

```
import java.util.TreeSet;
```

```
public class TreeSetCustomComparator
```

```
{
    public static void main(String[] args)
    {
        TreeSet<Employee> ts1 = new TreeSet<>((e1, e2)->
e1.employeeName().compareTo(e2.employeeName()));
```

```
ts1.add(new Employee(101, "Zaheer", 24000.90));
ts1.add(new Employee(201, "Aryan", 27678.67));
ts1.add(new Employee(301, "Pooja", 26567D));
System.out.println("Sorting the Employee based on Name in Ascending order" );
ts1.forEach(System.out::println);
```

```
TreeSet<Employee> ts2 = new TreeSet<>((e1, e2)->
e2.employeeName().compareTo(e1.employeeName()));
```

```
ts2.add(new Employee(101, "Zaheer", 24000.90));
ts2.add(new Employee(201, "Aryan", 27678.67));
ts2.add(new Employee(301, "Pooja", 26567D));
```

```
System.out.println("Sorting the Employee based on Name in Descending order" );
ts2.forEach(System.out::println);
```

```
}
```

```
}
```

Methods of SortedSet interface :

public E first() :- Will fetch first element

public E last() :- Will fetch last element

public SortedSet headSet(int range) :- Will fetch the values which are less than specified range

public SortedSet tailSet(int range) :- Will fetch the values which are equal and greater than the specified range.

public SortedSet subSet(int startRange, int endRange) :- Will fetch the range of values where startRange is inclusive and endRange is exclusive.

Note :- headSet(), tailSet() and subSet(), return type is SortedSet.

import java.util.*;
public class SortedSetMethodDemo
{
 public static void main(String[] args)
 {
 TreeSet<Integer> times = new TreeSet<>();
 times.add(1205);
 times.add(1505);
 times.add(1545);
times.add(1600);
 times.add(1830);
 times.add(2010);
 times.add(2100);

 SortedSet<Integer> sub = new TreeSet<>();

sub = times.subSet(1545,2100);
 System.out.println("Using subSet() :-"+sub);//[1545, 1600,1830,2010]
 System.out.println(sub.first());
 System.out.println(sub.last());

 sub = times.headSet(1545);
System.out.println("Using headSet() :-"+sub); //[1205, 1505]

 sub = times.tailSet(1545);
System.out.println("Using tailSet() :-"+sub); //[1545 to 2100]
 }
}

By using SortedSet interface we can find the range of values but navigation among the element is not

possible by using SortedSet interface so, Java software people has provided a new interface NavigableSet which is extending from SortedSet cam from 1.6v

NavigableSet :

It is a predefined interface available in java.util package and it is extending from SortedSet interface.

By using NavigableSet interface we can navigate among the elements as shown in the program.

```
import java.util.*;

public class NavigableSetDemo
{
    public static void main(String[] args)
    {
        NavigableSet<Integer> ns = new TreeSet<>();
        ns.add(1);
        ns.add(2);
        ns.add(3);
        ns.add(4);
        ns.add(5);
        ns.add(6);

        System.out.println("lower(3): " + ns.lower(3)); //Just below than the specified element or null

        System.out.println("floor(3): " + ns.floor(3)); //Equal less or null

        System.out.println("higher(3): " + ns.higher(3)); //Just greater than specified element or null

        System.out.println("ceiling(3): " + ns.ceiling(3)); //Equal or greater or null

    }
}
```

23-07-2024

Map interface :

As we know Collection interface is used to hold single Or individual object but Map interface will hold group of objects in the form key and value pair. {key = value}

Map interface is not the part the Collection.

Before Map interface We had Dictionary(abstract class) class and it is extended by Hashtable class in JDK 1.0V

Map interface works with key and value pair introduced from 1.2V.

Here key and value both are objects.

Here key must be unique and value may be duplicate.

Each key and value pair is creating one Entry.(Entry is nothing but the combination of key and value pair)

```

interface Map<K,V>
{
    interface Entry<K,V>
    {
        //key and value
    }
}

```

How to represent this entry interface (Map.Entry in .java) [Map\$Entry in .class]

In Map interface whenever we have a duplicate key then the old key value will be replaced by new key(duplicate key) value.

forEach(BiConsumer cons) method is available in Map interface.

Iterator and ListIterator we can't use directly using Map.

Map interface Hierarchy :

The hierarchy is available in paint diagram 23-JULY

Methods of Map Interface :

-
- 1) Object put(Object key, Object value) :- To insert one entry in the Map collection. It will return the old object key value if the key is already available(Duplicate key), If key is not available then it will return null.
 - 2) putIfAbsent(Object key, Object value) :- It will insert an entry if and only if key is not present , if the key is already available then it will not insert the Entry to the Map Collection
 - 3) Object get(Object key) :- It will return corresponding value of key, if the key is not present then it will return null.
 - 4) Object getOrDefault(Object key, Object defaultValue) :- To avoid null value this method has been introduced, here we can pass some defaultValue to avoid the null value.
 - 5) boolean containsKey(Object key) :- To Search a particular key
 - 6) boolean containsValue(Object value) :- To Search a particular value
 - 7) int size() :- To count the number of Entries.
 - 8) remove(Object key) :- One complete entry will be removed.
 - 9) void clear() :- Used to clear the Map
 - 10) boolean isEmpty() :- To verify Map is empty or not?
 - 11) void putAll(Map m) :- Merging of two Map collection

Methods of map interface to convert the map into Collection :

We have map interface methods through which we can convert map interface into collection interface which is known as collection views method.

- 1) public Set<Object> keySet() : It will retrieve all the keys.
- 2) public Collection values() : It will retrieve all the values.
- 3) public Set<Map.Entry> entrySet() : It will retrieve key and value both in a single object.

How to generate custom hashCode for String :

```
public class StringHashCode
{
    public static void main(String[] args)
    {
        String str = "India";
        System.out.println("Calculated Hash Code value :"+str.hashCode());

        System.out.println("System generated Hashcode :"+System.identityHashCode(str));
    }
}
```

package com.ravi.map;

```
public class CustomStringHashCode
{
    public static int customHashCode(String str)
    {
        if (str == null)
        {
            return 0; // default value for null is 0
        }

        int hashCode = 0;

        for (int i = 0; i < str.length(); i++)
        {
            char charValue = str.charAt(i);
            hashCode = 31 * hashCode + charValue;
        }

        return hashCode;
    }

    public static void main(String[] args)
    {
        String exampleString = "Pune";

        // Using the built-in hashCode method
        int hashCodeBuiltIn = exampleString.hashCode();
        System.out.println("Built-in hashCode: " + hashCodeBuiltIn);

        // Using the customHashCode method
```

```

        int customHashCode = customHashCode(exampleString);
        System.out.println("Custom hashCode: " + customHashCode);
    }
}

```

24-07-2024

**** How HashMap works internally ?

- a) While working with HashSet or HashMap every object must be compared because duplicate objects are not allowed.
- b) Whenever we add any new key to verify whether key is unique or duplicate, HashMap internally uses hashCode(), == operator and equals method.
- c) While adding the key object in the HashMap, first of all it will invoke the hashCode() method to retrieve the corresponding key hashcode value.
Example :- hm.put(key,value);
then internally key.hashCode();
- d) If the newly added key and existing key hashCode value both are same (Hash collision), then only == operator is used for comparing those keys by using reference or memory address, if both keys references are same then existing key value will be replaced with new key value.

If the reference of both keys are different then equals(Object obj) method is invoked to compare those keys by using state(data). [content comparison]

If the equals(Object obj) method returns true (content wise both keys are same), this new key is duplicate then existing key value will be replaced by new key value.

If equals(Object obj) method returns false, this new key is unique, new entry (key-value) will be inserted in the same
Bucket.

Note :- equals(Object obj) method is invoked only when two keys are having same hashcode as well as their references are different.

e) Actually by calling hashcode method we are not comparing the objects, we are just storing the objects in a group so the currently adding key object will be compared with its SAME HASHCODE GROUP objects, but not with all the keys which are available in the Map.

f) The main purpose of storing objects into the corresponding group to decrease the number of comparison so the efficiency of the program will increase.

g) To insert an entry in the HashMap, HashMap internally uses Hashtable data structure.

h) Now, for storing same hashcode object into a single group, hash table data structure internally uses one more data structure called Bucket.

i) The Hashtable data structure internally uses Node class array object.

j) The bucket data structure internally uses LinkedList data structure, It is a single linked list again implemented by Node class only.

*k) A bucket is group of entries of same hash code keys.

l) Performance wise LinkedList is not good to search, so from java 8 onwards LinkedList is changed to Binary tree to decrease the number of comparison within the same bucket hashCode if the number of entries are greater than 8.

* equals() and hashCode() method contract :

Both the methods are working together to find out the duplicate objects in the Map.

*If equals() method invoked on two objects and it returns true then hashCode of both the objects must be same.

Note : IF TWO OBJECTS ARE HAVING SAME HASH CODE THEN IT MAY BE SAME OR DIFFERENT BUT IF EQUALS(OBJECT OBJ) METHOD RETURN TRUE THEN BOTH OBJECTS MUST RETURN SAME HASHCODE.

What will happen if we don't follow the contract ?

Case 1 :

If we override only equals(Object obj)

If we override only equals(Object obj) method for content comparison then same object will have different hashCode (due to new keyword) hence same object (content wise) will move into two different buckets [Duplication].

Case 2 :

If we override only hashCode() method

If we override only hashCode() method then two object which are having same hashCode (due to overriding) will go to same bucket but == operator and equals(Object obj) method of Object class, both will return false hence duplicate object will be inserted into same bucket.

So, the conclusion is compulsory we need to override both the methods for removing duplicate elements.
=====

```
package com.ravi.collection;
```

```
import java.util.HashMap;
```

```
public class HashMapInternalsDemo1 {
```

```
    public static void main(String[] args)
    {
```

```
        HashMap<String,Integer> hm1 = new HashMap<>();
        hm1.put("A", 1);
        hm1.put("A", 2);
        hm1.put(new String("A"), 3);
        System.out.println("Size is :"+hm1.size());
        System.out.println(hm1);
```

```
        System.out.println(".....");
```

```

HashMap<Integer,Integer> hm2 = new HashMap<>();
hm2.put(128, 1);
hm2.put(128, 2);
System.out.println("Size is :"+hm2.size());
System.out.println(hm2);
System.out.println(".....");

```

```

HashMap hm3 = new HashMap();
hm3.put("A", 1);
hm3.put("A", 2);
hm3.put(new String("A"), 3);
hm3.put(65, 4);
System.out.println("Size is :"+hm3.size());
System.out.println(hm3);
}

```

```

}

```

```

-----
package com.ravi.collection;

```

```

import java.util.HashMap;
import java.util.Objects;

```

```

class Customer

```

```

{
    private int customerId;
    private String customerName;

```

```

    public Customer(int customerId, String customerName) {
        super();
        this.customerId = customerId;
        this.customerName = customerName;
    }

```

```

    @Override
    public int hashCode() {
        return Objects.hash(customerId, customerName);
    }

```

```

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (getClass() != obj.getClass())
            return false;
        Customer other = (Customer) obj;
        return customerId == other.customerId && Objects.equals(customerName, other.customerName);
    }
}

public class HashMapInternalsDemo2 {

```

```

public static void main(String[] args)
{
    Customer c1 = new Customer(111, "Scott");
    Customer c2 = new Customer(111, "Scott");

    HashMap<Customer,String> map = new HashMap<>();
    map.put(c1, "A");
    map.put(c2, "B");

    System.out.println(map.size());
}
}

```

```

import java.util.*;
class Employee
{
    int eid;
    String ename;

    Employee(int eid, String ename)
    {
        this.eid = eid;
        this.ename = ename;
    }

    @Override
    public boolean equals(Object obj) //obj = e2
    {
        if(obj instanceof Employee)
        {
            Employee e2 = (Employee) obj; //downcasting

            if(this.eid == e2.eid && this.ename.equals(e2.ename))
            {
                return true;
            }
            else
            {
                return false;
            }
        }
        else
        {
            System.out.println("Comparison is not possible");
            return false;
        }
    }
}

```

```

public String toString()
{
    return " "+eid+" "+ename;
}

```

```

}
public class HashMapDemo8
{
    public static void main(String[] args)
    {
        Employee e1 = new Employee(101,"Aryan");
        Employee e2 = new Employee(102,"Pooja");
        Employee e3 = new Employee(101,"Aryan");
        Employee e4 = e2;

        HashMap<Employee,String> hm = new HashMap<>();
        hm.put(e1,"Ameerpet");
        hm.put(e2,"S.R Nagar");
        hm.put(e3,"Begumpet");
        hm.put(e4,"Panjagutta");

        hm.forEach((k,v)-> System.out.println(k+" : "+v));
    }
}

```

```

package com.ravi.collection;

```

```

import java.util.HashSet;
import java.util.Objects;

```

```

record Student(int id, String name)
{

}

```

```

public class HashSetInternal {

    public static void main(String[] args)
    {
        HashSet<Student> hs = new HashSet<>();
        hs.add(new Student(1, "A"));
        hs.add(new Student(1, "A"));

        System.out.println(hs.size());

    }

}

```

25-07-2024

HashMap<K,V> :- [Unsorted, Unordered, No Duplicate keys]

public class HashMap<K,V> extends AbstractMap<K,V> implements Map<K,V>, Serializable, Cloneable

It is a predefined class available in java.util package under Map interface available from JDK 1.2v.

It gives us unsorted and Unordered map. when we need a map and we don't care about the order while iterating the elements through it then we should use HashMap.

It inserts the element based on the hashCode of the Object key using hashing technique [hasing alogorithm]

It does not accept duplicate keys but value may be duplicate.

It accepts only one null key(because duplicate keys are not allowed) but multiple null values are allowed.

HashMap is not synchronized.

Time complexcity of search, insert and delete will be $O(1)$

We should use HashMap to perform fast searching opeartion.

For eliminating duplicate keys in hashMap object we should compulsory follow the contract between hashCode and equals(Object obj)

It contains 4 types of constructor

1) HashMap hm1 = new HashMap();

It will create the HashMap Object with default capacity is 16. The default load fator or Fill Ratio is 0.75 (75% of HashMap is filled up then new HashMap Object will be created having double capacity)

2) HashMap hm2 = new HashMap(int initialCapacity);

will create the HashMap object with user specified capacity

3) HashMap hm3 = new HashMap(int initialCapacity, float loadFactor);

we can specify our own initialCapacity and loadFactor(by default load factor is 0.75)

4)HashMap hm4 = new HashMap(Map m);

Interconversion of Map Collection

import java.util.HashMap;

public class HashMapExample {

public static void main(String[] args)

{
HashMap<String,String> map1 = new HashMap<>();
map1.put("ravi", "1234");
map1.put(new String("ravi"), "5678");

System.out.println(map1.size());

System.out.println(".....");

HashMap<StringBuffer, String> map2 = new HashMap<>();
map2.put(new StringBuffer("NIT"), "8888");
map2.put(new StringBuffer("NIT"), "9999");

System.out.println(map2.size());

}

```
}
```

Note : While working with String object we will get the size as 1 because String class has overridden hashCode and equals method but StringBuffer class has not overridden hashCode and equals method so we will get the size as 2.

```
-----
//Program that shows HashMap is unordered
import java.util.*;
public class HashMapDemo
{
    public static void main(String[] a)
    {
        Map<String,String> map = new HashMap<>();
        map.put("Ravi", "12345");
        map.put("Rahul", "12345");
        map.put("Aswin", "5678");
        map.put(null, "6390");
        map.put("Ravi","1529");
        map.put("Aamir","890");

        System.out.println(map); //{}

        map.forEach((k,v)-> System.out.println("Key is :"+k+" value is "+v));

    }
}
-----
```

```
//Program to search a particular key and value in the Map collection
import java.util.*;
public class HashMapDemo1
{
    public static void main(String args[])
    {
        HashMap<Integer,String> hm = new HashMap<>();
        hm.put(1, "JSE");
        hm.put(2, "JEE");
        hm.put(3, "JME");
        hm.put(4,"JavaFX");
        hm.put(5,null);
        hm.put(6,null);

        System.out.println("Initial map elements: " + hm);
        System.out.println("key 2 is present or not :"+hm.containsKey(2));

        System.out.println("JME is present or not :"+hm.containsValue("JME"));

        System.out.println("Size of Map : " + hm.size());
        hm.clear();
        System.out.println("Map elements after clear: " + hm); //{}
    }
}
-----
```

```
//Collection view methods [keySet(), values(), Set<Map.Entry> entrySet()]
import java.util.*;
public class HashMapDemo2
```

```

{
public static void main(String args[])
{
    Map<Integer,String> map = new HashMap<>();
    map.put(1, "C");
    map.put(2, "C++");
    map.put(3, "Java");
    map.put(4, ".net");

    map.forEach((k,v)->System.out.println("Key :"+k+" Value :"+v ));

    System.out.println("Return Old Object value :"+map.put(4,"Python"));

    Set keys = map.keySet();
    System.out.println("All keys are :"+keys);

    Collection values = map.values();
    System.out.println("All values are :"+values);

    for(Map.Entry m : map.entrySet())
    {
        System.out.println(m.getKey()+" : "+m.getValue());
    }

    System.out.println("Retrieving using Iterator :");

    Iterator itr= map.entrySet().iterator();

    while(itr.hasNext())
    {
        System.out.println(itr.next());
    }
}
}

```

```

import java.util.*;
public class HashMapDemo3
{
    public static void main(String args[])
    {
        HashMap<Integer,String> map = new HashMap<>(26,0.95f);
        map.put(1, "Java");
        map.put(2, "is");
        map.put(3, "best");
        map.remove(3); //will remove the complete Entry
        String val=(String)map.get(3);
        System.out.println("Value for key 3 is: " + val);
        map.forEach((k,v)->System.out.println(k +" : "+v));
    }
}

```

By using get() method, if the key is not available then we will get null value.

```

//To merge two Map Collection (putAll)
import java.util.*;

```

```

public class HashMapDemo4
{
    public static void main(String args[])
    {
        HashMap<Integer,String> newmap1 = new HashMap<>();

        HashMap<Integer,String> newmap2 = new HashMap<>();

        newmap1.put(1, "OCPJP");
        newmap1.put(2, "is");
        newmap1.put(3, "best");

        System.out.println("Values in newmap1: "+ newmap1);

        newmap2.put(4, "Exam");

        newmap2.putAll(newmap1);

        newmap2.forEach((k,v)->System.out.println(k+" : "+v));
    }
}

```

```

-----
import java.util.*;
public class HashMapDemo5
{
    public static void main(String[] argv)
    {
        Map<String,String> map = new HashMap<>(9, 0.85f);
        map.put("key", "value");
        map.put("key2", "value2");
        map.put("key3", "value3");
        map.put("key7", "value7");

        Set keys = map.keySet();//keySet return type is Set
        System.out.println(keys ); //[]

        Collection val = map.values(); //values return type is collection
        System.out.println(val);

        map.forEach((k,v)-> System.out.println(k+" : "+v));

    }
}

```

```

-----
//getOrDefault() method
import java.util.*;
public class HashMapDemo6
{
    public static void main(String[] args)
    {
        Map<String, String> map = new HashMap<>();
        map.put("A", "1");
        map.put("B", "2");
    }
}

```

```

map.put("C", "3");
    //if the key is not present, it will return default value .It is used to avoid null
String value = map.getOrDefault("D","Key is not available");
System.out.println(value);
System.out.println(map);
}
}

```

```

//interconversion of two HashMap
import java.util.*;
public class HashMapDemo7
{
    public static void main(String args[])
    {
        HashMap<Integer, String> hm1 = new HashMap<>();

        hm1.put(1, "Ravi");
        hm1.put(2, "Rahul");
        hm1.put(3, "Rajen");

        HashMap<Integer, String> hm2 = new HashMap<>(hm1);

        System.out.println("Mapping of HashMap hm1 are : " + hm1);

        System.out.println("Mapping of HashMap hm2 are : " + hm2);
    }
}

```

LinkedHashMap<K,V>

public class LinkedHashMap<K,V> extends HashMap<K,V> implements Map<K,V>

It is a predefined class available in java.util package under Map interface available from 1.4.

It is the sub class of HashMap class.

It maintains insertion order. It contains a doubly linked with the elements or nodes so It will iterate more slowly in comparison to HashMap.

It uses Hashtable and LinkedList data structure.

If We want to fetch the elements in the same order as they were inserted then we should go with LinkedHashMap.

It accepts one null key and multiple null values.

It is not synchronized.

It has also 4 constructors same as HashMap

1) LinkedHashMap hm1 = new LinkedHashMap();
will create a LinkedHashMap with default capacity 16 and load factor 0.75

2) LinkedHashMap hm1 = new LinkedHashMap(int initialCapacity);

3) LinkedHashMap hm1 = new LinkedHashMap(iny initialCapacity, float loadFactor);

4) LinkedHashMap hm1 = new LinkedHashMap(Map m);

```
-----
import java.util.*;
public class LinkedHashMapDemo
{
    public static void main(String[] args)
    {
        LinkedHashMap<Integer,String> l = new LinkedHashMap<>();
        l.put(1,"abc");
        l.put(3,"xyz");
        l.put(2,"pqr");
        l.put(4,"def");
        l.put(null,"ghi");
        System.out.println(l);
    }
}
-----
```

```
import java.util.*;

public class LinkedHashMapDemo1
{
    public static void main(String[] a)
    {
        Map<String,String> map = new LinkedHashMap<>();
        map.put("Ravi", "1234");
        map.put("Rahul", "1234");
        map.put("Aswin", "1456");
        map.put("Samir", "1239");

        map.forEach((k,v)->System.out.println(k+" : "+v));
    }
}
-----
```

Hashtable<K,V>

public class Hashtable<K,V> extends Dictionary<K,V> implements Map<K,V>, Clonable, Serializable

It is predefined class available in java.util package under Map interface from JDK 1.0.

Like Vector, Hashtable is also form the birth of java so called legacy class.

It is the sub class of Dictionary class which is an abstract class.

*The major difference between HashMap and Hashtable is, HashMap methods are not synchronized where as Hastable methods are synchronized.

HashMap can accept one null key and multiple null values where as Hashtable does not contain anything as a null(key and value both). if we try to add null then JVM will throw an exception i.e NullPointerException.

The initial default capacity of Hashtable class is 11 where as loadFactor is 0.75.

It has also same constructor as we have in HashMap.(4 constructors)

1) Hashtable hs1 = new Hashtable();

It will create the Hashtable Object with default capacity as 11 as well as load factor is 0.75

2) Hashtable hs2 = new Hashtable(int initialCapacity);

will create the Hashtable object with specified capacity

3) Hashtable hs3 = new Hashtable(int initialCapacity, float loadFactor);

we can specify our own initialCapacity and loadFactor

4) Hashtable hs = new Hashtable(Map c);

Interconversion of Map Collection

```
-----
import java.util.*;
public class HashtableDemo
{
    public static void main(String args[])
    {
        Hashtable<Integer,String> map=new Hashtable<>();
        map.put(1, "Java");
        map.put(2, "is");
        map.put(3, "best");
        map.put(4,"language");

        //map.put(5,null);

        System.out.println(map);

        System.out.println(".....");

        for(Map.Entry m : map.entrySet())
        {
            System.out.println(m.getKey()+" = "+m.getValue());
        }
    }
}
```

```
-----
import java.util.*;
public class HashtableDemo1
{
    public static void main(String args[])
    {
        Hashtable<Integer,String> map=new Hashtable<>();
        map.put(1,"Priyanka");
        map.put(2,"Ruby");
        map.put(3,"Vibha");
        map.put(4,"Kanchan");

        map.putIfAbsent(5,"Bina");
        map.putIfAbsent(24,"Pooja");
        map.putIfAbsent(26,"Ankita");
    }
}
```

```

        map.putIfAbsent(1,"Sneha");
        System.out.println("Updated Map: "+map);
    }
}

```

WeakHashMap<K,V> :

public class WeakHashMap<K,V> extends AbstractMap<K,V> implements Map<K,V>

It is a predefined class in java.util package under Map interface. It was introduced from JDK 1.2v onwards.

While working with HashMap, keys of HashMap are of strong reference type. This means the entry of map will not be deleted by the garbage collector even though the key is set to be null and still it is not eligible for Garbage Collector.

On the other hand while working with WeakHashMap, keys of WeakHashMap are of weak reference type. This means the entry and corresponding object of a map is deleted by the garbage collector if the key value is set to be null because it is of weak type.

So, HashMap dominates over Garbage Collector where as Garbage Collector dominates over WeakHashMap.

It contains 4 types of Constructor :

1) WeakHashMap wm1 = new WeakHashMap();

Creates an empty WeakHashMap object with default capacity is 16 and load factor 0.75

2) WeakHashMap wm2 = new WeakHashMap(int initialCapacity);

3) WeakHashMap wm3 = new WeakHashMap(int initialCapacity, float loadFactor);

Eg:- WeakHashMap wm = new WeakHashMap(10,0.9);

capacity - The capacity of this map is 10. Meaning, it can store 10 entries.

loadFactor - The load factor of this map is 0.9. This means whenever our hashtable is filled up by 90%, the entries are moved to a new hashtable of double the size of the original hashtable.

4) WeakHashMap wm4 = new WeakHashMap(Map m);

```

package com.ravi.collection;

```

```

import java.util.WeakHashMap;

```

```

record Product(Integer productId, String productName, Double productPrice)
{
    @Override
    public void finalize()
    {
        System.out.println("Product object is eligible 4 GC");
    }
}

```



```
}
```

```
public class WeakHashMapDemo
{
    public static void main(String[] args) throws InterruptedException
    {
        WeakHashMap<Product,String> map = new WeakHashMap<>();

        Product p1 = new Product(1, "Laptop", 84000D);

        map.put(p1,"Hyderabad");
        System.out.println(map);

        p1 = null;

        System.gc();
        Thread.sleep(5000);

        System.out.println(map); //{}
    }
}
```

Note : If we work on inventory system then for frequent insertion and deletion of Product from the inventory, we should use WeakHashMap

27-07-2024

How to generate System hashCode :

We know Object class has provided a predefined method hashCode() to search the bucket location in the Hashtable so it generate Hashcode for each every object.

If we want to provide System hashCode then System class has provided a predefined static method identityHashCode(Object obj) which will provide System hashCode.

```
public static native int identityHashCode(Object obj)
```

```
package com.ravi.identity_hashcode;
```

```
public class IdentityHashCodeDemo {

    public static void main(String[] args)
    {
        String str = "Hyderabad";
        System.out.println("Object class hashCode :"+str.hashCode());
        System.out.println("System generated hashCode :"+System.identityHashCode(str));

    }

}
```

Note : From the above program it is clear that Object class hashCode method hashCode and System generated hashCode both are different.

IdentityHashMap<K,V> : [Comparing keys based on the Memory reference]

public class IdentityHashMap<K,V> extends AbstractMap<K,V> implements Map<K,V>, Clonable, Serializable.

It was introduced from JDK 1.4 onwards.

The IdentityHashMap uses == operator to compare keys.

As we know HashMap uses equals() and hashCode() method for comparing the keys based on the hashCode of the object it will search the bucket location and insert the entry there only.

So We should use IdentityHashMap where we need to check the reference or memory address instead of logical equality.

HashMap uses hashCode of the "Object key" to find out the bucket location in Hashtable, on the other hand IdentityHashMap does not use hashCode() method actually It uses System.identityHashCode(Object o)

IdentityHashMap is more faster than HashMap in case of key Comparison.

The default capacity is 32.

It has three constructors, It does not contain loadFactor specific constructor.

package com.ravi.identity_hashcode;

import java.util.HashMap;
import java.util.IdentityHashMap;

public class IdentityHashMapDemo {

public static void main(String[] args)
{
HashMap<String,String> hm = new HashMap<>();
hm.put("india", "Asia");
hm.put(new String ("india"), "Asia");
System.out.println(hm.size());
System.out.println(hm);

System.out.println(".....");

IdentityHashMap<String,String> ihm = new IdentityHashMap<>();
ihm.put("india", "Asia");
ihm.put(new String ("india"), "Asia");
System.out.println(ihm.size());
System.out.println(ihm);

}

}

SortedMap<K,V>

It is a predefined interface available in java.util package under Map interface.

We should use SortedMap interface when we want to insert the key element based on some sorting order i.e the default natural sorting order.

TreeMap<K,V>

public class TreeMap<K,V> extends AbstractMap<K,V> implements NavigableMap<K,V> , Clonable, Serializable

It is a predefined class available in java.util package under Map interface available for 1.2V.

It is a sorted map that means it will sort the elements by natural sorting order based on the key or by using Comparator interface as a constructor parameter.

It does not allow non comparable keys.

It does not accept null key but null value allowed.

TreeMap implements NavigableMap and NavigableMap extends SortedMap. SortedMap extends Map interface.

TreeMap contains 4 types of Constructors :

- 1) TreeMap tm1 = new TreeMap(); //creates an empty TreeMap
- 2) TreeMap tm2 = new TreeMap(Comparator cmp); //user defined sorting logic
- 3) TreeMap tm3 = new TreeMap(Map m);
- 4) TreeMap tm4 = new TreeMap(SortedMap m);

```
import java.util.*;
public class TreeMapDemo
{
    public static void main(String[] args)
    {
        TreeMap<Object,String> t = new TreeMap<>();
        t.put(4,"Ravi");
        t.put(7,"Aswin");
        t.put(2,"Ananya");
        t.put(1,"Dinesh");
        t.put(9,"Ravi");
        t.put(3,"Ankita");
        t.put(5,null);
```

```
        System.out.println(t);
    }
}
```

Note : It will sort the entry based on the key, In the above program, It uses Comparable interface.

```

import java.util.*;
public class TreeMapDemo1
{
    public static void main(String args[])
    {
        TreeMap map = new TreeMap();
        map.put("one","1");
        map.put("two",null);
        map.put("three","3");
        map.put("four",4);

        displayMap(map);

        map.forEach((k, v) -> System.out.println("Key = " + k + ", Value = " + v));

    }
    static void displayMap(TreeMap map)
    {
        Collection c = map.entrySet(); //Set<Map.Entry>

        Iterator i = c.iterator();
        i.forEachRemaining(x -> System.out.println(x));
    }
}

```

```

//firstKey() lastKey() headMap() tailMap() subMap()    SortedMap
// first()   last()   headSet() tailSet() subSet()    SortedSet

```

```

import java.util.*;
public class TreeMapDemo2
{
    public static void main(String[] argv)
    {
        Map map = new TreeMap();
        map.put("key2", "value2");
        map.put("key3", "value3");
        map.put("key1", "value1");

        System.out.println(map); //{

        SortedMap x = (SortedMap) map;
        System.out.println("First key is :"+x.firstKey());
        System.out.println("Last Key is :"+x.lastKey());
    }
}

```

```

package com.ravi.collection;

```

```

import java.util.TreeMap;

```

```

record Student(Integer studentId, String studentName) implements Comparable<Student>
{
    @Override
    public int compareTo(Student s2)
    {

```

```
return this.studentId.compareTo(s2.studentId);  
}
```

```
}
```

```
public class TreeMapDemo3  
{  
    public static void main(String[] args)  
    {  
        TreeMap<Student, String> tm = new TreeMap<>();  
        tm.put(new Student(2, "Sailesh"), "Hyderabad");  
        tm.put(new Student(1, "Zuber"), "BBSR");  
  
        tm.forEach((k,v)-> System.out.println("Key is :"+k+" and Address is :"+v));  
    }  
}
```

```
}
```

```
-----  
package com.ravi.collection;
```

```
import java.util.TreeMap;
```

```
record Employee(Integer employeeId, String employeeName)  
{  
  
}
```

```
public class TreeMapDemo4 {  
  
    public static void main(String[] args)  
    {  
        TreeMap<Employee,String> tm1 = new TreeMap<>((e1,e2)->  
e1.employeeId().compareTo(e2.employeeId()));
```

```
  
        tm1.put(new Employee(333, "Raj"),"TCS");  
        tm1.put(new Employee(111, "Dinesh"),"WIPRO");  
        tm1.put(new Employee(222, "Aryan"),"NIT");
```

```
  
        tm1.forEach((k,v)-> System.out.println("Key is :"+k+" value is :"+v));
```

```
    }  
  
}
```

```
}
```

```
-----  
package com.ravi.collection;
```

```
import java.util.TreeMap;
```

```
public class TreeMapDemo5 {
```

```

public static void main(String[] args)
{
    TreeMap<String,Integer> map = new TreeMap<>(String::compareTo);
    map.put("raj", 1);
    map.put("ravi", 2);
    map.put("rahul",3);

    System.out.println(map);

}

}

```

Methods of SortedMap interface :

- 1) firstKey() //first key
- 2) lastKey() //last key
- 3) headMap(int keyRange) //less than the specified range
- 4) tailMap(int keyRange) //equal or greater than the specified range
- 5) subMap(int startKeyRange, int endKeyRange) //the range of key where startKey will be inclusive and endKey will be exclusive.

return type of headMap(), tailMap() and subMap() return type would be SortedMap(l)

```

import java.util.*;
public class SortedMapMethodDemo
{
    public static void main(String args[])
    {
        SortedMap<Integer,String> map=new TreeMap<>();
        map.put(100,"Amit");
        map.put(101,"Ravi");

        map.put(102,"Vijay");
        map.put(103,"Rahul");

        System.out.println("First Key: "+map.firstKey()); //100
        System.out.println("Last Key "+map.lastKey()); //103
        System.out.println("headMap: "+map.headMap(102)); //100 101
        System.out.println("tailMap: "+map.tailMap(102)); //102 103
        System.out.println("subMap: "+map.subMap(100, 102)); //100 101

    }
}

```

Assignment for NavigableMap :

Properties :

```

public class Properties extends Hashtable<K,V>

```

Properties class is used to maintain the data in the key-value form. It takes both key and value as a string.

Properties class is a subclass of Hashtable. It provides the methods to store properties in a properties file (load method) and to get the properties from the properties file, we should use load method of Properties class.

FileReader reader = new FileReader("db.properties");

Properties p = new Properties();
p.load(reader);
p.getProperty(String key); //read the corresponding key value

getProperty() and load() both are non static method of Property class

System.getProperties() returns the all system properties.

Here we need to create a file with extension .properties

Here we need to create db.properties file

db.properties

driver = oracle.jdbc.driver.OracleDriver
userName = scott
password = tiger

```
import java.io.*;
import java.util.*;
public class PropertiesDemo1 {

    public static void main(String[] args) throws IOException, ClassNotFoundException
    {
        Properties p = new Properties();

        FileReader fr = new FileReader("C:\\new\\database.properties");

        p.load(fr);

        String driver = p.getProperty("driver");
        System.out.println(driver);

        String userName = p.getProperty("userName");
        System.out.println(userName);

        String pwd = p.getProperty("password");
        System.out.println(pwd);

    }
}
```

If we make changes in the properties file then directly (without compilation) we can take the the value in

our java file so after any modification in the properties file we need not to re-compile/re-deploy the file.

```
import java.util.*;
import java.io.*;
public class PropertiesExample2
{
    public static void main(String[] args)throws Exception
    {
        Properties p=System.getProperties();
        Set set=p.entrySet();

        Iterator itr=set.iterator();
        while(itr.hasNext())
        {
            Map.Entry entry=(Map.Entry)itr.next();
            System.out.println(entry.getKey()+" = "+entry.getValue());
        }
    }
}
```

here getProperties() is a static method of System class.

29-07-2024

Queue interface :-

- 1) It is sub interface of Collection(I) available from JDK 1.5V
- 2) It works in FIFO(First In first out)
- 3) It is an ordered collection.
- 4) In a queue, insertion is possible from last is called REAR where as deletion is possible from the starting is called FRONT of the queue.
- 5) From jdk 1.5 onwards LinkedList class implments Queue interface to handle the basic queue operations.

PriorityQueue :

```
public class PriorityQueue extends AbstractQueue implements Serializable
```

It is a predefined class in java.util package, available from Jdk 1.5 onwards.

It stores the elements using min-heap tree, meaning the smallest element is at the head of the queue.

The elements of the priority queue are ordered according to their natural ordering (min heap tree), or by using Comparator provided at queue construction time, depending on which constructor is used.

A priority queue does not permit null elements.

It provides natural sorting order so we can't take non-comparable objects(hetrogeneous types of Object)

The initial capacity of PriorityQueue is 11.

Constructor :

- 1) PriorityQueue pq1 = new PriorityQueue();
- 2) PriorityQueue pq2 = new PriorityQueue(int initialCapacity);
- 3) PriorityQueue pq3 = new PriorityQueue(int initialCapacity, Comparator cmp);
- 4) PriorityQueue pq3 = new PriorityQueue(Comparator cmp);
- 5) PriorityQueue pq4 = new PriorityQueue(Collection c);

Methods :-

add() / offer() :- Used to add an element in the Queue

poll() :- It is used to fetch the elements from head of the queue, after fetching it will delete the element.

peek() :- It is also used to fetch the elements from head of the queue, Unlike poll it will only fetch but not delete the element.

boolean remove(Object element) :- It is used to remove an element. The return type is boolean.

```
import java.util.PriorityQueue;

public class PriorityQueueDemo
{
    public static void main(String[] argv)
    {
        PriorityQueue<Object> pq = new PriorityQueue<>();
        pq.add("Orange");
        pq.add("Apple");
        pq.add("Mango");
        pq.add("Guava");
        pq.add("Grapes");

        System.out.println(pq);
    }
}
```

```
import java.util.PriorityQueue;
public class PriorityQueueDemo1
{
    public static void main(String[] argv)
    {
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        pq.add(11);
        pq.add(2);
        pq.add(4);
        pq.add(6);
        System.out.println(pq);
    }
}
```

```

}
}
----- import java.util.PriorityQueue;

public class PriorityQueueDemo2
{
    public static void main(String[] argv)
    {
        PriorityQueue<String> pq = new PriorityQueue<>();
        pq.add("2");
        pq.add("4");
        pq.add("6");
        System.out.print(pq.peek() + " "); //2 2 3 4 4
        pq.offer("1");
        pq.offer("9");
        pq.add("3"); //      9 6

        pq.remove("1");
        System.out.print(pq.poll() + " ");
        if (pq.remove("2"))
            System.out.print(pq.poll() + " ");
        System.out.println(pq.poll() + " " + pq.peek()+ " "+pq.poll());
    }
}

```

```

-----
import java.util.*;

```

```

record Task(String name, int priority)
{
}

```

```

public class PriorityQueueDemo3 {
    public static void main(String[] args)
    {
        PriorityQueue<Task> taskQueue = new PriorityQueue<>(new Comparator<Task>())
        {
            @Override
            public int compare(Task t1, Task t2)
            {
                return Integer.compare(t1.priority(), t2.priority());
            }
        });

        taskQueue.add(new Task("Submit report", 4));
        taskQueue.add(new Task("Find Bug", 2));
        taskQueue.add(new Task("Write Program", 1));
        taskQueue.add(new Task("Execute Program", 3));

        while (!taskQueue.isEmpty())
        {
            System.out.println("Executing: " + taskQueue.poll());
        }
    }
}
-----

```

14-07-2024

Method Reference in java :

Method Reference :

It is a new feature introduced from java 1.8 onwards.

It is mainly used to write concise coding.

By using method reference we can refer an existing method which is available at API level or Project level.

We can use this technique in the body of Lambda expression just to call method definition.

The entire method body will be automatically placed into Lambda Expression.

It is used to enhance the code reusability.

It uses :: (Double Colon Operator)

While working with Lambda expression we need to write the Lambda Method Body but while working with Method reference we can refer an existing method which is already available in the package or Project.

There are 4 types of method reference

- 1) Static Method Reference(ClassName::methodName)
- 2) Instance Method Reference(objectReference::methodName)
- 3) Constructor Reference (ClassName::new)
- 4) Arbitrary Referenc (ClassName::instanceMethodName)

Basic Example of Method Reference :

3 files :

Worker.java(I)

```
package com.ravi.method_ref;
```

```
@FunctionalInterface
public interface Worker
{
    void work();
}
```

Employee.java(C)

```
package com.ravi.method_ref;
```

```
public class Employee
{
    public void work()
    {
        System.out.println("Employee is working");
    }
}
```

```
}
```

MethodReferenceDemo1.java

```
-----  
package com.ravi.method_ref;  
  
public class MethodReferenceDemo1  
{  
    public static void main(String[] args)  
    {  
        //Writing Lambda Expression  
        Worker w = ()-> System.out.println("Worker is working");  
        w.work();  
  
        //Writing Method Reference  
        Worker w1 = new Employee()::work;  
        w1.work();  
    }  
}
```

Method reference can be called by Functional Interface . Here Functional interface method argument and method return type must be same otherwise we will get compilation error as shown in the program

3 files :

Worker.java(I)

package com.ravi.method_ref;

```
@FunctionalInterface  
public interface Worker  
{  
    void work();  
}
```

Employee.java(C)

```
-----  
package com.ravi.method_ref;  
  
public class Employee  
{  
    public static void salary()  
    {  
        System.out.println("Employee is working for salary");  
    }  
}
```

MethodReferenceDemo2.java(C)

```
-----  
package com.ravi.method_ref;  
  
public class MethodReferenceDemo1  
{  
    public static void main(String[] args)
```

```
{
Worker w1 = Employee::salary;
w1.work();

}
}
```

3 files :

Worker.java(I)

package com.ravi.method_ref;

```
@FunctionalInterface
public interface Worker
{
    void work(double salary);
}
```

Employee.java(C)

package com.ravi.method_ref;

```
public class Employee
{
    public static void salary(double salary)
    {
        System.out.println("Employee Salary is :"+salary);
    }
}
```

MethodReferenceDemo3.java

package com.ravi.method_ref;

```
public class MethodReferenceDemo3
{
    public static void main(String[] args)
    {
        Worker w1 = Employee::salary;
        w1.work(40000);

    }
}
```

Program on Static Method Reference :

package com.ravi.static_method_reference;

import java.util.Vector;

```
class EvenOrOdd
{
    public static void isEven(int number)
```

```

    {
        if (number % 2 == 0)
        {
            System.out.println(number + " is even");
        }
        else
        {
            System.out.println(number + " is odd");
        }
    }
}

public class StaticMethodReferenceDemo1
{
    public static void main(String[] args)
    {
        Vector<Integer> numbers = new Vector<>();
        numbers.add(5);
        numbers.add(2);
        numbers.add(9);
        numbers.add(12);
        System.out.println("By using Lambda....");
        numbers.forEach(x -> EvenOrOdd.isEven(x));

        System.out.println("_____");

        System.out.println("By using static method reference");
        numbers.forEach(EvenOrOdd::isEven);
    }
}

```

Program that describes how to work with instance method :

```

package com.ravi.instance_method_reference;

interface Trainer
{
    void getTraining(String name, int experience);
}

class InstanceMethod
{
    public void getTraining(String name, int experience)
    {
        System.out.println("Trainer name is :"+name+" having "+experience+" years of experience.");
    }
}

public class InstanceMethodReferenceDemo
{
    public static void main(String[] args)
    {
        //Using Lambda Expression
        Trainer t1 = (name, exp)-> System.out.println("Trainer name is :"+name+" and total experience is :"+exp);
        t1.getTraining("Smith", 5);
    }
}

```

```
//By using Method reference
Trainer t2 = new InstanceMethod().getTraining;
t2.getTraining("Scott", 10);
```

```
    }
}
```

By using constructor :

package com.ravi.constructor_reference;

```
@FunctionalInterface
interface A
{
    Test createObject();
}
```

```
class Test
{
    public Test()
    {
        System.out.println("Test class Constructor invoked");
    }
}
```

```
public class ConstructorReferenceDemo1
{
    public static void main(String[] args)
    {
        //By Using Lambda
        A obj = ()-> new Test();
        obj.createObject();

        System.out.println(".....");

        //By Using Constructor Reference
        A obj1 = Test::new;
        obj1.createObject();
    }
}
```

package com.ravi.constructor_reference;

```
import java.util.function.Function;
```

```
class MyClass
{
    private int value;

    public MyClass(int value)
    {
        this.value = value;
    }
}
```

```

    public int getValue()
    {
        return this.value;
    }
}

```

```

public class ConstructorReferenceDemo2
{
    public static void main(String[] args)
    {
        Function<Integer,MyClass> consRef = MyClass::new;
        MyClass obj = consRef.apply(9);
        System.out.println(obj.getValue());
    }
}

```

//How to work with array object i.e creating array using constructor :

```

package com.ravi.constructor_reference;

```

```

import java.util.Scanner;
import java.util.function.Function;

```

```

class Person
{
    private String name;

    public Person(String name)
    {
        this.name = name;
    }

    public String getName()
    {
        return name;
    }
}

```

```

public class ConstructorReferenceDemo3
{
    public static void main(String[] args)
    {
        Function<Integer,Person[]> consRef = Person[]::new;
        Person[] persons = consRef.apply(5); //5 is size of array

        Scanner sc = new Scanner(System.in);

        for(int i=0; i<persons.length; i++)
        {
            System.out.println("Enter the name at "+i+" position :");
            String name = sc.nextLine();
            persons[i] = new Person(name);
        }
    }
}

```



```

        System.out.println("Getting the name of the person :");
        for(Person person : persons)
        {
            System.out.println(person.getName());
        }
    }
}

```

How to craete an array object using constructor array reference :

```

package com.ravi.constructor_ref;

import java.util.Scanner;
import java.util.function.Function;

class Employee
{
    private int employeeId;
    private String employeeName;

    public Employee(int employeeId, String employeeName)
    {
        super();
        this.employeeId = employeeId;
        this.employeeName = employeeName;
    }

    @Override
    public String toString() {
        return "Employee [employeeId=" + employeeId + ", employeeName=" + employeeName + "]";
    }
}

public class ConstructorReferenceDemo4 {

    public static void main(String[] args)
    {
        //creating Employee Array
        Function<Integer,Employee[]> fn1 = Employee[]::new;

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the size of an array :");
        int size = sc.nextInt();

        Employee[] emps = fn1.apply(size); //here size is, size of array

        //Initialized the Employee array object
        for(int i=0; i<emps.length; i++)
        {
            System.out.print("Enter Employee id :");
            int id = sc.nextInt();

```

```
System.out.print("Enter Employee name :");
String name = sc.nextLine();
name = sc.nextLine();
```

```
emps[i] = new Employee(id, name);
```

```
}
```

```
//Retrieve the value from Array object
for(Employee e1 : emps)
{
    System.out.println(e1);
}
```

```
}
```

```
}
```

Arbitrary Reference [Reference to an instance method of an arbitrary object of a particular type]

In Arbitrary reference we can access a non static method with the help of class name.
Here internally this keyword i.e object of that particular class is passed by JVM to access non static method using Method reference.

package com.ravi.arbitrary_reference;

```
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
```

```
public class ArbitraryRefDemo1
{
    public static void main(String[] args)
    {
        List<Integer> list = Arrays.asList(9,5,11,3,7,2);
```

```
//Sort the given list using Lambda Approach by using comparator
Collections.sort(list, (i1,i2)-> i1.compareTo(i2));
list.forEach(System.out::println);
```

```
System.out.println(".....");
```

```
//Sort the given list using Arbitrary reference by using comparator
Collections.sort(list, Integer::compareTo);
list.forEach(System.out::println);
```

```
System.out.println(".....");
```

```
String [] players = {"Rohit", "Abhishek", "Virat", "Bhubneswar", "Zaheer"};
Arrays.sort(players, String::compareTo);
System.out.println(Arrays.toString(players));
```

```

}
}
-----
package com.ravi.arbitrary_reference;

import java.util.Arrays;
import java.util.Comparator;

class Person
{
    String name;

    public Person(String name)
    {
        this.name = name;
    }

    public int personInstanceMethod1(Person person)
    {
        return this.name.compareTo(person.name);
    }

    @Override
    public String toString() {
        return "Person [name=" + name + "]";
    }

}

public class ArbitraryRefDemo2
{
    public static void main (String[] args) throws Exception
    {

        Person[] personArray = {new Person("Zuber"),new Person("Raj"), new Person("Ankit"), new
        Person("Abhishek")};
        Arrays.sort(personArray, Person::personInstanceMethod1);

        System.out.println(Arrays.toString(personArray));

    }

}
-----

```

Generics :

Why generic came into picture :

As we know our compiler is known for Strict type checking because java is a statically typed checked language.

The basic problem with collection is It can hold any kind of Object.

```
ArrayList al = new ArrayList();
```

```

al.add("Ravi");
al.add("Aswin");
al.add("Rahul");
al.add("Raj");
al.add("Samir");

for(int i =0; i<al.size(); i++)
{
    String s = (String) al.get(i);
    System.out.println(s);
}

```

By looking the above code it is clear that Collection stores everything in the form of Object so here even after adding String type only we need to provide casting as shown below.

```

import java.util.*;
class Test1
{
    public static void main(String[] args)
    {
        ArrayList al = new ArrayList(); //raw type
        al.add("Ravi");
        al.add("Ajay");
        al.add("Vijay");

        for(int i=0; i<al.size(); i++)
        {
            String name = (String) al.get(i); //type casting is required
            System.out.println(name.toUpperCase());
        }

    }
}

```

Even after type casting there is no guarantee that the things which are coming from ArrayList Object is String only because we can add anything in the Collection as a result java.lang.ClassCastException as shown in the program below.

```

import java.util.*;
class Test2
{
    public static void main(String[] args)
    {
        ArrayList t = new ArrayList(); //raw type
        t.add("alpha");
        t.add("beta");
        for (int i = 0; i < t.size(); i++)
        {
            String str =(String) t.get(i);
            System.out.println(str);
        }

        t.add(1234);
        t.add(1256);
        for (int i = 0; i < t.size(); ++i)

```

```

    {
        String obj= (String)t.get(i); //we can't perform type casting here
        System.out.println(obj);
    }
}
}
}

```

To avoid all the above said problem Generics came into picture from JDK 1.5 onwards

-> It deals with type safe Object so there is a guarantee of both the end i.e putting inside and getting outside.

Example:-

```
ArrayList<String > al = new ArrayList<>();
```

Now here we have a guarantee that only String can be inserted as well as only String will come out from the Collection so we can perform String related operation.

Advantages of Generics :

-
- 1) Type Safe Object (No Compilation warning)
 - 2) No need of type casting
 - 3) Strict compile time checking. (*Type Erasure)
-

```

import java.util.*;
public class Test3
{
    public static void main(String[] args)
    {
        ArrayList<String> al = new ArrayList<>(); //Generic type
        al.add("Ravi");
        al.add("Ajay");
        al.add("Vijay");

        for(int i=0; i<al.size(); i++)
        {
            String name = al.get(i); //no type casting is required
            System.out.println(name.toUpperCase());
        }
    }
}

```

//Program that describes the return type of any method can be type safe
 //[We can apply generics on method return type]

```

import java.util.*;
public class Test4
{
    public static void main(String [] args)
    {
        Dog d1 = new Dog();
        Dog d2 = d1.getDogList().get(0);
        System.out.println(d2);
    }
}

```

```

class Dog
{
    public List<Dog> getDogList()
    {
        ArrayList<Dog> d = new ArrayList<>();
        d.add(new Dog());
        d.add(new Dog());
        d.add(new Dog());
        return d;
    }
}

```

Note :- In the above program the compiler will stop us from returning anything which is not compatible with List<Dog> and there is a guarantee that only "type safe list of Dog object" will be returned so we need not to provide type casting as shown below

```

Dog d2 = (Dog) d1.getDogList().get(0); //before generic.

```

Mixing generic with Non Generic :

```

//Mixing generic with non-generic
import java.util.*;

```

```

class Car
{
}
public class Test5
{
    public static void main(String [] args)
    {
        ArrayList<Car> a = new ArrayList<>();
        a.add(new Car());
        a.add(new Car());
        a.add(new Car());
    }
}

```

```

ArrayList b = a; //assigning Generic to raw type

```

```

    System.out.println(b);
}
}

```

```

//Mixing generic to non-generic
import java.util.*;
public class Test6
{
    public static void main(String[] args)
    {
        List<Integer> myList = new ArrayList<>();
        myList.add(4);
        myList.add(6);
        myList.add(5);
    }
}

```

```

UnknownClass u = new UnknownClass();
int total = u.addValue(myList);
System.out.println("The sum of Integer Object is :"+total);
}

```

```

}
class UnknownClass
{
    public int addValues(List list) //generic to raw type OR
    {
        Iterator it = list.iterator();
        int total = 0;
        while (it.hasNext())
        {
            int i = ((Integer)it.next());
            total += i;           //total = 15
        }
        return total;
    }
}

```

Note :-

In the above program the compiler will not generate any warning message because even though we are assigning type safe Integer Object to unsafe or raw type List Object but this List Object is not inserting anything new in the collection so there is no risk to the caller.

```

-----
//Mixing generic to non-generic
import java.util.*;
public class Test7
{
    public static void main(String[] args)
    {
        List<Integer> myList = new ArrayList<>();

        myList.add(4);
        myList.add(6);
        UnknownClass u = new UnknownClass();
        int total = u.addValues(myList);
        System.out.println(total);
    }
}

class UnknownClass
{
    public int addValues(List list)
    {
        list.add(5); //adding object to raw type
        Iterator it = list.iterator();
        int total = 0;
        while (it.hasNext())
        {
            int i = ((Integer)it.next());
            total += i;
        }
        return total;
    }
}

```

Here Compiler will generate warning message because the unsafe object is inserting the value 5 to safe object.

*Type Erasure

In the above program the compiler will generate warning message because the unsafe List Object is inserting the Integer object 5 so the type safe Integer object is getting value 5 from unsafe type so there is a problem to the caller method.

By writing `ArrayList<Integer>` actually JVM does not have any idea that our ArrayList was suppose to hold only Integers.

All the type safe generics information does not exist at runtime. All our generic code is Strictly for compiler.

There is a process done by java compiler called "Type erasure" in which the java compiler converts generic version to non-generic type.

```
List<Integer> myList = new ArrayList<Integer>();
```

At the compilation time it is fine but at runtime for JVM the code becomes

```
List myList = new ArrayList();
```

Note :- GENERIC IS STRICTLY COMPILE TIME PROTECTION.

30-07-2024

//Polymorphism with array

```
import java.util.*;
abstract class Animal
{
    public abstract void checkup();
}

class Dog extends Animal
{
    @Override
    public void checkup()
    {
        System.out.println("Dog checkup");
    }
}

class Cat extends Animal
{
    @Override
    public void checkup()
    {
        System.out.println("Cat checkup");
    }
}

class Bird extends Animal
{
    @Override
```



```
public void checkup()
{
    System.out.println("Bird checkup");
}
}
```

```
public class Test8
{
    public void checkAnimals(Animal animals[])
    {
        for(Animal animal : animals)
        {
            animal.checkup();
        }
    }
}
```

```
public static void main(String[] args)
{
    Dog []dogs={new Dog(), new Dog()};

    Cat []cats={new Cat(), new Cat(), new Cat()};

    Bird []birds = {new Bird(), new Bird()};

    Test8 t = new Test8();

    t.checkAnimals(dogs);
    t.checkAnimals(cats);
    t.checkAnimals(birds);
}
}
```

Note :-From the above program it is clear that polymorphism(Upcasting) concept works with array.

```
import java.util.*;
abstract class Animal
{
    public abstract void checkup();
}
```

```
class Dog extends Animal
{
    @Override
    public void checkup()
    {
        System.out.println("Dog checkup");
    }
}
```

```
class Cat extends Animal
{
    @Override
    public void checkup()
    {
        System.out.println("Cat checkup");
    }
}
```

```

}
class Bird extends Animal
{
    @Override
    public void checkup()
    {
        System.out.println("Bird checkup");
    }
}
public class Test9
{
    public void checkAnimals(List<Animal> animals)
    {
        for(Animal animal : animals)
        {
            animal.checkup();
        }
    }
    public static void main(String[] args)
    {
        List<Dog> dogs = new ArrayList<>();
        dogs.add(new Dog());
        dogs.add(new Dog());

        List<Cat> cats = new ArrayList<>();
        cats.add(new Cat());
        cats.add(new Cat());

        List<Bird> birds = new ArrayList<>();
        birds.add(new Bird());

        Test9 t = new Test9();
        t.checkAnimals(dogs);
        t.checkAnimals(cats);
        t.checkAnimals(birds);

    }
}

```

Note :- The above program will generate compilation error.

So from the above program it is clear that polymorphism does not work in the same way for generics as it does with arrays.

Example :

```

Parent [] arr = new Child[5]; //valid
Object [] arr = new String[5]; //valid

```

But in generics the same type is not valid

```

List<Object> list = new ArrayList<Integer>(); //Invalid
List<Parent> mylist = new ArrayList<Child>(); //Invalid

```

```

import java.util.*;
public class Test10
{
    public static void main(String [] args)
    {
        /*ArrayList<Object> al = new ArrayList<String>(); [Compile time]
        ArrayList al = new ArrayList(); [Runtime, Type Erasure]
        al.add("Ravi");*/

        Object []obj = new String[3]; //valid with Array
        obj[0] = "Ravi";
        obj[1] = "hyd";
        obj[2] = 90; //java.lang.ArrayStoreException
        System.out.println(Arrays.toString(obj));
    }
}

```

Note :- Program will generate java.lang.ArrayStoreException because we are trying to insert 90 (integer value) into String array.

In Array we have an Exception called ArrayStoreException (Which protect us to assign some illegal value in the array) but the same Exception or such type of exception, is not available with Generics (due to Type Erasure) that is the reason in generics, compiler does not allow upcasting concept. (It is a strict compile time checking)

 Wild card character(?) :

 <?> -: Many possibilities

<Animal>:- Only <Animal> can assign, but not Dog or sub type of animal

<? super Dog>:- Dog, Animal, Object can assign (Compiler has surity) It is also called lower bound

<? extends Animal> -: Below of Animal(Child of Animal) means, sub classes of Animal (But the compiler does not have surity because you can have many sub classes of Animal in the future, so chances of wrong collections). It is also called Upper bound.

 import java.util.*;
 class Parent
 {
 }
 class Child extends Parent
 {
 }

 public class Test11
 {
 public static void main(String [] args)
 {
 ArrayList<?> lp = new ArrayList<Parent>();

 ArrayList<Parent> lp1 = new ArrayList<Parent>();
 }
 }

```
ArrayList<Child> lp2 = new ArrayList<>();
```

```
System.out.println("Success");
}
}
```

//program on wild-card chracter

```
import java.util.*;
```

```
class Parent
```

```
{
```

```
}
```

```
class Child extends Parent
```

```
{
```

```
}
```

```
public class Test12
```

```
{
```

```
public static void main(String [] args)
```

```
{
```

```
List<?> lp = new ArrayList<Parent>();
```

```
System.out.println("Wild card....");
```

```
}
```

```
}
```

import java.util.*;

```
public class Test13
```

```
{
```

```
public static void main(String[] args)
```

```
{
```

```
List<? extends Number> list1 = new ArrayList<Double>();
```

```
List<? super String> list2 = new ArrayList<Object>();
```

```
List<? super Gamma> list3 = new ArrayList<Alpha>();
```

```
List list4 = new ArrayList();
```

```
System.out.println("yes");
```

```
}
```

```
}
```

```
class Alpha
```

```
{
```

```
}
```

```
class Beta extends Alpha
```

```
{
```

```
}
```

```
class Gamma extends Beta
```

```
{
```

```
}
```

We can also create our own functional interfaces :

package com.ravi.collection;

```
@FunctionalInterface
interface OwnFunction<T,U,V,R>
{
    public abstract R apply(T t, U u, V v);
}
```

```
public class MyFunctionalInterface {

    public static void main(String[] args)
    {
        OwnFunction<Integer,String,Integer,String> fun = (t, u, v)-> t + u + v;
        System.out.println(fun.apply(12, " india ", 15));

    }

}
```

```
class Test<T,U>
{
    private T r;
    public void set(U a)
    {
        r = (T)a;
    }

    public T get()
    {
        return this.r;
    }
}

public class Test14
{
    public static void main(String[] args)
    {
        Test<String,String> test = new Test();
        test.set("NIT");
        System.out.println(test.get());
    }
}
```

```
class MyClass<T>
{
    T obj;
    public MyClass(T obj)    //Student obj = new Student(
    {
        this.obj=obj;
    }

    T getObj()
    {
        return this.obj;
    }
}
```

```

}
}
public class Test15
{
    public static void main(String[] args)
    {
        Integer i=12;
        MyClass<Integer> mi = new MyClass<>(i);
        System.out.println("Integer object stored :"+mi.getObj());

        Float f=12.34f;
        MyClass<Float> mf = new MyClass<>(f);
        System.out.println("Float object stored :"+mf.getObj());

        MyClass<String> ms = new MyClass<>("Rahul");
        System.out.println("String object stored :"+ms.getObj());

        MyClass<Boolean> mb = new MyClass<>(false);
        System.out.println("Boolean object stored :"+mb.getObj());

        Double d=99.34;
        MyClass<Double> md = new MyClass<>(d);
        System.out.println("Double object stored :"+md.getObj());

        MyClass<Student> std = new MyClass<>(new Student(1,"A"));
        System.out.println("Student object stored :"+std.getObj());
    }
}

```

```

record Student(int id, String name)
{

}

```

//E stands for Element type

```

class Fruit
{
}
class Apple extends Fruit //Fruit is super, Apple is sub class
{
    @Override
    public String toString()
    {
        return "Apple";
    }
}

```

```

class Basket<E> // E is Fruit Type
{
    private E element;
    public void setElement(E element) //Fruit element = new Apple();
    {
        this.element = element;
    }
}

```

```

public E getElement() // E Fruit
{
    return this.element;
}
}

```

```

public class Test16
{
    public static void main(String[] args)
    {
        Basket<Fruit> b = new Basket<>();
        b.setElement(new Apple());

```

```

        Apple x = (Apple) b.getElement();
        System.out.println(x);

```

```

        Basket<Fruit> b1 = new Basket<>();
        b1.setElement(new Mango());
        Mango y = (Mango)b1.getElement();
        System.out.println(y);

```

```

    }
}
class Mango extends Fruit
{
    @Override
    public String toString()
    {
        return "Mango";
    }
}

```

Note : There is no difference between element and type parameter, only they are represented by different types

E : Element Type
T : Type Parameter

----- Concurrent collections in java

Concurrent Collections are introduced from JDK 1.5 onwards to enhance the performance of multithreaded application.

These are threadsafe collection and available in java.util.concurrent sub package.

Limitation of Traditional Collection :

1) In the Collection framework most of the Collection classes are not thread-safe because those are non-synchronized like ArrayList, LinkedList, HashSet, HashMap is non-synchronized in nature, So If multiple threads will perform any operation on the collection object simultaneously then we will get some wrong data this is known as Data race or Race condition.

2) Some Collection classes are synchronized like Vector, Hashtable but performance wise these classes

are slow in nature.

Collections class has provided static methods to make our List, Set and Map interface classes as a synchronized.

- a) public static List synchronizedList(List list)
- b) public static Set synchronizedSet(Set set)
- c) public static Map synchronizedMap(Map map)

3) Traditional Collection works with fail fast iterator that means while iterating the element, if there is a change in structure then we will get java.util.ConcurrentModificationException, On the other hand concurrent collection works with fail safe iterator where even though there is a change in structure but we will not get ConcurrentModificationException.

```
-----
import java.util.*;
public class Collection1
{
    public static void main(String args[])
    {
        ArrayList al = new ArrayList();
        al.add(10);
        al.add(20);
        al.add(30);
        al.add(40);
        al.add(50);
        al.add(50);
        System.out.println("Arraylist Elements : "+al);
        Set s = new HashSet(al);
        System.out.println("Set Elements are: "+s);
    }
}
-----
```

```
//Collections.synchronizedList(List list);
import java.util.*;
public class Collection2
{
    public static void main(String[] args)
    {
        ArrayList<String> arl = new ArrayList<>();
        arl.add("Apple");
        arl.add("Orange");
        arl.add("Grapes");
        arl.add("Mango");
        arl.add("Guava");
        arl.add("Mango");
```

```
List<String> syncCollection = Collections.synchronizedList(arl);
```

```
List<String> upperList = new ArrayList<>(); //New List
```

```
Runnable listOperations = () ->
```

```
{
    synchronized (syncCollection)
    {
        syncCollection.forEach(str -> upperList.add(str.toUpperCase()));
    }
}
```



```

    }
};

Thread t1 = new Thread(listOperations);
t1.start();

    upperList.forEach(x -> System.out.println(x));
}
}
-----
//Collections.synchronizedSet(Set set);
import java.util.*;
public class Collection3
{
    public static void main(String[] args)
    {
        Set<String> set = Collections.synchronizedSet(new HashSet<>());
        set.add("Apple");
        set.add("Orange");
        set.add("Grapes");
        set.add("Mango");
        set.add("Guava");
        set.add("Mango");
        System.out.println("Set after Synchronization :");
        synchronized (set)
        {
            Spliterator<String> itr = set.spliterator();
            itr.forEachRemaining(str -> System.out.println(str));
        }
    }
}
-----
//Collections.synchronizedMap(Map map);
import java.util.*;
public class Collection4
{
    public static void main(String[] args)
    {
        Map<String, String> map = new HashMap<String, String>();
        map.put("1", "Ravi");
        map.put("4", "Elina");
        map.put("3", "Aryan");
        Map<String, String> synmap = Collections.synchronizedMap(map);
        System.out.println("Synchronized map is ." + synmap);
    }
}
-----
import java.util.*;
class ConcurrentModification extends Thread
{
    ArrayList<String> al = null;

    public ConcurrentModification(ArrayList<String> al) //al = arl
    {
        this.al = al;
    }
}

```

```

}

@Override
public void run()
{
    try
    {
        Thread.sleep(1000);
    }
    catch (InterruptedException e)
    {
    }
    al.add("KIWI");
}
}
public class Collection5
{
    public static void main(String[] args) throws InterruptedException
    {
        ArrayList<String> arl = new ArrayList<>();
        arl.add("Apple");
        arl.add("Orange");
        arl.add("Grapes");
        arl.add("Mango");
        arl.add("Guava");

        ConcurrentModification cm = new ConcurrentModification(arl);

        cm.start();

        Iterator<String> itr = arl.iterator();
        while(itr.hasNext())
        {
            String str = itr.next();
            System.out.println(str);
            Thread.sleep(500);
        }
    }
}

```

Note :- In the above program we will get java.util.ConcurrentModificationException because Iterator is fail fast iterator.

Note : IMMUTABLE OBJECTS ARE ALWAYS THREADED SAFE.

31-07-2024

CopyOnWriteArrayList in java :

public class CopyOnWriteArrayList implements List, Cloneable, Serializable, RandomAccess

A CopyOnWriteArrayList is similar to an ArrayList but it has some additional features like thread-safe. This class is existing in java.util.concurrent sub package.

ArrayList is not thread-safe. We can't use ArrayList in the multi-threaded environment because it creates a

problem in ArrayList values (Data inconsistency).

*The CopyOnWriteArrayList is an enhanced version of ArrayList. If we are making any modifications(add, remove, etc.) in CopyOnWriteArrayList then JVM creates a new copy by use of Cloning.

The CopyOnWriteArrayList is costly, if we want to perform update operations so it is immutable object , because whenever we make any changes the JVM creates a cloned copy of the array and add/update element to new object.

It is a thread-safe version of ArrayList as well as here Iterator is fail safe iterator.

*CopyOnWriteArrayList is the best choice if we want to perform read operation frequently in multithreaded environment.

The CopyOnWriteArrayList is a replacement of a synchronized List, because it offers better concurrency.

Constructors of CopyOnWriteArrayList in java :

We have 3 constructors :

1) CopyOnWriteArrayList c = new CopyOnWriteArrayList();

It creates an empty list in memory. This constructor is useful when we want to create a list without any value.

2) CopyOnWriteArrayList c = new CopyOnWriteArrayList(Collection c);

Interconversion of collections.

3) CopyOnWriteArrayList c = new CopyOnWriteArrayList(Object[] obj) ;

It Creates a list that containing all the elements that is specified Array. This constructor is useful when we want to create a CopyOnWriteArrayList from Array.

Note : All the immutable objects are thread-safe because, On immutable objects if we perform any operation then another object will be created in a new memory location so at a time multiple threads can work.

All String Object, Wrapper classes object, Concurrent collection classes are Thread-safe.

import java.util.Arrays;
import java.util.Iterator;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

public class CopyOnWriteArrayListExample1

{
 public static void main(String[] args)
 {
 List<String> list = Arrays.asList("Apple", "Orange", "Mango","Kiwi", "Grapes");

 CopyOnWriteArrayList<String> copyOnWriteList = new CopyOnWriteArrayList<String>(list);

 System.out.println("Without modification = "+copyOnWriteList);

//Iterator1

```

        Iterator<String> iterator1 = copyOnWriteList.iterator();

        //Add one element and verify list is updated
        copyOnWriteList.add("Guava");

        System.out.println("After modification = "+copyOnWriteList);

        //Iterator2
        Iterator<String> iterator2 = copyOnWriteList.iterator();

        System.out.println("Element from first Iterator:");
        iterator1.forEachRemaining(System.out::println);

        System.out.println("Element from Second Iterator:");
        iterator2.forEachRemaining(System.out::println);
    }
}
-----
import java.util.*;
import java.util.concurrent.*;
class ConcurrentModification extends Thread
{
    CopyOnWriteArrayList<String> al = null;
    public ConcurrentModification(CopyOnWriteArrayList<String> al)
    {
        this.al = al;
    }
    @Override
    public void run()
    {
        try
        {
            Thread.sleep(1000);
        }
        catch (InterruptedException e)
        {
        }
        al.add("KIWI");
    }
}
public class CopyOnWriteArrayListExample2
{
    public static void main(String[] args) throws InterruptedException
    {
        CopyOnWriteArrayList<String> arl = new CopyOnWriteArrayList<>();
        arl.add("Apple");
        arl.add("Orange");
        arl.add("Grapes");
        arl.add("Mango");
        arl.add("Guava");
        ConcurrentModification cm = new ConcurrentModification(arl);
        cm.start();

        Iterator<String> itr = arl.iterator();
        while(itr.hasNext())

```

```

{
    String str = itr.next();
    System.out.println(str);
    Thread.sleep(1500);
}

    System.out.println(".....");

    Spliterator<String> spl = arl.spliterator();
    spl.forEachRemaining(x -> System.out.println(x));
}
}

```

CopyOnWriteArraySet :

public class CopyOnWriteArraySet extends AbstractSet implements Serializable

A CopyOnWriteArraySet is a thread-safe version of HashSet in Java and it works like CopyOnWriteArrayList in java.

The CopyOnWriteArraySet internally used CopyOnWriteArrayList to perform all type of operation. It means the CopyOnWriteArraySet internally creates an object of CopyOnWriteArrayList and perform operation on it.

Whenever we perform add, set, and remove operation on CopyOnWriteArraySet, it internally creates a new object of CopyOnWriteArrayList and copies all the data to the new object by eliminating duplicates so, when it is used in by multiple threads, it doesn't create a problem, but it is well suited if we have small size collection and want to perform only read operation by multiple threads.

The CopyOnWriteArraySet is the replacement of synchronizedSet and offers better concurrency.

It creates a new copy of the array every time iterator is created, so performance is slower than HashSet.

Constructors :

It has two constructors

1) CopyOnWriteArraySet set1 = new CopyOnWriteArraySet();
It will create an empty Set

2) CopyOnWriteArraySet set1 = new CopyOnWriteArraySet(Collection c); Interconversion of collection.

import java.util.*;
import java.util.concurrent.CopyOnWriteArraySet;
import java.util.concurrent.CopyOnWriteArraySet;

```

public class CopyOnWriteArraySetExample1
{
    public static void main(String[] args)
    {
        CopyOnWriteArraySet<String> set = new CopyOnWriteArraySet<>();

        set.add("Java");
        set.add("Python");
    }
}

```

```

        set.add("C++");
set.add("Java");

        Iterator itr = set.iterator();
while(itr.hasNext())
{
    System.out.println(itr.next());
}

        // Adding a new element
set.add("JavaScript");
System.out.println(".....");
for (String language : set)
{
    System.out.println(language);
}
}
}

```

```

import java.util.concurrent.CopyOnWriteArraySet;

public class CopyOnWriteArraySetExample2
{
    public static void main(String[] args)
    {
        CopyOnWriteArraySet<Integer> set = new CopyOnWriteArraySet<Integer>();
        set.add(1);
        set.add(2);
        set.add(3);
        set.add(4);
        set.add(5);

        System.out.println("Is element contains: "+set.contains(1));

        System.out.println("Is set empty: "+set.isEmpty());

        System.out.println("remove element from set: "+set.remove(3));

        System.out.println("Element from Set: "+ set);
    }
}

```

*** ConcurrentHashMap : [Bucket Level Locking]

```

public class ConcurrentHashMap<K,V> extends AbstractMap<K,V> implements ConcurrentMap<K,V>,
Serializable

```

Like HashMap, ConcurrentHashMap provides similar functionality except that it has internally maintained concurrency.

It is the concurrent version of the HashMap. It internally maintains a Hashtable that is divided into segments(Buckets).

The number of segments depends upon the level of concurrency required the Concurrent HashMap. By default, it divides into 16 segments and each Segment behaves independently. It doesn't lock the whole

HashMap as done in Hashtables/synchronizedMap, it only locks the particular segment(Bucket) of HashMap. [Bucket level locking]

ConcurrentHashMap allows multiple threads can perform read/write operation without locking the ConcurrentHashMap object.

It does not allow null as a key or even null as a value.

[Note :- TreeSet, TreeMap, Hashtable, PriorityQueue, ConcurrentHashMap , These 5 classes never containing null key or null element)

It contains 5 types of constructor :

-
- 1) ConcurrentHashMap chm1 = new ConcurrentHashMap();
- 2) ConcurrentHashMap chm2 = new ConcurrentHashMap(int initialCapacity);
- 3) ConcurrentHashMap chm3 = new ConcurrentHashMap(int initialCapacity, float loadFactor);
- 4) ConcurrentHashMap chm4 = new ConcurrentHashMap(int initialCapacity, float loadFactor, int concurrencyLevel);
- 5) ConcurrentHashMap chm5 = new ConcurrentHashMap(ConcurrentMap m);

Internal Working of ConcurrentHashMap :

Like HashMap and Hashtable, the ConcurrentHashMap is also used Hashtable data structure. But it is using the segment locking strategy to handle the multiple threads.

A segment(bucket) is a portion of ConcurrentHashMap and ConcurrentHashMap uses a separate lock for each thread. Unlike Hashtable or synchronized HashMap, it doesn't synchronize the whole HashMap or Hashtable for one thread.

As we have seen in the internal implementation of the HashMap, the default size of HashMap is 16 and it means there are 16 buckets. The ConcurrentHashMap uses the same concept is used in ConcurrentHashMap. It uses the 16 separate locks for 16 buckets by default because the default concurrency level is 16. It means a ConcurrentHashMap can be used by 16 threads at same time. If one thread is reading from one bucket(Segment), then the second bucket doesn't affect it.

Why we need ConcurrentHashMap in java?

As we know Hashtable and HashMap works based on key-value pairs. But why we are introducing another Map? As we know HashMap is not thread safe, but we can make it thread-safe by using Collections.synchronizedMap() method and Hashtable is thread-safe by default.

But a synchronized HashMap or Hashtable is accessible only by one thread at a time because the object get the lock for the whole HashMap or Hashtable. Even multiple threads can't perform read operations at the same time. It is the main disadvantage of Synchronized HashMap or Hashtable, which creates performance issues. So ConcurrentHashMap provides better performance than Synchronized HashMap or Hashtable.

```
import java.util.HashMap;
import java.util.concurrent.ConcurrentHashMap;
```

```
public class ConcurrentHashMapExample1
```

```

{
    public static void main(String args[])
    {

        HashMap<Integer, String> hashMap = new HashMap<Integer, String>();
        hashMap.put(1, "Ravi");
        hashMap.put(2, "Ankit");
        hashMap.put(3, "Prashant");
        hashMap.put(4, "Pallavi");

        ConcurrentHashMap<Integer, String> concurrentHashMap = new
        ConcurrentHashMap<>(hashMap);
        System.out.println("Object from ConcurrentHashMap: "+ concurrentHashMap);

    }
}

```

```

-----
import java.util.Iterator;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class ConcurrentHashMapExample2
{
    public static void main(String args[])
    {
        // Creating ConcurrentHashMap
        Map<String, String> cityTemperatureMap = new ConcurrentHashMap<>();

        cityTemperatureMap.put("Delhi", "30");
        cityTemperatureMap.put("Mumbai", "32");
        cityTemperatureMap.put("Chennai", "35");
        cityTemperatureMap.put("Bangalore", "22" );

        Iterator<String> iterator = cityTemperatureMap.keySet().iterator();

        while (iterator.hasNext())
        {
            System.out.println(cityTemperatureMap.get(iterator.next()));
            // adding new value, it won't throw error
            cityTemperatureMap.put("Hyderabad", "28");
        }
    }
}

```

01-08-2024

Streams in java :

It is introduced from Java 8 onwards, the Stream API is used to process the collection objects.

It contains classes for processing sequence of elements over Collection object and array object.

Stream is a predefined interface available in java.util.stream sub package.

Package Information :

java.util -> Base package
java.util.function -> Functional interfaces(Lambda)
java.util.concurrent -> Multithreaded support
java.util.stream -> Processing of Collection Object

forEach() method in java :

The Java forEach() method is a technique to iterate over a collection such as (list, set or map) and stream. It is used to perform a given action on each of the element of the collection.

The forEach() method has been added in following places:

Iterable interface This makes Iterable.forEach() method available to all collection classes. Iterable interface is the super interface of Collection interface

Map interface This makes forEach() operation available to all map classes.

Stream interface This makes forEach() operations available to all types of stream.

Creation of Streams to process the data :

We can create Stream from collection or array with the help of stream() and of() methods:

A stream() method is added to the Collection interface and allows creating a Stream<T> using any collection object as a source.

```
public java.util.Stream<E> stream();
```

The return type of this method is Stream interface available in java.util.stream sub package.

Eg:-

```
List<String> items = new ArrayList<String>();  
items.add("Apple");  
items.add("Orange");  
items.add("Mango");  
Stream<String> stream = items.stream();
```

package com.ravi.basic;
import java.util.*; //Base package
import java.util.stream.*; //Sub package
public class StreamDemo1
{
 public static void main(String[] args)
 {
 List<String> items = new ArrayList<>();

 items.add("Apple");
 items.add("Orange");
 items.add("Mango");

 //Collections Object to Stream
 Stream<String> strm = items.stream();
 strm.forEach(p -> System.out.println(p));
 }
}

```
}  
}
```

Stream.of()

```
public static java.util.stream.Stream of(T ...x)  
-----
```

It is a static method of Stream interface through which we can create Stream of arrays and Stream of Collection. The return type of this method is Stream interface.

```
//Stream.of()  
package com.ravi.basic;  
import java.util.stream.*;  
public class StreamDemo2  
{  
    public static void main(String[] args)  
    {  
        //Stream of Collection  
        Stream<Integer> stream = Stream.of(1,2,3,4,5,6,7,8,9);  
        stream.forEach(p -> System.out.println(p));  
  
        System.out.println(".....");  
  
        //Anonymous Array Object (Stream of Arrays)  
  
        Stream<Integer> strm = Stream.of( new Integer[]{15,29,45,8,16} );  
        strm.forEach(p -> System.out.println(p));  
    }  
}
```

Operation on Stream interface :

We have 2 types of operation :

- 1) Intermediate Operation
- 2) Terminal Operation

Intermediate Operations:

The following methods are available to perform intermediate operation.

filter(Predicate<T> predicate): Returns a new stream which contains filtered elements based on the boolean expression using Predicate.

map(Function<T, R> mapper): Transforms elements in the stream using the provided mapping function.

flatMap(Function<T, Stream<R>> mapper): Flattens a stream of streams into a single stream.

distinct(): Returns a stream with distinct elements (based on their equals method).

sorted(): Returns a stream with elements sorted in their natural order.

sorted(Comparator<T> comparator): Returns a stream with elements sorted using the specified comparator.

peek(Consumer<T> action): Allows us to perform an action on each element in the stream without

modifying the stream.

limit(long maxSize): Limits the number of elements in the stream to a specified maximum size.

skip(long n): Skips the first n elements in the stream.

takeWhile(Predicate<T> predicate): Returns a stream of elements from the beginning until the first element that does not satisfy the predicate.

dropWhile(Predicate<T> predicate): Returns a stream of elements after skipping elements at the beginning that satisfy the predicate.

public abstract Stream<T> filter(Predicate<T> p) :

It is a predefined method of Stream interface. It is used to select/filter elements as per the Predicate passed as an argument. It is basically used to filter the elements based on boolean condition.

public abstract <T> collect(java.util.stream.Collectors c)

It is a predefined method of Stream interface. It is used to return the result of the intermediate operations performed on the stream.

It is a terminal operation. It is used to collect the data after filtration and convert the data to the Collection(List/Set).

Collectors is a predefined final class available in java.util.stream sub package which contains static method toList() and toSet() to convert the data as a List/Set i.e Collection object. The return type of this method is List/Set interface.

//Filter all the even numbers from Collection
package com.ravi.basic;

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;
import java.util.stream.Stream;
```

```
public class StreamDemo3
{
    public static void main(String[] args)
    {
        //Printing even number without Stream interface
        List<Integer> listOfNumber = Arrays.asList(1,2,3,4,5,6,7,8,9,10,11,12);
```

```
List<Integer> evenList = new ArrayList<>();
```

```
for(Integer x : listOfNumber)
{
    if(x%2==0)
    {
        evenList.add(x);
    }
}
```

```

}
evenList.forEach(System.out::println);
System.out.println(".....");
Stream.of(1,2,3,4,5,6,7,8,9,10,11,12).filter(num -> num%2==0).forEach(System.out::println);

System.out.println(".....");
ArrayList<Integer> al = new ArrayList<>();
al.add(11);
al.add(12);
al.add(13);
al.add(14);
al.add(15);
al.add(16);
al.add(17);
al.add(18);
al.add(19);
al.add(20);
al.add(20);

Set<Integer> set = al.stream().filter(num -> num%2==0).collect(Collectors.toSet());
set.forEach(System.out::println);
}
}

```

```

//Filtering the name which starts with 'R' character.
package com.ravi.basic;

```

```

import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.Stream;

```

```

public class StreamDemo4
{
    public static void main(String[] args)
    {
        List<String> listOfName = Arrays.asList("Rahul","Ankit","Raj","Roshan","Scott");

        List<String> nameWithR = listOfName.stream().filter(str ->
str.startsWith("R")).collect(Collectors.toList());
        nameWithR.forEach(System.out::println);

        System.out.println(".....");

        Stream<String> names = Stream.of("Ravi","Rahul","Ankit","Raj","Roshan","Scott","Rahul","Raj");

        List<String> filteredName = names.filter(name -> name.startsWith("R")).collect(Collectors.toList());

        System.out.println(filteredName);

    }
}

```

public Stream sorted() :

It is a predefined method of Stream interface.

It provides default natural sorting order.

The return type of this method is Stream.

We have overloaded method which accepts Comparator<T>

//Sorting the data

```
package com.ravi.basic;
```

```
import java.util.*;
```

```
import java.util.stream.*;
```

```
public class StreamDemo5
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        List<String> names = Arrays.asList("Zaheer","Rahul","Aryan","Sailesh","Zaheer");
```

```
        List<String> sortedName =
```

```
            names.stream().sorted().collect(Collectors.toList());
```

```
        System.out.println(sortedName);
```

```
    }
```

```
}
```

package com.ravi.basic;

```
import java.util.ArrayList;
```

```
import java.util.List;
```

//Fetch all the Employees name whose salary is greater then 50k

```
record Employee(Integer employeeId, String employeeName, Double employeeSalary)
```

```
{
```

```
}
```

```
public class StreamDemo6
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        List<Employee> listOfEmployee = new ArrayList<>();
```

```
        listOfEmployee.add(new Employee(111, "Scott", 75000D));
```

```
        listOfEmployee.add(new Employee(222, "John", 45000D));
```

```
        listOfEmployee.add(new Employee(333, "Martin", 55000D));
```

```
        listOfEmployee.add(new Employee(444, "Smith", 65000D));
```

```
        listOfEmployee.add(new Employee(555, "Virat", 35000D));
```

```
        System.out.println("Name of the Employee whose salary is > 50K");
```

```
        listOfEmployee.stream().filter(emp -> emp.employeeSalary()>=50000).forEach(e ->  
        System.out.println(e.employeeName()));
```

```
    }
```

```
}
```

Assignment :

Print all the product with details whose price is < 25K

02-08-2024

public Stream map(Function<? super T,? extends R> mapper) :

It is a predefined method of Stream interface.

It takes Function (Predefined functional interface) as a parameter.

It performs intermediate operation and consumes single element from input Stream and produces single element to output Stream. (1:1 transformation)

Here mapper function is functional interface which takes one input and provides one output.

```
package com.ravi.basic;
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.Stream;
```

```
public class StreamDemo7
{
    public static void main(String[] args)
    {
        //add value 10 to each and every number
        List<Integer> asList = Arrays.asList(1,2,3,4,5,6,7,8,9);
        asList.stream().map(num -> num + 10).forEach(System.out::println);

        System.out.println(".....");

        //Find even numbers in stream and collect the cubes
        List<Integer> numbers = List.of(1,2,3,4,5,6,7,8,9,10,11,12);
        List<Integer> collect = numbers.stream().filter(n -> n % 2 == 0).map(num ->
num*num*num).collect(Collectors.toList());
        collect.forEach(System.out::println);

        System.out.println(".....");
        //Find the length of the name
        Stream.of("Raj","Scott","subramanyam","Rahul").map(str-> str.length()).forEach(System.out::println);

        //retrieve first character of all the given name

        List<Character> firstChar = Stream.of("Jyoti","Aman","Viraj","Ankit").map(str ->
str.charAt(0)).collect(Collectors.toList());
        System.out.println(firstChar);

    }
}
```

//Program on map(Function<T,R> mapped)
package com.ravi.basic;

```

import java.util.ArrayList;
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

public class StreamDemo8
{
    public static void main(String args[])
    {
        //Player object to name of the player
        List<String> nameOfPlayers = createPlayerList().stream().map(player
->player.name()).collect(Collectors.toList());
        nameOfPlayers.forEach(System.out::println);
    }

    public static List<Player> createPlayerList()
    {
        List<Player> al = new ArrayList<>();
        al.add(new Player(18, "Virat"));
        al.add(new Player(45, "Rohit"));
        al.add(new Player(7, "Dhoni"));
        al.add(new Player(18, "Virat"));
        al.add(new Player(90, "Bumrah"));
        al.add(new Player(67, "Hardik"));

        return al;
    }
}

```

```

record Player(Integer playerId, String name)
{

}

```

```

public Stream flatMap(Function<? super T,? extends Stream<? extends R>> mapper)

```

It is a predefined method of Stream interface.

The map() method produces one output value for each input value in the stream So if there are n elements in the stream, map() operation will produce a stream of n output elements.

flatMap() is two step process i.e. map() + Flattening. It helps in converting Collection<Collection<T>> into Collection<T> [to make flat i.e converting Collections of collection into single collection or merging of all the collection]

```

//flatMap()
//map + Flattening [Converting Collections of collection into single collection]
package com.ravi.basic;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.Collector;
import java.util.stream.Collectors;

```

```

public class StreamDemo9
{
    public static void main(String[] args)
    {
        List<String> list1 = Arrays.asList("A","B","C");
        List<String> list2 = Arrays.asList("D","E","F");
        List<String> list3 = Arrays.asList("G","H","I");

        List<List<String>> nestedColl = Arrays.asList(list1, list2, list3);
        System.out.println(nestedColl);

        List<String> flatColl = nestedColl.stream().flatMap(list -> list.stream()).collect(Collectors.toList());
        System.out.println(flatColl);

    }
}

```

Note : Whenever we are dealing with nested structure then we should use flatMap().

```

//Flattening of prime, even and odd number
package com.ravi.basic.flat_map;

```

```

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.Stream;

```

```

public class FlatMapDemo1
{
    public static void main(String[] args)
    {
        List<Integer> primeNumbers = Arrays.asList(5,7,11);
        List<Integer> evenNumbers = Arrays.asList(2,4,6);
        List<Integer> oddNumbers = Arrays.asList(1,3,5);

        List<List<Integer>> nestedColl = List.of(primeNumbers,evenNumbers,oddNumbers);
        System.out.println(nestedColl);

        List<Integer> flatNumber = nestedColl.stream().flatMap(number ->
number.stream()).collect(Collectors.toList());

        System.out.println(flatNumber);

    }
}

```

```

//Fetching first character using flatMap()
package com.ravi.basic.flat_map;

```

```

import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

```



```

import java.util.stream.Stream;

public class FlatMapDemo2
{
    public static void main(String[] args)
    {
        List<String> listOfNames = List.of("Aman","Venkatesh","Raj","Scott","Smith");

        Set<Character> firstChar = listOfNames.stream().flatMap(str ->
Stream.of(str.charAt(0))).collect(Collectors.toSet());

        System.out.println(firstChar);

    }
}

```

```

-----
package com.ravi.basic.flat_map;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

class Product
{
    private Integer productId;
    private List<String> listOfProducts;

    public Product(Integer productId, List<String> listOfProducts)
    {
        super();
        this.productId = productId;
        this.listOfProducts = listOfProducts;
    }

    public Integer getProductId() {
        return productId;
    }

    public List<String> getListOfProducts() {
        return listOfProducts;
    }
}

```

```

public class FlatMapDemo3
{
    public static void main(String[] args)
    {

        List<Product> listOfProduct = Arrays.asList(
new Product(1, Arrays.asList("Camera", "Mobile", "Laptop")),
new Product(2, Arrays.asList("Bat", "Ball", "Wicket")),
new Product(3, Arrays.asList("Chair", "Table", "Lamp")),
new Product(4, Arrays.asList("Cycle", "Bike", "Car"))

```

```
);
```

```
List<String> collect = listOfProduct.stream().flatMap(p -> p.getListOfProducts().stream())  
    .collect(Collectors.toList());  
System.out.println(collect);
```

```
}
```

```
}
```

Working with IntStream, LongStream and DoubleStream :

IntStream flatMapToInt(Function<? super T, ? extends IntStream> mapper)

It is a predefined method of Stream interface which comes under flattening.

It allows us to transform each element of the stream into an IntStream (a stream of primitive int values) and then flattens these resulting streams into a single IntStream.

Note : IntStream is a specialized stream for working with int values available in java.util.stream sub package.

package com.ravi.basic.flat_map;

```
import java.util.Arrays;  
import java.util.List;  
import java.util.stream.IntStream;
```

```
public class FlatMapToIntDemo1 {
```

```
    public static void main(String[] args)  
    {  
        int []a1 = new int[] {1,2,3};  
        int []a2 = new int[] {4,5,6};  
        int []a3 = new int[] {7,8,9};
```

```
        List<int[]> intArrays = Arrays.asList(a1,a2,a3);
```

```
        IntStream stream = intArrays.stream().flatMapToInt(array -> IntStream.of(array));
```

```
        stream.forEach(System.out::print);
```

```
    }
```

```
}
```

LongStream flatMapToLong(Function<? super T, ? extends LongStream> mapper) :

It is a predefined method of Stream interface which comes under flattening.

It allows us to transform each element of the stream into an LongStream (a stream of primitive long values) and then flattens these resulting streams into a single LongStream.

Note : LongStream is a specialized stream for working with long values available in java.util.stream sub package.

```
package com.ravi.basic.flat_map;
```

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.LongStream;
```

```
public class FlatMapToLongDemo1 {
```

```
    public static void main(String[] args)
    {
        long []arr1 = new long[] {23,33,43};
        long []arr2 = new long[] {53,63,73};
        long []arr3 = new long[] {83,93,103};
```

```
        List<long[]> longArray = Arrays.asList(arr1, arr2,arr3);
        LongStream flatMapToLong = longArray.stream().flatMapToLong(array -> LongStream.of(array));
```

```
        flatMapToLong.forEach(System.out::println);
    }
```

```
}
```

```
LongStream flatMapToDouble(Function<? super T, ? extends DoubleStream> mapper)
```

It is a predefined method of Stream interface which comes under flattening.

It allows us to transform each element of the stream into an DoubleStream (a stream of primitive double values) and then flattens these resulting streams into a single DoubleStream.

Note : DoubleStream is a specialized stream for working with double values available in java.util.stream sub package.

```
package com.ravi.basic.flat_map;
```

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.DoubleStream;
```

```
public class FlatMapToDoubleDemo1
```

```
{
    public static void main(String[] args)
    {
        double d1[] = new double[]{1.1, 1.2, 1.3};
        double d2[] = new double[]{2.1, 2.2, 2.3};
        double d3[] = new double[]{3.1, 3.2, 3.3};
```

```
        List<double[]> listOfDoubleArrays = Arrays.asList(d1,d2,d3);
```

```
        DoubleStream doubleStream = listOfDoubleArrays.stream()
```

```

        .flatMapToDouble(array -> DoubleStream.of(array));

        // Print each double value in the flattened stream
        doubleStream.forEach(System.out::println);
    }
}

```

****Difference between map() and flatMap()**

map() method transforms each element into another single element.

flatMap() transforms each element into a stream of elements and then flattens those streams into a single stream.

We should use map() when you want a one-to-one transformation, and we should use flatMap() when dealing with nested structures or when you need to produce multiple output elements for each input element.

 03-08-2024:

We have already completed flatMapToInt(), flatMapToLong() and flatMapToDouble(), In the same way we can use mapToInt(), mapToFloat() and mapToDouble().

Program to mapToInt()

```

package com.ravi.stream_demo;

import java.util.List;
import java.util.stream.IntStream;

public class StreapMapDemo {

    public static void main(String[] args)
    {
        //Filter all the name with character 'R' and print the length of filtered name (mapToInt())

        List<String> listOfName = List.of("Ravi","Rajen","Ram","Suraj","Scott");

        IntStream mapToInt = listOfName.stream().filter(str -> str.startsWith("R")).mapToInt(name ->
        name.length());
        mapToInt.forEach(System.out::println);

    }

}

```

 public Stream sorted() :

It is a predefined method of Stream interface.

It provides default natural sorting order.

The return type of this method is Stream.

It has an overloaded method which accept Comparator<T> as a parameter through which we can provide user-defined sorting logic

```
-----  
package com.ravi.basic;
```

```
import java.util.List;  
import java.util.stream.Stream;
```

```
public class StreamDemo10  
{  
    public static void main(String[] args)  
    {  
        //Print the numbers in ascending order  
        List<Integer> listOfNum = List.of(89,67,56,45,23,15);  
        listOfNum.stream().sorted(Integer::compareTo).forEach(System.out::println);  
        System.out.println("=====");  
  
        //Print the numbers in descending order  
        List<Integer> listOfNumber = List.of(89,67,56,45,23,15);  
        listOfNumber.stream().sorted((i1,i2)-> i2.compareTo(i1)).forEach(System.out::println);  
        System.out.println("=====");  
  
        //Print the names in Ascending order  
        Stream<String> strOfName = Stream.of("Ankit","Scott","Smith","James");  
        strOfName.sorted(String::compareTo).forEach(System.out::println);  
  
        System.out.println("=====");  
  
        //Print the names in Descending order  
        Stream<String> strmOfName = Stream.of("Ankit","Scott","Smith","James");  
        strmOfName.sorted((s1,s2)-> s2.compareTo(s1)).forEach(System.out::println);  
  
    }  
}
```

```
-----  
public Stream distinct() :
```

```
-----  
It is a predefined method of Stream interface.
```

If we want to return stream from another stream by removing all the duplicates then we should use distinct() method.

```
package com.ravi.basic;  
import java.util.stream.Stream;  
public class StreamDemo11  
{  
    public static void main(String[] args)  
    {  
        Stream<Integer> numbers = Stream.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 3, 4, 5);  
  
        numbers.distinct().forEach(System.out::println);  
  
    }  
}
```

```
public Stream<T> limit(long maxSize) :
```

It is a predefined method of Stream interface to work with sequence of elements.

The limit() method is used to limit the number of elements in a stream by providing maximum size.

It creates a new Stream by taking the data from original Stream.

Elements which are not in the range or beyond the range of specified limit will be ignored.

```
public Stream<T> skip(long n) :
```

It is a predefined method of Stream interface which is used to skip the elements from beginning of the Stream.

It returns a new stream that contains the remaining elements after skipping the specified number of elements which is passed as a parameter.

```
-----  
package com.ravi.basic;  
import java.util.stream.Stream;  
public class StreamDemo12  
{  
    public static void main(String[] args)  
    {  
        Stream<String> s = Stream.of("Virat", "Rohit", "Dhoni", "Zaheer", "Raina", "Sahwag", "Sachin", "Bumrah");  
        s.skip(2).limit(5).forEach(System.out::println);  
    }  
}
```

```
-----  
public Stream<T> peek(Consumer<? super T> action) :
```

It is a predefined method of Stream interface which is used to perform a side-effect operation on each element in the stream while the stream remains unchanged.

It is an intermediate operation that allows us to perform operation on each element of Stream without modifying original.

The peek() method takes a Consumer as an argument, and this function is applied to each element in the stream. The method returns a new stream with the same elements as the original stream.

```
-----  
package com.ravi.basic;  
  
import java.util.List;  
import java.util.stream.Collectors;  
import java.util.stream.Stream;  
  
public class StreamDemo13  
{  
    public static void main(String[] args)  
    {  
        Stream<String> listOfFruits = Stream.of("Apple", "Mango", "Grapes", "Kiwi", "pomogranate");  
  
        List<Integer> doubledNumbers = listOfFruits  
            .peek(str -> System.out.println("Peeking from Original: " + str.toUpperCase()))  
            .map(fruit -> fruit.length())
```

```

        .collect(Collectors.toList());
System.out.println("-----");
System.out.println(doubledNumbers);

}

}

```

Note :- peek(Consumer<T> cons) will not modify the Original Source.

```

-----
public Stream<T> takeWhile(Predicate<T> predicate) :
-----

```

It is a predefined method of Stream interface introduced from java 9 which is used to perform a side-effect operation on each element in the stream while the stream remains unchanged.

*It is used to create a new stream that includes elements from the original stream only as long as they satisfy a given predicate.

```

package com.ravi.basic;

import java.util.stream.Stream;

public class StreamDemo14
{
    public static void main(String[] args)
    {
        Stream<Integer> numbers = Stream.of(10,11,9,13,2,1,100);

        numbers.takeWhile(n -> n > 9).forEach(System.out::println);

        System.out.println(".....");

        numbers = Stream.of(12,2,10,3,4,5,6,7,8,9);

        numbers.takeWhile(n -> n%2==0).forEach(System.out::println);

        System.out.println(".....");

        numbers = Stream.of(1,2,3,4,5,6,7,8,9);

        numbers.takeWhile(n -> n < 9).forEach(System.out::println);

        System.out.println(".....");

        numbers = Stream.of(11,2,3,4,5,6,7,8,9);

        numbers.takeWhile(n -> n > 9).forEach(System.out::println);

        System.out.println(".....");

        Stream<String> stream = Stream.of("Ravi", "Ankit", "Rohan", "Aman", "Ravish");

        stream.takeWhile(str -> str.charAt(0)=='R').forEach(System.out::println);
    }
}

```

```

}
}
-----
public Stream<T> dropWhile(Predicate<T> predicate) :
-----

```

It is a predefined method of Stream interface introduced from java 9 which is used to create a new stream by excluding elements from the original stream as long as they satisfy a given predicate.

```

package com.ravi.basic;

import java.util.stream.Stream;

public class StreamDemo15 {

    public static void main(String[] args)
    {
        Stream<Integer> numbers = Stream.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

        numbers.dropWhile(num -> num < 5).forEach(System.out::println);

        System.out.println(".....");

        numbers = Stream.of(15, 8, 7, 9, 5, 6, 7, 8, 9, 10);

        numbers.dropWhile(num -> num > 5).forEach(System.out::println);

    }

}

```

```

=====
Optional<T> class :
-----

```

```

Optional<T> class in Java :
-----

```

It is a predefined final and immutable class available in java.util package from java 1.8v.

It is a container object which is used to represent an object (Optional object) that may or may not contain a non-null value.

If the value is available in the container, isPresent() method will return true and get() method will return the actual value.

It is very useful in industry to avoid NullPointerException.

```

Methods of Optional<T> class :
-----

```

```

1) public static Optional<T> ofNullable(T x) :
-----

```

It will return the object of Optional class with specified value. If the specified value is null then this method will return an empty object of the optional class.

2) public boolean isPresent() :

It will return true, if the value is available in the container otherwise it will return false.

3) public T get() :

It will get/fetch the value from the container, if the value is not available then it will throw java.util.NoSuchElementException.

4) public T orElse(T defaultValue) :

It will return the value, if available otherwise it will return the specified default value.

5) public static Optional<T> of (T value) :

It will return the optional object with the specified value that is non- null value.

6) public static Optional<T> empty() :

It will return an empty Optional Object.

7) public java.util.stream.Stream stream()

It will Convert optional to Stream.

//Program to verify whether the container has value or not
package com.ravi.optional_class_demo;

import java.util.Optional;

```
public class OptionalDemo1
{
    public static void main(String[] args)
    {
        String str = null;
```

```
        Optional<String> optional = Optional.ofNullable(str);
```

```
        String orElse = optional.orElse("No value in container");
        System.out.println("Value by orElse :"+orElse);
```

```

        //Optional is containing value or not?
        if(optional.isPresent())
        {
            System.out.println("Value by get :"+optional.get());
        }
        else
        {
            System.err.println("No value is available in the container");
        }
    }
}
```

```
}
```

```
}
```

```
-----  
//Writing different style of getter
```

```
package com.ravi.optional_class_demo;
```

```
import java.util.Optional;
```

```
class Employee
```

```
{
```

```
    private Integer empId;
```

```
    private String empName;
```

```
    public Employee() {}
```

```
    public Employee(Integer empId, String empName)
```

```
    {
```

```
        super();
```

```
        this.empId = empId;
```

```
        this.empName = empName;
```

```
    }
```

```
    //Changing the style of writing getter method
```

```
    public Optional<Integer> getEmpId()
```

```
    {
```

```
        return Optional.ofNullable(empId);
```

```
    }
```

```
    public Optional<String> getEmpName()
```

```
    {
```

```
        return Optional.ofNullable(empName);
```

```
    }
```

```
}
```

```
public class OptionalDemo2
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        //Employee emp = new Employee(111,"Ravi");
```

```
        Employee emp = new Employee();
```

```
        Optional<Integer> empId = emp.getEmpId();
```

```
        if(empId.isPresent())
```

```
        {
```

```
            System.out.println(empId.get());
```

```
        }
```

```
        else
```

```
        {
```

```
            System.err.println("No id value ");
```

```
        }
```

```
        Optional<String> empName = emp.getEmpName();
```

```
        String name = empName.orElse("name is not available");
```

```
        System.out.println(name);
```

```
}  
}
```

06-08-2024

Optional.empty()

```
//Program to verify value is available or not  
package com.ravi.optional_class_demo;
```

```
import java.util.ArrayList;  
import java.util.List;  
import java.util.Optional;
```

```
public class OptionalDemo3  
{  
    public static void main(String[] args)  
    {  
        List<Optional<String>> optionalList = new ArrayList<>();  
        optionalList.add(Optional.of("Hyderabad"));  
        optionalList.add(Optional.of("Delhi"));  
        optionalList.add(Optional.of("Mumbai"));  
        optionalList.add(Optional.of("Kolkata"));  
        optionalList.add(Optional.empty());  
  
        for (Optional<String> optional : optionalList)  
        {  
            if (optional.isPresent())  
            {  
                System.out.println(optional.get());  
            }  
            else  
            {  
                System.out.println("No data is available");  
            }  
        }  
    }  
}
```

Optional is final and immutable class.
[ALL IMMUTABLE OBJECTS ARE THREAD SAFE]

The following program explains it is an immutable object.

//Immutability of Optional class

```
package com.ravi.optional_class_demo;
```

```
import java.util.Optional;  
public class OptionalDemo4  
{  
    public static void main(String[] args)  
    {  
        Optional<String> optl = Optional.of("India");  
        System.out.println(optl.hashCode());  
    }  
}
```

```

Optional<String> newOptnl = modifyOptional(optl);
System.out.println(newOptnl.hashCode());

// Check if the original Optional is still the same
System.out.println("Address is :" + (optl == newOptnl));

}

public static Optional<String> modifyOptional(Optional<String> optional)
{
    if (optional.isPresent())
    {
        return Optional.of("Modified: " + optional.get());
    }
    else
    {
        return Optional.empty();
    }
}
}

```

Note : From the above program it is clear that Optional class is an immutable class.

New Date and Time API :

New Date and Time API :

LocalDate :

It is a predefined final class which represents only Date. The java.util.Date class is providing Date and Time both so, only to get the Date we need to use LocalDate class available in java.time package.

```

    LocalDate d = LocalDate.now();

```

Here now is a static method of LocalDate class and its return type is LocalDate class. (Factory Method)

LocalTime :

It is also a final class which will provide only time.

```

    LocalTime d = LocalTime.now();

```

Here now is a static method of LocalTime class and its return type is LocalTime (Factory Method).

LocalDateTime :

It is also a final class which will provide Date and Time both without a time zone. It is a combination of LocalDate and LocalTime class.

```

    LocalDateTime d = LocalDateTime.now();

```

package com.ravi.new_date_time;

```

import java.time.LocalDate;

```

```
import java.time.LocalDateTime;
import java.time.LocalDateTime;
import java.time.ZoneId;
import java.time.ZonedDateTime;
```

```
public class Demo1
{
    public static void main(String[] args)
    {
        LocalDate d = LocalDate.now();
        System.out.println(d);

        LocalDateTime t = LocalDateTime.now();
        System.out.println(t);

        LocalDateTime dt = LocalDateTime.now();
        System.out.println(dt);
    }
}
```

Now, based on our application requirement we can take only date OR only time or Date and time both

ZonedDateTime : (Date and time + Time zone)

It is a final class available in java.time package.

It also provides date and time along with time zone so, by using this class we can work with different time zone in a global way.

```
ZonedDateTime x = ZonedDateTime.now();
ZoneId zone = x.getZone();
```

getZone() is a predefined non static method of ZonedDateTime class which returns ZoneId class, this ZoneId class provides the different zones, by using getAvailableZoneIds() static method we can find out the total zone available using this ZoneId class.

package com.ravi.new_date_time;

```
import java.time.ZoneId;
import java.time.ZonedDateTime;
```

```
public class Demo2
{
    public static void main(String[] args)
    {
        ZonedDateTime z = ZonedDateTime.now();
        System.out.println(z);

        ZoneId zone = z.getZone();
        System.out.println(zone);

        System.out.println(ZoneId.getAvailableZoneIds());
    }
}
```

```

}

}
-----

```

Different of() static method :

```

-----
List.of();
Set.of();
Map.of();
Stream.of();
Optional.of();
ZoneId.of();
-----

```

```

package com.ravi.new_date_time;

```

```

import java.time.ZoneId;
import java.time.ZonedDateTime;

```

```

public class Demo3
{
    public static void main(String[] args)
    {
        ZoneId AusTimeZone = ZoneId.of("Australia/Hobart");
        ZonedDateTime aus = ZonedDateTime.now(AusTimeZone);
        System.out.println("Current Date and Time in Australia Time Zone: " + aus);

        ZoneId londonTimeZone = ZoneId.of("Europe/London");
        ZonedDateTime lon = ZonedDateTime.now(londonTimeZone);
        System.out.println("Current Date and Time in London Time Zone: " + lon);

    }
}

```

Note :- of(String zoneId) is a static method of ZoneId abstract class it will provide the zoneId of specified zone as a parameter.

Now with ZonedDateTime we can find out the specified zoned time.

```

-----
DateTimeFormatter :
-----

```

It is a predefined final class available in java.time.format sub package.

It is mainly used to formatting and parsing date and time objects according to Java Date and Time API.

Method :

```

-----
public static DateTimeFormatter ofPattern(String pattern) :

```

It is a static method of DateTimeFormatter class, It creates a formatter using user specified String pattern ("dd-MM-yyyy HH:mm:ss") and LocalDateTime class contains format method which takes DateTimeFormatter as a parameter and returns the String value.

```

package com.ravi.new_date_time;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class Demo4
{
    public static void main(String[] args)
    {
        LocalDateTime now = LocalDateTime.now();
        System.out.println(now);
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-YYYY");

        String formattedDateTime = now.format(formatter);
        System.out.println("Formatted DateTime: " + formattedDateTime);
    }
}

```

Terminal Operation in Stream in java:

In Java's Stream API, a terminal operation is an operation that produces a result or a side-effect operation.

Unlike intermediate operations, which return a new stream, terminal operation consumes the elements of the stream.

Once a terminal operation is applied to a stream, the stream is considered consumed and cannot be reused.

It is a final operation.

Methods of Terminal Operation :

-
- 1) public long count()
 - 2) public Optional<T> min(Comparator<? super T> comparator)
 - 3) public Optional<T> max(Comparator<? super T> comparator)
 - 4) public Optional<T> findAny()
 - 5) public Optional<T> findFirst()
 - 6) public boolean allMatch(Predicate<? super T> predicate)
 - 7) public boolean anyMatch(Predicate<? super T> predicate)
 - 8) public boolean noneMatch(Predicate<? super T> predicate)
 - 9) public void forEach(Consumer<T> cons)
 - 10) public R reduce(BinaryOperator<Integer> accumulator)
 - 11) public Optional<T> collect(Collector<? super Integer,A,R> collect)

public long count() :

The count operation returns the number of elements available in the stream. It is a terminal operation that terminates the stream after its execution.

Basically it used to count the number of elements after filter() method.

package com.ravi.advanced;

import java.util.stream.Stream;

```
public class CountDemo1
{
    public static void main(String[] args)
    {
        long count = Stream.of("Ravi","Raj","Elina","Aryan").count();
        System.out.println(count);
    }
}
```

package com.ravi.advanced;

//Count the name whose length is greater than 3

import java.util.List;

```
public class CountDemo2
{
    public static void main(String[] args) {
        List<String> listOfName = List.of("Raj","Ravi","Virat","Rohit","Ram","Bumrah","Sachin");

        long names = listOfName.stream().filter(name -> name.length()>3).count();
        System.out.println("Names whose length is > 3 are :"+names);
    }
}
```

package com.ravi.advanced;

//Count Unique elements by using Stream API

import java.util.List;

```
public class CountDemo3
{
    public static void main(String[] args) {
        List<String> listOfName = List.of("Raj","Ravi","Virat","Raj");

        long count = listOfName.stream()
                                .distinct()
                                .count();

        System.out.println("Count of unique elements: " + count);
    }
}
```



```

-----
package com.ravi.advanced;

//Count the names which are containing the character A
import java.util.Arrays;
import java.util.List;

public class CountDemo4
{
    public static void main(String[] args) {
        List<String> list = Arrays.asList("Raj","Ravi","Rohit","Virat","Raj");

        long count = list.stream()
            .map(String::toUpperCase)
            .filter(s -> s.contains("A"))
            .distinct()
            .count();

        System.out.println("Count of distinct strings containing 'A': " + count);
    }
}
-----

```

07-08-2024

Working with predefined functional interfaces :

Working with Predefined functional interfaces :

UnaryOperator<T> functional interface :

It is a predefined functional interface available in java.util.function sub package.

It is a functional interface in Java that represents an operation on a single operand that produces a result of the same type as its operand. This is a specialization of Function for the case where the operand and result are of the same type.

It has a single type parameter, T, which represents both the operand type and the result type.

```

@FunctionalInterface
public interface UnaryOperator<T> extends Function<T,T>
{
    public abstract T apply(T x);
}
-----

```

```

package com.ravi.functional_interface_demo;

```

```

import java.util.function.UnaryOperator;

```

```

public class UnaryOpeartorDemo
{
    public static void main(String[] args)
    {
        UnaryOperator<Integer> square = x -> x * x;
        System.out.println(square.apply(5));
    }
}

```

```
UnaryOperator<String> concat = str -> str.concat("base");
System.out.println(concat.apply("Data"));
```

```
}
```

```
}
```

BinaryOperator<T> Functional interface :

It is a predefined functional interface available in java.util.function sub package.

It is a functional interface in Java that represents an operation upon two operands of the same type, producing a result of the same type as the operands.

This is a specialization of BiFunction for the case where the operands and the result are all of the same type.

It has two parameters of same type, T, which represents both the operand types and the result type.

```
@FunctionalInterface
public interface BinaryOperator<T> extends BiFunction<T,U,R>
{
    public abstract T apply(T x, T y);
}
```

package com.ravi.functional_interface_demo;

```
import java.util.function.BinaryOperator;
```

```
public class BinaryOperatorDemo
{
    public static void main(String[] args)
    {
        BinaryOperator<Integer> add = (a, b) -> a + b;
        System.out.println(add.apply(3, 5));
    }
}
```

ToIntFunction<T> functional interface :

It is a predefined functional interface available in java.util.function sub package.

It is a functional interface in Java that represents a function that takes an argument of type T and returns an int. This is typically used in streams when you need to perform operations that result in primitive int value.

```
@FunctionalInterface
public interface ToIntFunction<T>
{
    int applyAsInt(T value);
}
```

package com.ravi.functional_interface_demo;

```
import java.util.ArrayList;
```

```

import java.util.function.ToIntFunction;

record Employee(String name, int experience)
{
}

public class ToIntFunctionDemo
{
    public static void main(String[] args)
    {
        ArrayList<Employee> listOfEmployee = new ArrayList<>();
        listOfEmployee.add(new Employee("Virat",12));
        listOfEmployee.add(new Employee("Rohit",12));
        listOfEmployee.add(new Employee("Bumrah",6));
        listOfEmployee.add(new Employee("Akshar",5));

        ToIntFunction<Employee> experienceFunction = employee -> employee.experience();

        int totalYearsOfExperience = listOfEmployee.stream()
            .mapToInt(experienceFunction)
            .sum();

        System.out.println("Total years of experience: " + totalYearsOfExperience);

    }
}

```

```

public Optional<T> min(Comparator<? super T> comparator)

```

It is a predefined method of Stream interface ,It is used to find the minimum element of the stream according to the provided Comparator.

This method is useful when we need to find out the smallest element in a stream, based on a specific comparison criteria using Comparator.

```

package com.ravi.advanced.min_demo;

import java.util.Arrays;
import java.util.List;
import java.util.Optional;

public class MinDemo1
{
    public static void main(String[] args)
    {
        List<Integer> listOfNumbers = Arrays.asList(10, 20, 5, 40, 25, 2);

        Optional<Integer> min = listOfNumbers.stream().min(Integer::compareTo);

        min.ifPresent(System.out::println);

    }
}

```

```
}
```

Note : Optional class has provided a predefined method `ifPresent(Consumer<T> cons)` as a parameter which will get the value from the Optional container.

```
package com.ravi.advanced.min_demo;
```

```
import java.util.Comparator;
import java.util.Optional;
import java.util.stream.Stream;
```

```
//Finding the minimum age of Employee
```

```
record Employee(Integer age, String name)
{

}
```

```
public class MinDemo2
```

```
{
    public static void main(String[] args)
    {
        Employee e1 = new Employee(23, "Scott");
        Employee e2 = new Employee(19, "Smith");
        Employee e3 = new Employee(12, "John");
```

```
        Stream<Employee> streamOfEmployee = Stream.of(e1,e2,e3);
        Optional<Employee> min = streamOfEmployee.min(Comparator.comparingInt(Employee::age));
```

```
        if(min.isPresent())
        {
            System.out.println("Employee with minimum Age is :"+min.get());
        }
        else
        {
            System.out.println("No Value is available");
        }
    }
}
```

```
}
```

Note : Comparator class contains a static method `comparingInt(ToIntFunction<T>)` as a parameter. This `ToIntFunction` functional interface will return int primitive value.

```
package com.ravi.advanced.min_demo;
```

```
import java.util.Comparator;
import java.util.List;
import java.util.Optional;
```

```
//Finding the Cheapest Product
```

```
record Product(Integer productId, String productName, Double productPrice)
```

```
{  
}
```

```
public class MinDemo3 {  
  
    public static void main(String[] args)  
    {  
        List<Product> listOfProduct = List.of(new Product(111, "Camera", 45000D), new Product(222, "Watch",  
23000D));  
  
        Optional<Product> min =  
listOfProduct.stream().min(Comparator.comparingDouble(Product::productPrice));  
  
        if (min.isPresent())  
        {  
            System.out.println("The cheapest product is: " + min.get());  
        }  
        else  
        {  
            System.out.println("No products found.");  
        }  
  
    }  
}
```

```
-----  
public Optional<T> max(Comparator<? super T> comparator)  
-----
```

It is a predefined method of Stream interface ,It is used to find the maximum element of the stream according to the provided Comparator.

This method is useful when we need to find out the largest element in a stream, based on a specific comparison criteria using Comparator.

```
-----  
package com.ravi.advanced.max_demo;
```

```
import java.util.Comparator;  
import java.util.List;  
import java.util.Optional;
```

```
public class MaxDemo1 {  
  
    public static void main(String[] args)  
    {  
        List<String> listOfFruits = List.of("Apple","Orange","Mango","Grapes","Pomogranate");  
  
        Optional<String> max = listOfFruits.stream().max(Comparator.comparingInt(String::length));  
  
        max.ifPresent(System.out::println);  
  
    }  
}
```

```
-----  
package com.ravi.advanced.max_demo;
```

```
import java.util.Comparator;  
import java.util.Optional;  
import java.util.stream.Stream;
```

```
//Finding the Employee with the Highest Salary
```

```
record Employee(Integer employeeId, String employeeName, Double employeeSalary)  
{  
}
```

```
public class MaxDemo2  
{  
    public static void main(String[] args)  
    {  
        Employee e1 = new Employee(111, "Aman", 23000D);  
        Employee e2 = new Employee(222, "Ramesh", 24000D);  
        Employee e3 = new Employee(333, "Suraj", 25000D);  
        Employee e4 = new Employee(444, "Raj", 26000D);  
        Employee e5 = new Employee(555, "Scott", 46000D);
```

```
        Stream<Employee> streamOfEmployees = Stream.of(e1,e2,e3,e4,e5);
```

```
        Optional<Employee> max =  
        streamOfEmployees.max(Comparator.comparingDouble(Employee::employeeSalary));
```

```
        if(max.isPresent())  
        {  
            System.out.println("Employee Having Maximum Salary is :"+max.get());  
        }  
        else  
        {  
            System.out.println("No record Available");  
        }  
    }  
}
```

```
}
```

```
-----  
public Optional<T> findAny() :
```

It is a predefined method of Stream interface ,It is used to return an Optional which describes some element of the stream, or an empty Optional if the stream is empty.

It's useful when we need any element from the stream but don't care which one.

Note : On Stream interface we can call two methods

- 1) public Stream stream() [It is 4 sequential Execution]
- 2) public Stream parallelStream() [It is for Multithreading]

```
package com.ravi.advanced.find_any;
```

```
import java.util.List;
```

```
import java.util.Optional;

public class FindAnyDemo1
{
    public static void main(String[] args)
    {
        List<String> listOfFruits = List.of("Raj", "Rahul", "Ankit");

        Optional<String> findAny = listOfFruits.parallelStream().findAny();

        findAny.ifPresent(System.out::println);

    }
}
```

```
-----
package com.ravi.advanced.find_any;

import java.util.List;
import java.util.Optional;

public class FindAnyDemo2 {

    public static void main(String[] args)
    {
        List<String> listOfName = List.of("Sachin", "Ankit", "Aman", "Rahul", "Ravi");

        Optional<String> anyElement = listOfName.parallelStream().filter(s -> s.startsWith("R")).findAny();

        anyElement.ifPresent(System.out::println);

    }
}
```

```
-----
public Optional<T> findFirst()
-----
```

It is a predefined method of Stream interface ,It is used to return an Optional which describes the first element of the stream, or an empty Optional if the stream is empty.

```
package com.ravi.advanced.find_first;

import java.util.stream.Stream;

public class FindFirstDemo1
{
    public static void main(String[] args)
    {
        Stream<String> fruitsStream = Stream.of("Virat", "Rohit", "Raj", "Bumrah", "Arshdeep");

        fruitsStream.findFirst().ifPresent(System.out::println);

    }
}
```

```
-----
Differences between findAny() and findFirst()
```

findFirst(): Always returns the first element of the stream, which is particularly useful for ordered streams.

findAny(): Returns any element from the stream, and is typically used in unordered streams where the order is not required. It may return elements faster because it does not have to maintain the order.

Note : Collection interface has provided parallelStream() method for fast execution [It uses multiple threads]

public boolean allMatch(Predicate<? super T> predicate)

It is a predefined method of Stream interface , It is used to check if all elements of the stream match a given predicate. This method is useful when we need to verify that every element in a stream satisfies a specific condition by using Predicate.

```
package com.ravi.advanced.match;
```

```
import java.util.Arrays;  
import java.util.List;  
import java.util.function.Predicate;  
import java.util.stream.Stream;
```

```
public class AllMatchDemo1 {
```

```
    public static void main(String[] args)  
    {  
        Stream<Integer> stream = Stream.of(1, 2, 3, 4, 5);  
        boolean allPositive = stream.allMatch(n -> n > 0);  
        System.out.println("All elements are positive: " + allPositive);  
  
        System.out.println(".....");  
  
        List<Integer> numbers = Arrays.asList(2, 4, 6, 8, 10);  
  
        Predicate<Integer> isEven = number -> number % 2 == 0;  
  
        boolean allEven = numbers.stream().allMatch(isEven);  
  
        if (allEven)  
        {  
            System.out.println("All numbers are even.");  
        }  
        else  
        {  
            System.out.println("Not all numbers are even.");  
        }  
    }  
}
```

public boolean anyMatch(Predicate<? super T> predicate)

It is a predefined method of Stream interface, It returns a boolean indicating whether any elements of the stream match the given predicate. It is useful when we need to verify if there exists at least one element

in the stream that satisfies the provided condition.

```
package com.ravi.advanced.match;

import java.util.Arrays;
import java.util.List;
import java.util.function.Predicate;

public class AnyMatchDemo1
{
    public static void main(String[] args)
    {
        List<String> listOfName = List.of("Virat","Rohit","Bumrah","Surya");

        boolean startsWithA = listOfName.stream().anyMatch(name -> name.startsWith("A"));

        System.out.println("Any name starts with letter 'A' : " + startsWithA);

        System.out.println("=====");

        List<Integer> numbers = Arrays.asList(1, 3, 5, 7, 8);

        Predicate<Integer> isEven = number -> number % 2 == 0;

        boolean anyEven = numbers.stream().anyMatch(isEven);

        if (anyEven)
        {
            System.out.println("There is at least one even number.");
        }
        else
        {
            System.out.println("There are no even numbers.");
        }
    }
}
```

```
-----
public boolean noneMatch(Predicate<? super T> predicate)
-----
```

It is a predefined method of Stream interface, It is used to check if no elements of the stream match a given predicate. It returns a boolean value true if no elements match the predicate, and false if at least one element matches the predicate or if the stream is empty.

```
package com.ravi.advanced.match;

import java.util.Arrays;
import java.util.List;
import java.util.function.Predicate;

public class NoneMatchDemo1
{
    public static void main(String[] args)
    {
        List<Integer> numbers = Arrays.asList(1, 3, 5, 7, 9, 2, 8);
```

```
Predicate<Integer> isEven = number -> number % 2 == 0;
```

```
boolean noneEven = numbers.stream().noneMatch(isEven);
```

```
if (noneEven)
{
    System.out.println("There are no even numbers.");
}
else
{
    System.out.println("There is at least one even number.");
}
}
}
```

```
-----
public void forEach(Consumer<T> cons) :
-----
```

It is a predefined method of Stream interface, the forEach operation allows us to perform an action on each element of a stream. It takes a Consumer as a parameter and executes it for each element of the stream.

```
package com.ravi.functional_interface_demo;
```

```
import java.util.stream.Stream;
```

```
public class ForEachDemo1
{

    public static void main(String[] args)
    {
        Stream.of(1,2,3,4,5,6,7).forEach(System.out::println);

    }

}
```

```
-----
08-08-2024
-----
```

```
R collect(Collector<? super T, A, R> collector);
-----
```

It is a predefined method of Stream interface. It is used to transform the elements of a stream into a different form like collection i.e. List, Set, or Map.

The collect() method takes an argument of type Collector, which is a functional interface that specifies how to perform the collection operation. The Collectors class has provided following static methods

- a) toList(): Collects the elements of the stream into a List.
 - b) toSet(): Collects the elements of the stream into a Set.
 - c) toMap(): Collects the elements of the stream into a Map.
 - d) joining(): Concatenates the elements of the stream into a String.
 - e) groupingBy(): Groups the elements of the stream by a classifier function.
 - f) partitioningBy(): Partitions the elements of the stream into two groups based on a predicate.
- ```

```

```
package com.ravi.advanced.collect;
```

```
//Join the elements by using joining() methods of Collectors class
```

```
import java.util.Arrays;
```

```
import java.util.List;
```

```
import java.util.stream.Collectors;
```

```
public class CollectDemo1
```

```
{
 public static void main(String[] args)
 {
 List<String> list = Arrays.asList("a", "b", "c", "d");
 String result = list.stream().collect(Collectors.joining("@"));
 System.out.println(result);
 }
}
```

```

** Map<K, List<T>> Collectors.groupingBy(Function<T,R> classifier) :
```

```

It is a predefined static method of Collectors class.
```

It is used to convert the stream into Map. Here Map keys are the function which are passing as a parameter to groupingBy() method and the value of Map is list of items.

```
package com.ravi.advanced.collect;
```

```
//Group the city name according to length of the city name
```

```
import java.util.*;
```

```
import java.util.stream.Collectors;
```

```
public class CollectDemo2
```

```
{
 public static void main(String[] args)
 {
 List<String> items = Arrays.asList("Delhi", "Indore", "Kolkata", "Pune",
"Hyderabad","Mumbai","Chennai");
```

```
 Map<Integer, List<String>> groupedByLength =
items.stream().collect(Collectors.groupingBy(String::length));
```

```
 groupedByLength.forEach((length, cities) ->
 {
 System.out.println("Length " + length + ": " + cities);
 });
 }
}
```

```

package com.ravi.advanced.collect;
```

```
//Print the length of the country
```

```
import java.util.*;
```

```
import java.util.stream.Collectors;
```

```
public class CollectDemo3
```

```

{
 public static void main(String[] args)
 {
 List<String> listOfCountry = List.of("India","Australia","USA","China","Japan");

 Map<String, Integer> map = listOfCountry.stream()
 .collect(Collectors.toMap(
 item -> item,
 item -> item.length()
));

 map.forEach((key, value) ->
 {
 System.out.println(key + ": " + value);
 });
 }
}

```

---

```

package com.ravi.advanced.collect;

```

```

//Based on the department, group all the employee

```

```

import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

```

```

record Employee(Integer empId, String empName, double salary, Department dept)
{

}

```

```

record Department(Integer deptId, String deptName)
{

}

```

```

public class CollectDemo4
{
 public static void main(String[] args)
 {
 Employee e1 = new Employee(111, "Raj", 23789.89, new Department(1, "IT"));
 Employee e2 = new Employee(222, "Rahul", 23789.89, new Department(1, "IT"));
 Employee e3 = new Employee(333, "Scott", 23789.89, new Department(2, "Sales"));
 Employee e4 = new Employee(444, "Smith", 23789.89, new Department(2, "Sales"));
 Employee e5 = new Employee(333, "Virat", 23789.89, new Department(3, "HR"));
 Employee e6 = new Employee(444, "Rohit", 23789.89, new Department(3, "HR"));
 }
}

```

```

List<Employee> list = List.of(e1,e2,e3,e4,e5,e6);

```

```

HashMap<Department,List<Employee>> map = new HashMap<>();
map.put(new Department(1, "IT"), list);
map.put(new Department(2, "Sales"), list);

```

```
map.put(new Department(3, "HR"), list);
```

```
Map<Department, List<Employee>> collect =
list.stream().collect(Collectors.groupingBy(Employee::dept));
```

```
collect.forEach((key, value) ->
{
 System.out.println(key + ": " + value);
});
}

}
```

```

Map<Boolean, List<T>> Collectors.partitioningBy(Predicate<T> p)

```

It is a predefined static method of Collectors class.

It is used to partition the elements of a stream into two groups based on a given predicate.

The result is a Map with Boolean keys, where the true key corresponds to elements that satisfy the predicate, and the false key corresponds to elements that do not satisfy the predicate.

```

package com.ravi.advanced.collect;
```

```
//Partition the given number based on the Predicate
```

```
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;
```

```
public class CollectDemo5
{
 public static void main(String[] args)
 {
 List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

 Map<Boolean, List<Integer>> partitioned = numbers.stream()
 .collect(Collectors.partitioningBy(n -> n % 2 == 0));

 System.out.println(partitioned);
 }
}
```

```

package com.ravi.advanced.collect;
```

```
//Partition the given number based on the Predicate and convert to Set
```

```
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.stream.Collectors;
```

```
public class CollectDemo6
{
 public static void main(String[] args)
```

```

{
 List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 1, 3);

 Map<Boolean, Set<Integer>> partitioned = numbers.stream()
 .collect(Collectors.partitioningBy(
 n -> n % 2 == 1,
 Collectors.toSet()));

 System.out.println(partitioned);

}
}

```

---

```

package com.ravi.advanced.collect;

```

```

//Count how many elements are partition based on given predicate

```

```

import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

```

```

public class CollectDemo7
{
 public static void main(String[] args) {
 List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11);

 Map<Boolean, Long> partitioned = numbers.stream()
 .collect(Collectors.partitioningBy
 (
 n -> n % 2 == 1,
 Collectors.counting()));

 System.out.println(partitioned);

 }
}

```

---

```

Optional<T> reduce(BinaryOperator<T> accumulator)

```

---

```

T reduce(T identity, BinaryOperator<T> accumulator);

```

---

It is a predefined method of Stream interface.

This method is useful for combining stream elements into a single result because it accept BinaryOperator<T> as a parameter, such as computing the sum, product, or concatenation of elements.

It performs a reduction on the elements of the stream, using an associative accumulation function, and returns an Optional.

---

```

package com.ravi.advanced.reduce;

```

```

//T reduce(T identity, BinaryOperator<T> accumulator);
//Finding the sum of all the numbers

```

```

import java.util.Optional;
import java.util.stream.Stream;

public class ReduceDemo1
{
 public static void main(String[] args)
 {
 Stream<Integer> integerStream = Stream.of(1, 2, 3, 4, 5);

 Optional<Integer> sum = integerStream.reduce(Integer::sum);

 System.out.println(sum.get());

 System.out.println(".....");

 integerStream = Stream.of(1, 2, 3, 4, 5);
 Integer sumWithIdentity = integerStream.reduce(0, Integer::sum);
 System.out.println(sumWithIdentity);

 }
}

```

The reduction operation takes two elements of same type and combines them into one using the provided accumulator.

```

package com.ravi.advanced.reduce;

//Finding the maximum number

import java.util.Arrays;
import java.util.List;
import java.util.Optional;

public class ReduceDemo2
{
 public static void main(String[] args) {
 List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

 Optional<Integer> max = numbers.stream()
 .reduce(Integer::max);

 max.ifPresent(System.out::println);
 }
}

```

```

package com.ravi.advanced.reduce;

//Finding the multiplication of all numbers

import java.util.Arrays;
import java.util.List;

```

```
import java.util.Optional;

public class ReduceDemo3
{
 public static void main(String[] args) {
 List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

 Optional<Integer> product = numbers.stream()
 .reduce((a, b) -> a * b);

 product.ifPresent(System.out::println);
 }
}
```

---

```
package com.ravi.advanced.reduce;
import java.util.Arrays;
import java.util.List;
import java.util.Optional;

public class ReduceDemo4
{
 public static void main(String[] args) {
 List<String> words = Arrays.asList("Java", "is", "Best");

 Optional<String> concatenated = words.stream()
 .reduce((a, b) -> a + " " + b);

 concatenated.ifPresent(System.out::println);
 }
}
```

---

```
package com.ravi.advanced.reduce;
```

```
//Finding the total sale amount
```

```
import java.util.stream.Stream;

record Sale(String item, Double amount)
{
}

}
```

```
public class ReduceDemo5
{
 public static void main(String[] args)
 {
 Stream<Sale> sales = Stream.of(
 new Sale("Camera", 10000.0),
 new Sale("Mobile", 50000.0),
 new Sale("Laptop", 80000.0),
 new Sale("LED", 20000.0)
);

 Double totalSale = sales.reduce(0.0, (sum, sale)-> sum + sale.amount(), Double::sum);
 }
}
```



```
System.out.println("Total Sale for today is :"+totalSale);
```

```
}
```

```
}
```

=====

Stream API Programs (84 Programs ):

-----

```
public class Employee {
 private String name;
 private int age;
 private double salary;
 private String gender;

 // Constructor
 public Employee(String name, int age, double salary, String gender) {
 this.name = name;
 this.age = age;
 this.salary = salary;
 this.gender = gender;
 }

 // Getters and setters for each variable
 public String getName() {
 return name;
 }

 public void setName(String name) {
 this.name = name;
 }

 public int getAge() {
 return age;
 }

 public void setAge(int age) {
 this.age = age;
 }

 public double getSalary() {
 return salary;
 }

 public void setSalary(double salary) {
 this.salary = salary;
 }

 public String getGender() {
 return gender;
 }

 public void setGender(String gender) {
 this.gender = gender;
 }
}
```

```

 }

 // Override toString() method for easy printing
 @Override
 public String toString() {
 return "Employee{" +
 "name=" + name + "\n" +
 ", age=" + age +
 ", salary=" + salary +
 ", gender=" + gender + "\n" +
 '}';
 }
}

public class EmployeeAdder {

 public static List<Employee> addDetails()
 {
 List<Employee> list = new ArrayList<>();
 Employee employee1 = new Employee("Anna", 27, 50000.0, "Male");
 Employee employee2 = new Employee("Employee 2", 27, 51000.0, "Female");
 Employee employee3 = new Employee("Bob", 27, 52000.0, "Male");
 Employee employee4 = new Employee("EmplSmithoyeeeee 4", 28, 53000.0, "Female");
 Employee employee5 = new Employee("Employee 5", 29, 53000.0, "Male");
 Employee employee6 = new Employee("Employee 6", 30, 55000.0, "Female");
 Employee employee7 = new Employee("EmSmithployee 7", 31, 56000.0, "Male");
 Employee employee8 = new Employee("Employee 8", 32, 57000.0, "Female");
 Employee employee9 = new Employee("Employee 9", 35, 58000.0, "Male");
 Employee employee10 = new Employee("EMPLOYEEeeseeee 10", 35, 59000.0, "Female");

 list.add(employee1);
 list.add(employee2);
 list.add(employee3);
 list.add(employee4);
 list.add(employee5);
 list.add(employee6);
 list.add(employee7);
 list.add(employee8);
 list.add(employee9);
 list.add(employee10);
 return list;
 }

}

public class Tester {
 private static final String RED = "\033[1;31m"; // RED
 private static final String RESET = "\033[0m"; // Text Reset
 public static void main(String[] args)
 {
 List<Employee> list = EmployeeAdder.addDetails();
 Set<Integer> set = new HashSet<>();
 Set<String> set2 = new HashSet<>();
 // 1. Filter Employees by Gender:

```

```

// - Retrieve a list of all female employees.
System.out.println(RED+"*****Retrieve a list of all female employees*****"+RESET);
list.stream()
 .filter(t -> t.getGender().equals("Female"))
 .forEach(System.out::println);
System.out.println(RED+"*****Get a list of employees older than 30 years.*****"+RESET);
// 2. Filter Employees by Age:
// - Get a list of employees older than 30 years.
list.stream()
 .filter(employee->employee.getAge()>30)
 .forEach(System.out::println);
System.out.println(RED+"*****Find employees with a salary greater than $50,000.*****"+RESET);
// 3. Filter Employees by Salary:
// - Find employees with a salary greater than $50,000.
list.stream()
 .filter(employee->employee.getSalary()>50000)
 .forEach(System.out::println);

// 4. Map Employee Names:
// - Create a list of employee names (Strings).
System.out.println(RED+"*****Create a list of employee names (Strings)*****"+RESET);
list.stream()
 .map(employee->employee.getName())
 .forEach(System.out::println);

// 5. Calculate Average Salary:
// - Calculate the average salary of all employees.
System.out.println(RED+"*****Calculate the average salary of all employees.*****"+RESET);
double orElseThrow = list.stream()
 .mapToDouble(Employee::getSalary)
 .average()
 .orElseThrow();
System.out.println("Average : "+orElseThrow);

// 6. Find Maximum Salary:
// - Find the employee with the highest salary.
System.out.println(RED+"*****Find the employee with the highest salary.*****"+RESET);
double max = list.stream()
 .mapToDouble(Employee::getSalary)
 .max()
 .orElseThrow();
System.out.println("Max Salary : "+max);

// 7. Group Employees by Gender:
// - Group employees by gender and return
// a map with gender as the key and a list of employees as the value.
System.out.println(RED+"*****Group employees by gender and return a map*****"+RESET);
list.stream()
 .collect(Collectors.groupingBy(Employee::getGender))
 .forEach((key,value)->System.out.println(key+" - "+value));

// 8. Count Male Employees:
// - Count the number of male employees.
System.out.println(RED+"*****Count the number of male employees.*****"+RESET);
long count = list.stream()

```

```

 .filter(employee -> employee.getGender().equals("Male"))
 .count();
System.out.println("Count : "+count);

// 9. Sum of All Salaries:
// - Calculate the total sum of salaries for all employees.
System.out.println(RED+"*****Calculate the total sum of salaries for all
employees.*****"+RESET);
double sum = list.stream()
 .mapToDouble(employee->employee.getSalary())
 .sum();
System.out.println("Sum : "+sum);

// 10. Sort Employees by Name:
// - Sort the employees by their names in alphabetical order.
System.out.println(RED+"*****Sort the employees by their names in alphabetical order.*****"+RESET);
list.stream().sorted(Comparator.comparing(Employee::getName))
 .forEach(System.out::println);

// 11. Sort Employees by Age:
// - Sort the employees by age in ascending order.
System.out.println(RED+"*****Sort the employees by age in ascending order.*****"+RESET);
list.stream()
 .sorted(Comparator.comparing(Employee::getAge))
 .forEach(System.out::println);

// 12. Sort Employees by Salary:
// - Sort the employees by salary in descending order.
System.out.println(RED+"*****Sort the employees by salary in descending order.*****"+RESET);
list.stream()
 .sorted(Comparator.comparing(Employee::getAge).reversed())
 .forEach(System.out::println);

// 13. Find Oldest Employee:
// - Find the oldest employee.
System.out.println(RED+"*****Find the oldest employee.*****"+RESET);
// int orElseThrow2 = list.stream().mapToInt(Employee::getAge).max().orElseThrow();
Employee employee = list.stream()
 .max((e1,e2)->(e1.getAge()-e2.getAge()))
 .get();
System.out.println(employee);
// System.out.println("Max Age : "+orElseThrow2);

// 14. Group Employees by Age:
// - Group employees into age groups (e.g., 20-30, 31-40, etc.)
// and return a map with age group as the key and a list of employees as the value.
System.out.println(RED+"*****Group employees into age groups (e.g., 20-30, 31-40, etc.)*****"+RESET);
list.stream()
 .collect(Collectors.groupingBy((t) ->
 {int age = t.getAge();
 if(age>=20 && age<=30)
 return "20-30";
 else if (age>=31 && age<=40)
 return "31-40";
 else

```

```

 return "40+";
 })).forEach((key,value)->System.out.println(key+ " - "+value));
// 15. Find Employees with a Specific Age:
// - Find all employees who are exactly 35 years old.
 System.out.println(RED+"***** Find all employees who are exactly 35 years
old.*****"+RESET);
 list.stream().filter(k->k.getAge()==35).forEach(System.out::println);

// 16. Calculate the Sum of Salaries by Gender:
// - Calculate the sum of salaries for each gender (i.e., male and female)
// and return a map with gender as the key and the sum of salaries as the value.
 System.out.println(RED+"***** Calculate the sum of salaries for each gender*****"+RESET);
 list.stream()
 .collect(Collectors.groupingBy(Employee::getGender ,
 Collectors.summingDouble(Employee::getSalary)))
 .forEach((key,value)->System.out.println(key + " - "+value));

// 17. Find Employees with Names Starting with "J":
// - Find all employees whose names start with the letter "E."
 System.out.println(RED+"*****Find all employees whose names start with the letter
J*****"+RESET);
 list.stream().filter(k->k.getName().startsWith("E")).forEach(System.out::println);

// 18. Calculate the Average Salary for Male and Female Employees:
// - Calculate the average salary separately for male
// and female employees and return a map with gender
// as the key and the average salary as the value.
 System.out.println(RED+"*****Calculate the average salary separately for male and
female*****"+RESET);
 list.stream()
 .collect(Collectors.groupingBy(Employee::getGender,
 Collectors.averagingDouble(Employee::getSalary)))
 .forEach((key,value)->System.out.println(key+" - "+value));

// 19. Find the Top N Highest Paid Employees:
// - Find the top N employees with the highest salaries.

 System.out.println(RED+"*****Find the top N employees with the highest salaries.*****"+RESET);
 list.stream().sorted((o1, o2) -> -(int)(o1.getSalary()-o2.getSalary()))
 .limit(5).forEach(System.out::println);

 list.stream()
 .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
 .limit(3)
 .forEach(System.out::println);

// 20. Retrieve Distinct Age Values:
// - Get a list of distinct ages of all employees.
 System.out.println(RED+"*****Get a list of distinct ages of all employees.*****"+RESET);

 list.stream().filter(k->!set.add(k.getAge())).forEach(System.out::println);

// 21. Find the Three Lowest-Paid Employees:
// - Find and display the names of the three employees with the lowest salaries.

```

```
System.out.println(RED+"*****Find and display the names of the three employees with the lowest salaries.*****"+RESET);
```

```
list.stream()
.sorted((o1,o2)->(int)(o1.getSalary()-o2.getSalary()))
.map(k->k.getName())
.limit(3)
.forEach(System.out::println);
```

```
// 22. Sort Employees by Name Length:
```

```
// - Sort employees by the length of their names (shortest to longest).
```

```
System.out.println(RED+"*****Sort employees by the length of their names (shortest to longest).*****"+RESET);
```

```
list.stream()
.sorted((o1, o2) ->(o1.getName().length()-o2.getName().length()))
.forEach(System.out::println);
```

```
// 23. Group Employees by Age Range:
```

```
// - Group employees into custom
```

```
// age ranges (e.g., 20-29, 30-39, etc.) and
```

```
// return a map with the age range as the key and a list of employees as the value.
```

```
System.out.println(RED+"*****Group employees into custom age ranges (e.g., 20-29, 30-39, etc.)*****"+RESET);
```

```
list.stream()
.collect(Collectors.groupingBy((t)->
{
 int age = (t).getAge();
 if(age>=20 && age<=29)
 return "20-29";
 else if(age>=30 && age <= 39)
 return "30-39";
 else
 return "40+";
}))
.forEach((key,value)->System.out.println(key+" - "+value));
```

```
// 24. Find the Average Salary of Employees Aged 30 or Younger:
```

```
// - Calculate the average salary of employees aged 30 or younger.
```

```
System.out.println(RED+"*****Calculate the average salary of employees aged 30 or younger.*****"+RESET);
```

```
double orElseThrow2 = list.stream()
.filter(emp->emp.getAge()<=30).mapToDouble(k->k.getSalary()).average().orElseThrow();
System.out.println(orElseThrow2);
```

```
// 25. Find the Names of Male Employees with Salaries Over $60,000:
```

```
// - Retrieve the names of male employees with salaries over $60,000.
```

```
System.out.println(RED+"*****Retrieve the names of male employees with salaries over $53,000.*****"+RESET);
```

```
list.stream()
.filter(e->e.getSalary()>=53000 && e.getGender().equals("Male"))
.map(k->k.getName())
.forEach(System.out::println);
```

```
// 26. Find the Youngest Female Employee:
```

```
// - Find the youngest female employee.
```

```
System.out.println(RED+"*****Find the youngest female employee.*****"+RESET);
Employee employee2 = list.stream()
 .filter(k->k.getGender().equals("Female"))
 .collect(Collectors.minBy((o1, o2) -> o1.getAge()-o2.getAge())).get();
System.out.println(employee2);
```

// 27. Retrieve the Names of Employees in Reverse Order:

```
// - Get a list of employee names in reverse order (from the last employee to the first).
System.out.println(RED+"*****Get a list of employee names in reverse order*****"+RESET);
List<String> collect = list.stream().map(k->k.getName()).collect(Collectors.toList());
Collections.reverse(collect);
System.out.println(collect);
```

// 28. Find the Highest Salary Among Female Employees:

```
// - Find the highest salary among female employees.
System.out.println(RED+"*****Find the highest salary among female employees."+RESET);
Employee employee3 = list.stream()
 .filter(k->k.getGender().equals("Female"))
 .collect(Collectors.maxBy((o1, o2) -> (int)(o1.getSalary()-o2.getSalary()))).get();
System.out.println(employee3);
```

// 29. Group Employees by Age and Gender:

```
// - Group employees by both age and gender and return a multi-level map.
System.out.println(RED+"*****Group employees by both age and gender and return a multi-level
map.*****"+RESET);
Map<String, Map<Integer, List<Employee>>> collect2 = list.stream()
 .collect(Collectors.groupingBy(Employee::getGender,Collectors.groupingBy(Employee::getAge)));

collect2.forEach((key,value)->
{
 value.forEach((k,v)->System.out.println(k+"-"+v));
 System.out.println(key+"-"+value);
});
```

// 30. Find the Sum of Salaries for Employees with Names Containing "Smith":

```
// - Calculate the sum of salaries for employees whose names contain the substring "Smith."
System.out.println(RED+"***** Calculate the sum of salaries for employees whose names contain the
substring Smith***"+RESET);
double sum2 =
list.stream().filter(k->k.getName().contains("Smith")).mapToDouble(k->k.getSalary()).sum();
System.out.println(sum2);
```

// 31. Find the Names of Employees Aged 30-40 with Salaries Between \$50,000 and \$60,000:

```
// - Retrieve the names of employees aged 30-40 with salaries between $50,000 and $60,000.
System.out.println(RED+"*****Retrieve the names of employees aged 30-40 with salaries between
$50,000 and $60,000.*****"+RESET);
list.stream()
 .filter(k->(k.getAge())>=30 && k.getAge()<=40)&&(k.getSalary())>=52000&&k.getSalary()<=60000))
 .forEach(System.out::println);
```

// 32. Calculate the Total Number of Employees:

```
// - Determine the total count of employees.
```

```
System.out.println(RED+"***** Determine the total count of employees.*****"+RESET);
System.out.println("Total Count of employees : "+list.stream().count());
```

```
// 33. Find the Most Common Age Among Employees:
// - Determine the age that is most common among the employees.
System.out.println(RED+"*****Determine the age that is most common among the
employees.*****"+RESET);
Integer orElseThrow3 = list.stream()
.collect(Collectors.groupingBy(Employee::getAge,Collectors.counting())) // grouping ages and count
.entrySet() // converting to set
.stream()
.max(Map.Entry.comparingByValue()) // finding max value in map
.map(Map.Entry::getKey) // getting key of max value
.orElseThrow(); // getting the key
System.out.println(orElseThrow3);
```

```
// 34. Find the Median Salary:
// - Calculate the median salary of all employees.
```

```
System.out.println(RED+"*****Calculate the median salary of all employees.*****"+RESET);
List<Employee> list2 = list.stream()
.sorted(Comparator.comparingDouble(Employee::getSalary)).toList();
int size = list2.size();
if(size%2==0)
{
double s = list2.get(size/2-1).getSalary();
double s1 = list2.get(size/2).getSalary();
System.out.println((s+s1)/2.0);
}
else {
System.out.println(list2.get(size/2).getSalary());
}
}
```

```
// 35. Group Employees by Age and Count:
// - Group employees by age and count the number of employees in each age group.
System.out.println(RED+"*****Group employees by age and count*****"+RESET);
list.stream().collect(Collectors.groupingBy(Employee::getAge,Collectors.counting()))
.forEach((key,value)->System.out.println(key+" - "+value));
```

```
// 36. Find the Employee with the Longest Name:
// - Find the employee with the longest name.
System.out.println(RED+"*****Find the employee with the longest name.*****"+RESET);
Employee employee4 = list.
stream()
.max((o1, o2) -> o1.getName().length() - o2.getName().length())
.get();
System.out.println(employee4);
```

```
// 37. Calculate the Sum of Salaries for Each Age:
// - Calculate the sum of salaries for each distinct age in a map.
System.out.println(RED+"*****Calculate the sum of salaries for each distinct age in a
map*****"+RESET);
set.clear();
list.stream()
.filter(k->!set.add(k.getAge()))
.collect(Collectors.groupingBy(Employee::getAge,Collectors.summingDouble(Employee::getSalary)))
```



```

.forEach((key,value)->System.out.println(key+" - "+value));

// 38. Sort Employees by Age, Then by Salary:
// - Sort employees first by age in ascending order, and then by salary in descending order.
System.out.println(RED+"*****Sort employees first by age in ascending order, and then by salary in
descending order*****"+RESET);
list.stream()
.sorted(Comparator.comparingInt(Employee::getAge)
.thenComparing(Comparator.comparingDouble(Employee::getSalary).reversed()))
.forEach(System.out::println);

// 39. Find Employees Whose Names Contain More Than One Word:
// - Retrieve employees whose names consist of more than one word.
System.out.println(RED+"*****Retrieve employees whose names consist of more than one
word*****"+RESET);
list.stream()
.filter(k->k.getName().length()>1)
.forEach(System.out::println);

// 40. Find the Two Highest Paid Female Employees:
// - Find and display the names of the two highest-paid female employees.
System.out.println(RED+"*****Find and display the names of the two highest-paid female
employees*****"+RESET);
list.stream()
.filter(k->k.getGender().equals("Female"))
.sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
.limit(2)
.forEach(System.out::println);

// 41. Find the Employee with the Highest Salary in Each Gender:
// - Find the employee with the highest salary for each gender (male and female).
System.out.println(RED+"*****Find the employee with the highest salary for each gender (male and
female)*****"+RESET);
list.stream()
.collect(Collectors.toMap(Employee::getGender, k->k,(e1, e2) ->
e1.getSalary()>=e2.getSalary()?e1:e2))
.forEach((key,value)->System.out.println(key+"-"+value));

// 42. Retrieve Employees with Unique Names:
// - Find employees with unique names (no duplicate names in the list).
System.out.println(RED+"*****Find employees with unique names (no duplicate names in the
list)*****"+RESET);
list.stream().filter(k->set2.add(k.getName()))
.forEach(System.out::println);
set2.clear();

// 43. Find the Names of Employees in Uppercase:
// - Get a list of employee names in uppercase.
System.out.println(RED+"*****Get a list of employee names in uppercase*****"+RESET);
list.stream()
.filter(k->k.getName().equals(k.getName().toUpperCase()))
.forEach(System.out::println);

// 44. Calculate the Salary Range (Min-Max) for Each Age Group:
// - Calculate the salary range (minimum and maximum) for each distinct age group.

```

```

System.out.println(RED+"*****Calculate the salary range (minimum and maximum) for each distinct age
group.*****"+RESET);
list.stream()
.collect(Collectors.
groupingBy(Employee::getAge,
Collectors.collectingAndThen(Collectors.toList(),
employees->
{
double min = employees.stream()
.mapToDouble(Employee::getSalary)
.min().orElseThrow();
double maxs = employees.stream()
.mapToDouble(Employee::getSalary)
.max().orElseThrow();
Map<String,Double> map = new HashMap<>();
map.put("min", min);
map.put("max", maxs);
return map;
}))).
forEach((age,salary)->
System.out.println
("Age : "+age+" - "+"Min Salary : "+salary.get("min")+ " Max Salary : "+salary.get("max"))));

```

// 45. Filter Employees by First Name Initial:

// - Retrieve employees whose first name starts with a specific initial (e.g., 'A').

```

System.out.println(RED+"*****Retrieve employees whose first name starts with a specific initial (e.g.,
'E').*****"+RESET);
list.stream()
.filter(k->k.getName().startsWith("E"))
.forEach(System.out::println);

```

// 46. Group Employees by Age and List Only Unique Salaries:

// - Group employees by age and display a list of unique salaries for each age group.

```

System.out.println(RED+"*****Group employees by age and display a list of unique salaries for each
age group.*****"+RESET);
list.stream()
.collect(Collectors
.groupingBy(Employee::getAge,
Collectors.mapping(Employee::getSalary, Collectors.toSet()))
.forEach((key,value)->System.out.println("Age : "+key +" Salary : "+value)));

```

// 47. Find Employees with the Same Salary:

// - Identify and display employees who have the same salary.

```

System.out.println(RED+"*****Identify and display employees who have the same salary.*****"+RESET);
list.stream()
.collect(Collectors.groupingBy(Employee::getSalary))
.entrySet()
.stream() // k is entry
.filter(k -> k.getValue().size()>1)
.forEach(entry -> // Map (Double , List<>)
{
System.out.println(entry.getKey());
entry.getValue().forEach(System.out::println);
});

```

```
// 48. Find the Employee with the Shortest Name Among Male Employees:
// - Find the male employee with the shortest name.
System.out.println(RED+"*****Find the male employee with the shortest name.*****"+RESET);
Employee employee5 = list.stream()
 .filter(k->k.getGender().equals("Male"))
 .min((o1, o2) -> Integer.compare(o1.getName().length(), o2.getName().length()))
 .orElseThrow();
System.out.println(employee5);

// 49. Find the Most Common Salary Value:
// - Determine the salary value that appears most frequently among the employees.
System.out.println(RED+"*****Determine the salary value that appears most frequently among the
employees.*****"+RESET);
Double orElseThrow4 = list.stream()
 .collect(Collectors.groupingBy(Employee::getSalary , Collectors.counting()))
 .entrySet()
 .stream()
 .max(Map.Entry.comparingByValue())
 .map(Map.Entry::getKey).orElseThrow();
System.out.println(orElseThrow4);

// 50. Find the Oldest Employee with the Lowest Salary:
// - Find the oldest employee with the lowest salary.
System.out.println(RED+"*****Find the oldest employee with the lowest salary.*****"+RESET);
Employee employee6 = list.stream()
 .filter(k-> k.getAge()==list.stream().mapToInt(Employee::getAge).max().orElseThrow())
 .min(Comparator.comparingDouble(Employee::getSalary)).get();
System.out.println(employee6);

// 51. Filter Employees by Gender:
// - Retrieve a list of all female employees.
System.out.println(RED+"***** Retrieve a list of all female employees.*****"+RESET);
list
 .stream()
 .filter(k->k.getGender().equals("Female"))
 .forEach(System.out::println);

// 52. Filter Employees by Age:
// - Get a list of employees older than 30 years.
System.out.println(RED+"*****Get a list of employees older than 30 years.*****"+RESET);
list
 .stream()
 .filter(k->k.getAge()>30)
 .forEach(System.out::println);

// 53. Filter Employees by Salary:
// - Find employees with a salary greater than $50,000.
System.out.println(RED+"*****Find employees with a salary greater than $50,000.*****"+RESET);
list.stream()
 .filter(k->k.getSalary()>50000)
 .forEach(System.out::println);
```

```
// 54. Map Employee Names:
// - Create a list of employee names (Strings).
System.out.println(RED+"*****Create a list of employee names (Strings).*****"+RESET);
list.stream()
.map(k->k.getName())
.toList().forEach(System.out::println);

// 55. Calculate Average Salary:
// - Calculate the average salary of all employees.
System.out.println(RED+"*****Calculate the average salary of all employees.*****"+RESET);
Double collect3 = list.stream()
.collect(Collectors.averagingDouble(Employee::getSalary));
System.out.println(collect3);

// 56. Find Maximum Salary:
// - Find the employee with the highest salary.
System.out.println(RED+"*****Find the employee with the highest salary.*****"+RESET);
Employee employee7= list.stream()
.collect(Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary))).get();
System.out.println(employee7);

// 57. Group Employees by Gender:
// - Group employees by gender
// and return a map with gender as the key and a list of employees as the value.
System.out.println(RED+"*****return a map with gender as the key and a list of
employees*****"+RESET);
list.stream()
.collect(Collectors.groupingBy(Employee::getGender))
.forEach((k,v)->System.out.println(k+" - "+v));

// 58. Count Male Employees:
// - Count the number of male and female employees.
System.out.println(RED+"*****Count the number of male and female employees.*****"+RESET);
list.stream()
.collect(Collectors.groupingBy(Employee::getGender , Collectors.counting()))
.forEach((k,v)->System.out.println(k+" - "+v));

// 59. Sum of All Salaries:
// - Calculate the total sum of salaries for all employees.
System.out.println(RED+"*****Calculate the total sum of salaries for all employees.*****"+RESET);
Double collect4 = list.stream()
.collect(Collectors.summingDouble(Employee::getSalary));
System.out.println(collect4);

// 60. Sort Employees by Name:
// - Sort the employees by their names in alphabetical order.
System.out.println(RED+"***** Sort the employees by their names in alphabetical
order.*****"+RESET);
list.stream()
.sorted(Comparator.comparing(Employee::getName)).forEach(System.out::println);

// 61. Find the Employee with the Highest Salary in Each Gender:
// - Find the employee with the highest salary for each gender (male and female).
System.out.println(RED+"*****Find the employee with the highest salary for each gender (male and
```

```

female).*****"+RESET);
list.stream()
.collect(Collectors.toMap(Employee::getGender, t -> t,(t, u) -> t.getSalary()>=u.getSalary() ? t : u))
.forEach((k,v)->System.out.println(k+" - "+v));

// 62. Retrieve Employees with Unique Names:
// - Find employees with unique names (no duplicate names in the list).
System.out.println(RED+"*****Find employees with unique names (no duplicate names in the
list).*****"+RESET);
list
.stream()
.collect(Collectors.groupingBy(Employee::getName , Collectors.counting()))
.entrySet()
.stream()
.filter(k->k.getValue()==1)
.forEach(k->System.out.println(k.getKey()));

// 63. Find the Names of Employees in Uppercase:
// - Get a list of employee names in uppercase.
System.out.println(RED+"*****Get a list of employee names in uppercase.*****"+RESET);
list
.stream()
.filter(k->k.getName().equals(k.getName().toUpperCase()))
.forEach(System.out::println);

// 64. Calculate the Salary Range (Min-Max) for Each Age Group:
// - Calculate the salary range (minimum and maximum) for each distinct age group.
System.out.println(RED+"*****Calculate the salary range (minimum and maximum) for each distinct
age group.*****"+RESET);
Map<Integer,Map<String,Double>> collect5 = list.stream()
.collect(Collectors.groupingBy(Employee::getAge,
Collectors.collectingAndThen(Collectors.toList(), k ->
{
Double maxx = k.stream().mapToDouble(k2->k2.getSalary()).max().getAsDouble();
Double min = k.stream().mapToDouble(k1->k1.getSalary()).min().getAsDouble();
Map<String, Double> map = new HashMap<>();
map.put("max", maxx);
map.put("min", min);
return map;
}
)));
collect5.forEach((age,map)->
{
System.out.print("Age : "+age+" - ");
System.out.println("Minimum : "+map.get("min")+" Maximum : "+map.get("max"));
});

// 65. Filter Employees by First Name Initial:
// - Retrieve employees whose first name starts with a specific initial (e.g., 'E').
System.out.println(RED+"*****Retrieve employees whose first name starts with a specific initial
*****"+RESET);
list.stream()
.filter(k->k.getName().startsWith("E"))
.forEach(System.out::println);

```

```
// 66. Group Employees by Age and List Only Unique Salaries:
// - Group employees by age and display a list of unique salaries for each age group.
System.out.println(RED+"*****Group employees by age and display a list of unique salaries for each
age group.*****"+RESET);
list.stream()
.collect(Collectors.groupingBy(Employee::getAge,
Collectors.mapping(Employee::getSalary, Collectors.toSet()))))
.forEach((k,v)->System.out.println(k+" - "+v));

// 67. Find Employees with the Same Salary:
// - Identify and display employees who have the same salary.
System.out.println(RED+"*****Identify and display employees who have the same
salary.*****"+RESET);
list.stream()
.collect(Collectors.groupingBy(Employee::getSalary))
.entrySet()
.stream()
.filter(k->k.getValue().size()>1)
.forEach(t ->
{
System.out.println(t.getKey() + " " + t.getValue());
});

// 68. Find the Employee with the Shortest Name Among Male Employees:
// - Find the male employee with the shortest name.
System.out.println(RED+"*****Find the male employee with the shortest name.*****"+RESET);
Employee employee8 = list.stream().filter(k->k.getGender().equals("Male"))
.min(Comparator.comparing(Employee::getName)).get();
System.out.println(employee8);

// 69. Find the Most Common Salary Value:
// - Determine the salary value that appears most frequently among the employees.
System.out.println(RED+"*****Determine the salary value that appears most frequently among the
employees.*****"+RESET);
Double double1 = list.stream()
.collect(Collectors.groupingBy(Employee::getSalary , Collectors.counting()))
.entrySet()
.stream()
.max(Map.Entry.comparingByValue())
.map(Map.Entry::getKey).get();
System.out.println(double1);

// 70. Find the Oldest Employee with the Lowest Salary:
// - Find the oldest employee with the lowest salary.
System.out.println(RED+"*****Find the oldest employee with the lowest salary.*****"+RESET);
Employee employee9 = list.stream()
.filter(k->k.getAge()==list.stream().mapToInt(Employee::getAge).max().getAsInt())
.min(Comparator.comparingDouble(Employee::getSalary)).get();
System.out.println(employee9);

// 71. Find the Most Common Age Among Employees:
// - Determine the age that is most common among the employees.
System.out.println(RED+"*****Determine the age that is most common among the
employees.*****"+RESET);
Integer integer = list.stream()
```

```
.collect(Collectors.groupingBy(Employee::getAge , Collectors.counting()))
.entrySet()
.stream()
.max(Map.Entry.comparingByValue())
.map(Map.Entry::getKey)
.get();
System.out.println(integer);
```

// 72. Find the Employee with the Longest Name:

// - Find the employee with the longest name.

```
System.out.println(RED+"*****Find the employee with the longest name.*****"+RESET);
Employee employee10 = list.stream()
.max(Comparator.comparingInt(value -> value.getName().length())).get();
System.out.println(employee10);
```

// 73. Find Employees with Palindromic Names:

// - Retrieve employees whose names are palindromes (e.g., "Anna" or "Bob").

```
System.out.println(RED+"*****Retrieve employees whose names are palindromes*****"+RESET);
list.stream()
.filter(k->
{
String mainString = k.getName().toLowerCase();
StringBuffer sr = new StringBuffer(mainString);
return mainString.contentEquals(sr.reverse());
}).forEach(System.out::println);
```

// 74. Calculate the Sum of Salaries for Employees with Odd Ages:

// - Calculate the sum of salaries for employees with odd ages.

```
System.out.println(RED+"*****the sum of salaries for employees with odd ages.*****"+RESET);
double sum3 = list.stream()
.filter(k->k.getAge()%2!=0)
.mapToDouble(Employee::getSalary)
.sum();
System.out.println(sum3);
```

// 75. Find the Employee with the Highest Salary Whose Name Contains "Smith":

// - Find the employee with the highest salary whose name contains the word "Smith."

```
System.out.println(RED+"*****Find the employee with the highest salary whose name contains the word
'Smith.'*****"+RESET);
Employee employee11 = list.stream()
.filter(k->k.getName().contains("Smith"))
.max(Comparator.comparingDouble(Employee::getSalary)).get();
System.out.println(employee11);
```

// 76. Group Employees by the First Letter of Their Names:

// - Group employees by the first letter

// of their names and return a map with

// the letter as the key and a list of employees as the value.

```
System.out.println(RED+"*****Group employees by the first letter of their names*****"+RESET);
list.stream()
.collect(Collectors.groupingBy(t -> t.getName().charAt(0)))
.forEach((k,v)->System.out.println(k+" - "+v));
```

// 77. Find the Employee with the Shortest Name:

// - Find the employee with the shortest name.

```
System.out.println(RED+"*****Find the employee with the shortest name.*****"+RESET);
Employee employee12 = list.stream()
 .min(Comparator.comparingInt(value -> value.getName().length()))
 .get();
System.out.println(employee12);
```

```
// 78. Calculate the Average Salary of Employees Whose Names Start with "E":
// - Calculate the average salary of employees whose names start with the letter "E."
System.out.println(RED+"*****Calculate the average salary of employees whose names start with the
letter \"E.\""+RESET);
Double collect6 = list.stream()
 .filter(k->k.getName().startsWith("E"))
 .collect(Collectors.averagingDouble(Employee::getSalary));
System.out.println(collect6);
```

```
// 79. Filter Employees by Age Range:
// - Filter employees
// based on a custom age range (e.g., between 25 and 35 years old).
System.out.println(RED+"*****based on a custom age range (e.g., between 25 and 35 years
old)*****"+RESET);
list.stream()
 .filter(k->k.getAge()>=25 && k.getAge()<=35)
 .forEach(System.out::println);
```

```
// 80. Group Employees by the First Two Letters of Their Names:
// - Group employees by the first two letters
// of their names and
// return a map with the letters as the key and a list of employees as the value.
System.out.println(RED+"*****Group employees by the first two letters*****"+RESET);
list.stream()
 .collect(Collectors.groupingBy(k->k.getName().substring(0, 2)))
 .forEach((k,v)->System.out.println(k+" - "+v));
```

```
// 81. Find the Employee with the Longest Name Whose Salary Is Below $70,000:
// - Find the employee with the longest name whose salary is below $70,000.
System.out.println(RED+"*****Find the employee with the longest name whose salary is below
$70,000.**"+RESET);
Employee employee13 = list.stream().filter(k->k.getSalary()<70000)
 .max(Comparator.comparingInt(k->k.getName().length())).get();
System.out.println(employee13);
```

```
// 82. Calculate the Average Salary of Employees Whose Names End with "son":
// - Calculate the average salary of employees whose names end with the suffix "son."
System.out.println(RED+"*****Calculate the average salary of employees whose names end with the
suffix \"son.\""+RESET);
Double collect7 = list.stream()
 .filter(k->k.getName()
 .endsWith("na")).collect(Collectors.averagingDouble(Employee::getSalary));
System.out.println(collect7);
```

```
// 83. Group employees by the number of
// words in their names (e.g., one-word names, two-word names, etc.)
// and return a map with the count as the key and a list of employees as the value.
list.stream()
 .collect(Collectors.groupingBy(k->k.getName().length()))
```



```
.forEach((k,v)->System.out.println(k+" - "+v));
```

```
// 84. Calculate the Average Salary of Employees Whose Names Contain Both "A" and "E":
```

```
// - Calculate the average salary of employees whose names contain both the letters "A" and "E."
```

```
System.out.println(RED+"*****the average salary of employees whose names contain both the letters
\"A\" and \"E.\"*****"+RESET);
```

```
double asDouble = list.stream()
```

```
.filter(k->k.getName().contains("A")&&k.getName().contains("E"))
```

```
.mapToDouble(Employee::getSalary).average().orElse(0.0);
```

```
System.out.println(asDouble);
```

```
}
```

```
}
```

